

Dokumentacja techniczna cz.1

1. Środowisko i technologie

Silnik	Unity
Wersja edytora	Unity 6000.0.58f2
Język programowania	C#
IDE	Visual Studio
Interfejs użytkownika	UI Toolkit (Unity)

2. Struktura projektu

Struktura folderów Assets zawiera wszystkie autorskie elementy projektu oraz wkład zespołu w jego realizację, w tym skrypty, zasoby graficzne, dźwiękowe oraz elementy interfejsu użytkownika wykorzystywane w aplikacji. Podczas tworzenia gry nie wykorzystano zewnętrznych darmowych assetów – wszystkie grafiki oraz materiały audio zostały przygotowane na potrzeby projektu. Katalog Assets został podzielony na podfoldery grupujące zasoby według ich przeznaczenia funkcjonalnego.

Assets/
├─ Animations
├─ Art
├─ Font
├─ Resources
│ └─ Prefabs
│ ├─ Layout
│ └─ SkillCheck
├─ Scenes
├─ Scripts
├─ Settings
├─ SFX
├─ UI
└─ UI Toolkit

Opis folderów:

Animations	Zawiera pliki animacji wykorzystywane przez obiekty w grze.
Art	Zasoby graficzne projektu (sprity, tekstury).
Font	Czcionki używane w interfejsie użytkownika.
Resources	Zasoby ładowane dynamicznie w czasie działania aplikacji.
Resources/Prefabs	Prefaby obiektów ładowanych w sposób dynamiczny.
Resources/Prefabs/Layout	Prefaby odpowiedzialne za układ elementów interfejsu.
Resources/Prefabs/SkillCheck	Prefaby związane z mechaniką Skill Check.
Scenes	Sceny Unity definiujące poszczególne etapy działania aplikacji.

Scripts	Autorskie skrypty języka C# implementujące logikę aplikacji.
Settings	Ustawienia konfiguracyjne projektu.
SFX	Pliki dźwiękowe i efekty audio.
UI	Zasoby graficzne interfejsu użytkownika.
UI Toolkit	Pliki i ustawienia związane z frameworkiem UI Toolkit.

3. Opis scen oraz obiektów na scenach

W projekcie znajdują się trzy sceny:

MainMenu
QuarkTestScene
SampleScene

Niektóre obiekty na scenach zostały utworzone jako Prefab, aby można było je użyć na innych scenach.

3.1 Main Menu

Scena odpowiada za wyświetlenie menu głównego aplikacji oraz obsługę interakcji użytkownika przed rozpoczęciem rozgrywki. Zawiera elementy interfejsu użytkownika, system dźwięku oraz podstawowe komponenty zarządzające przejściami pomiędzy scenami.

Scena zawiera:

Main Camera

Główna kamera sceny. Odpowiada za renderowanie interfejsu użytkownika oraz elementów wizualnych menu. Kamera posiada komponent `AudioListener`, umożliwiający odtwarzanie dźwięków w scenie.

Menu

Obiekt odpowiedzialny za wyświetlanie interfejsu użytkownika menu głównego. Został zrealizowany przy użyciu `Unity UI Toolkit`. Obiekt korzysta z ustawień panelu (`Panel Settings`) oraz zasobu UI definiującego układ menu. Dodatkowy skrypt zarządza obsługą dźwięków oraz komunikacją z kontrolerem scen.

Audio

Obiekt nadrzędny systemu dźwiękowego sceny. Zawiera źródła audio wykorzystywane do odtwarzania muzyki w tle oraz efektów dźwiękowych.

EventSystem

Obiekt systemowy odpowiedzialny za obsługę wejścia użytkownika (mysz, klawiatura). Umożliwia poprawne działanie elementów interfejsu użytkownika oraz interakcji z menu.

SceneChanger (Prefab)

Obiekt odpowiedzialny za zarządzanie przejściami pomiędzy scenami aplikacji. Wykorzystuje prefab zawierający logikę zmiany aktualnej sceny na podstawie akcji użytkownika.

Background (Prefab)

Obiekt typu canvas wykorzystany do wyświetlenia tła menu.

3.2 QuarkTestScene

Scena wykorzystywana wyłącznie do przetestowania zaimplementowanego typu danych (QuarkType).

Scena zawiera:

Main Camera

Główna kamera sceny (W przypadku tej sceny nie istotny element).

GameObject

Obiekt zawierający podpięty skrypt testujący „QuarkTestNode”. Przy kompilacji następuje asercja QuarkType’a, a na konsoli ukazane są poszczególne wyniki testów.

3.3 SampleScene

Główna scena rozgrywki. Odpowiada za wyświetlanie faktycznej gry oraz interfejsu użytkownika. W scenie zastosowano renderowanie 2D oraz globalne oświetlenie sceny.

Scena zawiera:

Main Camera

Główna kamera sceny odpowiedzialna za renderowanie elementów dwuwymiarowych oraz interfejsu użytkownika. Kamera działa w trybie posiada komponent AudioListener, umożliwiający odtwarzanie dźwięków w scenie.

Global Light 2D

Obiekt oświetlenia globalnego wykorzystywany w rendererze 2D. Zapewnia jednolite oświetlenie wszystkich elementów wizualnych na scenie.

SkillCheck (Prefab)

Obiekt odpowiedzialny za mechanikę „SkillCheck”. Domyślnie pozostaje nieaktywny i jest włączany jedynie w momencie wywołania przez odpowiedni trigger w grze.

Background (Prefab)

Obiekt odpowiedzialny za wyświetlanie tła sceny. Wykorzystuje prefab zawierający komponenty interfejsu użytkownika oraz logikę umożliwiającą dynamiczną zmianę tła w zależności od stanu aplikacji.

Layout (Prefab)

Główny kontener interfejsu użytkownika sceny. Zawiera dokument UI Toolkit definiujący układ elementów graficznych oraz skrypty zarządzające stanami interfejsu, reakcją na zdarzenia oraz integracją z centralnym systemem logiki aplikacji.

SceneChanger (Prefab)

Obiekt odpowiedzialny za obsługę przejść pomiędzy scenami aplikacji. Prefab zawiera logikę umożliwiającą zmianę aktywnej sceny w odpowiedzi na zdarzenia generowane przez interfejs użytkownika lub system gry.

Audio Source (Prefab)

Obiekt zawierający komponent AudioSource z przypisanym dźwiękiem odpowiedzialnym za obsługę standardowych interakcji kliknięcia w trakcie rozgrywki.

Crit SFX (Prefab)

Obiekt zawierający komponent AudioSource z przypisanym efektem dźwiękowym wyzwalanym w momencie wystąpienia kliknięcia krytycznego.

ClickerObject (Prefab)

Główny obiekt sceny, stanowiący centralny element rozgrywki. Reprezentuje surowiec, który jest podstawą mechaniki gry opartej na klikaniu. Do obiektu przypisany jest skrypt RaptorCore, pełniący rolę głównego kontrolera logiki rozgrywki.

ClickerObject agreguje kluczowe komponenty sceny, w tym obiekty odpowiedzialne za obsługę dźwięku, układ interfejsu użytkownika (Layout) oraz mechanikę testu umiejętności (SkillCheck). Obiekt przechowuje również informacje dotyczące postępu gracza, takie jak ilość zdobytego złota oraz liczba punktów prestiżu.

IdleManager (Prefab)

Obiekt odpowiedzialny za obsługę mechaniki fabryk oraz systemu rozgrywki typu idle. Zarządza procesami generowania zasobów w czasie, niezależnie od bezpośrednich interakcji użytkownika.

CharacterProgression (Prefab)

Obiekt odpowiedzialny za zarządzanie aktualnym postępem gracza. Przechowuje informacje dotyczące zdobytego doświadczenia, odblokowanych oraz aktualnie wykupionych umiejętności, a także aktywnych wzmocnień (buffów). Komponent ten integruje system rozwoju postaci z główną logiką rozgrywki oraz wpływa na dostępne mechaniki i parametry gry.

4. Dokumentacja kodu

Dokumentacja techniczna kodu dostępna jest w osobnym pliku wygenerowanym przez Doxygen.

Gra clicker - Timeless

Wygenerowano za pomocą Doxygen 1.16.1

1 Struktura katalogów	1
1.1 Katalogi	1
2 Indeks hierarchiczny	3
2.1 Hierarchia klas	3
3 Indeks klas	5
3.1 Lista klas	5
4 Indeks plików	9
4.1 Lista plików	9
5 Dokumentacja katalogów	11
5.1 Dokumentacja katalogu Assets	11
5.2 Dokumentacja katalogu Scripts	11
6 Dokumentacja klas	13
6.1 Dokumentacja klasy ActivePotionsUI	13
6.1.1 Opis szczegółowy	14
6.1.2 Dokumentacja funkcji składowych	14
6.1.2.1 CreatePotionDisplay()	14
6.1.2.2 GetPotionColor()	15
6.1.2.3 GetPotionSprite()	15
6.1.2.4 RebuildPanel()	15
6.1.2.5 RefreshPanel()	15
6.1.2.6 Start()	15
6.1.2.7 Update()	16
6.1.2.8 UpdateActivePotionsDisplay()	16
6.1.2.9 UpdatePotionTimers()	16
6.1.3 Dokumentacja atrybutów składowych	16
6.1.3.1 activePotionDisplays	16
6.1.3.2 activePotionsPanel	16
6.1.3.3 cincoPotionSprite	16
6.1.3.4 cuatroPotionSprite	16
6.1.3.5 dosPotionSprite	17
6.1.3.6 idleManager	17
6.1.3.7 seisPotionSprite	17
6.1.3.8 tresPotionSprite	17
6.1.3.9 uiDocument	17
6.1.3.10 unoPotionSprite	17
6.2 Dokumentacja klasy AutomatronSkillTree	17
6.2.1 Opis szczegółowy	19
6.2.2 Dokumentacja konstruktora i destruktor	19
6.2.2.1 AutomatronSkillTree()	19
6.2.3 Dokumentacja funkcji składowych	20

6.2.3.1 CanUnlock()	20
6.2.3.2 GetDependencies()	20
6.2.3.3 OnSkillClicked()	20
6.2.3.4 UpdateVisual()	21
6.3 Dokumentacja klasy AutoSaveSystem	21
6.3.1 Opis szczegółowy	22
6.3.2 Dokumentacja funkcji składowych	22
6.3.2.1 DeleteSave()	22
6.3.2.2 GetAutoSave()	22
6.3.2.3 LazyGetAutoSave()	22
6.3.2.4 LoadAll()	23
6.3.2.5 LoadGame()	23
6.3.2.6 SaveAll()	23
6.3.2.7 SaveGame()	23
6.3.2.8 SavePathS()	23
6.3.2.9 SetAutoSave()	24
6.3.3 Dokumentacja atrybutów składowych	24
6.3.3.1 __AutoSave	24
6.3.3.2 loaded	24
6.3.3.3 path	24
6.3.4 Dokumentacja właściwości	24
6.3.4.1 AutoSave	24
6.3.4.2 SaveFileName	24
6.3.4.3 SavePath	25
6.4 Dokumentacja klasy BrickMovementScript	25
6.4.1 Opis szczegółowy	26
6.4.2 Dokumentacja funkcji składowych	26
6.4.2.1 OnTriggerEnter2D()	26
6.4.2.2 OnTriggerExit2D()	26
6.4.2.3 Start()	26
6.4.2.4 Update()	26
6.4.3 Dokumentacja atrybutów składowych	27
6.4.3.1 overlap	27
6.4.3.2 speed	27
6.4.3.3 sr	27
6.5 Dokumentacja klasy BuffManager	27
6.5.1 Opis szczegółowy	28
6.5.2 Dokumentacja konstruktora i destruktor	28
6.5.2.1 BuffManager()	28
6.5.3 Dokumentacja funkcji składowych	28
6.5.3.1 ApplyBuff()	28
6.5.3.2 RegisterBuff()	29
6.5.3.3 RemoveBuff()	29

6.5.4 Dokumentacja atrybutów składowych	29
6.5.4.1 buffBar	29
6.5.4.2 buffs	29
6.6 Dokumentacja klasy BuffVisual	29
6.6.1 Opis szczegółowy	30
6.6.2 Dokumentacja konstruktora i destruktora	30
6.6.2.1 BuffVisual()	30
6.6.3 Dokumentacja funkcji składowych	30
6.6.3.1 Hide()	30
6.6.3.2 Show()	31
6.6.4 Dokumentacja atrybutów składowych	31
6.6.4.1 root	31
6.6.4.2 skillId	31
6.6.4.3 valueLabel	31
6.7 Dokumentacja klasy CharacterClass	31
6.7.1 Opis szczegółowy	36
6.7.2 Dokumentacja funkcji składowych	36
6.7.2.1 ActiveIdle()	36
6.7.2.2 Addict()	37
6.7.2.3 AlanWake()	37
6.7.2.4 AlwaysWinner()	37
6.7.2.5 ApplySkillChanges()	37
6.7.2.6 ChampionOfClicks()	38
6.7.2.7 ChickenDinner()	38
6.7.2.8 ChristmasBonus()	38
6.7.2.9 CriticalMass()	38
6.7.2.10 DeathDose()	39
6.7.2.11 DeserializeFromStream()	39
6.7.2.12 EnableAlanWake()	39
6.7.2.13 EnableExpBoost()	39
6.7.2.14 EnableGenius()	39
6.7.2.15 Entrepreneur()	40
6.7.2.16 FailToWin()	40
6.7.2.17 Failure()	40
6.7.2.18 FailureGrind()	40
6.7.2.19 GainExp()	41
6.7.2.20 Genius()	41
6.7.2.21 GetCriticalChance()	41
6.7.2.22 GetCurrentExp()	41
6.7.2.23 GetLevel()	41
6.7.2.24 GetMaxExpCap()	41
6.7.2.25 GetSkillCheckChance()	41
6.7.2.26 HardWorker()	42

6.7.2.27 HungryForMore()	42
6.7.2.28 HungryWolf()	42
6.7.2.29 JustBastard()	42
6.7.2.30 LuckyBastard()	43
6.7.2.31 MarketplaceGenius()	43
6.7.2.32 MichealScott()	43
6.7.2.33 MortalClicker()	43
6.7.2.34 Multitasking()	44
6.7.2.35 NewLevelCap()	44
6.7.2.36 NoMatterWhat()	44
6.7.2.37 OneForEveryone()	44
6.7.2.38 PassiveAgressive()	45
6.7.2.39 Pedator()	45
6.7.2.40 PushToTheLimit()	45
6.7.2.41 ReactionTest()	45
6.7.2.42 ResetCharacter()	46
6.7.2.43 SerializeToStream()	46
6.7.2.44 SetLevel()	46
6.7.2.45 Shark()	46
6.7.2.46 SkillBasedClicking()	47
6.7.2.47 Start()	47
6.7.2.48 Symbiosis()	47
6.7.2.49 UnskilledPredator()	47
6.7.2.50 Update()	47
6.7.2.51 UpdateTreeCounter()	48
6.7.2.52 WhatEyesDontSee()	48
6.7.3 Dokumentacja atrybutów składowych	48
6.7.3.1 activeIdle	48
6.7.3.2 alwaysWinner	48
6.7.3.3 boostedChance	48
6.7.3.4 buffManager	48
6.7.3.5 championOfClicks	49
6.7.3.6 chickenDinner	49
6.7.3.7 christmasBonus	49
6.7.3.8 criticalChance	49
6.7.3.9 currentExp	49
6.7.3.10 deathDose	49
6.7.3.11 expBoost	49
6.7.3.12 failToWin	49
6.7.3.13 failureGrind	50
6.7.3.14 hardWorker	50
6.7.3.15 hungryWolf	50
6.7.3.16 komboBoost	50

6.7.3.17 level	50
6.7.3.18 marketplaceGenius	50
6.7.3.19 maxExpCap	50
6.7.3.20 maxLvlCap	50
6.7.3.21 michealScott	51
6.7.3.22 mortalClicker	51
6.7.3.23 multitasking	51
6.7.3.24 noMatterWhat	51
6.7.3.25 oneForEveryone	51
6.7.3.26 passiveAgressive	51
6.7.3.27 potionBoost	51
6.7.3.28 potionRandomChance	51
6.7.3.29 productionIdleAgressiveBonus	52
6.7.3.30 productionIdleBonus	52
6.7.3.31 productionIdlePedatorBonus	52
6.7.3.32 Raptorcore	52
6.7.3.33 reactionTest	52
6.7.3.34 sellingBonus	52
6.7.3.35 sellingHardBonus	52
6.7.3.36 skillCheckChance	52
6.7.3.37 skillCheckReduce	53
6.7.3.38 skillpoints	53
6.7.3.39 skillPointsLimit	53
6.7.3.40 symbiosis	53
6.7.3.41 T1	53
6.7.3.42 T2	53
6.7.3.43 T3	53
6.7.3.44 unskilledPredator	53
6.7.3.45 whatEyesDontSee	54
6.7.4 Dokumentacja właściwości	54
6.7.4.1 SaveKey	54
6.8 Dokumentacja klasy ComboVisual	54
6.8.1 Opis szczegółowy	55
6.8.2 Dokumentacja funkcji składowych	55
6.8.2.1 Start()	55
6.8.2.2 UpdateCombo()	55
6.8.3 Dokumentacja atrybutów składowych	55
6.8.3.1 comboContainer	55
6.8.3.2 comboLabel	56
6.8.3.3 isVisible	56
6.8.3.4 ui	56
6.9 Dokumentacja klasy ContentPageManager	56
6.9.1 Opis szczegółowy	57

6.9.2 Dokumentacja konstruktora i destruktora	58
6.9.2.1 <code>ContentPageManager()</code>	58
6.9.3 Dokumentacja funkcji składowych	58
6.9.3.1 <code>Initialize()</code>	58
6.9.3.2 <code>ShowPage()</code>	58
6.9.4 Dokumentacja atrybutów składowych	58
6.9.4.1 <code>audioSource</code>	58
6.9.4.2 <code>btn1</code>	58
6.9.4.3 <code>btn2</code>	59
6.9.4.4 <code>btn3</code>	59
6.9.4.5 <code>btn4</code>	59
6.9.4.6 <code>btn5</code>	59
6.9.4.7 <code>btn6</code>	59
6.9.4.8 <code>btn7</code>	59
6.9.4.9 <code>contentPage1</code>	59
6.9.4.10 <code>contentPage2</code>	59
6.9.4.11 <code>contentPage3</code>	60
6.9.4.12 <code>contentPage4</code>	60
6.9.4.13 <code>contentPage5</code>	60
6.9.4.14 <code>contentPage6</code>	60
6.9.4.15 <code>contentPage7</code>	60
6.9.4.16 <code>ui</code>	60
6.10 Dokumentacja klasy <code>CritVisualGenerator</code>	60
6.10.1 Opis szczegółowy	61
6.10.2 Dokumentacja funkcji składowych	61
6.10.2.1 <code>SpawnCrit()</code>	61
6.10.2.2 <code>Start()</code>	61
6.10.3 Dokumentacja atrybutów składowych	62
6.10.3.1 <code>critContainer</code>	62
6.10.3.2 <code>ui</code>	62
6.11 Dokumentacja klasy <code>DynamicBackground</code>	62
6.11.1 Opis szczegółowy	63
6.11.2 Dokumentacja funkcji składowych	63
6.11.2.1 <code>ChangeBackground()</code>	63
6.11.2.2 <code>ChangeBackgroundRoutine()</code>	63
6.11.2.3 <code>Start()</code>	64
6.11.2.4 <code>ZoomEffect()</code>	64
6.11.3 Dokumentacja atrybutów składowych	64
6.11.3.1 <code>backgrounds</code>	64
6.11.3.2 <code>changeInterval</code>	64
6.11.3.3 <code>currentIndex</code>	64
6.11.3.4 <code>enableZoom</code>	64
6.11.3.5 <code>fadeDuration</code>	64

6.11.3.6 maxZoomValue	65
6.11.3.7 rawImage	65
6.11.3.8 rectTransform	65
6.11.3.9 zoomValue	65
6.12 Dokumentacja klasy Factory	65
6.12.1 Opis szczegółowy	66
6.12.2 Dokumentacja konstruktora i destruktor	66
6.12.2.1 Factory() [1/2]	66
6.12.2.2 Factory() [2/2]	67
6.12.3 Dokumentacja funkcji składowych	67
6.12.3.1 costOverEstimate()	67
6.12.3.2 DeserializeFromStream()	67
6.12.3.3 GetProduction()	68
6.12.3.4 GetSellValue()	68
6.12.3.5 SerializeToStream()	68
6.12.4 Dokumentacja atrybutów składowych	68
6.12.4.1 baseCost	68
6.12.4.2 baseProduction	69
6.12.4.3 costMultiplier	69
6.12.4.4 count	69
6.12.4.5 isUnlocked	69
6.12.4.6 name	69
6.12.4.7 productionMultiplier	69
6.12.4.8 resource	69
6.12.4.9 unlockCost	69
6.12.5 Dokumentacja właściwości	70
6.12.5.1 currentCost	70
6.13 Dokumentacja klasy FactoryUIManager.FactoryUI	70
6.13.1 Opis szczegółowy	71
6.13.2 Dokumentacja atrybutów składowych	71
6.13.2.1 buyButtons	71
6.13.2.2 costLabel	71
6.13.2.3 countLabel	71
6.13.2.4 detailsPanel	71
6.13.2.5 isVisible	71
6.13.2.6 multiplierLevelLabel	71
6.13.2.7 nameLabel	71
6.13.2.8 productionLevelLabel	72
6.13.2.9 sellButtons	72
6.13.2.10 toggleButton	72
6.13.2.11 unlockButton	72
6.13.2.12 upgradeMainBtn	72
6.13.2.13 upgradeMultiplierBtn	72

6.13.2.14 upgradePanel	72
6.13.2.15 upgradeProductionBtn	72
6.13.2.16 upgradesVisible	73
6.14 Dokumentacja klasy FactoryUIManager	73
6.14.1 Opis szczegółowy	75
6.14.2 Dokumentacja konstruktora i destruktora	76
6.14.2.1 FactoryUIManager()	76
6.14.3 Dokumentacja funkcji składowych	76
6.14.3.1 BuyFactory()	76
6.14.3.2 BuyFactoryMax()	76
6.14.3.3 CalculateUpgradeCost()	77
6.14.3.4 CycleToNextFactory()	77
6.14.3.5 CycleToPreviousFactory()	77
6.14.3.6 DeserializeFromStream()	77
6.14.3.7 GetFactory()	78
6.14.3.8 Initialize()	78
6.14.3.9 InitializeFactories()	78
6.14.3.10 InitializeUnlockButtons()	78
6.14.3.11 IsFactoryUnlocked()	78
6.14.3.12 ResetFactoryGUI()	79
6.14.3.13 SellFactory()	79
6.14.3.14 SellFactoryMax()	79
6.14.3.15 SerializeToStream()	79
6.14.3.16 SwitchToFactory()	80
6.14.3.17 ToggleFactoryDetails()	80
6.14.3.18 UnlockFactory()	80
6.14.3.19 UnlockFactoryFromButton()	80
6.14.3.20 Update()	81
6.14.3.21 UpdateButton()	81
6.14.3.22 UpdateFactoryUI()	81
6.14.3.23 UpdateNavigationButtons()	81
6.14.3.24 UpdateUnlockButton()	81
6.14.3.25 UpdateUnlockButtons()	82
6.14.3.26 UpgradeFactoryBaseProduction()	82
6.14.3.27 UpgradeFactoryMultiplier()	82
6.14.4 Dokumentacja atrybutów składowych	82
6.14.4.1 audioSource	82
6.14.4.2 clickerObject	82
6.14.4.3 currencyIcon	82
6.14.4.4 currentFactoryIndex	83
6.14.4.5 factoryLabel	83
6.14.4.6 factoryUIs	83
6.14.4.7 factoryUpgradeLevels	83

6.14.4.8 idleManager	83
6.14.4.9 images	83
6.14.4.10 nextFactoryBtn	83
6.14.4.11 numberOfFactories	83
6.14.4.12 prevFactoryBtn	84
6.14.4.13 raptorCore	84
6.14.4.14 scrollView	84
6.14.4.15 shopIcon	84
6.14.4.16 ui	84
6.14.4.17 unlockFactory2Btn	84
6.14.4.18 unlockFactory3Btn	84
6.14.4.19 unlockFactory4Btn	84
6.14.4.20 unlockFactory5Btn	85
6.14.4.21 unlockFactory6Btn	85
6.15 Dokumentacja klasy FactoryUIManager.FactoryUpgradeLevels	85
6.15.1 Opis szczegółowy	85
6.15.2 Dokumentacja funkcji składowych	86
6.15.2.1 DeserializeFromStream()	86
6.15.2.2 SerializeToStream()	86
6.15.3 Dokumentacja atrybutów składowych	86
6.15.3.1 baseMultiplierCost	86
6.15.3.2 baseProductionCost	86
6.15.3.3 multiplierLevel	86
6.15.3.4 productionLevel	87
6.16 Dokumentacja interfejsu IBinarySaveable	87
6.16.1 Opis szczegółowy	87
6.16.2 Dokumentacja funkcji składowych	88
6.16.2.1 DeserializeFromStream()	88
6.16.2.2 SerializeToStream()	88
6.16.3 Dokumentacja właściwości	88
6.16.3.1 SaveKey	88
6.17 Dokumentacja klasy IdleManager	88
6.17.1 Opis szczegółowy	90
6.17.2 Dokumentacja funkcji składowych	91
6.17.2.1 BuyFactory()	91
6.17.2.2 BuyPotion()	91
6.17.2.3 DeserializeFromStream()	91
6.17.2.4 DisablePotionEffect()	92
6.17.2.5 EnablePotionEffect()	92
6.17.2.6 GetFactory()	92
6.17.2.7 GetFactoryCount()	92
6.17.2.8 GetFocusedFactory()	93
6.17.2.9 GetPotion()	93

6.17.2.10	GetUnlockedFactoryCount()	93
6.17.2.11	InitFactories()	94
6.17.2.12	InitPotions()	94
6.17.2.13	InitResources()	94
6.17.2.14	OnDisable()	94
6.17.2.15	OnEnable()	94
6.17.2.16	ProcessTick()	94
6.17.2.17	ResetFactory()	94
6.17.2.18	SellFactory()	95
6.17.2.19	SerializeToStream()	95
6.17.2.20	Start()	95
6.17.2.21	TickLoop()	95
6.17.2.22	UnlockFactory()	96
6.17.3	Dokumentacja atrybutów składowych	96
6.17.3.1	characterClass	96
6.17.3.2	factories	96
6.17.3.3	potionFactoryBonus1	96
6.17.3.4	potionFactoryBonus2	96
6.17.3.5	potions	96
6.17.3.6	raptorCore	97
6.17.3.7	resources	97
6.17.3.8	tickCoroutine	97
6.17.3.9	tickInterval	97
6.17.4	Dokumentacja właściwości	97
6.17.4.1	SaveKey	97
6.18	Dokumentacja klasy JackSkillTree	97
6.18.1	Opis szczegółowy	99
6.18.2	Dokumentacja konstruktora i destruktor	99
6.18.2.1	JackSkillTree()	99
6.18.3	Dokumentacja funkcji składowych	100
6.18.3.1	CanUnlock()	100
6.18.3.2	GetDependencies()	100
6.18.3.3	OnSkillClicked()	100
6.18.3.4	UpdateVisual()	101
6.19	Dokumentacja klasy LayoutController	101
6.19.1	Opis szczegółowy	103
6.19.2	Dokumentacja funkcji składowych	103
6.19.2.1	Awake()	103
6.19.2.2	DeserializeFromStream()	103
6.19.2.3	InitializeManagers()	104
6.19.2.4	InitializeTooltips()	104
6.19.2.5	OnEnable()	104
6.19.2.6	ResetGui()	104

6.19.2.7 SerializeToStream()	104
6.19.2.8 SetCurrencyText()	105
6.19.2.9 SetGoldText()	105
6.19.2.10 Update()	105
6.19.2.11 UpdateUI()	105
6.19.3 Dokumentacja atrybutów składowych	105
6.19.3.1 audioSource	105
6.19.3.2 bufficons	105
6.19.3.3 buffManager	106
6.19.3.4 characterClass	106
6.19.3.5 clickerObject	106
6.19.3.6 contentPageManager	106
6.19.3.7 critSource	106
6.19.3.8 factoryUIManager	106
6.19.3.9 idleManager	106
6.19.3.10 images	106
6.19.3.11 learnedAutomatron	107
6.19.3.12 learnedJackOfAllClicks	107
6.19.3.13 learnedOneClickArmy	107
6.19.3.14 levelUIManager	107
6.19.3.15 optionsManager	107
6.19.3.16 potionUIManager	107
6.19.3.17 prestigeUIManager	107
6.19.3.18 raptorCore	107
6.19.3.19 resourceShopManager	108
6.19.3.20 skillPointsLimit	108
6.19.3.21 skillTreesUIManager	108
6.19.3.22 tooltipAsset	108
6.19.3.23 ui	108
6.19.4 Dokumentacja właściwości	108
6.19.4.1 Instance	108
6.19.4.2 SaveKey	108
6.20 Dokumentacja klasy LevelUIManager	109
6.20.1 Opis szczegółowy	109
6.20.2 Dokumentacja konstruktora i destruktoru	110
6.20.2.1 LevelUIManager()	110
6.20.3 Dokumentacja funkcji składowych	110
6.20.3.1 Initialize()	110
6.20.3.2 Update()	110
6.20.3.3 UpdateLvlBar()	110
6.20.4 Dokumentacja atrybutów składowych	110
6.20.4.1 barMask	110
6.20.4.2 barTexture	110

6.20.4.3	characterClass	111
6.20.4.4	expBarController	111
6.20.4.5	lvlNumber	111
6.20.4.6	ui	111
6.21	Dokumentacja klasy MainMenuController	111
6.21.1	Opis szczegółowy	113
6.21.2	Dokumentacja funkcji składowych	113
6.21.2.1	Awake()	113
6.21.2.2	OnBackButtonClicked()	113
6.21.2.3	OnClick()	113
6.21.2.4	OnEnable()	113
6.21.2.5	OnExitButtonClicked()	113
6.21.2.6	OnMusicVolumeChanged()	114
6.21.2.7	OnSettingsButtonClicked()	114
6.21.2.8	OnSfxVolumeChanged()	114
6.21.2.9	OnStartButtonClicked()	114
6.21.2.10	SetFullScreen()	114
6.21.2.11	SetResolution()	115
6.21.3	Dokumentacja atrybutów składowych	115
6.21.3.1	AutoSavetoggle	115
6.21.3.2	backButton	115
6.21.3.3	DeleteSaveButton	115
6.21.3.4	fullScreen	115
6.21.3.5	mainContainer	115
6.21.3.6	mainMenu	115
6.21.3.7	musicSlider	116
6.21.3.8	musicSource	116
6.21.3.9	quitButton	116
6.21.3.10	resolutionList	116
6.21.3.11	resolutions	116
6.21.3.12	sceneController	116
6.21.3.13	settingsButton	116
6.21.3.14	settingsContainer	116
6.21.3.15	sfxSlider	117
6.21.3.16	sfxSource	117
6.21.3.17	startButton	117
6.22	Dokumentacja klasy OneClickArmySkillTree	117
6.22.1	Opis szczegółowy	119
6.22.2	Dokumentacja konstruktora i destruktor	119
6.22.2.1	OneClickArmySkillTree()	119
6.22.3	Dokumentacja funkcji składowych	120
6.22.3.1	CanUnlock()	120
6.22.3.2	GetDependencies()	120

6.22.3.3 OnSkillClicked()	120
6.22.3.4 UpdateVisual()	121
6.23 Dokumentacja klasy OptionsManager	121
6.23.1 Opis szczegółowy	122
6.23.2 Dokumentacja konstruktora i destruktor	123
6.23.2.1 OptionsManager()	123
6.23.3 Dokumentacja funkcji składowych	123
6.23.3.1 EnableOptions()	123
6.23.3.2 ExitGame()	123
6.23.3.3 Initialize()	123
6.23.3.4 InitializeAudioSettings()	123
6.23.3.5 InitializeDisplaySettings()	123
6.23.3.6 InitializeSaveSettings()	124
6.23.3.7 LoadSettings()	124
6.23.3.8 OnMusicVolumeChanged()	124
6.23.3.9 OnSfxVolumeChanged()	124
6.23.3.10 ResumeGame()	124
6.23.3.11 ReturnSettings()	124
6.23.3.12 SetFullScreen()	125
6.23.3.13 SetResolution()	125
6.23.4 Dokumentacja atrybutów składowych	125
6.23.4.1 audioSource	125
6.23.4.2 autoSaveToggle	125
6.23.4.3 backSettings	125
6.23.4.4 critSource	125
6.23.4.5 deleteSaveButton	126
6.23.4.6 exit	126
6.23.4.7 fullScreen	126
6.23.4.8 mainMenu	126
6.23.4.9 menu	126
6.23.4.10 musicSlider	126
6.23.4.11 optionBtn	126
6.23.4.12 options	126
6.23.4.13 resolutionList	127
6.23.4.14 resolutions	127
6.23.4.15 resume	127
6.23.4.16 settings	127
6.23.4.17 sfxSlider	127
6.23.4.18 ui	127
6.24 Dokumentacja klasy Potion	127
6.24.1 Opis szczegółowy	128
6.24.2 Dokumentacja konstruktora i destruktor	128
6.24.2.1 Potion()	128

6.24.3 Dokumentacja atrybutów składowych	129
6.24.3.1 boughtCount	129
6.24.3.2 cost1	129
6.24.3.3 cost2	129
6.24.3.4 duration	129
6.24.3.5 effect	129
6.24.3.6 effectDescription	130
6.24.3.7 isActive	130
6.24.3.8 linkedFactory	130
6.24.3.9 name	130
6.24.3.10 resourceType1	130
6.24.3.11 resourceType2	130
6.24.3.12 timeRemaining	130
6.25 Dokumentacja klasy PotionUIManager.PotionUI	131
6.25.1 Opis szczegółowy	131
6.25.2 Dokumentacja atrybutów składowych	131
6.25.2.1 buyButton	131
6.25.2.2 costLabel	131
6.25.2.3 effectLabel	131
6.25.2.4 nameLabel	131
6.25.2.5 potionPanel	132
6.26 Dokumentacja klasy PotionUIManager	132
6.26.1 Opis szczegółowy	133
6.26.2 Dokumentacja konstruktora i destruktora	133
6.26.2.1 PotionUIManager()	133
6.26.3 Dokumentacja funkcji składowych	133
6.26.3.1 BuyPotion()	133
6.26.3.2 Initialize()	133
6.26.3.3 InitializePotions()	134
6.26.3.4 Update()	134
6.26.3.5 UpdatePotionUI()	134
6.26.4 Dokumentacja atrybutów składowych	134
6.26.4.1 audioSource	134
6.26.4.2 characterClass	134
6.26.4.3 idleManager	134
6.26.4.4 numberOfPotions	134
6.26.4.5 potionUIs	135
6.26.4.6 raptorCore	135
6.26.4.7 scrollView	135
6.26.4.8 ui	135
6.27 Dokumentacja klasy PrestigeSystem	135
6.27.1 Opis szczegółowy	136
6.27.2 Dokumentacja funkcji składowych	136

6.27.2.1 CheckPrestige()	136
6.27.2.2 ExecutePrestige()	136
6.27.2.3 Init()	137
6.27.3 Dokumentacja atrybutów składowych	137
6.27.3.1 _core	137
6.27.3.2 _initialized	137
6.28 Dokumentacja klasy PrestigeUIManager	137
6.28.1 Opis szczegółowy	138
6.28.2 Dokumentacja konstruktora i destruktora	139
6.28.2.1 PrestigeUIManager()	139
6.28.3 Dokumentacja funkcji składowych	139
6.28.3.1 DisablePrestige()	139
6.28.3.2 EnablePrestige()	139
6.28.3.3 Hide()	139
6.28.3.4 Initialize()	139
6.28.3.5 PrestigeTrigger()	140
6.28.3.6 Trigger()	140
6.28.3.7 UpdateContentPage()	140
6.28.4 Dokumentacja atrybutów składowych	140
6.28.4.1 audioSource	140
6.28.4.2 BarContainer	140
6.28.4.3 BarTexture	140
6.28.4.4 contentPage	141
6.28.4.5 ExpContainer	141
6.28.4.6 PrestigeAccept	141
6.28.4.7 PrestigeButtonMenu	141
6.28.4.8 PrestigeCancel	141
6.28.4.9 PrestigeRoot	141
6.28.4.10 rp	141
6.28.4.11 ui_ref	142
6.29 Dokumentacja klasy QuarkTestNode	142
6.29.1 Opis szczegółowy	143
6.29.2 Dokumentacja funkcji składowych	144
6.29.2.1 Assert()	144
6.29.2.2 IsApproximatelyEqual()	144
6.29.2.3 Start()	144
6.29.2.4 TestAddition()	144
6.29.2.5 TestComparisonOperators()	145
6.29.2.6 TestDivision()	145
6.29.2.7 TestMultiplication()	145
6.29.2.8 TestMultiplicationEXT()	145
6.29.2.9 TestNormalizationAndConstructors()	145
6.29.2.10 TestPrecisionLossAndEdgeCases()	145

6.29.2.11 TestPrimitiveAddition()	145
6.29.2.12 TestPrimitiveConversions()	146
6.29.2.13 TestPrimitiveMultiplication()	146
6.29.2.14 TestPrimitiveSubtraction()	146
6.29.2.15 TestSignedAndRangeArithmetic()	146
6.29.2.16 TestSubtraction()	146
6.29.2.17 Update()	146
6.29.3 Dokumentacja atrybutów składowych	146
6.29.3.1 _testsFailed	146
6.29.3.2 _testsRun	147
6.29.3.3 MANTISSA_TOLERANCE	147
6.30 Dokumentacja klasy QuarkType	147
6.30.1 Opis szczegółowy	150
6.30.2 Dokumentacja konstruktora i destruktora	150
6.30.2.1 QuarkType() [1/5]	150
6.30.2.2 QuarkType() [2/5]	150
6.30.2.3 QuarkType() [3/5]	150
6.30.2.4 QuarkType() [4/5]	150
6.30.2.5 QuarkType() [5/5]	150
6.30.3 Dokumentacja funkcji składowych	151
6.30.3.1 Ceil()	151
6.30.3.2 CompareMagnitudeTo()	151
6.30.3.3 CompareTo()	151
6.30.3.4 DeserializeFromStream()	151
6.30.3.5 Equals() [1/2]	151
6.30.3.6 Equals() [2/2]	151
6.30.3.7 Floor()	151
6.30.3.8 GetHashCode()	152
6.30.3.9 Normalize()	152
6.30.3.10 operator double()	152
6.30.3.11 operator QuarkType() [1/5]	152
6.30.3.12 operator QuarkType() [2/5]	152
6.30.3.13 operator QuarkType() [3/5]	152
6.30.3.14 operator QuarkType() [4/5]	152
6.30.3.15 operator QuarkType() [5/5]	153
6.30.3.16 operator "!="()	153
6.30.3.17 operator*() [1/3]	153
6.30.3.18 operator*() [2/3]	153
6.30.3.19 operator*() [3/3]	153
6.30.3.20 operator+() [1/3]	153
6.30.3.21 operator+() [2/3]	154
6.30.3.22 operator+() [3/3]	154
6.30.3.23 operator++()	154

6.30.3.24 operator-() [1/4]	154
6.30.3.25 operator-() [2/4]	154
6.30.3.26 operator-() [3/4]	154
6.30.3.27 operator-() [4/4]	155
6.30.3.28 operator/() [1/3]	155
6.30.3.29 operator/() [2/3]	155
6.30.3.30 operator/() [3/3]	155
6.30.3.31 operator==()	155
6.30.3.32 Pow()	155
6.30.3.33 SerializeToStream()	156
6.30.3.34 ToString()	156
6.30.4 Dokumentacja atrybutów składowych	156
6.30.4.1 Exponent	156
6.30.4.2 ExponentRangeForPrecision	156
6.30.4.3 Mantissa	156
6.30.4.4 MantissaLength	156
6.30.4.5 NormalizeDivisor	156
6.30.5 Dokumentacja właściwości	157
6.30.5.1 IsNegative	157
6.30.5.2 NegativeOne	157
6.30.5.3 One	157
6.30.5.4 Zero	157
6.31 Dokumentacja klasy RaptorCore	157
6.31.1 Opis szczegółowy	160
6.31.2 Dokumentacja składowych wyliczanych	160
6.31.2.1 ClickSource	160
6.31.3 Dokumentacja funkcji składowych	160
6.31.3.1 AddCurrency()	160
6.31.3.2 Awake()	160
6.31.3.3 click()	160
6.31.3.4 DeserializeFromStream()	160
6.31.3.5 FixedUpdate()	161
6.31.3.6 OnMouseDown()	161
6.31.3.7 PerformClick()	161
6.31.3.8 ResetCore()	161
6.31.3.9 SerializeToStream()	161
6.31.3.10 Start()	161
6.31.3.11 SubCurrency()	161
6.31.3.12 Update()	162
6.31.4 Dokumentacja atrybutów składowych	162
6.31.4.1 activeClickSource	162
6.31.4.2 anim	162
6.31.4.3 audioSource	162

6.31.4.4 AutoPrestige	162
6.31.4.5 CBasevalue	162
6.31.4.6 characterClass	162
6.31.4.7 clickCooldown	163
6.31.4.8 clickingDebuff	163
6.31.4.9 CMultiplier	163
6.31.4.10 critGen	163
6.31.4.11 CritGenerator	163
6.31.4.12 critSource	163
6.31.4.13 GBasevalue	163
6.31.4.14 GMultiplier	163
6.31.4.15 Gold	164
6.31.4.16 idleManager	164
6.31.4.17 JackPrestige	164
6.31.4.18 OneClickPrestige	164
6.31.4.19 potionClickBonus1	164
6.31.4.20 potionClickBonus2	164
6.31.4.21 PrestigeCount	164
6.31.4.22 skillCheck	164
6.31.4.23 SkillMultiplier	165
6.31.4.24 sourceBlockEndTime	165
6.31.4.25 tickCount	165
6.31.5 Dokumentacja właściwości	165
6.31.5.1 Currency	165
6.31.5.2 ResourceManager	165
6.31.5.3 SaveKey	165
6.31.5.4 SellManager	165
6.31.5.5 SkillManager	166
6.32 Dokumentacja klasy Resource	166
6.32.1 Opis szczegółowy	167
6.32.2 Dokumentacja konstruktora i destruktora	167
6.32.2.1 Resource() [1/2]	167
6.32.2.2 Resource() [2/2]	167
6.32.3 Dokumentacja funkcji składowych	167
6.32.3.1 DeserializeFromStream()	167
6.32.3.2 SerializeToStream()	168
6.32.4 Dokumentacja atrybutów składowych	168
6.32.4.1 _value	168
6.32.4.2 BaseLimit	168
6.32.4.3 displayName	168
6.32.4.4 LvlBoost	168
6.32.4.5 name	168
6.32.4.6 SkillBoost	168

6.32.5 Dokumentacja właściwości	169
6.32.5.1 Limit	169
6.32.5.2 value	169
6.33 Dokumentacja klasy ResourceManager	169
6.33.1 Opis szczegółowy	172
6.33.2 Dokumentacja funkcji składowych	172
6.33.2.1 AddResource()	172
6.33.2.2 CanClickCurrentResource()	172
6.33.2.3 DeserializeFromStream()	172
6.33.2.4 GetBaseLimitResource()	172
6.33.2.5 GetCurrentFactory()	173
6.33.2.6 GetLimitResource()	173
6.33.2.7 GetResourceValue()	173
6.33.2.8 GetResourceValueDirect()	173
6.33.2.9 HasResource()	173
6.33.2.10 IncrementSkillBoostResource()	173
6.33.2.11 IncrementSkillBoostResourceAll()	173
6.33.2.12 Initialize()	174
6.33.2.13 LoadAllResources()	174
6.33.2.14 LoadResource()	174
6.33.2.15 MullLimitResourceAll()	174
6.33.2.16 RegisterResource()	174
6.33.2.17 RemoveResource()	175
6.33.2.18 ResetResources()	175
6.33.2.19 SaveResource()	175
6.33.2.20 SaveResourceValues()	175
6.33.2.21 SerializeToStream()	175
6.33.2.22 SetCurrentResource()	175
6.33.2.23 SetLvlBoostResource()	176
6.33.2.24 SetResourceValueDirect()	176
6.33.3 Dokumentacja atrybutów składowych	176
6.33.3.1 currentResource	176
6.33.3.2 idleManager	176
6.33.3.3 resource1Limit	176
6.33.3.4 resource1Value	176
6.33.3.5 resource2Limit	176
6.33.3.6 resource2Value	177
6.33.3.7 resource3Limit	177
6.33.3.8 resource3Value	177
6.33.3.9 resource4Limit	177
6.33.3.10 resource4Value	177
6.33.3.11 resource5Limit	177
6.33.3.12 resource5Value	177

6.33.3.13 resource6Limit	177
6.33.3.14 resource6Value	178
6.33.3.15 resources	178
6.33.4 Dokumentacja właściwości	178
6.33.4.1 SaveKey	178
6.34 Dokumentacja klasy ResourceShopManager	178
6.34.1 Opis szczegółowy	179
6.34.2 Dokumentacja konstruktora i destruktora	180
6.34.2.1 ResourceShopManager()	180
6.34.3 Dokumentacja funkcji składowych	180
6.34.3.1 ClickSell1()	180
6.34.3.2 Initialize()	180
6.34.3.3 SetCurrencyText()	180
6.34.3.4 SetGoldText()	181
6.34.3.5 Update()	181
6.34.4 Dokumentacja atrybutów składowych	181
6.34.4.1 audioSource	181
6.34.4.2 characterClass	181
6.34.4.3 currencySlider	181
6.34.4.4 goldLabel	181
6.34.4.5 raptorCore	181
6.34.4.6 res1curr	182
6.34.4.7 res2curr	182
6.34.4.8 res3curr	182
6.34.4.9 res4curr	182
6.34.4.10 res5curr	182
6.34.4.11 res6curr	182
6.34.4.12 sell1	182
6.34.4.13 sliderText	182
6.34.4.14 ui	183
6.35 Dokumentacja klasy SceneController	183
6.35.1 Opis szczegółowy	183
6.35.2 Dokumentacja funkcji składowych	184
6.35.2.1 Awake()	184
6.35.2.2 LoadScene()	184
6.35.2.3 LoadSceneCoroutine()	184
6.35.2.4 Start()	184
6.35.3 Dokumentacja atrybutów składowych	184
6.35.3.1 fadeDuration	184
6.35.3.2 sceneFade	185
6.36 Dokumentacja klasy SceneFade	185
6.36.1 Opis szczegółowy	186
6.36.2 Dokumentacja funkcji składowych	186

6.36.2.1 Awake()	186
6.36.2.2 FadeCoroutine()	186
6.36.2.3 FadeInCoroutine()	186
6.36.2.4 FadeOutCoroutine()	187
6.36.2.5 LateUpdate()	187
6.36.3 Dokumentacja atrybutów składowych	187
6.36.3.1 backgroundTransform	187
6.36.3.2 rectTransform	187
6.36.3.3 sceneImage	187
6.37 Dokumentacja klasy SellingManager	188
6.37.1 Opis szczegółowy	189
6.37.2 Dokumentacja funkcji składowych	189
6.37.2.1 Initialize()	189
6.37.2.2 SellMaterials()	189
6.37.2.3 SellResource()	190
6.37.2.4 Update()	190
6.37.3 Dokumentacja atrybutów składowych	190
6.37.3.1 characterClass	190
6.37.3.2 idleManager	190
6.37.3.3 potionSellBonus1	190
6.37.3.4 potionSellBonus2	191
6.37.3.5 raptorCore	191
6.37.3.6 resourceManager	191
6.38 Dokumentacja klasy SkillCheckScript	191
6.38.1 Opis szczegółowy	192
6.38.2 Dokumentacja funkcji składowych	193
6.38.2.1 CheckMouse0()	193
6.38.2.2 CheckMouse1()	193
6.38.2.3 ClearBricks()	193
6.38.2.4 ResetSkillCheck()	193
6.38.2.5 SpawnBricks()	193
6.38.2.6 Start()	194
6.38.2.7 StartSkillCheck()	194
6.38.2.8 Update()	194
6.38.3 Dokumentacja atrybutów składowych	194
6.38.3.1 Bonus	194
6.38.3.2 brickCount	194
6.38.3.3 brickPrefab	194
6.38.3.4 bricks	194
6.38.3.5 characterClass	195
6.38.3.6 combovisual	195
6.38.3.7 flag	195
6.38.3.8 kombo	195

6.38.3.9 Layout	195
6.38.3.10 raptorCore	195
6.38.3.11 waitTime	195
6.39 Dokumentacja klasy SkillManager	196
6.39.1 Opis szczegółowy	197
6.39.2 Dokumentacja funkcji składowych	197
6.39.2.1 CheckAndTriggerSkillCheck()	197
6.39.2.2 CheckReactionTest()	197
6.39.2.3 ChickenDinnerEffect()	198
6.39.2.4 EnableActiveldle()	198
6.39.2.5 EnableChickenDinner()	198
6.39.2.6 GetActiveldleValue()	198
6.39.2.7 Initialize()	198
6.39.2.8 IsSkillCheckActive()	198
6.39.2.9 RandomPotionEffect()	199
6.39.2.10 TriggerSkillCheck()	199
6.39.2.11 UpdateSkillCheckCooldown()	199
6.39.3 Dokumentacja atrybutów składowych	199
6.39.3.1 characterClass	199
6.39.3.2 idleManager	199
6.39.3.3 raptorCore	199
6.39.3.4 skillCheck	199
6.39.3.5 skillCheckCooldown	200
6.39.3.6 skillCheckCooldownCount	200
6.39.4 Dokumentacja właściwości	200
6.39.4.1 SkillCheckCooldown	200
6.40 Dokumentacja klasy SkillPointsLimit	200
6.40.1 Opis szczegółowy	201
6.40.2 Dokumentacja funkcji składowych	201
6.40.2.1 Awake()	201
6.40.2.2 DecSkillPoints()	201
6.40.2.3 IncSkillPoints()	201
6.40.2.4 UpdateButton()	202
6.40.3 Dokumentacja atrybutów składowych	202
6.40.3.1 availableColor	202
6.40.3.2 characterClass	202
6.40.3.3 critContainer	202
6.40.3.4 Skillbtn	202
6.40.3.5 ui	202
6.40.3.6 unavailableColor	202
6.41 Dokumentacja klasy SkillTreeManager	203
6.41.1 Opis szczegółowy	204
6.41.2 Dokumentacja konstruktora i destruktor	204

6.41.2.1 SkillTreeManager()	204
6.41.3 Dokumentacja funkcji składowych	205
6.41.3.1 CanUnlock()	205
6.41.3.2 ClearTree()	205
6.41.3.3 DeserializeFromStream()	205
6.41.3.4 DrawAllLines()	205
6.41.3.5 DrawLine()	206
6.41.3.6 DrawLineBetweenButtons()	206
6.41.3.7 InitializeLineLayer()	206
6.41.3.8 InitializeSkills()	206
6.41.3.9 OnSkillClicked()	207
6.41.3.10 OnSkillUnlearned()	207
6.41.3.11 RegisterTooltip()	207
6.41.3.12 SerializeToStream()	207
6.41.3.13 UpdateAll()	208
6.41.3.14 UpdateVisual()	208
6.41.4 Dokumentacja atrybutów składowych	208
6.41.4.1 availableColor	208
6.41.4.2 buttons	208
6.41.4.3 characterClass	208
6.41.4.4 dependencies	208
6.41.4.5 learned	208
6.41.4.6 learnedColor	209
6.41.4.7 lineLayer	209
6.41.4.8 lockedColor	209
6.41.4.9 skillPointsLimit	209
6.41.4.10 skillTreeContainer	209
6.41.4.11 ui	209
6.42 Dokumentacja klasy SkillTreesUIManager	209
6.42.1 Opis szczegółowy	211
6.42.2 Dokumentacja konstruktora i destruktor	211
6.42.2.1 SkillTreesUIManager()	211
6.42.3 Dokumentacja funkcji składowych	211
6.42.3.1 ClearTrees()	211
6.42.3.2 DeserializeFromStream()	211
6.42.3.3 InitializeAllTrees()	212
6.42.3.4 InitializeAutomatronTree()	212
6.42.3.5 InitializeJackTree()	212
6.42.3.6 InitializeOneClickArmyTree()	212
6.42.3.7 SerializeToStream()	212
6.42.3.8 Update()	212
6.42.4 Dokumentacja atrybutów składowych	212
6.42.4.1 automatronTree	212

6.42.4.2	characterClass	213
6.42.4.3	jackTree	213
6.42.4.4	learnedAutomatron	213
6.42.4.5	learnedJack	213
6.42.4.6	learnedOneClick	213
6.42.4.7	oneClickArmyTree	213
6.42.4.8	skillPointsLimit	213
6.42.4.9	tooltipAsset	213
6.42.4.10	ui	214
6.43	Dokumentacja klasy StaticBackground	214
6.43.1	Opis szczegółowy	214
6.43.2	Dokumentacja funkcji składowych	215
6.43.2.1	Start()	215
6.43.2.2	Update()	215
6.43.2.3	UpdateBackground()	215
6.43.3	Dokumentacja atrybutów składowych	215
6.43.3.1	backgrounds	215
6.43.3.2	currentFactory	215
6.43.3.3	raptorCore	215
6.43.3.4	rawImage	215
6.43.3.5	rectTransform	216
6.44	Dokumentacja klasy Tooltip	216
6.44.1	Opis szczegółowy	217
6.44.2	Dokumentacja funkcji składowych	217
6.44.2.1	Hide()	217
6.44.2.2	Init()	217
6.44.2.3	Move()	217
6.44.2.4	Register()	218
6.44.2.5	Show()	218
6.44.2.6	SizeRefresh()	218
6.44.3	Dokumentacja atrybutów składowych	219
6.44.3.1	Content	219
6.44.3.2	Frame	219
6.44.3.3	Lore	219
6.44.3.4	root	219
6.44.3.5	Tittle	219
7	Dokumentacja plików	221
7.1	Dokumentacja pliku ActivePotionsUI.cs	221
7.2	Dokumentacja pliku BrickMovementScript.cs	221
7.3	Dokumentacja pliku BuffManager.cs	221
7.4	Dokumentacja pliku BuffVisual.cs	221
7.5	Dokumentacja pliku CharacterClass.cs	222

7.6 Dokumentacja pliku ComboVisual.cs	222
7.7 Dokumentacja pliku ContentPageManager.cs	222
7.8 Dokumentacja pliku CritVisualGenerator.cs	222
7.9 Dokumentacja pliku DynamicBackground.cs	222
7.10 Dokumentacja pliku Factory.cs	222
7.11 Dokumentacja pliku FactoryUIManager.cs	223
7.12 Dokumentacja pliku IdleManager.cs	223
7.13 Dokumentacja pliku LayoutController.cs	223
7.14 Dokumentacja pliku LevelUIManager.cs	223
7.15 Dokumentacja pliku MainMenuController.cs	223
7.16 Dokumentacja pliku OptionsManager.cs	224
7.17 Dokumentacja pliku Potion.cs	224
7.18 Dokumentacja pliku PotionUIManager.cs	224
7.19 Dokumentacja pliku PrestigeSystem.cs	224
7.20 Dokumentacja pliku PrestigeUIManager.cs	224
7.21 Dokumentacja pliku QuarkTestNode.cs	224
7.22 Dokumentacja pliku QuarkType.cs	225
7.23 Dokumentacja pliku RaptorCore.cs	225
7.24 Dokumentacja pliku Resource.cs	225
7.25 Dokumentacja pliku ResourceManager.cs	225
7.26 Dokumentacja pliku ResourceShopManager.cs	225
7.27 Dokumentacja pliku SaveManager.cs	225
7.28 Dokumentacja pliku SceneController.cs	226
7.29 Dokumentacja pliku SceneFade.cs	226
7.30 Dokumentacja pliku SellManager.cs	226
7.31 Dokumentacja pliku SkillCheckScript.cs	226
7.32 Dokumentacja pliku SkillManager.cs	226
7.33 Dokumentacja pliku SkillPointsLimit.cs	226
7.34 Dokumentacja pliku SkillTreesUI.cs	227
7.35 Dokumentacja pliku SkillTreesUIManager.cs	227
7.36 Dokumentacja pliku StaticBackground.cs	227
7.37 Dokumentacja pliku ToolTipStatic.cs	227

Rozdział 1

Struktura katalogów

1.1 Katalogi

Assets	11
Scripts	11
ActivePotionsUI.cs	221
BrickMovementScript.cs	221
BuffManager.cs	221
BuffVisual.cs	221
CharacterClass.cs	222
ComboVisual.cs	222
ContentPageManager.cs	222
CritVisualGenerator.cs	222
DynamicBackground.cs	222
Factory.cs	222
FactoryUIManager.cs	223
IdleManager.cs	223
LayoutController.cs	223
LevelUIManager.cs	223
MainMenuController.cs	223
OptionsManager.cs	224
Potion.cs	224
PotionUIManager.cs	224
PrestigeSystem.cs	224
PrestigeUIManager.cs	224
QuarkTestNode.cs	224
QuarkType.cs	225
RaptorCore.cs	225
Resource.cs	225
ResourceManager.cs	225
ResourceShopManager.cs	225
SaveManager.cs	225
SceneController.cs	226
SceneFade.cs	226
SellManager.cs	226
SkillCheckScript.cs	226
SkillManager.cs	226
SkillPointsLimit.cs	226
SkillTreesUI.cs	227
SkillTreesUIManager.cs	227
StaticBackground.cs	227

ToolTipStatic.cs	227
Scripts	11
ActivePotionsUI.cs	221
BrickMovementScript.cs	221
BuffManager.cs	221
BuffVisual.cs	221
CharacterClass.cs	222
ComboVisual.cs	222
ContentPageManager.cs	222
CritVisualGenerator.cs	222
DynamicBackground.cs	222
Factory.cs	222
FactoryUIManager.cs	223
IdleManager.cs	223
LayoutController.cs	223
LevelUIManager.cs	223
MainMenuController.cs	223
OptionsManager.cs	224
Potion.cs	224
PotionUIManager.cs	224
PrestigeSystem.cs	224
PrestigeUIManager.cs	224
QuarkTestNode.cs	224
QuarkType.cs	225
RaptorCore.cs	225
Resource.cs	225
ResourceManager.cs	225
ResourceShopManager.cs	225
SaveManager.cs	225
SceneController.cs	226
SceneFade.cs	226
SellManager.cs	226
SkillCheckScript.cs	226
SkillManager.cs	226
SkillPointsLimit.cs	226
SkillTreesUI.cs	227
SkillTreesUIManager.cs	227
StaticBackground.cs	227
ToolTipStatic.cs	227

Rozdział 2

Indeks hierarchiczny

2.1 Hierarchia klas

Ta lista dziedziczenia posortowana jest z grubsza, choć nie całkowicie, alfabetycznie:

AutoSaveSystem	21
BuffManager	27
BuffVisual	29
ContentPageManager	56
Factory	65
FactoryUIManager.FactoryUI	70
FactoryUIManager	73
FactoryUIManager.FactoryUpgradeLevels	85
IBinarySaveable	87
CharacterClass	31
IdleManager	88
LayoutController	101
RaptorCore	157
ResourceManager	169
IComparable	
QuarkType	147
IEquatable	
QuarkType	147
LevelUIManager	109
MonoBehaviour	
ActivePotionsUI	13
BrickMovementScript	25
CharacterClass	31
ComboVisual	54
CritVisualGenerator	60
DynamicBackground	62
IdleManager	88
LayoutController	101
MainMenuController	111
QuarkTestNode	142
RaptorCore	157
ResourceManager	169
SceneController	183
SceneFade	185
SellingManager	188

SkillCheckScript	191
SkillManager	196
SkillPointsLimit	200
StaticBackground	214
OptionsManager	121
Potion	127
PotionUIManager.PotionUI	131
PotionUIManager	132
PrestigeSystem	135
PrestigeUIManager	137
Resource	166
ResourceShopManager	178
SkillTreeManager	203
AutomatronSkillTree	17
JackSkillTree	97
OneClickArmySkillTree	117
SkillTreesUIManager	209
Tooltip	216

Rozdział 3

Indeks klas

3.1 Lista klas

Tutaj znajdują się klasy, struktury, unie i interfejsy wraz z ich krótkimi opisami:

ActivePotionsUI	Zarządza wyświetlaniem aktywnych mikstur (Potions) w UI gry. Pobiera dane z IdleManager i dynamicznie aktualizuje panel mikstur	13
AutomatronSkillTree	Drzewko umiejętności typu "Automatron". Zarządza odblokowywaniem i wizualizacją poszczególnych umiejętności dla tej gałęzi	17
AutoSaveSystem	Statyczny system autozapisu gry. Umożliwia zapis i odczyt wszystkich obiektów implementujących IBinarySaveable , obsługę autopoziomu zapisu i domyślnej ścieżki do pliku	21
BrickMovementScript	Odpowiada za ruch bloczka oraz wykrywanie kolizji z granicami. Skrypt losuje kierunek ruchu bloczka podczas aktywacji "SkillChecka", zmienia jego wygląd oraz aktualizuje status overlapowania w zależności od kolizji z obszarami granicznymi	25
BuffManager	Zarządza buffami wyświetlanymi w interfejsie użytkownika	27
BuffVisual	Reprezentuje pojedynczy buff wyświetlany w interfejsie użytkownika	29
CharacterClass	Główna klasa reprezentująca postać gracza w systemie RPG	31
ComboVisual	Odpowiada za wizualne wyświetlanie aktualnego mnożnika combo	54
ContentPageManager	Zarządza przełączaniem stron zawartości w interfejsie użytkownika	56
CritVisualGenerator	Generator wizualizacji efektu krytycznego trafienia	60
DynamicBackground	Dynamicznie zmienia tło UI z efektami fade i opcjonalnym zoomem	62
Factory	Reprezentuje fabrykę produkującą określony zasób w grze	65
FactoryUIManager.FactoryUI	Przechowuje referencje do elementów UI jednej fabryki	70
FactoryUIManager	Zarządza interfejsem użytkownika fabryk w grze	73
FactoryUIManager.FactoryUpgradeLevels	Przechowuje poziomy ulepszeń fabryki	85

IBinarySaveable	Interfejs dla wszystkich klas, które można zapisać/odczytać w formacie binarnym	87
IdleManager	Klasa odpowiedzialna za zarządzanie fabrykami, zasobami i miksturami w grze. Obsługuje ticki produkcji, kupno i sprzedaż fabryk oraz aktywację efektów mikstur. Implementuje IBinarySaveable do zapisu i wczytywania stanu gry	88
JackSkillTree	Drzewko umiejętności typu "Jack of All Clicks". Zarządza odblokowywaniem i wizualizacją poszczególnych umiejętności dla tej gałęzi	97
LayoutController	Kontroler głównego layoutu gry. Zarządza wszystkimi menedżerami UI, zasobami i tooltippami	101
LevelUIManager	Menedżer UI poziomu postaci. Aktualizuje pasek doświadczenia i wyświetla aktualny poziom postaci	109
MainMenuController	Kontroler głównego menu gry, obsługuje przyciski Start, Ustawienia, Wyjście oraz slidery i toggle dla ustawień audio i ekranu	111
OneClickArmySkillTree	Drzewko umiejętności typu "One Click Army". Zarządza odblokowywaniem i wizualizacją poszczególnych umiejętności dla tej gałęzi	117
OptionsManager	Zarządza menu opcji gry, w tym ustawieniami audio, wyświetlania i zapisu	121
Potion	Reprezentuje miksturę/potion w grze, którą można kupić i aktywować. Mikstury mają koszty w określonych zasobach, czas trwania oraz przypisany efekt	127
PotionUIManager.PotionUI	Reprezentuje pojedynczy element UI dla mikstury	131
PotionUIManager	Zarządza wyświetlaniem panelu mikstur w UI, aktualizuje ceny, efekty i przyciski zakupu	132
PrestigeSystem	Statyczny system prestiżu odpowiedzialny za obsługę resetu postępu i przyznawanie punktów prestiżu	135
PrestigeUIManager	Manager interfejsu użytkownika odpowiedzialny za system prestiżu. Obsługuje wyświetlanie statystyk prestiżu, pasek prestiżu oraz okno potwierdzenia resetu	137
QuarkTestNode	MonoBehaviour odpowiedzialny za testowanie struktury QuarkType . Uruchamia zestaw testów jednostkowych sprawdzających poprawność:	142
QuarkType	QuarkType to wysokozakresowy typ liczbowy oparty o:	147
RaptorCore	Główny rdzeń gry. Odpowiada za: walutę, kliknięcia, ticki idle, prestiż, serializację stanu gry	157
Resource	Reprezentuje pojedynczy zasób w grze. Zawiera aktualną wartość, limit oraz modyfikatory limitu	166
ResourceManager	Zarządza wszystkimi zasobami w grze. Odpowiada za ich wartości, limity, zapis/odczyt, aktualizację UI oraz integrację z fabrykami	169
ResourceShopManager	Manager UI dla sklepu zasobów. Odpowiada za aktualizację wyświetlanych wartości zasobów, obsługę przycisków sprzedaży i suwaka procentowego sprzedaży	178
SceneController	Kontroler scen w grze. Zarządza przejściami między scenami z użyciem efektu fade in/out	183
SceneFade	Klasa obsługująca efekt zanikania i pojawiania obrazu w scenie. Może być używana do przejść między scenami lub efektów fade-in/fade-out	185

SellingManager	
Klasa odpowiedzialna za sprzedaż zasobów przez gracza i przeliczanie zysków w złocie. Uwzględnia bonusy z mikstur, prestiż i klasy postaci	188
SkillCheckScript	
Skrypt obsługujący mini-grę Skill Check w systemie klików. Gracz musi kliknąć odpowiednim przyciskiem myszy na nadlatujące klocki, aby zdobywać bonusy i utrzymywać combo . . .	191
SkillManager	
Manager umiejętności postaci gracza. Odpowiada za skill check, aktywne idle, efekty mikstur i tymczasowe buffy	196
SkillPointsLimit	
Manager przycisku Skill Points. Odpowiada za wyświetlanie liczby punktów umiejętności, zmianę koloru przycisku w zależności od dostępności oraz zwiększanie i zmniejszanie punktów postaci	200
SkillTreeManager	
Bazowa klasa dla drzewek umiejętności. Zarządza przyciskami, zależnościami, liniami wizualnymi między umiejętnościami i zapis/odczyt stanu nauczonych umiejętności	203
SkillTreesUIManager	
Menedżer wszystkich trzech drzewek umiejętności w UI. Odpowiada za inicjalizację, aktualizację, serializację i rysowanie linii między przyciskami	209
StaticBackground	
Klasa odpowiedzialna za dynamiczną zmianę tła w UI w zależności od aktualnie wybranej fabryki. Sprawdza co klatkę, która fabryka jest w fokusie, i aktualizuje RawImage tła odpowiednią teksturą	214
Tooltip	
Klasa statyczna odpowiedzialna za wyświetlanie tooltipów w UI. Umożliwia inicjalizację tooltipu, jego pokazanie, ukrycie, przesuwanie oraz rejestrowanie elementów, które będą wywoływać tooltip po najechaniu myszką	216

Rozdział 4

Indeks plików

4.1 Lista plików

Tutaj znajduje się lista wszystkich plików wraz z ich krótkimi opisami:

ActivePotionsUI.cs	221
BrickMovementScript.cs	221
BuffManager.cs	221
BuffVisual.cs	221
CharacterClass.cs	222
ComboVisual.cs	222
ContentPageManager.cs	222
CritVisualGenerator.cs	222
DynamicBackground.cs	222
Factory.cs	222
FactoryUIManager.cs	223
IdleManager.cs	223
LayoutController.cs	223
LevelUIManager.cs	223
MainMenuController.cs	223
OptionsManager.cs	224
Potion.cs	224
PotionUIManager.cs	224
PrestigeSystem.cs	224
PrestigeUIManager.cs	224
QuarkTestNode.cs	224
QuarkType.cs	225
RaptorCore.cs	225
Resource.cs	225
ResourceManager.cs	225
ResourceShopManager.cs	225
SaveManager.cs	225
SceneController.cs	226
SceneFade.cs	226
SellManager.cs	226
SkillCheckScript.cs	226
SkillManager.cs	226
SkillPointsLimit.cs	226
SkillTreesUI.cs	227
SkillTreesUIManager.cs	227
StaticBackground.cs	227
ToolTipStatic.cs	227

Rozdział 5

Dokumentacja katalogów

5.1 Dokumentacja katalogu Assets

Katalogi

- katalog [Scripts](#)

5.2 Dokumentacja katalogu Scripts

Pliki

- plik [ActivePotionsUI.cs](#)
- plik [BrickMovementScript.cs](#)
- plik [BuffManager.cs](#)
- plik [BuffVisual.cs](#)
- plik [CharacterClass.cs](#)
- plik [ComboVisual.cs](#)
- plik [ContentPageManager.cs](#)
- plik [CritVisualGenerator.cs](#)
- plik [DynamicBackground.cs](#)
- plik [Factory.cs](#)
- plik [FactoryUIManager.cs](#)
- plik [IdleManager.cs](#)
- plik [LayoutController.cs](#)
- plik [LevelUIManager.cs](#)
- plik [MainMenuController.cs](#)
- plik [OptionsManager.cs](#)
- plik [Potion.cs](#)
- plik [PotionUIManager.cs](#)
- plik [PrestigeSystem.cs](#)
- plik [PrestigeUIManager.cs](#)
- plik [QuarkTestNode.cs](#)
- plik [QuarkType.cs](#)
- plik [RaptorCore.cs](#)
- plik [Resource.cs](#)
- plik [ResourceManager.cs](#)

- plik [ResourceShopManager.cs](#)
- plik [SaveManager.cs](#)
- plik [SceneController.cs](#)
- plik [SceneFade.cs](#)
- plik [SellManager.cs](#)
- plik [SkillCheckScript.cs](#)
- plik [SkillManager.cs](#)
- plik [SkillPointsLimit.cs](#)
- plik [SkillTreesUI.cs](#)
- plik [SkillTreesUIManager.cs](#)
- plik [StaticBackground.cs](#)
- plik [ToolTipStatic.cs](#)

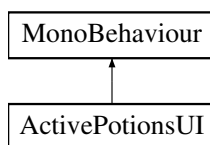
Rozdział 6

Dokumentacja klas

6.1 Dokumentacja klasy ActivePotionsUI

Zarządza wyświetlaniem aktywnych mikstur (Potions) w UI gry. Pobiera dane z [IdleManager](#) i dynamicznie aktualizuje panel mikstur.

Diagram dziedziczenia dla ActivePotionsUI



Metody publiczne

- void [RefreshPanel](#) ()
Odświeża panel mikstur ręcznie (np. po zmianie stanu mikstur).

Metody prywatne

- void [Start](#) ()
Inicjalizacja panelu aktywnych mikstur. Pobiera root VisualElement z UIDocument.
- void [Update](#) ()
Aktualizuje wyświetlanie mikstur i timery co klatkę.
- void [UpdateActivePotionsDisplay](#) ()
Sprawdza, czy liczba aktywnych mikstur różni się od liczby wyświetlanych elementów w panelu i przebudowuje panel, jeśli to konieczne.
- void [RebuildPanel](#) ()
Przebudowuje panel mikstur, dodaje tylko aktywne mikstury.
- void [CreatePotionDisplay](#) ([Potion](#) potion, int index)
Tworzy element wizualny dla pojedynczej mikstury i dodaje go do panelu.
- void [UpdatePotionTimers](#) ()
Aktualizuje timery mikstur w panelu co klatkę. Zmienia kolor tekstu w zależności od czasu pozostałego.
- Sprite [GetPotionSprite](#) (string potionName)
Zwraca sprite dla mikstury według jej nazwy.
- Color [GetPotionColor](#) (string potionName)
Zwraca kolor borderu dla mikstury według jej nazwy.

Atrybuty prywatne

- [IdleManager](#) `idleManager`
Referencja do menedżera Idle, który trzyma informacje o miksturach.
- `UIDocument` [uiDocument](#)
Referencja do dokumentu UI (UIDocument).
- `Sprite` [unoPotionSprite](#)
Sprite dla pierwszej mikstury.
- `Sprite` [dosPotionSprite](#)
Sprite dla drugiej mikstury.
- `Sprite` [tresPotionSprite](#)
Sprite dla trzeciej mikstury.
- `Sprite` [cuatroPotionSprite](#)
Sprite dla czwartej mikstury.
- `Sprite` [cincoPotionSprite](#)
Sprite dla piątej mikstury.
- `Sprite` [seisPotionSprite](#)
Sprite dla szóstej mikstury.
- `VisualElement` [activePotionsPanel](#)
Obiekt w interfejsie odpowiadający za miejsce aktywnych mikstur.
- `List< VisualElement >` [activePotionDisplays](#) = `new List<VisualElement>()`
Lista reprezentująca wszystkie aktywne mikstury.

6.1.1 Opis szczegółowy

Zarządza wyświetlaniem aktywnych mikstur (Potions) w UI gry. Pobiera dane z [IdleManager](#) i dynamicznie aktualizuje panel mikstur.

6.1.2 Dokumentacja funkcji składowych

6.1.2.1 CreatePotionDisplay()

```
void ActivePotionsUI.CreatePotionDisplay (
    Potion potion,
    int index) [private]
```

Tworzy element wizualny dla pojedynczej mikstury i dodaje go do panelu.

Parametry

<i>potion</i>	Obiekt mikstury
<i>index</i>	Indeks mikstury w liście

6.1.2.2 GetPotionColor()

```
Color ActivePotionsUI.GetPotionColor (
    string potionName) [private]
```

Zwraca kolor borderu dla mikstury według jej nazwy.

Parametry

<i>potionName</i>	Nazwa mikstury
-------------------	----------------

Zwraca

Kolor borderu mikstury

6.1.2.3 GetPotionSprite()

```
Sprite ActivePotionsUI.GetPotionSprite (
    string potionName) [private]
```

Zwraca sprite dla mikstury według jej nazwy.

Parametry

<i>potionName</i>	Nazwa mikstury
-------------------	----------------

Zwraca

Sprite odpowiadający miksturze lub null

6.1.2.4 RebuildPanel()

```
void ActivePotionsUI.RebuildPanel () [private]
```

Przebudowuje panel mikstur, dodaje tylko aktywne mikstury.

6.1.2.5 RefreshPanel()

```
void ActivePotionsUI.RefreshPanel ()
```

Odświeża panel mikstur ręcznie (np. po zmianie stanu mikstur).

6.1.2.6 Start()

```
void ActivePotionsUI.Start () [private]
```

Inicjalizacja panelu aktywnych mikstur. Pobiera root VisualElement z UIDocument.

6.1.2.7 Update()

```
void ActivePotionsUI.Update () [private]
```

Aktualizuje wyświetlanie mikstur i timery co klatkę.

6.1.2.8 UpdateActivePotionsDisplay()

```
void ActivePotionsUI.UpdateActivePotionsDisplay () [private]
```

Sprawdza, czy liczba aktywnych mikstur różni się od liczby wyświetlanych elementów w panelu i przebudowuje panel, jeśli to konieczne.

6.1.2.9 UpdatePotionTimers()

```
void ActivePotionsUI.UpdatePotionTimers () [private]
```

Aktualizuje timery mikstur w panelu co klatkę. Zmienia kolor tekstu w zależności od czasu pozostałego.

6.1.3 Dokumentacja atrybutów składowych

6.1.3.1 activePotionDisplays

```
List<VisualElement> ActivePotionsUI.activePotionDisplays = new List<VisualElement>() [private]
```

Lista reprezentująca wszystkie aktywne mikstury.

6.1.3.2 activePotionsPanel

```
VisualElement ActivePotionsUI.activePotionsPanel [private]
```

Obiekt w interfejsie odpowiadający za miejsce aktywnych mikstur.

6.1.3.3 cincoPotionSprite

```
Sprite ActivePotionsUI.cincoPotionSprite [private]
```

Sprite dla piątej mikstury.

6.1.3.4 cuatroPotionSprite

```
Sprite ActivePotionsUI.cuatroPotionSprite [private]
```

Sprite dla czwartej mikstury.

6.1.3.5 dosPotionSprite

`Sprite ActivePotionsUI.dosPotionSprite [private]`

Sprite dla drugiej mikstury.

6.1.3.6 idleManager

`IdleManager ActivePotionsUI.idleManager [private]`

Referencja do menedżera Idle, który trzyma informacje o miksturach.

6.1.3.7 seisPotionSprite

`Sprite ActivePotionsUI.seisPotionSprite [private]`

Sprite dla szóstej mikstury.

6.1.3.8 tresPotionSprite

`Sprite ActivePotionsUI.tresPotionSprite [private]`

Sprite dla trzeciej mikstury.

6.1.3.9 uiDocument

`UIDocument ActivePotionsUI.uiDocument [private]`

Referencja do dokumentu UI (UIDocument).

6.1.3.10 unoPotionSprite

`Sprite ActivePotionsUI.unoPotionSprite [private]`

Sprite dla pierwszej mikstury.

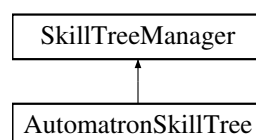
Dokumentacja dla tej klasy została wygenerowana z pliku:

- [ActivePotionsUI.cs](#)

6.2 Dokumentacja klasy AutomatronSkillTree

Drzewko umiejętności typu "Automatron". Zarządza odblokowywaniem i wizualizacją poszczególnych umiejętności dla tej gałęzi.

Diagram dziedziczenia dla AutomatronSkillTree



Metody publiczne

- [AutomatronSkillTree](#) (VisualElement root, [CharacterClass](#) charClass, HashSet< string > learnedSet, [SkillPointsLimit](#) skillPointsLimit)

Konstruktor drzewka Automatron.

Metody publiczne dziedziczone z [SkillTreeManager](#)

- [SkillTreeManager](#) (VisualElement root, [CharacterClass](#) charClass, Dictionary< string, List< string > > treeDependencies, string containerName, HashSet< string > learnedSet, [SkillPointsLimit](#) skillPointsLimit)

Konstruktor bazowy [SkillTreeManager](#).

- void [ClearTree](#) ()
Czyści nauczone umiejętności.
- void [InitializeSkills](#) (string[] skillIds)
Tworzy przyciski dla wszystkich umiejętności w drzewku.
- void [RegisterTooltip](#) (string skillId, string title, string description, string lore)
Rejestruje tooltip dla przycisku umiejętności.
- void [DrawLine](#) (string fromId, string toId, Color color)
Rysuje pojedynczą linię między dwoma przyciskami.
- void [DrawAllLines](#) ()
Rysuje wszystkie linie zależności w drzewku.
- void [UpdateAll](#) ()
Aktualizuje wszystkie przyciski wizualnie.
- void [SerializeToStream](#) (BinaryWriter writer)
Zapisuje stan nauczonych umiejętności do strumienia.
- void [DeserializeFromStream](#) (BinaryReader reader)
Wczytuje stan nauczonych umiejętności ze strumienia.

Metody chronione

- override void [OnSkillClicked](#) (string id)
Obsługuje kliknięcie lewym przyciskiem na umiejętność.
- override bool [CanUnlock](#) (string id)
Sprawdza, czy umiejętność może zostać odblokowana.
- override void [UpdateVisual](#) (string id)
Aktualizuje wygląd przycisku umiejętności w zależności od stanu (odblokowana, dostępna, zablokowana).

Metody chronione dziedziczone z [SkillTreeManager](#)

- void [OnSkillClicked](#) (string id)
Wywoływane po kliknięciu w umiejętność
- bool [CanUnlock](#) (string id)
Sprawdza, czy umiejętność może zostać nauczona.
- void [UpdateVisual](#) (string id)
Aktualizuje wizualną reprezentację przycisku umiejętności.
- virtual void [OnSkillUnlearned](#) (string id)
Wywoływane przy "odnauczaniu" umiejętności (klik prawym przyciskiem).

Statyczne metody prywatne

- static `Dictionary< string, List< string > >` `GetDependencies` ()
Zależności między umiejętnościami.

Dodatkowe dziedziczone składowe

Atrybuty chronione dziedziczone z `SkillTreeManager`

- `VisualElement` `ui`
Root UI dokumentu.
- `CharacterClass` `characterClass`
Klasa postaci, której drzewko dotyczy.
- `Dictionary< string, Button >` `buttons` = `new()`
Mapa przycisków umiejętności według ID.
- `HashSet< string >` `learned` = `new()`
Zbiór nauczonych umiejętności.
- `Dictionary< string, List< string > >` `dependencies` = `new()`
Zależności umiejętności.
- `VisualElement` `skillTreeContainer`
Kontener UI dla całego drzewka.
- `VisualElement` `lineLayer`
Warstwa linii rysowanych między umiejętnościami.
- `Color` `lockedColor` = `new Color(0.3f, 0.3f, 0.3f)`
Kolor przycisku zablokowanego.
- `Color` `availableColor` = `new Color(0.8f, 0.6f, 0.3f)`
Kolor przycisku dostępnego do nauki.
- `Color` `learnedColor` = `new Color(0.2f, 0.8f, 0.2f)`
Kolor przycisku nauczonego.
- `SkillPointsLimit` `skillPointsLimit`
Referencja do menedżera punktów umiejętności.

6.2.1 Opis szczegółowy

Drzewko umiejętności typu "Automatron". Zarządza odblokowywaniem i wizualizacją poszczególnych umiejętności dla tej gałęzi.

6.2.2 Dokumentacja konstruktora i destruktora

6.2.2.1 `AutomatronSkillTree()`

```
AutomatronSkillTree.AutomatronSkillTree (  
    VisualElement root,  
    CharacterClass charClass,  
    HashSet< string > learnedSet,  
    SkillPointsLimit skillPointsLimit)
```

Konstruktor drzewka Automatron.

Parametry

<i>root</i>	Root VisualElement UI drzewka
<i>charClass</i>	Instancja CharacterClass gracza
<i>learnedSet</i>	Zestaw już odblokowanych umiejętności
<i>skillPointsLimit</i>	Obiekt zarządzający punktami umiejętności

6.2.3 Dokumentacja funkcji składowych

6.2.3.1 CanUnlock()

```
override bool AutomatronSkillTree.CanUnlock (  
    string id) [protected]
```

Sprawdza, czy umiejętność może zostać odblokowana.

Parametry

<i>id</i>	ID umiejętności
-----------	-----------------

Zwraca

true jeśli odblokowanie jest możliwe

6.2.3.2 GetDependencies()

```
Dictionary< string, List< string > > AutomatronSkillTree.GetDependencies () [static], [private]
```

Zależności między umiejętnościami.

6.2.3.3 OnSkillClicked()

```
override void AutomatronSkillTree.OnSkillClicked (  
    string id) [protected]
```

Obsługuje kliknięcie lewym przyciskiem na umiejętność.

Parametry

<i>id</i>	ID klikniętej umiejętności
-----------	----------------------------

6.2.3.4 UpdateVisual()

```
override void AutomatronSkillTree.UpdateVisual (  
    string id) [protected]
```

Aktualizuje wygląd przycisku umiejętności w zależności od stanu (odblokowana, dostępna, zablokowana).

Parametry

<i>id</i>	ID umiejętności
-----------	-----------------

Dokumentacja dla tej klasy została wygenerowana z pliku:

- [SkillTreesUI.cs](#)

6.3 Dokumentacja klasy AutoSaveSystem

Statyczny system autozapisu gry. Umożliwia zapis i odczyt wszystkich obiektów implementujących [IBinarySaveable](#), obsługę autopoziomu zapisu i domyślnej ścieżki do pliku.

Statyczne metody publiczne

- static void [LazyGetAutoSave](#) ()
Wczytuje stan AutoSave tylko raz przy pierwszym odczycie.
- static void [DeleteSave](#) ()
Usuwa plik zapisu gry.
- static void [SaveGame](#) ()
Zapisuje wszystkie obiekty implementujące [IBinarySaveable](#) do pliku zapisu.
- static void [LoadGame](#) ()
Ładuje wszystkie obiekty implementujące [IBinarySaveable](#) z pliku zapisu.
- static void [SaveAll](#) (BinaryWriter writer)
Zapisuje wszystkie obiekty implementujące [IBinarySaveable](#) w scenie do strumienia binarnego.
- static void [LoadAll](#) (BinaryReader reader)
Wczytuje wszystkie obiekty implementujące [IBinarySaveable](#) w scenie ze strumienia binarnego.

Właściwości

- static string [SaveFileName](#) = "autosave.dat" [get, set]
Nazwa pliku, w którym zapisywana jest gra.
- static string [SavePath](#) [get]
Pełna ścieżka do pliku zapisu.
- static bool [AutoSave](#) [get, set]
Właściwość umożliwiająca odczyt lub ustawienie trybu AutoSave.

Statyczne metody prywatne

- static string `SavePathS` (string FileName)
Generuje pełną ścieżkę do dowolnego pliku zapisu.
- static bool `GetAutoSave` ()
Odczytuje stan `AutoSave` z pliku `MetaData.dat`.
- static void `SetAutoSave` (bool state)
Zapisuje stan `AutoSave` do pliku `MetaData.dat`.

Statyczne atrybuty prywatne

- static readonly string `path`
Ścieżka do pliku z metadanymi, np. stanu `AutoSave`.
- static bool `loaded` = false
Flaga, czy ustawienia `AutoSave` zostały już wczytane.
- static bool `_AutoSave`
Wewnętrzna flaga przechowująca stan `AutoSave`.

6.3.1 Opis szczegółowy

Statyczny system autozapisu gry. Umożliwia zapis i odczyt wszystkich obiektów implementujących `IBinarySaveable`, obsługę autopoziomu zapisu i domyślnej ścieżki do pliku.

6.3.2 Dokumentacja funkcji składowych

6.3.2.1 DeleteSave()

```
void AutoSaveSystem.DeleteSave () [static]
```

Usuwa plik zapisu gry.

6.3.2.2 GetAutoSave()

```
bool AutoSaveSystem.GetAutoSave () [static], [private]
```

Odczytuje stan `AutoSave` z pliku `MetaData.dat`.

Zwraca

True, jeśli `AutoSave` jest włączony, false w przeciwnym wypadku.

6.3.2.3 LazyGetAutoSave()

```
void AutoSaveSystem.LazyGetAutoSave () [static]
```

Wczytuje stan `AutoSave` tylko raz przy pierwszym odczycie.

6.3.2.4 `LoadAll()`

```
void AutoSaveSystem.LoadAll (  
    BinaryReader reader) [static]
```

Wczytuje wszystkie obiekty implementujące [IBinarySaveable](#) w scenie ze strumienia binarnego.

Parametry

<i>reader</i>	BinaryReader do odczytu danych.
---------------	---------------------------------

6.3.2.5 `LoadGame()`

```
void AutoSaveSystem.LoadGame () [static]
```

Ładuje wszystkie obiekty implementujące [IBinarySaveable](#) z pliku zapisu.

6.3.2.6 `SaveAll()`

```
void AutoSaveSystem.SaveAll (  
    BinaryWriter writer) [static]
```

Zapisuje wszystkie obiekty implementujące [IBinarySaveable](#) w scenie do strumienia binarnego.

Parametry

<i>writer</i>	BinaryWriter do zapisu danych.
---------------	--------------------------------

6.3.2.7 `SaveGame()`

```
void AutoSaveSystem.SaveGame () [static]
```

Zapisuje wszystkie obiekty implementujące [IBinarySaveable](#) do pliku zapisu.

6.3.2.8 `SavePathS()`

```
string AutoSaveSystem.SavePathS (  
    string FileName) [static], [private]
```

Generuje pełną ścieżkę do dowolnego pliku zapisu.

6.3.2.9 SetAutoSave()

```
void AutoSaveSystem.SetAutoSave (  
    bool state) [static], [private]
```

Zapisuje stan AutoSave do pliku MetaData.dat.

Parametry

<i>state</i>	Flaga włączająca lub wyłączająca AutoSave.
--------------	--

6.3.3 Dokumentacja atrybutów składowych

6.3.3.1 _AutoSave

```
bool AutoSaveSystem._AutoSave [static], [private]
```

Wewnętrzna flaga przechowująca stan AutoSave.

6.3.3.2 loaded

```
bool AutoSaveSystem.loaded = false [static], [private]
```

Flaga, czy ustawienia AutoSave zostały już wczytane.

6.3.3.3 path

```
readonly string AutoSaveSystem.path [static], [private]
```

Wartość początkowa:

```
=  
Path.Combine(Application.persistentDataPath, "MetaData.dat")
```

Ścieżka do pliku z metadanymi, np. stanu AutoSave.

6.3.4 Dokumentacja właściwości

6.3.4.1 AutoSave

```
bool AutoSaveSystem.AutoSave [static], [get], [set]
```

Właściwość umożliwiająca odczyt lub ustawienie trybu AutoSave.

6.3.4.2 SaveFileName

```
string AutoSaveSystem.SaveFileName = "autosave.dat" [static], [get], [set]
```

Nazwa pliku, w którym zapisywana jest gra.

6.3.4.3 SavePath

```
string AutoSaveSystem.SavePath [static], [get], [private]
```

Pełna ścieżka do pliku zapisu.

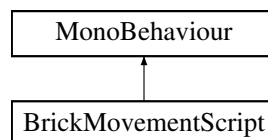
Dokumentacja dla tej klasy została wygenerowana z pliku:

- [SaveManager.cs](#)

6.4 Dokumentacja klasy BrickMovementScript

Odpowiada za ruch bloczka oraz wykrywanie kolizji z granicami. Skrypt losuje kierunek ruchu bloczka podczas aktywacji "SkillChecka", zmienia jego wygląd oraz aktualizuje status overlapowania w zależności od kolizji z obszarami granicznymi.

Diagram dziedziczenia dla BrickMovementScript



Atrybuty publiczne

- int [overlap](#) = 0
Status overlapowania z granicami.

Metody prywatne

- void [Start](#) ()
Metoda wywoływana przy starcie obiektu. Losuje kierunek ruchu bloczka oraz ustawia jego wygląd.
- void [Update](#) ()
Metoda wywoływana co klatkę. Odpowiada za przesuwanie bloczka.
- void [OnTriggerEnter2D](#) (Collider2D collision)
Wywoływana w momencie wejścia w obszar kolizji.
- void [OnTriggerExit2D](#) (Collider2D collision)
Wywoływana w momencie wyjścia z obszaru kolizji.

Atrybuty prywatne

- float [speed](#) = 0f
Prędkość poruszania się bloczka. Wartość dodatnia oznacza ruch w lewo, ujemna – ruch w prawo.
- SpriteRenderer [sr](#)
Referencja do komponentu SpriteRenderer odpowiadający za wygląd klocka.

6.4.1 Opis szczegółowy

Odpowiada za ruch bloczka oraz wykrywanie kolizji z granicami. Skrypt losuje kierunek ruchu bloczka podczas aktywacji "SkillChecka", zmienia jego wygląd oraz aktualizuje status overlapowania w zależności od kolizji z obszarami granicznymi.

6.4.2 Dokumentacja funkcji składowych

6.4.2.1 OnTriggerEnter2D()

```
void BrickMovementScript.OnTriggerEnter2D (  
    Collider2D collision) [private]
```

Wywoływana w momencie wejścia w obszar kolizji.

Parametry

<i>collision</i>	Strefa, z którą nastąpiła kolizja
------------------	-----------------------------------

6.4.2.2 OnTriggerExit2D()

```
void BrickMovementScript.OnTriggerExit2D (  
    Collider2D collision) [private]
```

Wywoływana w momencie wyjścia z obszaru kolizji.

Parametry

<i>collision</i>	Strefa, z którą zakończono kolizję
------------------	------------------------------------

6.4.2.3 Start()

```
void BrickMovementScript.Start () [private]
```

Metoda wywoływana przy starcie obiektu. Losuje kierunek ruchu bloczka oraz ustawia jego wygląd.

6.4.2.4 Update()

```
void BrickMovementScript.Update () [private]
```

Metoda wywoływana co klatkę. Odpowiada za przesuwanie bloczka.

6.4.3 Dokumentacja atrybutów składowych

6.4.3.1 overlap

```
int BrickMovementScript.overlap = 0
```

Status overlapowania z granicami.

Możliwe wartości: 0 – brak kolizji

1 – dotknięcie lewej granicy

2 – dotknięcie prawej granicy

3 – wyjście poza granice

6.4.3.2 speed

```
float BrickMovementScript.speed = 0f [private]
```

Prędkość poruszania się bloczka. Wartość dodatnia oznacza ruch w lewo, ujemna – ruch w prawo.

6.4.3.3 sr

```
SpriteRenderer BrickMovementScript.sr [private]
```

Referencja do komponentu SpriteRenderer odpowiadający za wygląd klocka.

Dokumentacja dla tej klasy została wygenerowana z pliku:

- [BrickMovementScript.cs](#)

6.5 Dokumentacja klasy BuffManager

Zarządza buffami wyświetlanymi w interfejsie użytkownika.

Metody publiczne

- [BuffManager](#) (VisualElement root, Texture2D[] [buffs](#))
Tworzy nową instancję menedżera buffów.
- void [RegisterBuff](#) (string skillId, Texture2D icon)
Rejestruje nowy buff w systemie.
- void [ApplyBuff](#) (string skillId, double value)
Wyświetla buff jako aktywny.
- void [RemoveBuff](#) (string skillId)
Ukrywa buff.

Atrybuty prywatne

- VisualElement `buffBar`
Kontener UI przechowujący ikony buffów.
- Dictionary< string, BuffVisual > `buffs` = new()
Słownik przechowujący aktywne buffy. Kluczem jest identyfikator umiejętności.

6.5.1 Opis szczegółowy

Zarządza buffami wyświetlanymi w interfejsie użytkownika.

Klasa odpowiada za rejestrowanie buffów, ich wyświetlanie oraz ukrywanie na pasku buffów.

6.5.2 Dokumentacja konstruktora i destruktora

6.5.2.1 BuffManager()

```
BuffManager.BuffManager (
    VisualElement root,
    Texture2D[] buffs)
```

Tworzy nową instancję menedżera buffów.

Parametry

<i>root</i>	Główny element UI, z którego pobierany jest pasek buffów
<i>buffs</i>	Tablica tekstur ikon buffów

6.5.3 Dokumentacja funkcji składowych

6.5.3.1 ApplyBuff()

```
void BuffManager.ApplyBuff (
    string skillId,
    double value)
```

Wyświetla buff jako aktywny.

Parametry

<i>skillId</i>	Identyfikator umiejętności
<i>value</i>	Wartość buffa do wyświetlenia

6.5.3.2 RegisterBuff()

```
void BuffManager.RegisterBuff (  
    string skillId,  
    Texture2D icon)
```

Rejestruje nowy buff w systemie.

Parametry

<i>skillId</i>	Identyfikator umiejętności
<i>icon</i>	Ikona reprezentująca buff

6.5.3.3 RemoveBuff()

```
void BuffManager.RemoveBuff (  
    string skillId)
```

Ukrywa buff.

Parametry

<i>skillId</i>	Identyfikator umiejętności
----------------	----------------------------

6.5.4 Dokumentacja atrybutów składowych

6.5.4.1 buffBar

```
VisualElement BuffManager.buffBar [private]
```

Kontener UI przechowujący ikony buffów.

6.5.4.2 buffs

```
Dictionary<string, BuffVisual> BuffManager.buffs = new() [private]
```

Słownik przechowujący aktywne buffy. Kluczem jest identyfikator umiejętności.

Dokumentacja dla tej klasy została wygenerowana z pliku:

- [BuffManager.cs](#)

6.6 Dokumentacja klasy BuffVisual

Reprezentuje pojedynczy buff wyświetlany w interfejsie użytkownika.

Metody publiczne

- `BuffVisual` (string `skillId`, Texture2D `icon`)
Tworzy nową wizualną reprezentację buffa.
- void `Show` (double `value`)
Wyświetla buff oraz aktualizuje jego wartość.
- void `Hide` ()
Ukrywa buff w interfejsie użytkownika.

Atrybuty publiczne

- string `skillId`
Identyfikator umiejętności powiązanej z buffem.
- VisualElement `root`
Główny element UI buffa.

Atrybuty prywatne

- Label `valueLabel`
Etykieta wyświetlająca aktualną wartość buffa.

6.6.1 Opis szczegółowy

Reprezentuje pojedynczy buff wyświetlany w interfejsie użytkownika.

Klasa odpowiada za wizualną reprezentację buffa, w tym ikonę, wartość oraz widoczność elementu UI.

6.6.2 Dokumentacja konstruktora i destruktora

6.6.2.1 BuffVisual()

```
BuffVisual.BuffVisual (
    string skillId,
    Texture2D icon)
```

Tworzy nową wizualną reprezentację buffa.

Parametry

<i>skillId</i>	Identyfikator umiejętności
<i>icon</i>	Ikona buffa

6.6.3 Dokumentacja funkcji składowych

6.6.3.1 Hide()

```
void BuffVisual.Hide ()
```

Ukrywa buff w interfejsie użytkownika.

6.6.3.2 Show()

```
void BuffVisual.Show (  
    double value)
```

Wyświetla buff oraz aktualizuje jego wartość.

Parametry

<i>value</i>	Wartość buffa do wyświetlenia
--------------	-------------------------------

6.6.4 Dokumentacja atrybutów składowych

6.6.4.1 root

```
VisualElement BuffVisual.root
```

Główny element UI buffa.

6.6.4.2 skillId

```
string BuffVisual.skillId
```

Identyfikator umiejętności powiązanej z buffem.

6.6.4.3 valueLabel

```
Label BuffVisual.valueLabel [private]
```

Etykieta wyświetlająca aktualną wartość buffa.

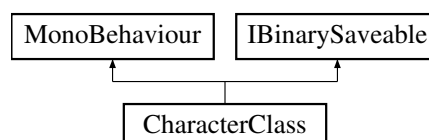
Dokumentacja dla tej klasy została wygenerowana z pliku:

- [BuffVisual.cs](#)

6.7 Dokumentacja klasy CharacterClass

Główna klasa reprezentująca postać gracza w systemie RPG.

Diagram dziedziczenia dla CharacterClass



Metody publiczne

- void [GainExp](#) (ulong exp)
Dodaje doświadczenie postaci i obsługuje awans poziomu.
- ulong [GetCurrentExp](#) ()
Zwraca aktualne doświadczenie.
- ulong [GetMaxExpCap](#) ()
Zwraca limit doświadczenia.
- uint [GetLevel](#) ()
Zwraca aktualny poziom.
- float [GetCriticalChance](#) ()
Zwraca całkowitą szansę krytyczną.
- float [GetSkillCheckChance](#) ()
Zwraca szansę na wystąpienie "SkillChecka".
- void [SetLevel](#) (uint __level)
Ustawia poziom postaci ręcznie.
- void [ApplySkillChanges](#) (string id, bool revert)
Aplikuje lub cofa zmiany wynikające z umiejętności.
- void [EnableAlanWake](#) ()
Aktywuje umiejętności Alan Wake.
- void [Pedator](#) (bool reset)
Aktywuje lub cofa efekt umiejętności Predator.
- void [EnableGenius](#) (bool reset)
Aktywuje umiejętności Marketplace Genius.
- void [EnableExpBoost](#) ()
Aktywuje boost do doświadczenia w zależności od posiadanych umiejętności.
- void [SerializeToStream](#) (BinaryWriter writer)
Serializuje dane postaci do strumienia binarnego.
- void [DeserializeFromStream](#) (BinaryReader reader)
Deserializuje dane postaci ze strumienia binarnego.
- void [ResetCharacter](#) ()
Resetuje postać do stanu początkowego.

Atrybuty publiczne

- [BuffManager buffManager](#)
Menedżer buffów odpowiedzialny za UI.
- [RaptorCore Raptorcore](#)
Referencja do rdzenia gry.
- uint [skillpoints](#) = 0
Aktualna liczba punktów umiejętności.
- float [boostedChance](#)
Bonus do szansy krytycznej.
- int [skillCheckReduce](#)
Redukcja wywołania "SkillChecka".
- double [productionIdleBonus](#)
Bonus do produkcji idle.
- double [productionIdlePedatorBonus](#)
Bonus do produkcji idle umiejętności Predator.
- double [productionIdleAgressiveBonus](#)

- Bonus do produkcji idle umiejętności Passive Agressive.*
- double `sellingBonus`
Bonus do sprzedaży.
- double `sellingHardBonus`
Bonus do sprzedaży wykorzystywany przy umiejętności Marketplace Genius.
- double `potionBoost`
Bonus do mikstur.
- double `potionRandomChance`
Szansa losowa miksture.
- ulong `expBoost`
Bonus do doświadczenia.
- ulong `komboBoost`
Bonus kombo.
- bool `activeIdle` = false
Flaga umiejętności Active Idle.
- bool `noMatterWhat` = false
Flaga umiejętności No Matter What.
- bool `chickenDinner` = false
Flaga umiejętności Chicken Dinner.
- bool `alwaysWinner` = false
Flaga umiejętności Always Winner.
- bool `sybiosis` = false
Flaga umiejętności Sybiosis.
- bool `mortalClicker` = false
Flaga umiejętności Mortal Clicker.
- bool `championOfClicks` = false
Flaga umiejętności Champion Of Clicks.
- bool `reactionTest` = false
Flaga umiejętności Reaction Test.
- bool `unskilledPredator` = false
Flaga umiejętności Unskilled Predator.
- bool `oneForEveryone` = false
Flaga umiejętności One For Everyone.
- bool `whatEyesDontSee` = false
Flaga umiejętności What Eyes Dont See.
- bool `passiveAgressive` = false
Flaga umiejętności Passive Agressive.
- bool `multitasking` = false
Flaga umiejętności Multitasking.
- bool `christmasBonus` = false
Flaga umiejętności Christmas Bonus.
- bool `hungryWolf` = false
Flaga umiejętności Hungry Wolf.
- bool `marketplaceGenius` = false
Flaga umiejętności Marketplace Genius.
- bool `hardWorker` = false
Flaga umiejętności HardWorker.
- bool `deathDose` = false
Flaga umiejętności DeathDose.
- bool `failToWin` = false
Flaga umiejętności Fail To Win.

- bool `failureGrind` = false
Flaga umiejętności Failure Grind.
- bool `michealScott` = false
Flaga umiejętności Micheal Scott.
- int `T1` = 0
Licznik umiejętności drzewa 1.
- int `T2` = 0
Licznik umiejętności drzewa 2.
- int `T3` = 0
Licznik umiejętności drzewa 3.

Właściwości

- int `SaveKey` [get]
Klucz zapisu używany w systemie zapisu stanu gry.

Metody prywatne

- void `Start` ()
Inicjalizacja podstawowych statystyk postaci.
- void `NewLevelCap` ()
Oblicza nowy limit doświadczenia po awansie.
- void `UpdateTreeCounter` (string id, bool revert)
Aktualizuje liczniki drzew umiejętności.
- void `SkillBasedClicking` (bool revert)
Aktywuje lub cofa efekt umiejętności Skill Based Clicking.
- void `CriticalMass` (bool revert)
Aktywuje lub cofa efekt umiejętności Critical Mass.
- void `ActiveIdle` (bool revert)
Aktywuje lub cofa efekt umiejętności Active Idle.
- void `NoMatterWhat` (bool revert)
Aktywuje lub cofa efekt umiejętności No Matter What.
- void `ChickenDinner` (bool revert)
Aktywuje lub cofa efekt umiejętności Chicken Dinner.
- void `HungryForMore` (bool revert)
Aktywuje lub cofa efekt umiejętności Hungry For More.
- void `AlwaysWinner` (bool revert)
Aktywuje lub cofa efekt umiejętności Always Winner.
- void `Symbiosis` (bool revert)
Aktywuje lub cofa efekt umiejętności Symbiosis.
- void `MortalClicker` (bool revert)
Aktywuje lub cofa efekt umiejętności Mortal Clicker.
- void `ChampionOfClicks` (bool revert)
Aktywuje lub cofa efekt umiejętności Champion Of Clicks.
- IEnumerator `AlanWake` ()
Enumerator pomocniczy do boosta krytycznego do umiejętności Alan Wake.
- void `Entrepreneur` (bool revert)
Aktywuje lub cofa efekt umiejętności Entrepreneur.
- void `PushToTheLimit` (bool revert)
Aktywuje lub cofa efekt umiejętności PushToTheLimit.

- void [ReactionTest](#) (bool revert)
Aktywuje lub cofa efekt umiejętności Reaction Test.
- void [UnskilledPredator](#) (bool revert)
Aktywuje lub cofa efekt umiejętności UnskilledPredator.
- void [OneForEveryone](#) (bool revert)
Aktywuje lub cofa efekt umiejętności OneForEveryone.
- void [WhatEyesDontSee](#) (bool revert)
Aktywuje lub cofa efekt umiejętności What Eyes Dont See.
- void [PassiveAgressive](#) (bool revert)
Aktywuje lub cofa efekt umiejętności Passive Agressive.
- void [Multitasking](#) (bool revert)
Aktywuje lub cofa efekt umiejętności Multitasking.
- void [ChristmasBonus](#) (bool revert)
Aktywuje lub cofa efekt umiejętności Christmas Bonus.
- void [HungryWolf](#) (bool revert)
Aktywuje lub cofa efekt umiejętności Hungry Wolf.
- void [Shark](#) (bool revert)
Aktywuje lub cofa efekt umiejętności Shark.
- void [MarketplaceGenius](#) (bool revert)
Aktywuje lub cofa efekt umiejętności Marketplace Genius.
- IEnumerator [Genius](#) ()
Enumerator pomocniczy do boosta sprzedaży do umiejętności Marketplace Genius.
- void [HardWorker](#) (bool revert)
Aktywuje lub cofa efekt umiejętności HardWorker.
- void [Addict](#) (bool revert)
Aktywuje lub cofa efekt umiejętności Addict.
- void [DeathDose](#) (bool revert)
Aktywuje lub cofa efekt umiejętności DeathDose.
- void [LuckyBastard](#) (bool revert)
Aktywuje lub cofa efekt umiejętności Lucky Bastard.
- void [JustBastard](#) (bool revert)
Aktywuje lub cofa efekt umiejętności Just Bastard.
- void [FailToWin](#) (bool revert)
Aktywuje lub cofa efekt umiejętności Fail To Win.
- IEnumerator [Failure](#) ()
Enumerator pomocniczy do boosta od doświadczenia.
- void [FailureGrind](#) (bool revert)
Aktywuje lub cofa efekt umiejętności Failure Grind.
- void [MichealScott](#) (bool revert)
Aktywuje lub cofa efekt umiejętności Micheal Scott.
- void [Update](#) ()
Aktualizacja wyświetlania niektórych umiejętności na ekranie użytkownika.

Atrybuty prywatne

- [SkillPointsLimit](#) skillPointsLimit
Ograniczenie punktów umiejętności.
- uint [level](#) = 1
Aktualny poziom gracza.
- ulong [currentExp](#) = 0

- *Aktualna ilość doświadczenia.*
▪ `ulong maxExpCap = 100`
- *Maksymalna ilość doświadczenia wymagana do awansu.*
▪ `uint maxLvlCap = 20`
- *Maksymalny poziom postaci.*
▪ `float criticalChance`
- *Bazowa szansa na trafienie krytyczne.*
▪ `float skillCheckChance`
- *Szansa na wywołanie "SkillChecka".*

6.7.1 Opis szczegółowy

Główna klasa reprezentująca postać gracza w systemie RPG.

Odpowiada za:

- poziom postaci i doświadczenie
- punkty umiejętności
- logikę skillów oraz ich efektów
- buffy wizualne
- zapis i odczyt stanu postaci

6.7.2 Dokumentacja funkcji składowych

6.7.2.1 ActiveIdle()

```
void CharacterClass.ActiveIdle (  
    bool revert) [private]
```

Aktywuje lub cofa efekt umiejętności Active Idle.

Parametry

<i>revert</i>	True – cofa efekt skilla False – aktywuje efekt skilla
---------------	---

6.7.2.2 Addict()

```
void CharacterClass.Addict (
    bool revert) [private]
```

Aktywuje lub cofa efekt umiejętności Addict.

Parametry

<i>revert</i>	True – cofa efekt skilla False – aktywuje efekt skilla
---------------	---

6.7.2.3 AlanWake()

```
IEnumerator CharacterClass.AlanWake () [private]
```

Enumerator pomocniczy do boosta krytycznego do umiejętności Alan Wake.

6.7.2.4 AlwaysWinner()

```
void CharacterClass.AlwaysWinner (
    bool revert) [private]
```

Aktywuje lub cofa efekt umiejętności Always Winner.

Parametry

<i>revert</i>	True – cofa efekt skilla False – aktywuje efekt skilla
---------------	---

6.7.2.5 ApplySkillChanges()

```
void CharacterClass.ApplySkillChanges (
    string id,
    bool revert)
```

Aplikuje lub cofa zmiany wynikające z umiejętności.

Parametry

<i>id</i>	Identyfikator skilla
<i>revert</i>	Czy cofnąć efekt

6.7.2.6 ChampionOfClicks()

```
void CharacterClass.ChampionOfClicks (  
    bool revert) [private]
```

Aktywuje lub cofa efekt umiejętności Champion Of Clicks.

Parametry

<i>revert</i>	True – cofa efekt skilla False – aktywuje efekt skilla
---------------	---

6.7.2.7 ChickenDinner()

```
void CharacterClass.ChickenDinner (  
    bool revert) [private]
```

Aktywuje lub cofa efekt umiejętności Chicken Dinner.

Parametry

<i>revert</i>	True – cofa efekt skilla False – aktywuje efekt skilla
---------------	---

6.7.2.8 ChristmasBonus()

```
void CharacterClass.ChristmasBonus (  
    bool revert) [private]
```

Aktywuje lub cofa efekt umiejętności Christmas Bonus.

Parametry

<i>revert</i>	True – cofa efekt skilla False – aktywuje efekt skilla
---------------	---

6.7.2.9 CriticalMass()

```
void CharacterClass.CriticalMass (  
    bool revert) [private]
```

Aktywuje lub cofa efekt umiejętności Critical Mass.

Parametry

<i>revert</i>	True – cofa efekt skilla False – aktywuje efekt skilla
---------------	---

6.7.2.10 DeathDose()

```
void CharacterClass.DeathDose (
    bool revert) [private]
```

Aktywuje lub cofa efekt umiejętności DeathDose.

Parametry

<i>revert</i>	True – cofa efekt skilla False – aktywuje efekt skilla
---------------	---

6.7.2.11 DeserializeFromStream()

```
void CharacterClass.DeserializeFromStream (
    BinaryReader reader)
```

Deserializuje dane postaci ze strumienia binarnego.

Implementuje [IBinarySaveable](#).

6.7.2.12 EnableAlanWake()

```
void CharacterClass.EnableAlanWake ()
```

Aktywuje umiejętności Alan Wake.

6.7.2.13 EnableExpBoost()

```
void CharacterClass.EnableExpBoost ()
```

Aktywuje boost do doświadczenia w zależności od posiadanych umiejętności.

6.7.2.14 EnableGenius()

```
void CharacterClass.EnableGenius (
    bool reset)
```

Aktywuje umiejętności Marketplace Genius.

6.7.2.15 Entrepreneur()

```
void CharacterClass.Entrepreneur (
    bool revert) [private]
```

Aktywuje lub cofa efekt umiejętności Entrepreneur.

Parametry

<i>revert</i>	True – cofa efekt skilla False – aktywuje efekt skilla
---------------	---

6.7.2.16 FailToWin()

```
void CharacterClass.FailToWin (
    bool revert) [private]
```

Aktywuje lub cofa efekt umiejętności Fail To Win.

Parametry

<i>revert</i>	True – cofa efekt skilla False – aktywuje efekt skilla
---------------	---

6.7.2.17 Failure()

```
IEnumerator CharacterClass.Failure () [private]
```

Enumerator pomocniczy do boosta od doświadczenia.

6.7.2.18 FailureGrind()

```
void CharacterClass.FailureGrind (
    bool revert) [private]
```

Aktywuje lub cofa efekt umiejętności Failure Grind.

Parametry

<i>revert</i>	True – cofa efekt skilla False – aktywuje efekt skilla
---------------	---

6.7.2.19 GainExp()

```
void CharacterClass.GainExp (
    ulong exp)
```

Dodaje doświadczenie postaci i obsługuje awans poziomu.

Parametry

<i>exp</i>	Ilość zdobytego doświadczenia
------------	-------------------------------

6.7.2.20 Genius()

```
IEnumerator CharacterClass.Genius () [private]
```

Enumerator pomocniczy do boosta sprzedaży do umiejętności Marketplace Genius.

6.7.2.21 GetCriticalChance()

```
float CharacterClass.GetCriticalChance ()
```

Zwraca całkowitą szansę krytyczną.

6.7.2.22 GetCurrentExp()

```
ulong CharacterClass.GetCurrentExp ()
```

Zwraca aktualne doświadczenie.

6.7.2.23 GetLevel()

```
uint CharacterClass.GetLevel ()
```

Zwraca aktualny poziom.

6.7.2.24 GetMaxExpCap()

```
ulong CharacterClass.GetMaxExpCap ()
```

Zwraca limit doświadczenia.

6.7.2.25 GetSkillCheckChance()

```
float CharacterClass.GetSkillCheckChance ()
```

Zwraca szansę na wystąpienie "SkillChecka".

6.7.2.26 HardWorker()

```
void CharacterClass.HardWorker (  
    bool revert) [private]
```

Aktywuje lub cofa efekt umiejętności HardWorker.

Parametry

<i>revert</i>	True – cofa efekt skilla False – aktywuje efekt skilla
---------------	---

6.7.2.27 HungryForMore()

```
void CharacterClass.HungryForMore (  
    bool revert) [private]
```

Aktywuje lub cofa efekt umiejętności Hungry For More.

Parametry

<i>revert</i>	True – cofa efekt skilla False – aktywuje efekt skilla
---------------	---

6.7.2.28 HungryWolf()

```
void CharacterClass.HungryWolf (  
    bool revert) [private]
```

Aktywuje lub cofa efekt umiejętności Hungry Wolf.

Parametry

<i>revert</i>	True – cofa efekt skilla False – aktywuje efekt skilla
---------------	---

6.7.2.29 JustBastard()

```
void CharacterClass.JustBastard (  
    bool revert) [private]
```

Aktywuje lub cofa efekt umiejętności Just Bastard.

Parametry

<i>revert</i>	True – cofa efekt skilla False – aktywuje efekt skilla
---------------	---

6.7.2.30 LuckyBastard()

```
void CharacterClass.LuckyBastard (
    bool revert) [private]
```

Aktywuje lub cofa efekt umiejętności Lucky Bastard.

Parametry

<i>revert</i>	True – cofa efekt skilla False – aktywuje efekt skilla
---------------	---

6.7.2.31 MarketplaceGenius()

```
void CharacterClass.MarketplaceGenius (
    bool revert) [private]
```

Aktywuje lub cofa efekt umiejętności Marketplace Genius.

Parametry

<i>revert</i>	True – cofa efekt skilla False – aktywuje efekt skilla
---------------	---

6.7.2.32 MichealScott()

```
void CharacterClass.MichealScott (
    bool revert) [private]
```

Aktywuje lub cofa efekt umiejętności Micheal Scott.

Parametry

<i>revert</i>	True – cofa efekt skilla False – aktywuje efekt skilla
---------------	---

6.7.2.33 MortalClicker()

```
void CharacterClass.MortalClicker (
    bool revert) [private]
```

Aktywuje lub cofa efekt umiejętności Mortal Clicker.

Parametry

<i>revert</i>	True – cofa efekt skilla False – aktywuje efekt skilla
---------------	---

6.7.2.34 Multitasking()

```
void CharacterClass.Multitasking (
    bool revert) [private]
```

Aktywuje lub cofa efekt umiejętności Multitasking.

Parametry

<i>revert</i>	True – cofa efekt skilla False – aktywuje efekt skilla
---------------	---

6.7.2.35 NewLevelCap()

```
void CharacterClass.NewLevelCap () [private]
```

Oblicza nowy limit doświadczenia po awansie.

6.7.2.36 NoMatterWhat()

```
void CharacterClass.NoMatterWhat (
    bool revert) [private]
```

Aktywuje lub cofa efekt umiejętności No Matter What.

Parametry

<i>revert</i>	True – cofa efekt skilla False – aktywuje efekt skilla
---------------	---

6.7.2.37 OneForEveryone()

```
void CharacterClass.OneForEveryone (
    bool revert) [private]
```

Aktywuje lub cofa efekt umiejętności OneForEveryone.

Parametry

<i>revert</i>	True – cofa efekt skilla False – aktywuje efekt skilla
---------------	---

6.7.2.38 PassiveAgressive()

```
void CharacterClass.PassiveAgressive (  
    bool revert) [private]
```

Aktywuje lub cofa efekt umiejętności Passive Agressive.

Parametry

<i>revert</i>	True – cofa efekt skilla False – aktywuje efekt skilla
---------------	---

6.7.2.39 Pedator()

```
void CharacterClass.Pedator (  
    bool reset)
```

Aktywuje lub cofa efekt umiejętności Predator.

Parametry

<i>reset</i>	True – cofa efekt skilla False – aktywuje efekt skilla
--------------	---

6.7.2.40 PushToTheLimit()

```
void CharacterClass.PushToTheLimit (  
    bool revert) [private]
```

Aktywuje lub cofa efekt umiejętności PushToTheLimit.

Parametry

<i>revert</i>	True – cofa efekt skilla False – aktywuje efekt skilla
---------------	---

6.7.2.41 ReactionTest()

```
void CharacterClass.ReactionTest (  
    bool revert) [private]
```

Aktywuje lub cofa efekt umiejętności Reaction Test.

Parametry

<i>revert</i>	True – cofa efekt skilla False – aktywuje efekt skilla
---------------	---

6.7.2.42 ResetCharacter()

```
void CharacterClass.ResetCharacter ()
```

Resetuje postać do stanu początkowego.

6.7.2.43 SerializeToStream()

```
void CharacterClass.SerializeToStream (  
    BinaryWriter writer)
```

Serializuje dane postaci do strumienia binarnego.

Implementuje [IBinarySaveable](#).

6.7.2.44 SetLevel()

```
void CharacterClass.SetLevel (  
    uint _level)
```

Ustawia poziom postaci ręcznie.

Parametry

<code>_level</code>	Docelowy poziom
---------------------	-----------------

6.7.2.45 Shark()

```
void CharacterClass.Shark (  
    bool revert) [private]
```

Aktywuje lub cofa efekt umiejętności Shark.

Parametry

<code>revert</code>	True – cofa efekt skilla False – aktywuje efekt skilla
---------------------	---

6.7.2.46 SkillBasedClicking()

```
void CharacterClass.SkillBasedClicking (  
    bool revert) [private]
```

Aktywuje lub cofa efekt umiejętności Skill Based Clicking.

Parametry

<i>revert</i>	True – cofa efekt skilla False – aktywuje efekt skilla
---------------	---

6.7.2.47 Start()

```
void CharacterClass.Start () [private]
```

Inicjalizacja podstawowych statystyk postaci.

6.7.2.48 Symbiosis()

```
void CharacterClass.Symbiosis (  
    bool revert) [private]
```

Aktywuje lub cofa efekt umiejętności Symbiosis.

Parametry

<i>revert</i>	True – cofa efekt skilla False – aktywuje efekt skilla
---------------	---

6.7.2.49 UnskilledPredator()

```
void CharacterClass.UnskilledPredator (  
    bool revert) [private]
```

Aktywuje lub cofa efekt umiejętności UnskilledPredator.

Parametry

<i>revert</i>	True – cofa efekt skilla False – aktywuje efekt skilla
---------------	---

6.7.2.50 Update()

```
void CharacterClass.Update () [private]
```

Aktualizacja wyświetlania niektórych umiejętności na ekranie użytkownika.

6.7.2.51 UpdateTreeCounter()

```
void CharacterClass.UpdateTreeCounter (
    string id,
    bool revert) [private]
```

Aktualizuje liczniki drzew umiejętności.

6.7.2.52 WhatEyesDontSee()

```
void CharacterClass.WhatEyesDontSee (
    bool revert) [private]
```

Aktywuje lub cofa efekt umiejętności What Eyes Dont See.

Parametry

<i>revert</i>	True – cofa efekt skilla False – aktywuje efekt skilla
---------------	---

6.7.3 Dokumentacja atrybutów składowych

6.7.3.1 activeIdle

```
bool CharacterClass.activeIdle = false
```

Flaga umiejętności Active Idle.

6.7.3.2 alwaysWinner

```
bool CharacterClass.alwaysWinner = false
```

Flaga umiejętności Always Winner.

6.7.3.3 boostedChance

```
float CharacterClass.boostedChance
```

Bonus do szansy krytycznej.

6.7.3.4 buffManager

```
BuffManager CharacterClass.buffManager
```

Menedżer buffów odpowiedzialny za UI.

6.7.3.5 championOfClicks

```
bool CharacterClass.championOfClicks = false
```

Flaga umiejętności Champion Of Clicks.

6.7.3.6 chickenDinner

```
bool CharacterClass.chickenDinner = false
```

Flaga umiejętności Chicken Dinner.

6.7.3.7 christmasBonus

```
bool CharacterClass.christmasBonus = false
```

Flaga umiejętności Christmas Bonus.

6.7.3.8 criticalChance

```
float CharacterClass.criticalChance [private]
```

Bazowa szansa na trafienie krytyczne.

6.7.3.9 currentExp

```
ulong CharacterClass.currentExp = 0 [private]
```

Aktualna ilość doświadczenia.

6.7.3.10 deathDose

```
bool CharacterClass.deathDose = false
```

Flaga umiejętności DeathDose.

6.7.3.11 expBoost

```
ulong CharacterClass.expBoost
```

Bonus do doświadczenia.

6.7.3.12 failToWin

```
bool CharacterClass.failToWin = false
```

Flaga umiejętności Fail To Win.

6.7.3.13 failureGrind

```
bool CharacterClass.failureGrind = false
```

Flaga umiejętności Failure Grind.

6.7.3.14 hardWorker

```
bool CharacterClass.hardWorker = false
```

Flaga umiejętności HardWorker.

6.7.3.15 hungryWolf

```
bool CharacterClass.hungryWolf = false
```

Flaga umiejętności Hungry Wolf.

6.7.3.16 komboBoost

```
ulong CharacterClass.komboBoost
```

Bonus kombo.

6.7.3.17 level

```
uint CharacterClass.level = 1 [private]
```

Aktualny poziom gracza.

6.7.3.18 marketplaceGenius

```
bool CharacterClass.marketplaceGenius = false
```

Flaga umiejętności Marketplace Genius.

6.7.3.19 maxExpCap

```
ulong CharacterClass.maxExpCap = 100 [private]
```

Maksymalna ilość doświadczenia wymagana do awansu.

6.7.3.20 maxLvlCap

```
uint CharacterClass.maxLvlCap = 20 [private]
```

Maksymalny poziom postaci.

6.7.3.21 michealScott

```
bool CharacterClass.michealScott = false
```

Flaga umiejętności Micheal Scott.

6.7.3.22 mortalClicker

```
bool CharacterClass.mortalClicker = false
```

Flaga umiejętności Mortal Clicker.

6.7.3.23 multitasking

```
bool CharacterClass.multitasking = false
```

Flaga umiejętności Multitasking.

6.7.3.24 noMatterWhat

```
bool CharacterClass.noMatterWhat = false
```

Flaga umiejętności No Matter What.

6.7.3.25 oneForEveryone

```
bool CharacterClass.oneForEveryone = false
```

Flaga umiejętności One For Everyone.

6.7.3.26 passiveAgressive

```
bool CharacterClass.passiveAgressive = false
```

Flaga umiejętności Passive Agressive.

6.7.3.27 potionBoost

```
double CharacterClass.potionBoost
```

Bonus do mikstur.

6.7.3.28 potionRandomChance

```
double CharacterClass.potionRandomChance
```

Szansa losowa miksture.

6.7.3.29 productionIdleAgressiveBonus

```
double CharacterClass.productionIdleAgressiveBonus
```

Bonus do produkcji idle umiejętności Passive Agressive.

6.7.3.30 productionIdleBonus

```
double CharacterClass.productionIdleBonus
```

Bonus do produkcji idle.

6.7.3.31 productionIdlePedatorBonus

```
double CharacterClass.productionIdlePedatorBonus
```

Bonus do produkcji idle umiejętności Predator.

6.7.3.32 Raptorcore

```
RaptorCore CharacterClass.Raptorcore
```

Referencja do rdzenia gry.

6.7.3.33 reactionTest

```
bool CharacterClass.reactionTest = false
```

Flaga umiejętności Reaction Test.

6.7.3.34 sellingBonus

```
double CharacterClass.sellingBonus
```

Bonus do sprzedaży.

6.7.3.35 sellingHardBonus

```
double CharacterClass.sellingHardBonus
```

Bonus do sprzedaży wykorzystywany przy umiejętności Marketplace Genius.

6.7.3.36 skillCheckChance

```
float CharacterClass.skillCheckChance [private]
```

Szansa na wywołanie "SkillChecka".

6.7.3.37 skillCheckReduce

```
int CharacterClass.skillCheckReduce
```

Redukcja wywołania "SkillChecka".

6.7.3.38 skillpoints

```
uint CharacterClass.skillpoints = 0
```

Aktualna liczba punktów umiejętności.

6.7.3.39 skillPointsLimit

```
SkillPointsLimit CharacterClass.skillPointsLimit [private]
```

Ograniczenie punktów umiejętności.

6.7.3.40 symbiosis

```
bool CharacterClass.symbiosis = false
```

Flaga umiejętności Symbiosis.

6.7.3.41 T1

```
int CharacterClass.T1 = 0
```

Licznik umiejętności drzewa 1.

6.7.3.42 T2

```
int CharacterClass.T2 = 0
```

Licznik umiejętności drzewa 2.

6.7.3.43 T3

```
int CharacterClass.T3 = 0
```

Licznik umiejętności drzewa 3.

6.7.3.44 unskilledPredator

```
bool CharacterClass.unskilledPredator = false
```

Flaga umiejętności Unskilled Predator.

6.7.3.45 whatEyesDontSee

```
bool CharacterClass.whatEyesDontSee = false
```

Flaga umiejętności What Eyes Dont See.

6.7.4 Dokumentacja właściwości

6.7.4.1 SaveKey

```
int CharacterClass.SaveKey [get]
```

Klucz zapisu używany w systemie zapisu stanu gry.

Implementuje [IBinarySaveable](#).

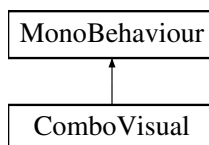
Dokumentacja dla tej klasy została wygenerowana z pliku:

- [CharacterClass.cs](#)

6.8 Dokumentacja klasy ComboVisual

Odpowiada za wizualne wyświetlanie aktualnego mnożnika combo.

Diagram dziedziczenia dla ComboVisual



Metody publiczne

- void [UpdateCombo](#) (int acKombo)
Aktualizuje wyświetlaną wartość combo.

Metody prywatne

- void [Start](#) ()
Inicjalizuje elementy interfejsu użytkownika oraz ich stan początkowy.

Atrybuty prywatne

- VisualElement `ui`
Główny element UI dokumentu.
- VisualElement `comboContainer`
Kontener wyświetlający combo.
- Label `comboLabel`
Etykieta wyświetlająca wartość combo.
- bool `isVisible` = false
Informuje, czy element combo jest aktualnie widoczny.

6.8.1 Opis szczegółowy

Odpowiada za wizualne wyświetlanie aktualnego mnożnika combo.

Klasa steruje widocznością, skalą oraz przezroczystością elementu UI reprezentującego combo.

6.8.2 Dokumentacja funkcji składowych

6.8.2.1 Start()

```
void ComboVisual.Start () [private]
```

Inicjalizuje elementy interfejsu użytkownika oraz ich stan początkowy.

6.8.2.2 UpdateCombo()

```
void ComboVisual.UpdateCombo (  
    int acKombo)
```

Aktualizuje wyświetlaną wartość combo.

Parametry

<code>acKombo</code>	Aktualna wartość mnożnika combo
----------------------	---------------------------------

6.8.3 Dokumentacja atrybutów składowych

6.8.3.1 comboContainer

```
VisualElement ComboVisual.comboContainer [private]
```

Kontener wyświetlający combo.

6.8.3.2 comboLabel

Label ComboVisual.comboLabel [private]

Etykieta wyświetlająca wartość combo.

6.8.3.3 isVisible

bool ComboVisual.isVisible = false [private]

Informuje, czy element combo jest aktualnie widoczny.

6.8.3.4 ui

VisualElement ComboVisual.ui [private]

Główny element UI dokumentu.

Dokumentacja dla tej klasy została wygenerowana z pliku:

- [ComboVisual.cs](#)

6.9 Dokumentacja klasy ContentPageManager

Zarządza przełączaniem stron zawartości w interfejsie użytkownika.

Metody publiczne

- [ContentPageManager](#) (VisualElement root, AudioSource audio)
Tworzy menedżera stron interfejsu gry.
- void [Initialize](#) ()
Inicjalizuje przyciski nawigacyjne oraz ich odpowiadające strony zawartości.

Metody prywatne

- void [ShowPage](#) (int pageNumber)
Wyświetla wybraną zakładkę interfejsu gry.

Atrybuty prywatne

- VisualElement [ui](#)
Główny element UI, z którego pobierane są komponenty.
- AudioSource [audioSource](#)
Źródło dźwięku odtwarzane przy zmianie strony.
- Button [btn1](#)
Przycisk zakładki Sklep.
- Button [btn2](#)
Przycisk zakładki Fabryki.
- Button [btn3](#)
Przycisk drzewka rozwoju One Click Army.
- Button [btn4](#)
Przycisk drzewka rozwoju Jack of All Click.
- Button [btn5](#)
Przycisk drzewka rozwoju Automatron.
- Button [btn6](#)
Przycisk zakładki Mikstury (potki).
- Button [btn7](#)
Przycisk zakładki Prestiżu.
- VisualElement [contentPage1](#)
Zawartość zakładki Sklep.
- VisualElement [contentPage2](#)
Zawartość zakładki Fabryki.
- VisualElement [contentPage3](#)
Zawartość drzewka One Click Army.
- VisualElement [contentPage4](#)
Zawartość drzewka Jack of All Click.
- VisualElement [contentPage5](#)
Zawartość drzewka Automatron.
- VisualElement [contentPage6](#)
Zawartość zakładki Mikstury.
- VisualElement [contentPage7](#)
Zawartość zakładki Prestiżu.

6.9.1 Opis szczegółowy

Zarządza przełączaniem stron zawartości w interfejsie użytkownika.

Klasa obsługuje przyciski nawigacyjne, wyświetlanie odpowiednich sekcji UI oraz odtwarzanie dźwięku przy zmianie strony.

6.9.2 Dokumentacja konstruktora i destruktor

6.9.2.1 ContentPageManager()

```
ContentPageManager.ContentPageManager (  
    VisualElement root,  
    AudioSource audio)
```

Tworzy menedżera stron interfejsu gry.

Parametry

<i>root</i>	Główny element UI
<i>audio</i>	Źródło dźwięku przy zmianie zakładki

6.9.3 Dokumentacja funkcji składowych

6.9.3.1 Initialize()

```
void ContentPageManager.Initialize ()
```

Inicjalizuje przyciski nawigacyjne oraz ich odpowiadające strony zawartości.

Metoda przypisuje zdarzenia kliknięcia oraz łączy przyciski z odpowiednimi zakładkami gry.

6.9.3.2 ShowPage()

```
void ContentPageManager.ShowPage (  
    int pageNumber) [private]
```

Wyświetla wybraną zakładkę interfejsu gry.

Parametry

<i>pageNumber</i>	Numer strony odpowiadający konkretnej zakładce (sklep, fabryki, drzewka, prestiż).
-------------------	--

6.9.4 Dokumentacja atrybutów składowych

6.9.4.1 audioSource

```
AudioSource ContentPageManager.audioSource [private]
```

Źródło dźwięku odtwarzane przy zmianie strony.

6.9.4.2 btn1

```
Button ContentPageManager.btn1 [private]
```

Przycisk zakładki Sklep.

6.9.4.3 btn2

Button ContentPageManager.btn2 [private]

Przycisk zakładki Fabryki.

6.9.4.4 btn3

Button ContentPageManager.btn3 [private]

Przycisk drzewka rozwoju One Click Army.

6.9.4.5 btn4

Button ContentPageManager.btn4 [private]

Przycisk drzewka rozwoju Jack of All Click.

6.9.4.6 btn5

Button ContentPageManager.btn5 [private]

Przycisk drzewka rozwoju Automatron.

6.9.4.7 btn6

Button ContentPageManager.btn6 [private]

Przycisk zakładki Mikstury (potki).

6.9.4.8 btn7

Button ContentPageManager.btn7 [private]

Przycisk zakładki Prestiżu.

6.9.4.9 contentPage1

VisualElement ContentPageManager.contentPage1 [private]

Zawartość zakładki Sklep.

6.9.4.10 contentPage2

VisualElement ContentPageManager.contentPage2 [private]

Zawartość zakładki Fabryki.

6.9.4.11 contentPage3

VisualElement ContentPageManager.contentPage3 [private]

Zawartość drzewka One Click Army.

6.9.4.12 contentPage4

VisualElement ContentPageManager.contentPage4 [private]

Zawartość drzewka Jack of All Click.

6.9.4.13 contentPage5

VisualElement ContentPageManager.contentPage5 [private]

Zawartość drzewka Automatron.

6.9.4.14 contentPage6

VisualElement ContentPageManager.contentPage6 [private]

Zawartość zakładki Mikstury.

6.9.4.15 contentPage7

VisualElement ContentPageManager.contentPage7 [private]

Zawartość zakładki Prestiżu.

6.9.4.16 ui

VisualElement ContentPageManager.ui [private]

Główny element UI, z którego pobierane są komponenty.

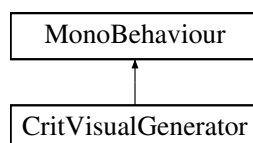
Dokumentacja dla tej klasy została wygenerowana z pliku:

- [ContentPageManager.cs](#)

6.10 Dokumentacja klasy CritVisualGenerator

Generator wizualizacji efektu krytycznego trafienia.

Diagram dziedziczenia dla CritVisualGenerator



Metody publiczne

- void [SpawnCrit](#) (Vector2 mouseScreenPos)
Tworzy wizualny napis "CRITICAL!" w pozycji kursora myszy.

Metody prywatne

- void [Start](#) ()
Inicjalizuje referencje do elementów UI.

Atrybuty prywatne

- VisualElement [ui](#)
Główny element UI dokumentu.
- VisualElement [critContainer](#)
Kontener, do którego dodawane są etykiety krytycznych trafień.

6.10.1 Opis szczegółowy

Generator wizualizacji efektu krytycznego trafienia.

Tworzy dynamiczne napisy "CRITICAL!" w miejscu kliknięcia myszy. Napisy znikają po krótkiej animacji zanikania.

6.10.2 Dokumentacja funkcji składowych

6.10.2.1 SpawnCrit()

```
void CritVisualGenerator.SpawnCrit (  
    Vector2 mouseScreenPos)
```

Tworzy wizualny napis "CRITICAL!" w pozycji kursora myszy.

Parametry

<code>mouseScreenPos</code>	Pozycja kursora w pikselach ekranu (Screen Space).
-----------------------------	--

6.10.2.2 Start()

```
void CritVisualGenerator.Start () [private]
```

Inicjalizuje referencje do elementów UI.

6.10.3 Dokumentacja atrybutów składowych

6.10.3.1 critContainer

`VisualElement CritVisualGenerator.critContainer [private]`

Kontener, do którego dodawane są etykiety krytycznych trafień.

6.10.3.2 ui

`VisualElement CritVisualGenerator.ui [private]`

Główny element UI dokumentu.

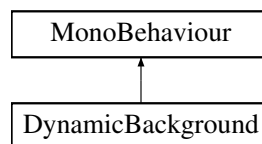
Dokumentacja dla tej klasy została wygenerowana z pliku:

- [CritVisualGenerator.cs](#)

6.11 Dokumentacja klasy DynamicBackground

Dynamicznie zmienia tło UI z efektami fade i opcjonalnym zoomem.

Diagram dziedziczenia dla DynamicBackground



Metody prywatne

- void [Start](#) ()
Inicjalizuje komponenty `RawImage` i `RectTransform` oraz uruchamia enumerator.
- IEnumerator [ChangeBackgroundRoutine](#) ()
Enumerator zmieniający tło co określony interwał czasu.
- IEnumerator [ChangeBackground](#) (Texture2D backgroundImage)
Enumerator wykonujący efekt fade przy zmianie tła.
- IEnumerator [ZoomEffect](#) ()
Enumerator wykonujący efekt powiększania (zoom) tła.

Atrybuty prywatne

- Texture2D[] `backgrounds`
Lista tekstur do wyświetlenia jako tła.
- float `changeInterval` = 5f
Czas w sekundach pomiędzy zmianami tła.
- float `fadeDuration` = 1f
Czas trwania efektu fade przy zmianie tła.
- bool `enableZoom` = true
Włącza lub wyłącza efekt powiększania tła.
- float `zoomValue` = 0.03f
Prędkość powiększania tła.
- float `maxZoomValue` = 3f
Maksymalna wartość skali dla efektu zoom.
- RawImage `rawImage`
Komponent RawImage, do którego przypisywane jest tło.
- RectTransform `rectTransform`
Transformacja tła do skalowania (zoom).
- int `currentIndex`
Indeks aktualnie wyświetlanej tekstury w tablicy backgrounds.

6.11.1 Opis szczegółowy

Dynamicznie zmienia tło UI z efektami fade i opcjonalnym zoomem.

Klasa zmienia obrazy tła w określonym interwale czasowym. Obsługuje efekt płynnego zanikania (fade) oraz opcjonalny efekt powiększania (zoom).

6.11.2 Dokumentacja funkcji składowych

6.11.2.1 ChangeBackground()

```
IEnumerator DynamicBackground.ChangeBackground (
    Texture2D backgroundImage) [private]
```

Enumerator wykonujący efekt fade przy zmianie tła.

Parametry

<code>backgroundImage</code>	Nowa tekstura tła do wyświetlenia.
------------------------------	------------------------------------

6.11.2.2 ChangeBackgroundRoutine()

```
IEnumerator DynamicBackground.ChangeBackgroundRoutine () [private]
```

Enumerator zmieniający tło co określony interwał czasu.

6.11.2.3 Start()

```
void DynamicBackground.Start () [private]
```

Inicjalizuje komponenty RawImage i RectTransform oraz uruchamia enumerator.

6.11.2.4 ZoomEffect()

```
IEnumerator DynamicBackground.ZoomEffect () [private]
```

Enumerator wykonujący efekt powiększania (zoom) tła.

6.11.3 Dokumentacja atrybutów składowych

6.11.3.1 backgrounds

```
Texture2D [] DynamicBackground.backgrounds [private]
```

Lista tekstur do wyświetlenia jako tło.

6.11.3.2 changeInterval

```
float DynamicBackground.changeInterval = 5f [private]
```

Czas w sekundach pomiędzy zmianami tła.

6.11.3.3 currentIndex

```
int DynamicBackground.currentIndex [private]
```

Indeks aktualnie wyświetlanej tekstury w tablicy backgrounds.

6.11.3.4 enableZoom

```
bool DynamicBackground.enableZoom = true [private]
```

Włącza lub wyłącza efekt powiększania tła.

6.11.3.5 fadeDuration

```
float DynamicBackground.fadeDuration = 1f [private]
```

Czas trwania efektu fade przy zmianie tła.

6.11.3.6 maxZoomValue

```
float DynamicBackground.maxZoomValue = 3f [private]
```

Maksymalna wartość skali dla efektu zoom.

6.11.3.7 rawImage

```
RawImage DynamicBackground.rawImage [private]
```

Komponent RawImage, do którego przypisywane jest tło.

6.11.3.8 rectTransform

```
RectTransform DynamicBackground.rectTransform [private]
```

Transformacja tła do skalowania (zoom).

6.11.3.9 zoomValue

```
float DynamicBackground.zoomValue = 0.03f [private]
```

Prędkość powiększania tła.

Dokumentacja dla tej klasy została wygenerowana z pliku:

- [DynamicBackground.cs](#)

6.12 Dokumentacja klasy Factory

Reprezentuje fabrykę produkującą określony zasób w grze.

Metody publiczne

- double [costOverEstimate](#) (int amount)
Oblicza przybliżony koszt zakupu określonej liczby fabryk.
- double [GetProduction](#) ()
Oblicza aktualną produkcję wszystkich fabryk tego typu.
- double [GetSellValue](#) ()
Oblicza wartość sprzedaży jednej fabryki.
- [Factory](#) ()
Konstruktor domyślny (bezparametrowy).
- [Factory](#) (Resource _resource, string _name, double _baseCost, double _costMultiplier, double _baseProduction, double _unlockCost)
Konstruktor tworzący fabrykę z podstawowymi parametrami.
- void [SerializeToStream](#) (BinaryWriter writer)
Serializuje dane fabryki do binarnego strumienia.
- void [DeserializeFromStream](#) (BinaryReader reader)
Deserializuje dane fabryki ze strumienia binarnego.

Atrybuty publiczne

- string `name`
Nazwa fabryki.
- Resource `resource`
Zasób produkowany przez fabrykę.
- double `baseCost`
Podstawowy koszt zakupu pierwszej fabryki.
- double `costMultiplier`
Mnożnik kosztu dla kolejnych fabryk.
- double `baseProduction`
Podstawowa produkcja jednej fabryki.
- double `productionMultiplier`
Mnożnik produkcji fabryki.
- bool `isUnlocked`
Informacja, czy fabryka została odblokowana.
- double `unlockCost`
Koszt odblokowania fabryki.
- ulong `count`
Liczba fabryk posiadanych przez gracza.

Właściwości

- double `currentCost` [get]
Oblicza aktualny koszt zakupu kolejnej fabryki tego typu.

6.12.1 Opis szczegółowy

Reprezentuje fabrykę produkującą określony zasób w grze.

Klasa zawiera informacje o koszcie, mnożniku produkcji, liczbie posiadanych fabryk, odblokowaniu oraz metodach obliczania produkcji i wartości sprzedaży. Obsługuje także serializację i deserializację do strumienia binarnego.

6.12.2 Dokumentacja konstruktora i destruktora

6.12.2.1 `Factory()` [1/2]

`Factory.Factory ()`

Konstruktor domyślny (bezparametrowy).

6.12.2.2 Factory() [2/2]

```
Factory.Factory (
    Resource _resource,
    string _name,
    double _baseCost,
    double _costMultiplier,
    double _baseProduction,
    double _unlockCost)
```

Konstruktor tworzący fabrykę z podstawowymi parametrami.

Parametry

<code>_resource</code>	Zasób produkowany przez fabrykę.
<code>_name</code>	Nazwa fabryki.
<code>_baseCost</code>	Podstawowy koszt zakupu.
<code>_costMultiplier</code>	Mnożnik kosztu dla kolejnych fabryk.
<code>_baseProduction</code>	Podstawowa produkcja fabryki.
<code>_unlockCost</code>	Koszt odblokowania fabryki.

6.12.3 Dokumentacja funkcji składowych

6.12.3.1 costOverEstimate()

```
double Factory.costOverEstimate (
    int amount)
```

Oblicza przybliżony koszt zakupu określonej liczby fabryk.

Parametry

<code>amount</code>	Liczba fabryk, które chcemy kupić.
---------------------	------------------------------------

Zwraca

Łączny koszt zakupu wskazanej liczby fabryk.

6.12.3.2 DeserializeFromStream()

```
void Factory.DeserializeFromStream (
    BinaryReader reader)
```

Deserializuje dane fabryki ze strumienia binarnego.

Parametry

<i>reader</i>	BinaryReader do odczytu danych.
---------------	---------------------------------

6.12.3.3 GetProduction()

```
double Factory.GetProduction ()
```

Oblicza aktualną produkcję wszystkich fabryk tego typu.

Zwraca

Całkowita produkcja z tej fabryki.

6.12.3.4 GetSellValue()

```
double Factory.GetSellValue ()
```

Oblicza wartość sprzedaży jednej fabryki.

Zwraca

Wartość w grze, którą gracz otrzyma przy sprzedaży fabryki.

6.12.3.5 SerializeToStream()

```
void Factory.SerializeToStream (  
    BinaryWriter writer)
```

Serializuje dane fabryki do binarnego strumienia.

Parametry

<i>writer</i>	BinaryWriter do zapisu danych.
---------------	--------------------------------

6.12.4 Dokumentacja atrybutów składowych**6.12.4.1 baseCost**

```
double Factory.baseCost
```

Podstawowy koszt zakupu pierwszej fabryki.

6.12.4.2 baseProduction

`double Factory.baseProduction`

Podstawowa produkcja jednej fabryki.

6.12.4.3 costMultiplier

`double Factory.costMultiplier`

Mnożnik kosztu dla kolejnych fabryk.

6.12.4.4 count

`ulong Factory.count`

Liczba fabryk posiadanych przez gracza.

6.12.4.5 isUnlocked

`bool Factory.isUnlocked`

Informacja, czy fabryka została odblokowana.

6.12.4.6 name

`string Factory.name`

Nazwa fabryki.

6.12.4.7 productionMultiplier

`double Factory.productionMultiplier`

Mnożnik produkcji fabryki.

6.12.4.8 resource

`Resource Factory.resource`

Zasób produkowany przez fabrykę.

6.12.4.9 unlockCost

`double Factory.unlockCost`

Koszt odblokowania fabryki.

6.12.5 Dokumentacja właściwości

6.12.5.1 currentCost

`double Factory.currentCost [get]`

Oblicza aktualny koszt zakupu kolejnej fabryki tego typu.

Dokumentacja dla tej klasy została wygenerowana z pliku:

- [Factory.cs](#)

6.13 Dokumentacja klasy FactoryUIManager.FactoryUI

Przechowuje referencje do elementów UI jednej fabryki.

Atrybuty publiczne

- Button [toggleButton](#)
Przycisk do rozwijania szczegółów fabryki.
- VisualElement [detailsPanel](#)
Panel zawierający szczegóły fabryki.
- Label [nameLabel](#)
Label z nazwą fabryki.
- Label [countLabel](#)
Label z ilością posiadanych fabryk.
- Label [costLabel](#)
Label z kosztem fabryki.
- Button [unlockButton](#)
Przycisk odblokowania fabryki.
- Button[] [buyButtons](#) = new Button[4]
Przyciski kupna fabryk w różnych ilościach.
- Button[] [sellButtons](#) = new Button[4]
Przyciski sprzedaży fabryk w różnych ilościach.
- bool [isVisible](#) = false
Określa czy panel szczegółów fabryki jest widoczny.
- Button [upgradeMainBtn](#)
Główny przycisk otwierania panelu ulepszeń.
- VisualElement [upgradePanel](#)
Panel zawierający ulepszenia fabryki.
- Button [upgradeProductionBtn](#)
Przycisk do ulepszania produkcji.
- Button [upgradeMultiplierBtn](#)
Przycisk do ulepszania mnożnika produkcji.
- Label [productionLevelLabel](#)
Label pokazujący poziom ulepszenia produkcji.
- Label [multiplierLevelLabel](#)
Label pokazujący poziom ulepszenia mnożnika produkcji.
- bool [upgradesVisible](#) = false
Określa, czy panel ulepszeń jest widoczny.

6.13.1 Opis szczegółowy

Przechowuje referencje do elementów UI jednej fabryki.

6.13.2 Dokumentacja atrybutów składowych

6.13.2.1 `buyButtons`

```
Button [] FactoryUIManager.FactoryUI.buyButtons = new Button[4]
```

Przyciski kupna fabryk w różnych ilościach.

6.13.2.2 `costLabel`

```
Label FactoryUIManager.FactoryUI.costLabel
```

Label z kosztem fabryki.

6.13.2.3 `countLabel`

```
Label FactoryUIManager.FactoryUI.countLabel
```

Label z ilością posiadanych fabryk.

6.13.2.4 `detailsPanel`

```
VisualElement FactoryUIManager.FactoryUI.detailsPanel
```

Panel zawierający szczegóły fabryki.

6.13.2.5 `isVisible`

```
bool FactoryUIManager.FactoryUI.isVisible = false
```

Określa czy panel szczegółów fabryki jest widoczny.

6.13.2.6 `multiplierLevelLabel`

```
Label FactoryUIManager.FactoryUI.multiplierLevelLabel
```

Label pokazujący poziom ulepszenia mnożnika produkcji.

6.13.2.7 `nameLabel`

```
Label FactoryUIManager.FactoryUI.nameLabel
```

Label z nazwą fabryki.

6.13.2.8 productionLevelLabel

`Label FactoryUIManager.FactoryUI.productionLevelLabel`

Label pokazujący poziom ulepszenia produkcji.

6.13.2.9 sellButtons

`Button [] FactoryUIManager.FactoryUI.sellButtons = new Button[4]`

Przyciski sprzedaży fabryk w różnych ilościach.

6.13.2.10 toggleButton

`Button FactoryUIManager.FactoryUI.toggleButton`

Przycisk do rozwijania szczegółów fabryki.

6.13.2.11 unlockButton

`Button FactoryUIManager.FactoryUI.unlockButton`

Przycisk odblokowania fabryki.

6.13.2.12 upgradeMainBtn

`Button FactoryUIManager.FactoryUI.upgradeMainBtn`

Główny przycisk otwierania panelu ulepszeń.

6.13.2.13 upgradeMultiplierBtn

`Button FactoryUIManager.FactoryUI.upgradeMultiplierBtn`

Przycisk do ulepszania mnożnika produkcji.

6.13.2.14 upgradePanel

`VisualElement FactoryUIManager.FactoryUI.upgradePanel`

Panel zawierający ulepszenia fabryki.

6.13.2.15 upgradeProductionBtn

`Button FactoryUIManager.FactoryUI.upgradeProductionBtn`

Przycisk do ulepszania produkcji.

6.13.2.16 upgradesVisible

```
bool FactoryUIManager.FactoryUI.upgradesVisible = false
```

Określa, czy panel ulepszeń jest widoczny.

Dokumentacja dla tej klasy została wygenerowana z pliku:

- [FactoryUIManager.cs](#)

6.14 Dokumentacja klasy FactoryUIManager

Zarządza interfejsem użytkownika fabryk w grze.

Komponenty

- class [FactoryUI](#)
Przechowuje referencje do elementów UI jednej fabryki.
- class [FactoryUpgradeLevels](#)
Przechowuje poziomy ulepszeń fabryki.

Metody publiczne

- [FactoryUIManager](#) (VisualElement root, [RaptorCore](#) core, [IdleManager](#) manager, AudioSource audio, SpriteRenderer clicker, Sprite[] factoryImages)
Konstruktor [FactoryUIManager](#).
- void [Initialize](#) ()
Inicjalizuje wszystkie elementy UI, przyciski fabryk i odblokowania.
- void [Update](#) ()
Aktualizacja interfejsu użytkownika w każdej klatce.
- void [UpdateFactoryUI](#) ()
Aktualizuje UI fabryk według stanu gry.
- void [SerializeToStream](#) (BinaryWriter writer)
Serializuje poziomy ulepszeń fabryk do strumienia binarnego.
- void [DeserializeFromStream](#) (BinaryReader reader)
Deserializuje poziomy ulepszeń fabryk ze strumienia binarnego.
- void [ResetFactoryGUI](#) ()
Resetuje UI fabryk i poziomy ulepszeń.

Metody prywatne

- void [InitializeUnlockButtons](#) ()
Inicjalizuje przyciski odblokowania fabryk.
- void [InitializeFactories](#) ()
- void [ToggleFactoryDetails](#) (int factoryIndex)
Włącza lub wyłącza panel szczegółów fabryki.
- void [BuyFactory](#) (int factoryIndex, int amount)
Kupuje określoną ilość fabryk.
- void [BuyFactoryMax](#) (int factoryIndex)
Kupuje maksymalną ilość fabryk możliwą do zakupu przy aktualnym złocie gracza.
- void [SellFactory](#) (int factoryIndex, int amount)
Sprzedaje określoną ilość fabryk.
- void [SellFactoryMax](#) (int factoryIndex)
Sprzedaje maksymalną ilość fabryk.
- void [UpdateButton](#) (Button button, bool enabled)
Odsiewez przycisk, ustawiając jego dostępność.
- [Factory GetFactory](#) (int index)
Pobiera fabrykę z [IdleManager](#).
- void [UnlockFactory](#) (int factoryIndex)
Odblokowuje fabrykę o podanym indeksie przy użyciu [IdleManagera](#). Aktualizuje UI i wyświetla komunikat w konsoli.
- double [CalculateUpgradeCost](#) (double baseCost, int currentLevel)
Oblicza koszt ulepszenia na podstawie kosztu bazowego i aktualnego poziomu.
- void [UpgradeFactoryBaseProduction](#) (int factoryIndex)
Zwiększa bazową produkcję fabryki o 1, jeśli gracz ma wystarczająco złota. Aktualizuje poziom ulepszenia i UI.
- void [UpgradeFactoryMultiplier](#) (int factoryIndex)
Zwiększa mnożnik produkcji fabryki o 0.1, jeśli gracz ma wystarczająco złota. Aktualizuje poziom ulepszenia i UI.
- void [CycleToPreviousFactory](#) ()
Przełącza widoczność poprzedniej odblokowanej fabryki w interfejsie użytkownika.
- void [CycleToNextFactory](#) ()
Przełącza widoczność następnej odblokowanej fabryki w interfejsie użytkownika.
- bool [IsFactoryUnlocked](#) (int factoryIndex)
Sprawdza, czy fabryka o podanym indeksie jest odblokowana.
- void [SwitchToFactory](#) (int factoryIndex)
Przełącza interfejs szczegółów fabryki (pokazuje/ukrywa panel szczegółów).
- void [UpdateNavigationButtons](#) ()
Aktualizuje widoczność i dostępność przycisków nawigacji między fabrykami.
- void [UpdateUnlockButtons](#) ()
Aktualizuje widoczność przycisków odblokowania fabryk na ekranie.
- void [UpdateUnlockButton](#) (Button button, int factoryIndex)
Aktualizuje pojedynczy przycisk odblokowania fabryki (widoczność i czy jest aktywny).
- void [UnlockFactoryFromButton](#) (int factoryIndex)
Obsługuje kliknięcie przycisku odblokowania fabryki. Sprawdza czy gracz ma wystarczająco złota i odblokowuje fabrykę.

Atrybuty prywatne

- VisualElement [ui](#)
Główny root UI dla wszystkich fabryk.
- ScrollView [scrollView](#)
ScrollView zawierający wszystkie elementy fabryk.
- List< [FactoryUI](#) > [factoryUIs](#) = new List<[FactoryUI](#)>()
Lista obiektów UI reprezentujących fabryki.
- Dictionary< int, [FactoryUpgradeLevels](#) > [factoryUpgradeLevels](#) = new Dictionary<int, [FactoryUpgradeLevels](#)>()
Słownik poziomów ulepszeń fabryk według indeksu fabryki.
- int [numberOfFactories](#) = 6
Liczba fabryk w grze.
- int [currentFactoryIndex](#) = 0
Indeks aktualnie wybranej fabryki.
- [RaptorCore](#) [raptorCore](#)
Główny kontroler gry do zarządzania zasobami.
- [IdleManager](#) [idleManager](#)
Manager fabryk dla logiki idle.
- AudioSource [audioSource](#)
Źródło audio dla efektów kliknięć.
- Button [prevFactoryBtn](#)
Przycisk poprzedniej fabryki.
- Button [nextFactoryBtn](#)
Przycisk następnej fabryki.
- Label [factoryLabel](#)
Label z nazwą aktualnej fabryki.
- VisualElement [currencyIcon](#)
Ikona waluty.
- VisualElement [shopIcon](#)
Ikona sklepu.
- SpriteRenderer [clickerObject](#)
Sprite fabryki w grze.
- Sprite[] [images](#)
Tablica sprite'ów dla fabryk.
- Button [unlockFactory2Btn](#)
Przycisk odblokowania drugiej fabryki.
- Button [unlockFactory3Btn](#)
Przycisk odblokowania trzeciej fabryki.
- Button [unlockFactory4Btn](#)
Przycisk odblokowania czwartej fabryki.
- Button [unlockFactory5Btn](#)
Przycisk odblokowania piątej fabryki.
- Button [unlockFactory6Btn](#)
Przycisk odblokowania szóstej fabryki.

6.14.1 Opis szczegółowy

Zarządza interfejsem użytkownika fabryk w grze.

Obsługuje wyświetlanie fabryk, kupowanie i sprzedawanie jednostek, odblokowywanie nowych fabryk, aktualizację przycisków oraz poziomów ulepszeń. Zawiera także serializację i deserializację poziomów ulepszeń fabryk.

6.14.2 Dokumentacja konstruktora i destruktora

6.14.2.1 FactoryUIManager()

```
FactoryUIManager.FactoryUIManager (
    VisualElement root,
    RaptorCore core,
    IdleManager manager,
    AudioSource audio,
    SpriteRenderer clicker,
    Sprite[] factoryImages)
```

Konstruktor [FactoryUIManager](#).

Parametry

<i>root</i>	Główny root VisualElement UI.
<i>core</i>	RaptorCore instancja do zarządzania zasobami.
<i>manager</i>	IdleManager do obsługi fabryk.
<i>audio</i>	AudioSource do efektów kliknięć.
<i>clicker</i>	SpriteRenderer reprezentujący fabrykę w grze.
<i>factoryImages</i>	Tablica sprite'ów dla fabryk.

6.14.3 Dokumentacja funkcji składowych

6.14.3.1 BuyFactory()

```
void FactoryUIManager.BuyFactory (
    int factoryIndex,
    int amount) [private]
```

Kupuje określoną ilość fabryk.

Parametry

<i>factoryIndex</i>	Indeks fabryki
<i>amount</i>	Ilość do zakupu

6.14.3.2 BuyFactoryMax()

```
void FactoryUIManager.BuyFactoryMax (
    int factoryIndex) [private]
```

Kupuje maksymalną ilość fabryk możliwą do zakupu przy aktualnym złocie gracza.

Parametry

<i>factoryIndex</i>	Indeks fabryki
---------------------	----------------

6.14.3.3 CalculateUpgradeCost()

```
double FactoryUIManager.CalculateUpgradeCost (
    double baseCost,
    int currentLevel) [private]
```

Oblicza koszt ulepszenia na podstawie kosztu bazowego i aktualnego poziomu.

Parametry

<i>baseCost</i>	Bazowy koszt ulepszenia.
<i>currentLevel</i>	Aktualny poziom ulepszenia.

Zwraca

Zaktualizowany koszt kolejnego ulepszenia.

6.14.3.4 CycleToNextFactory()

```
void FactoryUIManager.CycleToNextFactory () [private]
```

Przełącza widoczność następnej odblokowanej fabryki w interfejsie użytkownika.

6.14.3.5 CycleToPreviousFactory()

```
void FactoryUIManager.CycleToPreviousFactory () [private]
```

Przełącza widoczność poprzedniej odblokowanej fabryki w interfejsie użytkownika.

6.14.3.6 DeserializeFromStream()

```
void FactoryUIManager.DeserializeFromStream (
    BinaryReader reader)
```

Deserializuje poziomy ulepszeń fabryk ze strumienia binarnego.

Parametry

<i>reader</i>	BinaryReader do odczytu danych.
---------------	---------------------------------

6.14.3.7 GetFactory()

```
Factory FactoryUIManager.GetFactory (  
    int index) [private]
```

Pobiera fabrykę z [IdleManager](#).

Parametry

<i>index</i>	Indeks fabryki
--------------	----------------

Zwraca

Obiekt fabryki lub null jeśli nie istnieje

6.14.3.8 Initialize()

```
void FactoryUIManager.Initialize ()
```

Inicjalizuje wszystkie elementy UI, przyciski fabryk i odblokowania.

6.14.3.9 InitializeFactories()

```
void FactoryUIManager.InitializeFactories () [private]
```

Inicjalizuje fabryki oraz ich funkcjonalność.

6.14.3.10 InitializeUnlockButtons()

```
void FactoryUIManager.InitializeUnlockButtons () [private]
```

Inicjalizuje przyciski odblokowania fabryk.

6.14.3.11 IsFactoryUnlocked()

```
bool FactoryUIManager.IsFactoryUnlocked (  
    int factoryIndex) [private]
```

Sprawdza, czy fabryka o podanym indeksie jest odblokowana.

Parametry

<i>factoryIndex</i>	Indeks fabryki do sprawdzenia.
---------------------	--------------------------------

Zwraca

True jeśli fabryka jest odblokowana, false w przeciwnym razie.

6.14.3.12 ResetFactoryGUI()

```
void FactoryUIManager.ResetFactoryGUI ()
```

Resetuje UI fabryk i poziomy ulepszeń.

6.14.3.13 SellFactory()

```
void FactoryUIManager.SellFactory (  
    int factoryIndex,  
    int amount) [private]
```

Sprzedaje określoną ilość fabryk.

Parametry

<i>factoryIndex</i>	Indeks fabryki
<i>amount</i>	Ilość do sprzedaży

6.14.3.14 SellFactoryMax()

```
void FactoryUIManager.SellFactoryMax (  
    int factoryIndex) [private]
```

Sprzedaje maksymalną ilość fabryk.

Parametry

<i>factoryIndex</i>	Indeks fabryki
---------------------	----------------

6.14.3.15 SerializeToStream()

```
void FactoryUIManager.SerializeToStream (  
    BinaryWriter writer)
```

Serializuje poziomy ulepszeń fabryk do strumienia binarnego.

Parametry

<i>writer</i>	BinaryWriter do zapisu danych.
---------------	--------------------------------

6.14.3.16 SwitchToFactory()

```
void FactoryUIManager.SwitchToFactory (  
    int factoryIndex) [private]
```

Przełącza interfejs szczegółów fabryki (pokazuje/ukrywa panel szczegółów).

Parametry

<i>factoryIndex</i>	Indeks fabryki, której szczegóły mają być przełączone.
---------------------	--

6.14.3.17 ToggleFactoryDetails()

```
void FactoryUIManager.ToggleFactoryDetails (  
    int factoryIndex) [private]
```

Włącza lub wyłącza panel szczegółów fabryki.

Parametry

<i>factoryIndex</i>	Indeks fabryki
---------------------	----------------

6.14.3.18 UnlockFactory()

```
void FactoryUIManager.UnlockFactory (  
    int factoryIndex) [private]
```

Odblokowuje fabrykę o podanym indeksie przy użyciu IdleManagera. Aktualizuje UI i wyświetla komunikat w konsoli.

Parametry

<i>factoryIndex</i>	Indeks fabryki do odblokowania.
---------------------	---------------------------------

6.14.3.19 UnlockFactoryFromButton()

```
void FactoryUIManager.UnlockFactoryFromButton (  
    int factoryIndex) [private]
```

Obsługuje kliknięcie przycisku odblokowania fabryki. Sprawdza czy gracz ma wystarczająco złota i odblokowuje fabrykę.

Parametry

<i>factoryIndex</i>	Indeks fabryki do odblokowania.
---------------------	---------------------------------

6.14.3.20 Update()

```
void FactoryUIManager.Update ()
```

Aktualizacja interfejsu użytkownika w każdej klatce.

6.14.3.21 UpdateButton()

```
void FactoryUIManager.UpdateButton (  
    Button button,  
    bool enabled) [private]
```

Odswież przycisk, ustawiając jego dostępność.

Parametry

<i>button</i>	Przycisk do aktualizacji
<i>enabled</i>	Czy przycisk ma być aktywny

6.14.3.22 UpdateFactoryUI()

```
void FactoryUIManager.UpdateFactoryUI ()
```

Aktualizuje UI fabryk według stanu gry.

6.14.3.23 UpdateNavigationButtons()

```
void FactoryUIManager.UpdateNavigationButtons () [private]
```

Aktualizuje widoczność i dostępność przycisków nawigacji między fabrykami.

6.14.3.24 UpdateUnlockButton()

```
void FactoryUIManager.UpdateUnlockButton (  
    Button button,  
    int factoryIndex) [private]
```

Aktualizuje pojedynczy przycisk odblokowania fabryki (widoczność i czy jest aktywny).

Parametry

<i>button</i>	Przycisk odblokowania do aktualizacji.
<i>factoryIndex</i>	Indeks fabryki powiązanej z tym przyciskiem.

6.14.3.25 UpdateUnlockButtons()

```
void FactoryUIManager.UpdateUnlockButtons () [private]
```

Aktualizuje widoczność przycisków odblokowania fabryk na ekranie.

6.14.3.26 UpgradeFactoryBaseProduction()

```
void FactoryUIManager.UpgradeFactoryBaseProduction (  
    int factoryIndex) [private]
```

Zwiększa bazową produkcję fabryki o 1, jeśli gracz ma wystarczająco złota. Aktualizuje poziom ulepszenia i UI.

Parametry

<i>factoryIndex</i>	Indeks fabryki do ulepszenia produkcji.
---------------------	---

6.14.3.27 UpgradeFactoryMultiplier()

```
void FactoryUIManager.UpgradeFactoryMultiplier (  
    int factoryIndex) [private]
```

Zwiększa mnożnik produkcji fabryki o 0.1, jeśli gracz ma wystarczająco złota. Aktualizuje poziom ulepszenia i UI.

Parametry

<i>factoryIndex</i>	Indeks fabryki do ulepszenia mnożnika.
---------------------	--

6.14.4 Dokumentacja atrybutów składowych

6.14.4.1 audioSource

```
AudioSource FactoryUIManager.audioSource [private]
```

Źródło audio dla efektów kliknięć.

6.14.4.2 clickerObject

```
SpriteRenderer FactoryUIManager.clickerObject [private]
```

Sprite fabryki w grze.

6.14.4.3 currencyIcon

```
VisualElement FactoryUIManager.currencyIcon [private]
```

Ikona waluty.

6.14.4.4 currentFactoryIndex

```
int FactoryUIManager.currentFactoryIndex = 0 [private]
```

Indeks aktualnie wybranej fabryki.

6.14.4.5 factoryLabel

```
Label FactoryUIManager.factoryLabel [private]
```

Label z nazwą aktualnej fabryki.

6.14.4.6 factoryUIs

```
List<FactoryUI> FactoryUIManager.factoryUIs = new List<FactoryUI>() [private]
```

Lista obiektów UI reprezentujących fabryki.

6.14.4.7 factoryUpgradeLevels

```
Dictionary<int, FactoryUpgradeLevels> FactoryUIManager.factoryUpgradeLevels = new Dictionary<int, FactoryUpgradeLevels>() [private]
```

Słownik poziomów ulepszeń fabryk według indeksu fabryki.

6.14.4.8 idleManager

```
IdleManager FactoryUIManager.idleManager [private]
```

Manager fabryk dla logiki idle.

6.14.4.9 images

```
Sprite [] FactoryUIManager.images [private]
```

Tablica sprite'ów dla fabryk.

6.14.4.10 nextFactoryBtn

```
Button FactoryUIManager.nextFactoryBtn [private]
```

Przycisk następnej fabryki.

6.14.4.11 numberOfFactories

```
int FactoryUIManager.numberOfFactories = 6 [private]
```

Liczba fabryk w grze.

6.14.4.12 prevFactoryBtn

Button FactoryUIManager.prevFactoryBtn [private]

Przycisk poprzedniej fabryki.

6.14.4.13 raptorCore

[RaptorCore](#) FactoryUIManager.raptorCore [private]

Główny kontroler gry do zarządzania zasobami.

6.14.4.14 scrollView

ScrollView FactoryUIManager.scrollView [private]

ScrollView zawierający wszystkie elementy fabryk.

6.14.4.15 shopIcon

VisualElement FactoryUIManager.shopIcon [private]

Ikona sklepu.

6.14.4.16 ui

VisualElement FactoryUIManager.ui [private]

Główny root UI dla wszystkich fabryk.

6.14.4.17 unlockFactory2Btn

Button FactoryUIManager.unlockFactory2Btn [private]

Przycisk odblokowania drugiej fabryki.

6.14.4.18 unlockFactory3Btn

Button FactoryUIManager.unlockFactory3Btn [private]

Przycisk odblokowania trzeciej fabryki.

6.14.4.19 unlockFactory4Btn

Button FactoryUIManager.unlockFactory4Btn [private]

Przycisk odblokowania czwartej fabryki.

6.14.4.20 unlockFactory5Btn

Button FactoryUIManager.unlockFactory5Btn [private]

Przycisk odblokowania piątej fabryki.

6.14.4.21 unlockFactory6Btn

Button FactoryUIManager.unlockFactory6Btn [private]

Przycisk odblokowania szóstej fabryki.

Dokumentacja dla tej klasy została wygenerowana z pliku:

- [FactoryUIManager.cs](#)

6.15 Dokumentacja klasy FactoryUIManager.FactoryUpgradeLevels

Przechowuje poziomy ulepszeń fabryki.

Metody publiczne

- void [DeserializeFromStream](#) (BinaryReader reader)
Deserializuje poziomy ulepszeń z binarnego strumienia.
- void [SerializeToStream](#) (BinaryWriter writer)
Serializuje poziomy ulepszeń do binarnego strumienia.

Atrybuty publiczne

- int [productionLevel](#) = 0
Poziom ulepszenia produkcji fabryki.
- int [multiplierLevel](#) = 0
Poziom ulepszenia mnożnika fabryki.
- double [baseProductionCost](#) = 0
Podstawowy koszt ulepszenia produkcji fabryki.
- double [baseMultiplierCost](#) = 0
Podstawowy koszt ulepszenia mnożnika fabryki.

6.15.1 Opis szczegółowy

Przechowuje poziomy ulepszeń fabryki.

6.15.2 Dokumentacja funkcji składowych

6.15.2.1 DeserializeFromStream()

```
void FactoryUIManager.FactoryUpgradeLevels.DeserializeFromStream (  
    BinaryReader reader)
```

Deserializuje poziomy ulepszeń z binarnego strumienia.

Parametry

<i>reader</i>	BinaryReader do odczytu danych.
---------------	---------------------------------

6.15.2.2 SerializeToStream()

```
void FactoryUIManager.FactoryUpgradeLevels.SerializeToStream (  
    BinaryWriter writer)
```

Serializuje poziomy ulepszeń do binarnego strumienia.

Parametry

<i>writer</i>	BinaryWriter do zapisu danych.
---------------	--------------------------------

6.15.3 Dokumentacja atrybutów składowych

6.15.3.1 baseMultiplierCost

```
double FactoryUIManager.FactoryUpgradeLevels.baseMultiplierCost = 0
```

Podstawowy koszt ulepszenia mnożnika fabryki.

6.15.3.2 baseProductionCost

```
double FactoryUIManager.FactoryUpgradeLevels.baseProductionCost = 0
```

Podstawowy koszt ulepszenia produkcji fabryki.

6.15.3.3 multiplierLevel

```
int FactoryUIManager.FactoryUpgradeLevels.multiplierLevel = 0
```

Poziom ulepszenia mnożnika fabryki.

6.15.3.4 productionLevel

```
int FactoryUIManager.FactoryUpgradeLevels.productionLevel = 0
```

Poziom ulepszenia produkcji fabryki.

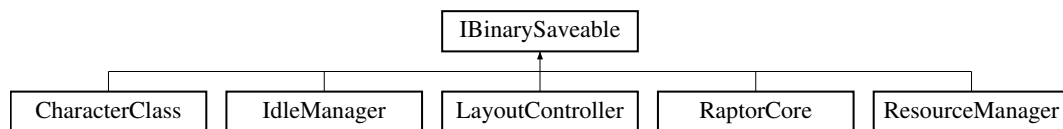
Dokumentacja dla tej klasy została wygenerowana z pliku:

- [FactoryUIManager.cs](#)

6.16 Dokumentacja interfejsu IBinarySaveable

Interfejs dla wszystkich klas, które można zapisać/odczytać w formacie binarnym.

Diagram dziedziczenia dla IBinarySaveable



Metody publiczne

- void [SerializeToStream](#) (BinaryWriter writer)
Serializuje dane obiektu do strumienia binarnego.
- void [DeserializeFromStream](#) (BinaryReader reader)
Deserializuje dane obiektu ze strumienia binarnego.

Właściwości

- int [SaveKey](#) [get]
Unikalny klucz zapisu. Używany do sortowania podczas zapisywania/ładowania.

6.16.1 Opis szczegółowy

Interfejs dla wszystkich klas, które można zapisać/odczytać w formacie binarnym.

6.16.2 Dokumentacja funkcji składowych

6.16.2.1 DeserializeFromStream()

```
void IBinarySaveable.DeserializeFromStream (
    BinaryReader reader)
```

Deserializuje dane obiektu ze strumienia binarnego.

Parametry

<i>reader</i>	Reader binarny do odczytu danych.
---------------	-----------------------------------

Implementowany w [CharacterClass](#), [IdleManager](#), [LayoutController](#), [RaptorCore](#) i [ResourceManager](#).

6.16.2.2 SerializeToStream()

```
void IBinarySaveable.SerializeToStream (
    BinaryWriter writer)
```

Serializuje dane obiektu do strumienia binarnego.

Parametry

<i>writer</i>	Writer binarny do zapisu danych.
---------------	----------------------------------

Implementowany w [CharacterClass](#), [IdleManager](#), [LayoutController](#), [RaptorCore](#) i [ResourceManager](#).

6.16.3 Dokumentacja właściwości

6.16.3.1 SaveKey

```
int IBinarySaveable.SaveKey [get]
```

Unikalny klucz zapisu. Używany do sortowania podczas zapisywania/ładowania.

Implementowany w [CharacterClass](#), [IdleManager](#), [LayoutController](#), [RaptorCore](#) i [ResourceManager](#).

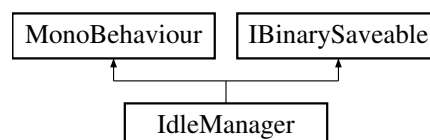
Dokumentacja dla tego interfejsu została wygenerowana z pliku:

- [SaveManager.cs](#)

6.17 Dokumentacja klasy IdleManager

Klasa odpowiedzialna za zarządzanie fabrykami, zasobami i miksturami w grze. Obsługuje ticki produkcji, kupno i sprzedaż fabryk oraz aktywację efektów mikstur. Implementuje [IBinarySaveable](#) do zapisu i wczytywania stanu gry.

Diagram dziedziczenia dla IdleManager



Metody publiczne

- bool [UnlockFactory](#) (int index)
Próbuje odblokować fabrykę o podanym indeksie.
- bool [BuyFactory](#) (int index)
Próbuje kupić jedną sztukę fabryki o podanym indeksie.
- bool [SellFactory](#) (int index)
Próbuje sprzedać jedną sztukę fabryki o podanym indeksie.
- [Factory GetFactory](#) (int index)
Zwraca fabrykę o podanym indeksie.
- int [GetFactoryCount](#) ()
Zwraca liczbę wszystkich fabryk.
- [Potion GetPotion](#) (int index)
Zwraca miksturę o podanym indeksie.
- bool [BuyPotion](#) (int index)
Próbuje kupić miksturę o podanym indeksie. Sprawdza wymagane zasoby i aktywuje efekt mikstury.
- void [EnablePotionEffect](#) ([Potion](#) potion)
Aktywuje efekt mikstury (np. bonus do klików, produkcji lub sprzedaży).
- void [DisablePotionEffect](#) ([Potion](#) potion)
Dezaktywuje efekt mikstury i resetuje wszelkie bonusy.
- void [SerializeToStream](#) (BinaryWriter writer)
Zapisuje stan fabryk do strumienia binarnego.
- void [DeserializeFromStream](#) (BinaryReader reader)
Wczytuje stan fabryk ze strumienia binarnego.

Atrybuty publiczne

- List< [Factory](#) > [factories](#)
Lista wszystkich fabryk w grze.
- List< [Resource](#) > [resources](#)
Lista wszystkich zasobów w grze.
- List< [Potion](#) > [potions](#)
Lista wszystkich mikstur w grze.
- [QuarkType](#) [potionFactoryBonus1](#) = 1
Bonus mnożnika produkcji dla mikstury 1.
- [QuarkType](#) [potionFactoryBonus2](#) = 1
Bonus mnożnika produkcji dla mikstury 2.

Funkcje pakietu

- void [ResetFactory](#) ()
Resetuje wszystkie fabryki do stanu początkowego.

Właściwości

- int [SaveKey](#) [get]
Identyfikator zapisu używany w systemie save/load.

Metody prywatne

- void [Start](#) ()
Inicjalizacja managera: odnajduje [RaptorCore](#) oraz inicjalizuje zasoby, fabryki i mikstury.
- void [InitResources](#) ()
Tworzy i rejestruje wszystkie zasoby w [ResourceManager](#).
- void [InitFactories](#) ()
Tworzy fabryki z domyślnymi parametrami i przypisuje je do listy.
- void [InitPotions](#) ()
Tworzy mikstury z domyślnymi właściwościami, czasem trwania i opisem efektu.
- void [OnEnable](#) ()
Obsługuje włączenie skryptu i uruchamia tick loop.
- void [OnDisable](#) ()
Obsługuje wyłączenie skryptu i zatrzymuje tick loop.
- IEnumerator [TickLoop](#) ()
Pętla ticków wywoływana co tickInterval sekund, przetwarza produkcję i mikstury.
- void [ProcessTick](#) ()
Przetwarza tick: aktualizuje produkcję fabryk, efekty mikstur i wyświetla zasoby w UI.
- int [GetUnlockedFactoryCount](#) (string resource)
Zwraca liczbę odblokowanych fabryk dla podanego zasobu.
- bool [GetFocusedFactory](#) (string resource)
Sprawdza, czy gracz aktualnie skupia się na fabryce danego zasobu.

Atrybuty prywatne

- [RaptorCore](#) [raptorCore](#)
Referencja do głównego skryptu gry zawierającego złoto i managerów.
- [CharacterClass](#) [characterClass](#)
Referencja do klasy postaci określająca różne bonusy i zdolności pasywne.
- float [tickInterval](#) = 1f
Interwał ticka w sekundach, po którym następuje aktualizacja produkcji fabryk.
- Coroutine [tickCoroutine](#)
Referencja do Coroutine obsługującej ticki.

6.17.1 Opis szczegółowy

Klasa odpowiedzialna za zarządzanie fabrykami, zasobami i miksturami w grze. Obsługuje ticki produkcji, kupno i sprzedaż fabryk oraz aktywację efektów mikstur. Implementuje [IBinarySaveable](#) do zapisu i wczytywania stanu gry.

6.17.2 Dokumentacja funkcji składowych

6.17.2.1 BuyFactory()

```
bool IdleManager.BuyFactory (  
    int index)
```

Próbuje kupić jedną sztukę fabryki o podanym indeksie.

Parametry

<i>index</i>	Indeks fabryki
--------------	----------------

Zwraca

True jeśli zakup udany, false jeśli brak środków lub indeks nieprawidłowy

6.17.2.2 BuyPotion()

```
bool IdleManager.BuyPotion (  
    int index)
```

Próbuje kupić miksturę o podanym indeksie. Sprawdza wymagane zasoby i aktywuje efekt mikstury.

Parametry

<i>index</i>	Indeks mikstury
--------------	-----------------

Zwraca

True jeśli zakup i aktywacja mikstury udana, false jeśli brak zasobów

6.17.2.3 DeserializeFromStream()

```
void IdleManager.DeserializeFromStream (  
    BinaryReader reader)
```

Wczytuje stan fabryk ze strumienia binarnego.

Parametry

<i>reader</i>	BinaryReader do odczytu
---------------	-------------------------

Implementuje [IBinarySaveable](#).

6.17.2.4 DisablePotionEffect()

```
void IdleManager.DisablePotionEffect (  
    Potion potion)
```

Dezaktywuje efekt mikstury i resetuje wszelkie bonusy.

Parametry

<i>potion</i>	Mikstura do dezaktywacji
---------------	--------------------------

6.17.2.5 EnablePotionEffect()

```
void IdleManager.EnablePotionEffect (  
    Potion potion)
```

Aktywuje efekt mikstury (np. bonus do klików, produkcji lub sprzedaży).

Parametry

<i>potion</i>	Mikstura do aktywacji
---------------	-----------------------

6.17.2.6 GetFactory()

```
Factory IdleManager.GetFactory (  
    int index)
```

Zwraca fabrykę o podanym indeksie.

Parametry

<i>index</i>	Indeks fabryki
--------------	----------------

Zwraca

Fabryka lub null

6.17.2.7 GetFactoryCount()

```
int IdleManager.GetFactoryCount ()
```

Zwraca liczbę wszystkich fabryk.

Zwraca

Liczba fabryk

6.17.2.8 GetFocusedFactory()

```
bool IdleManager.GetFocusedFactory (  
    string resource) [private]
```

Sprawdza, czy gracz aktualnie skupia się na fabryce danego zasobu.

Parametry

<i>resource</i>	Nazwa zasobu
-----------------	--------------

Zwraca

True jeśli fabryka jest aktualnie focusowana, false w przeciwnym wypadku

6.17.2.9 GetPotion()

```
Potion IdleManager.GetPotion (  
    int index)
```

Zwraca miksturę o podanym indeksie.

Parametry

<i>index</i>	Indeks mikstury
--------------	-----------------

Zwraca

Mikstura lub null

6.17.2.10 GetUnlockedFactoryCount()

```
int IdleManager.GetUnlockedFactoryCount (  
    string resource) [private]
```

Zwraca liczbę odblokowanych fabryk dla podanego zasobu.

Parametry

<i>resource</i>	Nazwa zasobu
-----------------	--------------

Zwraca

Liczba odblokowanych fabryk dla zasobu

6.17.2.11 InitFactories()

```
void IdleManager.InitFactories () [private]
```

Tworzy fabryki z domyślnymi parametrami i przypisuje je do listy.

6.17.2.12 InitPotions()

```
void IdleManager.InitPotions () [private]
```

Tworzy mikstury z domyślnymi właściwościami, czasem trwania i opisem efektu.

6.17.2.13 InitResources()

```
void IdleManager.InitResources () [private]
```

Tworzy i rejestruje wszystkie zasoby w [ResourceManager](#).

6.17.2.14 OnDisable()

```
void IdleManager.OnDisable () [private]
```

Obsługuje wyłączenie skryptu i zatrzymuje tick loop.

6.17.2.15 OnEnable()

```
void IdleManager.OnEnable () [private]
```

Obsługuje włączenie skryptu i uruchamia tick loop.

6.17.2.16 ProcessTick()

```
void IdleManager.ProcessTick () [private]
```

Przetwarza tick: aktualizuje produkcję fabryk, efekty mikstur i wyświetla zasoby w UI.

6.17.2.17 ResetFactory()

```
void IdleManager.ResetFactory () [package]
```

Resetuje wszystkie fabryki do stanu początkowego.

6.17.2.18 SellFactory()

```
bool IdleManager.SellFactory (  
    int index)
```

Próbuje sprzedać jedną sztukę fabryki o podanym indeksie.

Parametry

<i>index</i>	Indeks fabryki
--------------	----------------

Zwraca

True jeśli sprzedaż udana, false jeśli brak fabryk lub indeks nieprawidłowy

6.17.2.19 SerializeToStream()

```
void IdleManager.SerializeToStream (  
    BinaryWriter writer)
```

Zapisuje stan fabryk do strumienia binarnego.

Parametry

<i>writer</i>	BinaryWriter do zapisu
---------------	------------------------

Implementuje [IBinarySaveable](#).

6.17.2.20 Start()

```
void IdleManager.Start () [private]
```

Inicjalizacja managera: odnajduje [RaptorCore](#) oraz inicjalizuje zasoby, fabryki i mikstury.

6.17.2.21 TickLoop()

```
IEnumerator IdleManager.TickLoop () [private]
```

Pętla ticków wywoływana co tickInterval sekund, przetwarza produkcję i mikstury.

6.17.2.22 UnlockFactory()

```
bool IdleManager.UnlockFactory (  
    int index)
```

Próbuje odblokować fabrykę o podanym indeksie.

Parametry

<i>index</i>	Indeks fabryki do odblokowania
--------------	--------------------------------

Zwraca

True jeśli odblokowano fabrykę, false jeśli brak złota lub indeks nieprawidłowy

6.17.3 Dokumentacja atrybutów składowych

6.17.3.1 characterClass

```
CharacterClass IdleManager.characterClass [private]
```

Referencja do klasy postaci określająca różne bonusy i zdolności pasywne.

6.17.3.2 factories

```
List<Factory> IdleManager.factories
```

Lista wszystkich fabryk w grze.

6.17.3.3 potionFactoryBonus1

```
QuarkType IdleManager.potionFactoryBonus1 = 1
```

Bonus mnożnika produkcji dla mikstury 1.

6.17.3.4 potionFactoryBonus2

```
QuarkType IdleManager.potionFactoryBonus2 = 1
```

Bonus mnożnika produkcji dla mikstury 2.

6.17.3.5 potions

```
List<Potion> IdleManager.potions
```

Lista wszystkich mikstur w grze.

6.17.3.6 raptorCore

`RaptorCore IdleManager.raptorCore [private]`

Referencja do głównego skryptu gry zawierającego złoto i managerów.

6.17.3.7 resources

`List<Resource> IdleManager.resources`

Lista wszystkich zasobów w grze.

6.17.3.8 tickCoroutine

`Coroutine IdleManager.tickCoroutine [private]`

Referencja do Coroutine obsługującej ticki.

6.17.3.9 tickInterval

`float IdleManager.tickInterval = 1f [private]`

Interwał ticka w sekundach, po którym następuje aktualizacja produkcji fabryk.

6.17.4 Dokumentacja właściwości

6.17.4.1 SaveKey

`int IdleManager.SaveKey [get]`

Identyfikator zapisu używany w systemie save/load.

Implementuje [IBinarySaveable](#).

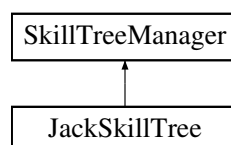
Dokumentacja dla tej klasy została wygenerowana z pliku:

- [IdleManager.cs](#)

6.18 Dokumentacja klasy JackSkillTree

Drzewko umiejętności typu "Jack of All Clicks". Zarządza odblokowywaniem i wizualizacją poszczególnych umiejętności dla tej gałęzi.

Diagram dziedziczenia dla JackSkillTree



Metody publiczne

- **JackSkillTree** (VisualElement root, **CharacterClass** charClass, HashSet< string > learnedSet, SkillPointsLimit skillPointsLimit)

Metody publiczne dziedziczone z **SkillTreeManager**

- **SkillTreeManager** (VisualElement root, **CharacterClass** charClass, Dictionary< string, List< string > > treeDependencies, string containerName, HashSet< string > learnedSet, **SkillPointsLimit** skillPointsLimit)
*Konstruktor bazowy **SkillTreeManager**.*
- void **ClearTree** ()
Czyści nauczone umiejętności.
- void **InitializeSkills** (string[] skillIds)
Tworzy przyciski dla wszystkich umiejętności w drzewku.
- void **RegisterTooltip** (string skillId, string title, string description, string lore)
Rejestruje tooltip dla przycisku umiejętności.
- void **DrawLine** (string fromId, string toId, Color color)
Rysuje pojedynczą linię między dwoma przyciskami.
- void **DrawAllLines** ()
Rysuje wszystkie linie zależności w drzewku.
- void **UpdateAll** ()
Aktualizuje wszystkie przyciski wizualnie.
- void **SerializeToStream** (BinaryWriter writer)
Zapisuje stan nauczonych umiejętności do strumienia.
- void **DeserializeFromStream** (BinaryReader reader)
Wczytuje stan nauczonych umiejętności ze strumienia.

Metody chronione

- override void **OnSkillClicked** (string id)
Obsługuje kliknięcie lewym przyciskiem na umiejętność.
- override bool **CanUnlock** (string id)
Sprawdza, czy umiejętność może zostać odblokowana.
- override void **UpdateVisual** (string id)
Aktualizuje wygląd przycisku umiejętności w zależności od stanu (odblokowana, dostępna, zablokowana).

Metody chronione dziedziczone z **SkillTreeManager**

- void **OnSkillClicked** (string id)
Wywoływane po kliknięciu w umiejętność
- bool **CanUnlock** (string id)
Sprawdza, czy umiejętność może zostać nauczona.
- void **UpdateVisual** (string id)
Aktualizuje wizualną reprezentację przycisku umiejętności.
- virtual void **OnSkillUnlearned** (string id)
Wywoływane przy "odnauczaniu" umiejętności (klik prawym przyciskiem).

Statyczne metody prywatne

- static Dictionary< string, List< string > > [GetDependencies](#) ()
Zależności między umiejętnościami.

Dodatkowe dziedziczone składowe

Atrybuty chronione dziedziczone z [SkillTreeManager](#)

- VisualElement [ui](#)
Root UI dokumentu.
- [CharacterClass](#) [characterClass](#)
Klasa postaci, której drzewko dotyczy.
- Dictionary< string, Button > [buttons](#) = new()
Mapa przycisków umiejętności według ID.
- HashSet< string > [learned](#) = new()
Zbiór nauczonych umiejętności.
- Dictionary< string, List< string > > [dependencies](#) = new()
Zależności umiejętności.
- VisualElement [skillTreeContainer](#)
Kontener UI dla całego drzewka.
- VisualElement [lineLayer](#)
Warstwa linii rysowanych między umiejętnościami.
- Color [lockedColor](#) = new Color(0.3f, 0.3f, 0.3f)
Kolor przycisku zablokowanego.
- Color [availableColor](#) = new Color(0.8f, 0.6f, 0.3f)
Kolor przycisku dostępnego do nauki.
- Color [learnedColor](#) = new Color(0.2f, 0.8f, 0.2f)
Kolor przycisku nauczonego.
- [SkillPointsLimit](#) [skillPointsLimit](#)
Referencja do menedżera punktów umiejętności.

6.18.1 Opis szczegółowy

Drzewko umiejętności typu "Jack of All Clicks". Zarządza odblokowywaniem i wizualizacją poszczególnych umiejętności dla tej gałęzi.

6.18.2 Dokumentacja konstruktora i destruktora

6.18.2.1 JackSkillTree()

```
JackSkillTree.JackSkillTree (
    VisualElement root,
    CharacterClass charClass,
    HashSet< string > learnedSet,
    SkillPointsLimit skillPointsLimit)
```

Konstruktor drzewka Jack Of All Clicks.

Parametry

<i>root</i>	Root VisualElement UI drzewka
<i>charClass</i>	Instancja CharacterClass gracza
<i>learnedSet</i>	Zestaw już odblokowanych umiejętności
<i>skillPointsLimit</i>	Obiekt zarządzający punktami umiejętności

6.18.3 Dokumentacja funkcji składowych

6.18.3.1 CanUnlock()

```
override bool JackSkillTree.CanUnlock (  
    string id) [protected]
```

Sprawdza, czy umiejętność może zostać odblokowana.

Parametry

<i>id</i>	ID umiejętności
-----------	-----------------

Zwraca

true jeśli odblokowanie jest możliwe

6.18.3.2 GetDependencies()

```
Dictionary< string, List< string > > JackSkillTree.GetDependencies () [static], [private]
```

Zależności między umiejętnościami.

6.18.3.3 OnSkillClicked()

```
override void JackSkillTree.OnSkillClicked (  
    string id) [protected]
```

Obsługuje kliknięcie lewym przyciskiem na umiejętność.

Parametry

<i>id</i>	ID klikniętej umiejętności
-----------	----------------------------

6.18.3.4 UpdateVisual()

```
override void JackSkillTree.UpdateVisual (
    string id) [protected]
```

Aktualizuje wygląd przycisku umiejętności w zależności od stanu (odblokowana, dostępna, zablokowana).

Parametry

<i>id</i>	ID umiejętności
-----------	-----------------

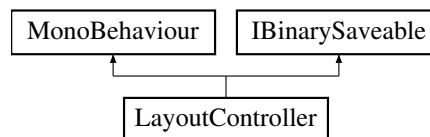
Dokumentacja dla tej klasy została wygenerowana z pliku:

- [SkillTreesUI.cs](#)

6.19 Dokumentacja klasy LayoutController

Kontroler głównego layoutu gry. Zarządza wszystkimi menedżerami UI, zasobami i tooltipami.

Diagram dziedziczenia dla LayoutController



Metody publiczne

- void [ResetGui](#) ()
Resetuje GUI: drzewka umiejętności, fabryki oraz przyciski punktów umiejętności.
- void [Update](#) ()
Aktualizacja każdego frame'a wybranych menedżerów UI oraz obsługa klawisza Escape.
- void [SetCurrencyText](#) (List< [QuarkType](#) > lista)
Ustawia tekst wyświetlający wartości wszystkich zasobów.
- void [SetGoldText](#) (string value)
Ustawia tekst wyświetlający ilość złota gracza.
- void [UpdateUI](#) ()
Aktualizuje UI zasobów i złota.
- void [SerializeToStream](#) (BinaryWriter writer)
Zapisuje stan UI fabryk i drzewek umiejętności do strumienia binarnego.
- void [DeserializeFromStream](#) (BinaryReader reader)
Wczytuje stan UI fabryk i drzewek umiejętności ze strumienia binarnego.

Atrybuty publiczne

- VisualTreeAsset [tooltipAsset](#)
Asset używany do tworzenia tooltipów.
- Sprite[] [images](#)
Tablica obrazków wykorzystywana w UI.
- VisualElement [ui](#)
Root VisualElement dla UI.
- FactoryUIManager [factoryUIManager](#)
Menedżer UI fabryk.
- PrestigeUIManager [prestigeUIManager](#)
Menedżer UI prestiżu.

Właściwości

- static [LayoutController Instance](#) [get, private set]
Instancja singletona [LayoutController](#).
- int [SaveKey](#) [get]
Klucz używany do zapisu stanu (SaveID) w systemie binarnym.

Metody prywatne

- void [Awake](#) ()
Inicjalizacja singletona oraz root UI.
- void [OnEnable](#) ()
Wywoływane przy włączaniu [LayoutController](#). Inicjalizuje menedżery i tooltipy.
- void [InitializeManagers](#) ()
Tworzy i inicjalizuje wszystkie menedżery UI.
- void [InitializeTooltips](#) ()
Inicjalizuje tooltipy i rejestruje callback odświeżający ich rozmiar.

Atrybuty prywatne

- SpriteRenderer [clickerObject](#)
Obiekt sprite wykorzystywany w clickerze.
- [RaptorCore](#) [raptorCore](#)
Główny rdzeń gry, przechowujący stan gracza i zasobów.
- [CharacterClass](#) [characterClass](#)
Klasa postaci gracza.
- [IdleManager](#) [idleManager](#)
Menedżer fabryk i produkcji idle.
- AudioSource [audioSource](#)
Źródło dźwięku do ogólnych efektów UI.
- AudioSource [critSource](#)
Źródło dźwięku do krytycznych akcji (crit).
- [SkillPointsLimit](#) [skillPointsLimit](#)
Limit punktów umiejętności.
- [PotionUIManager](#) [potionUIManager](#)
Menedżer UI mikstur.

- [OptionsManager](#) `optionsManager`
Menedżer opcji gry (menu, ustawienia).
- [LevelUIManager](#) `levelUIManager`
Menedżer UI poziomów postaci.
- [ContentPageManager](#) `contentPageManager`
Menedżer stron zawartości (content pages).
- [ResourceShopManager](#) `resourceShopManager`
Menedżer sklepu zasobów.
- [SkillTreesUIManager](#) `skillTreesUIManager`
Menedżer UI drzewek umiejętności.
- [BuffManager](#) `buffManager`
Menedżer buffów aktywnych w grze.
- `HashSet< string > learnedOneClickArmy = new()`
Zbiór nauczonych umiejętności OneClickArmy.
- `HashSet< string > learnedJackOfAllClicks = new()`
Zbiór nauczonych umiejętności JackOfAllClicks.
- `HashSet< string > learnedAutomatron = new()`
Zbiór nauczonych umiejętności Automatron.
- `Texture2D[] bufficons`
Ikony buffów w UI.

6.19.1 Opis szczegółowy

Kontroler głównego layoutu gry. Zarządza wszystkimi menedżerami UI, zasobami i tooltipami.

6.19.2 Dokumentacja funkcji składowych

6.19.2.1 Awake()

```
void LayoutController.Awake () [private]
```

Inicjalizacja singletona oraz root UI.

6.19.2.2 DeserializeFromStream()

```
void LayoutController.DeserializeFromStream (  
    BinaryReader reader)
```

Wczytuje stan UI fabryk i drzewek umiejętności ze strumienia binarnego.

Parametry

<code>reader</code>	BinaryReader do odczytu danych.
---------------------	---------------------------------

Implementuje [IBinarySaveable](#).

6.19.2.3 InitializeManagers()

```
void LayoutController.InitializeManagers () [private]
```

Tworzy i inicjalizuje wszystkie menedżery UI.

6.19.2.4 InitializeTooltips()

```
void LayoutController.InitializeTooltips () [private]
```

Inicjalizuje tooltipy i rejestruje callback odświeżający ich rozmiar.

6.19.2.5 OnEnable()

```
void LayoutController.OnEnable () [private]
```

Wywoływane przy włączaniu [LayoutController](#). Inicjalizuje menedżery i tooltipy.

6.19.2.6 ResetGui()

```
void LayoutController.ResetGui ()
```

Resetuje GUI: drzewka umiejętności, fabryki oraz przyciski punktów umiejętności.

6.19.2.7 SerializeToStream()

```
void LayoutController.SerializeToStream (  
    BinaryWriter writer)
```

Zapisuje stan UI fabryk i drzewek umiejętności do strumienia binarnego.

Parametry

<i>writer</i>	BinaryWriter do zapisu danych.
---------------	--------------------------------

Implementuje [IBinarySaveable](#).

6.19.2.8 SetCurrencyText()

```
void LayoutController.SetCurrencyText (  
    List< QuarkType > lista)
```

Ustawia tekst wyświetlający wartości wszystkich zasobów.

Parametry

<i>lista</i>	Lista wartości zasobów typu QuarkType .
--------------	---

6.19.2.9 SetGoldText()

```
void LayoutController.SetGoldText (  
    string value)
```

Ustawia tekst wyświetlający ilość złota gracza.

Parametry

<i>value</i>	Wartość złota w formacie string.
--------------	----------------------------------

6.19.2.10 Update()

```
void LayoutController.Update ()
```

Aktualizacja każdego frame'a wybranych menedżerów UI oraz obsługa klawisza Escape.

6.19.2.11 UpdateUI()

```
void LayoutController.UpdateUI ()
```

Aktualizuje UI zasobów i złota.

6.19.3 Dokumentacja atrybutów składowych

6.19.3.1 audioSource

```
AudioSource LayoutController.audioSource [private]
```

Źródło dźwięku do ogólnych efektów UI.

6.19.3.2 bufficons

```
Texture2D [] LayoutController.bufficons [private]
```

Ikony buffów w UI.

6.19.3.3 buffManager

`BuffManager` `LayoutController.buffManager` [private]

Menedżer buffów aktywnych w grze.

6.19.3.4 characterClass

`CharacterClass` `LayoutController.characterClass` [private]

Klasa postaci gracza.

6.19.3.5 clickerObject

`SpriteRenderer` `LayoutController.clickerObject` [private]

Obiekt sprite wykorzystywany w clickerze.

6.19.3.6 contentPageManager

`ContentPageManager` `LayoutController.contentPageManager` [private]

Menedżer stron zawartości (content pages).

6.19.3.7 critSource

`AudioSource` `LayoutController.critSource` [private]

Źródło dźwięku do krytycznych akcji (crit).

6.19.3.8 factoryUIManager

`FactoryUIManager` `LayoutController.factoryUIManager`

Menedżer UI fabryk.

6.19.3.9 idleManager

`IdleManager` `LayoutController.idleManager` [private]

Menedżer fabryk i produkcji idle.

6.19.3.10 images

`Sprite` [] `LayoutController.images`

Tablica obrazków wykorzystywana w UI.

6.19.3.11 `learnedAutomatron`

```
HashSet<string> LayoutController.learnedAutomatron = new() [private]
```

Zbiór nauczonych umiejętności `Automatron`.

6.19.3.12 `learnedJackOfAllClicks`

```
HashSet<string> LayoutController.learnedJackOfAllClicks = new() [private]
```

Zbiór nauczonych umiejętności `JackOfAllClicks`.

6.19.3.13 `learnedOneClickArmy`

```
HashSet<string> LayoutController.learnedOneClickArmy = new() [private]
```

Zbiór nauczonych umiejętności `OneClickArmy`.

6.19.3.14 `levelUIManager`

```
LevelUIManager LayoutController.levelUIManager [private]
```

Menedżer UI poziomów postaci.

6.19.3.15 `optionsManager`

```
OptionsManager LayoutController.optionsManager [private]
```

Menedżer opcji gry (menu, ustawienia).

6.19.3.16 `potionUIManager`

```
PotionUIManager LayoutController.potionUIManager [private]
```

Menedżer UI mikstur.

6.19.3.17 `prestigeUIManager`

```
PrestigeUIManager LayoutController.prestigeUIManager
```

Menedżer UI prestiżu.

6.19.3.18 `raptorCore`

```
RaptorCore LayoutController.raptorCore [private]
```

Główny rdzeń gry, przechowujący stan gracza i zasobów.

6.19.3.19 resourceShopManager

[ResourceShopManager](#) LayoutController.resourceShopManager [private]

Menedżer sklepu zasobów.

6.19.3.20 skillPointsLimit

[SkillPointsLimit](#) LayoutController.skillPointsLimit [private]

Limit punktów umiejętności.

6.19.3.21 skillTreesUIManager

[SkillTreesUIManager](#) LayoutController.skillTreesUIManager [private]

Menedżer UI drzewek umiejętności.

6.19.3.22 tooltipAsset

[VisualTreeAsset](#) LayoutController.tooltipAsset

Asset używany do tworzenia tooltipów.

6.19.3.23 ui

[VisualElement](#) LayoutController.ui

Root VisualElement dla UI.

6.19.4 Dokumentacja właściwości

6.19.4.1 Instance

[LayoutController](#) LayoutController.Instance [static], [get], [private set]

Instancja singletona [LayoutController](#).

6.19.4.2 SaveKey

int LayoutController.SaveKey [get]

Klucz używany do zapisu stanu (SaveID) w systemie binarnym.

Implementuje [IBinarySaveable](#).

Dokumentacja dla tej klasy została wygenerowana z pliku:

- [LayoutController.cs](#)

6.20 Dokumentacja klasy LevelUIManager

Menedżer UI poziomu postaci. Aktualizuje pasek doświadczenia i wyświetla aktualny poziom postaci.

Metody publiczne

- [LevelUIManager](#) (VisualElement root, [CharacterClass](#) charClass)
Konstruktor menedżera poziomu UI.
- void [Initialize](#) ()
Inicjalizuje elementy UI (Label poziomu, pasek doświadczenia i jego maskę).
- void [Update](#) ()
Aktualizuje pasek poziomu w każdym frame.

Metody prywatne

- void [UpdateLvlBar](#) ()
Aktualizuje pasek doświadczenia i tekst poziomu postaci. Oblicza procent wypełnienia paska na podstawie aktualnego doświadczenia.

Atrybuty prywatne

- VisualElement [ui](#)
Root element UI, z którego pobierane są wszystkie VisualElements.
- [CharacterClass](#) [characterClass](#)
Obiekt klasy gracza, zawierający poziom i doświadczenie.
- VisualElement [barMask](#)
Element maski paska doświadczenia – kontroluje wypełnienie paska.
- Label [lvlNumber](#)
Label wyświetlający numer aktualnego poziomu postaci.
- VisualElement [expBarContainer](#)
Kontener paska doświadczenia.
- VisualElement [barTexture](#)
Tekstura wypełnienia paska doświadczenia.

6.20.1 Opis szczegółowy

Menedżer UI poziomu postaci. Aktualizuje pasek doświadczenia i wyświetla aktualny poziom postaci.

6.20.2 Dokumentacja konstruktora i destruktora

6.20.2.1 LevelUIManager()

```
LevelUIManager.LevelUIManager (  
    VisualElement root,  
    CharacterClass charClass)
```

Konstruktor menedżera poziomu UI.

Parametry

<i>root</i>	Root VisualElement dla UI.
<i>charClass</i>	Instancja CharacterClass postaci gracza.

6.20.3 Dokumentacja funkcji składowych

6.20.3.1 Initialize()

```
void LevelUIManager.Initialize ()
```

Inicjalizuje elementy UI (Label poziomu, pasek doświadczenia i jego maskę).

6.20.3.2 Update()

```
void LevelUIManager.Update ()
```

Aktualizuje pasek poziomu w każdym frame.

6.20.3.3 UpdateLvlBar()

```
void LevelUIManager.UpdateLvlBar () [private]
```

Aktualizuje pasek doświadczenia i tekst poziomu postaci. Oblicza procent wypełnienia paska na podstawie aktualnego doświadczenia.

6.20.4 Dokumentacja atrybutów składowych

6.20.4.1 barMask

```
VisualElement LevelUIManager.barMask [private]
```

Element maski paska doświadczenia – kontroluje wypełnienie paska.

6.20.4.2 barTexture

```
VisualElement LevelUIManager.barTexture [private]
```

Tekstura wypełnienia paska doświadczenia.

6.20.4.3 characterClass

`CharacterClass` `LevelUIManager.characterClass` [private]

Obiekt klasy gracza, zawierający poziom i doświadczenie.

6.20.4.4 expBarContainer

`VisualElement` `LevelUIManager.expBarContainer` [private]

Kontener paska doświadczenia.

6.20.4.5 lvlNumber

`Label` `LevelUIManager.lvlNumber` [private]

Label wyświetlający numer aktualnego poziomu postaci.

6.20.4.6 ui

`VisualElement` `LevelUIManager.ui` [private]

Root element UI, z którego pobierane są wszystkie VisualElements.

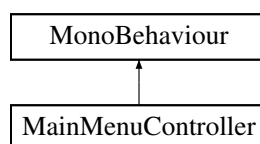
Dokumentacja dla tej klasy została wygenerowana z pliku:

- [LevelUIManager.cs](#)

6.21 Dokumentacja klasy MainMenuController

Kontroler głównego menu gry, obsługuje przyciski Start, Ustawienia, Wyjście oraz slidery i toggle dla ustawień audio i ekranu.

Diagram dziedziczenia dla MainMenuController



Metody prywatne

- void [Awake](#) ()
Inicjalizacja komponentów Unity.
- void [OnEnable](#) ()
Inicjalizacja menu i ustawień, rejestracja callbacków dla przycisków, sliderów i toggle.
- void [OnClick](#) ()
Odtwarza efekt kliknięcia przycisku z losowym pitch.
- void [OnStartButtonClicked](#) ()
Wywoływane po kliknięciu przycisku Start – ładuje scenę gry.
- void [OnSettingsButtonClicked](#) ()
Wywoływane po kliknięciu przycisku Settings – otwiera menu ustawień.
- void [OnBackButtonClicked](#) ()
Wywoływane po kliknięciu przycisku Back – powrót do głównego menu.
- void [OnExitButtonClicked](#) ()
Wywoływane po kliknięciu przycisku Quit – zamyka aplikację.
- void [OnMusicVolumeChanged](#) (ChangeEvent< float > evt)
Callback zmiany wartości suwaka muzyki – zapisuje głośność i ustawia źródło audio.
- void [OnSfxVolumeChanged](#) (ChangeEvent< float > evt)
Callback zmiany wartości suwaka SFX – zapisuje głośność i ustawia źródło audio.
- void [SetResolution](#) (int resIndex)
Ustawia rozdzielczość ekranu na podstawie indeksu w tablicy resolutions.
- void [SetFullScreen](#) (bool flag)
Ustawia tryb pełnoekranowy.

Atrybuty prywatne

- VisualElement [mainMenu](#)
Root VisualElement głównego menu.
- VisualElement [mainContainer](#)
Kontener głównych opcji menu.
- VisualElement [settingsContainer](#)
Kontener ustawień.
- AudioSource [sfxSource](#)
Źródło audio efektów dźwiękowych (SFX).
- AudioSource [musicSource](#)
Źródło audio muzyki tła.
- [SceneController](#) [sceneController](#)
Kontroler sceny używany do wczytywania scen.
- Button [startButton](#)
Przycisk Start gry.
- Button [settingsButton](#)
Przycisk otwierający ustawienia.
- Button [quitButton](#)
Przycisk wyjścia z gry.
- Button [backButton](#)
Przycisk powrotu z ustawień do głównego menu.
- Button [DeleteSaveButton](#)
Przycisk usuwania zapisu gry.
- Toggle [fullScreen](#)

- Toggle trybu pełnoekranowego.*
- Slider [musicSlider](#)
Slider regulacji głośności muzyki.
- Slider [sfxSlider](#)
Slider regulacji głośności efektów dźwiękowych.
- Toggle [AutoSavetoggle](#)
Toggle automatycznego zapisu gry.
- DropdownField [resolutionList](#)
Dropdown list wyboru rozdzielczości ekranu.
- Resolution[] [resolutions](#)
Lista dostępnych rozdzielczości ekranu.

6.21.1 Opis szczegółowy

Kontroler głównego menu gry, obsługuje przyciski Start, Ustawienia, Wyjście oraz slidery i toggle dla ustawień audio i ekranu.

6.21.2 Dokumentacja funkcji składowych

6.21.2.1 Awake()

```
void MainMenuController.Awake () [private]
```

Inicjalizacja komponentów Unity.

6.21.2.2 OnBackButtonClicked()

```
void MainMenuController.OnBackButtonClicked () [private]
```

Wywoływane po kliknięciu przycisku Back – powrót do głównego menu.

6.21.2.3 OnClick()

```
void MainMenuController.OnClick () [private]
```

Odtwarza efekt kliknięcia przycisku z losowym pitch.

6.21.2.4 OnEnable()

```
void MainMenuController.OnEnable () [private]
```

Inicjalizacja menu i ustawień, rejestracja callbacków dla przycisków, sliderów i toggle.

6.21.2.5 OnExitButtonClicked()

```
void MainMenuController.OnExitButtonClicked () [private]
```

Wywoływane po kliknięciu przycisku Quit – zamyka aplikację.

6.21.2.6 OnMusicVolumeChanged()

```
void MainMenuController.OnMusicVolumeChanged (
    ChangeEvent< float > evt) [private]
```

Callback zmiany wartości suwaka muzyki – zapisuje głośność i ustawia źródło audio.

Parametry

evt	Event zmiany wartości suwaka.
-----	-------------------------------

6.21.2.7 OnSettingsButtonClicked()

```
void MainMenuController.OnSettingsButtonClicked () [private]
```

Wywoływane po kliknięciu przycisku Settings – otwiera menu ustawień.

6.21.2.8 OnSfxVolumeChanged()

```
void MainMenuController.OnSfxVolumeChanged (
    ChangeEvent< float > evt) [private]
```

Callback zmiany wartości suwaka SFX – zapisuje głośność i ustawia źródło audio.

Parametry

evt	Event zmiany wartości suwaka.
-----	-------------------------------

6.21.2.9 OnStartButtonClicked()

```
void MainMenuController.OnStartButtonClicked () [private]
```

Wywoływane po kliknięciu przycisku Start – ładuje scenę gry.

6.21.2.10 SetFullScreen()

```
void MainMenuController.SetFullScreen (
    bool flag) [private]
```

Ustawia tryb pełnoekranowy.

Parametry

flag	True = pełny ekran, False = okno.
------	-----------------------------------

6.21.2.11 SetResolution()

```
void MainMenuController.SetResolution (
    int resIndex) [private]
```

Ustawia rozdzielczość ekranu na podstawie indeksu w tablicy resolutions.

Parametry

<i>resIndex</i>	Indeks wybranej rozdzielczości.
-----------------	---------------------------------

6.21.3 Dokumentacja atrybutów składowych

6.21.3.1 AutoSavetoggle

```
Toggle MainMenuController.AutoSavetoggle [private]
```

Toggle automatycznego zapisu gry.

6.21.3.2 backButton

```
Button MainMenuController.backButton [private]
```

Przycisk powrotu z ustawień do głównego menu.

6.21.3.3 DeleteSaveButton

```
Button MainMenuController.DeleteSaveButton [private]
```

Przycisk usuwania zapisu gry.

6.21.3.4 fullScreen

```
Toggle MainMenuController.fullScreen [private]
```

Toggle trybu pełnoekranowego.

6.21.3.5 mainContainer

```
VisualElement MainMenuController.mainContainer [private]
```

Kontener głównych opcji menu.

6.21.3.6 mainMenu

```
VisualElement MainMenuController.mainMenu [private]
```

Root VisualElement głównego menu.

6.21.3.7 musicSlider

`Slider MainMenuController.musicSlider [private]`

Slider regulacji głośności muzyki.

6.21.3.8 musicSource

`AudioSource MainMenuController.musicSource [private]`

Źródło audio muzyki tła.

6.21.3.9 quitButton

`Button MainMenuController.quitButton [private]`

Przycisk wyjścia z gry.

6.21.3.10 resolutionList

`DropDownField MainMenuController.resolutionList [private]`

Dropdown list wyboru rozdzielczości ekranu.

6.21.3.11 resolutions

`Resolution [] MainMenuController.resolutions [private]`

Lista dostępnych rozdzielczości ekranu.

6.21.3.12 sceneController

`SceneController MainMenuController.sceneController [private]`

Kontroler sceny używany do wczytywania scen.

6.21.3.13 settingsButton

`Button MainMenuController.settingsButton [private]`

Przycisk otwierający ustawienia.

6.21.3.14 settingsContainer

`VisualElement MainMenuController.settingsContainer [private]`

Kontener ustawień.

6.21.3.15 sfxSlider

Slider MainMenuController.sfxSlider [private]

Slider regulacji głośności efektów dźwiękowych.

6.21.3.16 sfxSource

AudioSource MainMenuController.sfxSource [private]

Źródło audio efektów dźwiękowych (SFX).

6.21.3.17 startButton

Button MainMenuController.startButton [private]

Przycisk Start gry.

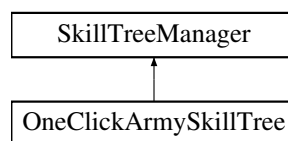
Dokumentacja dla tej klasy została wygenerowana z pliku:

- [MainMenuController.cs](#)

6.22 Dokumentacja klasy OneClickArmySkillTree

Drzewko umiejętności typu "One Click Army". Zarządza odblokowywaniem i wizualizacją poszczególnych umiejętności dla tej gałęzi.

Diagram dziedziczenia dla OneClickArmySkillTree



Metody publiczne

- `OneClickArmySkillTree` (VisualElement root, [CharacterClass](#) charClass, HashSet< string > learnedSet, SkillPointsLimit skillPointsLimit)

Konstruktor drzewka One Click Army.

Metody publiczne dziedziczone z [SkillTreeManager](#)

- [SkillTreeManager](#) (VisualElement root, [CharacterClass](#) charClass, Dictionary< string, List< string > > treeDependencies, string containerName, HashSet< string > learnedSet, [SkillPointsLimit](#) skillPointsLimit)
Konstruktor bazowy [SkillTreeManager](#).
- void [ClearTree](#) ()
Czyści nauczone umiejętności.
- void [InitializeSkills](#) (string[] skillIds)
Tworzy przyciski dla wszystkich umiejętności w drzewku.
- void [RegisterTooltip](#) (string skillId, string title, string description, string lore)
Rejestruje tooltip dla przycisku umiejętności.
- void [DrawLine](#) (string fromId, string toId, Color color)
Rysuje pojedynczą linię między dwoma przyciskami.
- void [DrawAllLines](#) ()
Rysuje wszystkie linie zależności w drzewku.
- void [UpdateAll](#) ()
Aktualizuje wszystkie przyciski wizualnie.
- void [SerializeToStream](#) (BinaryWriter writer)
Zapisuje stan nauczonych umiejętności do strumienia.
- void [DeserializeFromStream](#) (BinaryReader reader)
Wczytuje stan nauczonych umiejętności ze strumienia.

Metody chronione

- override void [OnSkillClicked](#) (string id)
Obsługuje kliknięcie lewym przyciskiem na umiejętność.
- override bool [CanUnlock](#) (string id)
Sprawdza, czy umiejętność może zostać odblokowana.
- override void [UpdateVisual](#) (string id)
Aktualizuje wygląd przycisku umiejętności w zależności od stanu (odblokowana, dostępna, zablokowana).

Metody chronione dziedziczone z [SkillTreeManager](#)

- void [OnSkillClicked](#) (string id)
Wywoływane po kliknięciu w umiejętność
- bool [CanUnlock](#) (string id)
Sprawdza, czy umiejętność może zostać nauczona.
- void [UpdateVisual](#) (string id)
Aktualizuje wizualną reprezentację przycisku umiejętności.
- virtual void [OnSkillUnlearned](#) (string id)
Wywoływane przy "odnauczaniu" umiejętności (klik prawym przyciskiem).

Statyczne metody prywatne

- static Dictionary< string, List< string > > [GetDependencies](#) ()
Zależności między umiejętnościami.

Dodatkowe dziedziczone składowe

Atrybuty chronione dziedziczone z SkillTreeManager

- VisualElement `ui`
Root UI dokumentu.
- CharacterClass `characterClass`
Klasa postaci, której drzewko dotyczy.
- Dictionary< string, Button > `buttons` = new()
Mapa przycisków umiejętności według ID.
- HashSet< string > `learned` = new()
Zbiór nauczonych umiejętności.
- Dictionary< string, List< string > > `dependencies` = new()
Zależności umiejętności.
- VisualElement `skillTreeContainer`
Kontener UI dla całego drzewka.
- VisualElement `lineLayer`
Warstwa linii rysowanych między umiejętnościami.
- Color `lockedColor` = new Color(0.3f, 0.3f, 0.3f)
Kolor przycisku zablokowanego.
- Color `availableColor` = new Color(0.8f, 0.6f, 0.3f)
Kolor przycisku dostępnego do nauki.
- Color `learnedColor` = new Color(0.2f, 0.8f, 0.2f)
Kolor przycisku nauczonego.
- SkillPointsLimit `skillPointsLimit`
Referencja do menedżera punktów umiejętności.

6.22.1 Opis szczegółowy

Drzewko umiejętności typu "One Click Army". Zarządza odblokowywaniem i wizualizacją poszczególnych umiejętności dla tej gałęzi.

6.22.2 Dokumentacja konstruktora i destruktora

6.22.2.1 OneClickArmySkillTree()

```
OneClickArmySkillTree.OneClickArmySkillTree (
    VisualElement root,
    CharacterClass charClass,
    HashSet< string > learnedSet,
    SkillPointsLimit skillPointsLimit)
```

Konstruktor drzewka One Click Army.

Parametry

<code>root</code>	Root VisualElement UI drzewka
<code>charClass</code>	Instancja CharacterClass gracza

<i>learnedSet</i>	Zestaw już odblokowanych umiejętności
<i>skillPointsLimit</i>	Obiekt zarządzający punktami umiejętności

6.22.3 Dokumentacja funkcji składowych

6.22.3.1 CanUnlock()

```
override bool OneClickArmySkillTree.CanUnlock (  
    string id) [protected]
```

Sprawdza, czy umiejętność może zostać odblokowana.

Parametry

<i>id</i>	ID umiejętności
-----------	-----------------

Zwraca

true jeśli odblokowanie jest możliwe

6.22.3.2 GetDependencies()

```
Dictionary< string, List< string > > OneClickArmySkillTree.GetDependencies () [static], [private]
```

Zależności między umiejętnościami.

6.22.3.3 OnSkillClicked()

```
override void OneClickArmySkillTree.OnSkillClicked (  
    string id) [protected]
```

Obsługuje kliknięcie lewym przyciskiem na umiejętność.

Parametry

<i>id</i>	ID klikniętej umiejętności
-----------	----------------------------

6.22.3.4 UpdateVisual()

```
override void OneClickArmySkillTree.UpdateVisual (
    string id) [protected]
```

Aktualizuje wygląd przycisku umiejętności w zależności od stanu (odblokowana, dostępna, zablokowana).

Parametry

<i>id</i>	ID umiejętności
-----------	-----------------

Dokumentacja dla tej klasy została wygenerowana z pliku:

- [SkillTreesUI.cs](#)

6.23 Dokumentacja klasy OptionsManager

Zarządza menu opcji gry, w tym ustawieniami audio, wyświetlania i zapisu.

Metody publiczne

- [OptionsManager](#) (VisualElement root, AudioSource audio, AudioSource crit)
Konstruktor klasy [OptionsManager](#).
- void [Initialize](#) ()
Inicjalizuje wszystkie elementy menu opcji i rejestruje zdarzenia.
- void [EnableOptions](#) ()
Włącza menu opcji (pokazuje panel).

Metody prywatne

- void [InitializeAudioSettings](#) ()
Inicjalizacja ustawień audio (slidery muzyki i SFX).
- void [InitializeDisplaySettings](#) ()
Inicjalizacja ustawień wyświetlania (fullscreen i rozdzielczość).
- void [InitializeSaveSettings](#) ()
Inicjalizacja przycisku usuwania zapisu i toggle automatycznego zapisu.
- void [ResumeGame](#) ()
Wznawia grę i ukrywa menu opcji.
- void [ExitGame](#) ()
Wyjście z gry.
- void [LoadSettings](#) ()
Pokazuje panel ustawień.
- void [ReturnSettings](#) ()
Powrót z ustawień do głównego menu.
- void [OnSfxVolumeChanged](#) (ChangeEvent< float > evt)
Callback zmiany głośności SFX.
- void [OnMusicVolumeChanged](#) (ChangeEvent< float > evt)
Callback zmiany głośności muzyki.
- void [SetResolution](#) (int resIndex)
Ustawia rozdzielczość ekranu.
- void [SetFullScreen](#) (bool flag)
Włącza lub wyłącza tryb pełnoekranowy.

Atrybuty prywatne

- VisualElement [ui](#)
Root UI (UIDocument) dla menu opcji.
- AudioSource [audioSource](#)
Główne źródło audio dla efektów w menu.
- VisualElement [menu](#)
Kontener całego menu pauzy/opcji.
- VisualElement [options](#)
Panel ustawień.
- VisualElement [mainMenu](#)
Panel głównego menu.
- Button [optionBtn](#)
Przycisk otwierający menu opcji (obecnie nie używany).
- Button [resume](#)
Przycisk wznowienia gry.
- Button [settings](#)
Przycisk otwierający panel ustawień.
- Button [exit](#)
Przycisk wyjścia z gry.
- Button [backSettings](#)
Przycisk powrotu z ustawień do głównego menu.
- Slider [musicSlider](#)
Slider do ustawiania głośności muzyki.
- Slider [sfxSlider](#)
Slider do ustawiania głośności efektów SFX.
- Toggle [fullScreen](#)
Toggle włączania/wyłączania pełnego ekranu.
- DropdownField [resolutionList](#)
Dropdown do wyboru rozdzielczości.
- Resolution[] [resolutions](#)
Lista dostępnych rozdzielczości.
- Button [deleteSaveButton](#)
Przycisk usuwania zapisu gry.
- Toggle [autoSaveToggle](#)
Toggle automatycznego zapisu gry.
- AudioSource [critSource](#)
Źródło audio dla efektów krytycznych.

6.23.1 Opis szczegółowy

Zarządza menu opcji gry, w tym ustawieniami audio, wyświetlania i zapisu.

6.23.2 Dokumentacja konstruktora i destruktora

6.23.2.1 OptionsManager()

```
OptionsManager.OptionsManager (  
    VisualElement root,  
    AudioSource audio,  
    AudioSource crit)
```

Konstruktor klasy [OptionsManager](#).

Parametry

<i>root</i>	Root UI (VisualElement) dla menu.
<i>audio</i>	Główne źródło audio dla przycisków i SFX.
<i>crit</i>	Źródło audio dla efektów krytycznych.

6.23.3 Dokumentacja funkcji składowych

6.23.3.1 EnableOptions()

```
void OptionsManager.EnableOptions ()
```

Włącza menu opcji (pokazuje panel).

6.23.3.2 ExitGame()

```
void OptionsManager.ExitGame () [private]
```

Wyjście z gry.

6.23.3.3 Initialize()

```
void OptionsManager.Initialize ()
```

Inicjalizuje wszystkie elementy menu opcji i rejestruje zdarzenia.

6.23.3.4 InitializeAudioSettings()

```
void OptionsManager.InitializeAudioSettings () [private]
```

Inicjalizacja ustawień audio (slidery muzyki i SFX).

6.23.3.5 InitializeDisplaySettings()

```
void OptionsManager.InitializeDisplaySettings () [private]
```

Inicjalizacja ustawień wyświetlania (fullscreen i rozdzielczość).

6.23.3.6 InitializeSaveSettings()

```
void OptionsManager.InitializeSaveSettings () [private]
```

Inicjalizacja przycisku usuwania zapisu i toggle automatycznego zapisu.

6.23.3.7 LoadSettings()

```
void OptionsManager.LoadSettings () [private]
```

Pokazuje panel ustawień.

6.23.3.8 OnMusicVolumeChanged()

```
void OptionsManager.OnMusicVolumeChanged (  
    ChangeEvent< float > evt) [private]
```

Callback zmiany głośności muzyki.

Parametry

evt	Event zmiany wartości slidera.
-----	--------------------------------

6.23.3.9 OnSfxVolumeChanged()

```
void OptionsManager.OnSfxVolumeChanged (  
    ChangeEvent< float > evt) [private]
```

Callback zmiany głośności SFX.

Parametry

evt	Event zmiany wartości slidera.
-----	--------------------------------

6.23.3.10 ResumeGame()

```
void OptionsManager.ResumeGame () [private]
```

Wznawia grę i ukrywa menu opcji.

6.23.3.11 ReturnSettings()

```
void OptionsManager.ReturnSettings () [private]
```

Powrót z ustawień do głównego menu.

6.23.3.12 SetFullScreen()

```
void OptionsManager.SetFullScreen (
    bool flag) [private]
```

Włącza lub wyłącza tryb pełnoekranowy.

Parametry

<i>flag</i>	True = fullscreen, False = okno.
-------------	----------------------------------

6.23.3.13 SetResolution()

```
void OptionsManager.SetResolution (
    int resIndex) [private]
```

Ustawia rozdzielczość ekranu.

Parametry

<i>resIndex</i>	Indeks wybranej rozdzielczości w tablicy resolutions.
-----------------	---

6.23.4 Dokumentacja atrybutów składowych

6.23.4.1 audioSource

```
AudioSource OptionsManager.audioSource [private]
```

Główne źródło audio dla efektów w menu.

6.23.4.2 autoSaveToggle

```
Toggle OptionsManager.autoSaveToggle [private]
```

Toggle automatycznego zapisu gry.

6.23.4.3 backSettings

```
Button OptionsManager.backSettings [private]
```

Przycisk powrotu z ustawień do głównego menu.

6.23.4.4 critSource

```
AudioSource OptionsManager.critSource [private]
```

Źródło audio dla efektów krytycznych.

6.23.4.5 deleteSaveButton

Button OptionsManager.deleteSaveButton [private]

Przycisk usuwania zapisu gry.

6.23.4.6 exit

Button OptionsManager.exit [private]

Przycisk wyjścia z gry.

6.23.4.7 fullScreen

Toggle OptionsManager.fullScreen [private]

Toggle włączania/wyłączania pełnego ekranu.

6.23.4.8 mainMenu

VisualElement OptionsManager.mainMenu [private]

Panel głównego menu.

6.23.4.9 menu

VisualElement OptionsManager.menu [private]

Kontener całego menu pauzy/opcji.

6.23.4.10 musicSlider

Slider OptionsManager.musicSlider [private]

Slider do ustawiania głośności muzyki.

6.23.4.11 optionBtn

Button OptionsManager.optionBtn [private]

Przycisk otwierający menu opcji (obecnie nie używany).

6.23.4.12 options

VisualElement OptionsManager.options [private]

Panel ustawień.

6.23.4.13 resolutionList

DropDownField OptionsManager.resolutionList [private]

Dropdown do wyboru rozdzielczości.

6.23.4.14 resolutions

Resolution [] OptionsManager.resolutions [private]

Lista dostępnych rozdzielczości.

6.23.4.15 resume

Button OptionsManager.resume [private]

Przycisk wznowienia gry.

6.23.4.16 settings

Button OptionsManager.settings [private]

Przycisk otwierający panel ustawień.

6.23.4.17 sfxSlider

Slider OptionsManager.sfxSlider [private]

Slider do ustawiania głośności efektów SFX.

6.23.4.18 ui

VisualElement OptionsManager.ui [private]

Root UI (UIDocument) dla menu opcji.

Dokumentacja dla tej klasy została wygenerowana z pliku:

- [OptionsManager.cs](#)

6.24 Dokumentacja klasy Potion

Reprezentuje miksturę/potion w grze, którą można kupić i aktywować. Mikstury mają koszty w określonych zasobach, czas trwania oraz przypisany efekt.

Metody publiczne

- **Potion** (string _name, [Resource](#) _resource1, [Resource](#) _resource2, [QuarkType](#) _cost1, [QuarkType](#) _cost2, string _effect, double _duration, string _effectDescription)

Tworzy nową miksturę z określonymi parametrami.

Atrybuty publiczne

- string [name](#)
Nazwa mikstury.
- [Resource](#) [resourceType1](#)
Pierwszy typ zasobu wymagany do zakupu mikstury.
- [Resource](#) [resourceType2](#)
Drugi typ zasobu wymagany do zakupu mikstury.
- [QuarkType](#) [cost1](#)
Koszt mikstury w pierwszym typie zasobu.
- [QuarkType](#) [cost2](#)
Koszt mikstury w drugim typie zasobu.
- string [effect](#)
Nazwa efektu mikstury.
- double [duration](#)
Czas trwania efektu mikstury w sekundach.
- bool [isActive](#) = false
Czy mikstura jest obecnie aktywna.
- double [timeRemaining](#)
Pozostały czas działania mikstury w sekundach.
- [Factory](#) [linkedFactory](#)
Fabryka, na którą aktualnie działa mikstura.
- string [effectDescription](#)
Opis działania mikstury wyświetlany w UI.
- int [boughtCount](#)
Liczba zakupionych egzemplarzy tej mikstury.

6.24.1 Opis szczegółowy

Reprezentuje miksturę/potion w grze, którą można kupić i aktywować. Mikstury mają koszty w określonych zasobach, czas trwania oraz przypisany efekt.

6.24.2 Dokumentacja konstruktora i destruktoru

6.24.2.1 Potion()

```
Potion.Potion (
    string _name,
    Resource _resource1,
    Resource _resource2,
    QuarkType _cost1,
    QuarkType _cost2,
    string _effect,
```

```
double _duration,  
string _effectDescription)
```

Tworzy nową miksturę z określonymi parametrami.

Parametry

<code>_name</code>	Nazwa mikstury.
<code>_resource1</code>	Pierwszy typ zasobu wymagany do zakupu.
<code>_resource2</code>	Drugi typ zasobu wymagany do zakupu.
<code>_cost1</code>	Koszt mikstury w pierwszym zasobie.
<code>_cost2</code>	Koszt mikstury w drugim zasobie.
<code>_effect</code>	Efekt mikstury.
<code>_duration</code>	Czas trwania efektu w sekundach.
<code>_effectDescription</code>	Opis działania mikstury wyświetlany w UI.

6.24.3 Dokumentacja atrybutów składowych

6.24.3.1 boughtCount

```
int Potion.boughtCount
```

Liczba zakupionych egzemplarzy tej mikstury.

6.24.3.2 cost1

```
QuarkType Potion.cost1
```

Koszt mikstury w pierwszym typie zasobu.

6.24.3.3 cost2

```
QuarkType Potion.cost2
```

Koszt mikstury w drugim typie zasobu.

6.24.3.4 duration

```
double Potion.duration
```

Czas trwania efektu mikstury w sekundach.

6.24.3.5 effect

```
string Potion.effect
```

Nazwa efektu mikstury.

6.24.3.6 effectDescription

```
string Potion.effectDescription
```

Opis działania mikstury wyświetlany w UI.

6.24.3.7 isActive

```
bool Potion.isActive = false
```

Czy mikstura jest obecnie aktywna.

6.24.3.8 linkedFactory

```
Factory Potion.linkedFactory
```

Fabryka, na którą aktualnie działa mikstura.

6.24.3.9 name

```
string Potion.name
```

Nazwa mikstury.

6.24.3.10 resourceType1

```
Resource Potion.resourceType1
```

Pierwszy typ zasobu wymagany do zakupu mikstury.

6.24.3.11 resourceType2

```
Resource Potion.resourceType2
```

Drugi typ zasobu wymagany do zakupu mikstury.

6.24.3.12 timeRemaining

```
double Potion.timeRemaining
```

Pozostały czas działania mikstury w sekundach.

Dokumentacja dla tej klasy została wygenerowana z pliku:

- [Potion.cs](#)

6.25 Dokumentacja klasy PotionUIManager.PotionUI

Reprezentuje pojedynczy element UI dla mikstury.

Atrybuty publiczne

- VisualElement [potionPanel](#)
Panel wizualny mikstury w UI.
- Label [nameLabel](#)
Etykieta z nazwą mikstury.
- Label [costLabel](#)
Etykieta z ceną mikstury.
- Label [effectLabel](#)
Etykieta z opisem efektu mikstury.
- Button [buyButton](#)
Przycisk zakupu mikstury.

6.25.1 Opis szczegółowy

Reprezentuje pojedynczy element UI dla mikstury.

6.25.2 Dokumentacja atrybutów składowych

6.25.2.1 buyButton

Button `PotionUIManager.PotionUI.buyButton`

Przycisk zakupu mikstury.

6.25.2.2 costLabel

Label `PotionUIManager.PotionUI.costLabel`

Etykieta z ceną mikstury.

6.25.2.3 effectLabel

Label `PotionUIManager.PotionUI.effectLabel`

Etykieta z opisem efektu mikstury.

6.25.2.4 nameLabel

Label `PotionUIManager.PotionUI.nameLabel`

Etykieta z nazwą mikstury.

6.25.2.5 potionPanel

VisualElement PotionUIManager.PotionUI.potionPanel

Panel wizualny mikstury w UI.

Dokumentacja dla tej klasy została wygenerowana z pliku:

- [PotionUIManager.cs](#)

6.26 Dokumentacja klasy PotionUIManager

Zarządza wyświetlaniem panelu mikstur w UI, aktualizuje ceny, efekty i przyciski zakupu.

Komponenty

- class [PotionUI](#)
Reprezentuje pojedynczy element UI dla mikstury.

Metody publiczne

- [PotionUIManager](#) (VisualElement root, [RaptorCore](#) core, [IdleManager](#) manager, AudioSource audio, CharacterClass character)
Tworzy manager UI dla mikstur.
- void [Initialize](#) ()
Inicjalizuje ScrollView i elementy mikstur w UI.
- void [Update](#) ()
Aktualizuje wszystkie elementy UI mikstur, w tym nazwy, efekty i przyciski zakupu.

Metody prywatne

- void [InitializePotions](#) ()
Tworzy wszystkie elementy UI mikstur, przypisuje przyciski i etykiety.
- void [UpdatePotionUI](#) ()
Sprawdza stan każdej mikstury i aktualizuje jej UI.
- void [BuyPotion](#) (int potionIndex)
Próbuje zakupić miksturę o podanym indeksie i aktualizuje UI.

Atrybuty prywatne

- VisualElement [ui](#)
Root UI, do którego przypięte są wszystkie elementy.
- ScrollView [scrollView](#)
ScrollView panelu z miksturami.
- List< [PotionUI](#) > [potionUIs](#) = new List<[PotionUI](#)>()
Lista wszystkich elementów UI mikstur.
- int [numberOfPotions](#) = 6
Liczba mikstur dostępnych w grze.
- [RaptorCore](#) [raptorCore](#)
Główny komponent gry, umożliwia dostęp do zasobów gracza.
- [IdleManager](#) [idleManager](#)
IdleManager odpowiedzialny za logikę mikstur i fabryk.
- AudioSource [audioSource](#)
Źródło dźwięku dla kliknięć w UI.
- [CharacterClass](#) [characterClass](#)
Klasa postaci gracza (np. jej umiejętności, bonusy itp.).

6.26.1 Opis szczegółowy

Zarządza wyświetlaniem panelu mikstur w UI, aktualizuje ceny, efekty i przyciski zakupu.

6.26.2 Dokumentacja konstruktora i destruktora

6.26.2.1 PotionUIManager()

```
PotionUIManager.PotionUIManager (  
    VisualElement root,  
    RaptorCore core,  
    IdleManager manager,  
    AudioSource audio,  
    CharacterClass character)
```

Tworzy manager UI dla mikstur.

Parametry

<i>root</i>	Root VisualElement dla UI.
<i>core</i>	RaptorCore do obsługi zasobów.
<i>manager</i>	IdleManager do obsługi logiki mikstur.
<i>audio</i>	Źródło dźwięku do kliknięć UI.
<i>character</i>	Obiekt klasy postaci gracza.

6.26.3 Dokumentacja funkcji składowych

6.26.3.1 BuyPotion()

```
void PotionUIManager.BuyPotion (  
    int potionIndex) [private]
```

Próbuje zakupić miksturę o podanym indeksie i aktualizuje UI.

Parametry

<i>potionIndex</i>	Indeks mikstury w liście IdleManager .
--------------------	--

6.26.3.2 Initialize()

```
void PotionUIManager.Initialize ()
```

Inicjalizuje ScrollView i elementy mikstur w UI.

6.26.3.3 InitializePotions()

```
void PotionUIManager.InitializePotions () [private]
```

Tworzy wszystkie elementy UI mikstur, przypisuje przyciski i etykiety.

6.26.3.4 Update()

```
void PotionUIManager.Update ()
```

Aktualizuje wszystkie elementy UI mikstur, w tym nazwy, efekty i przyciski zakupu.

6.26.3.5 UpdatePotionUI()

```
void PotionUIManager.UpdatePotionUI () [private]
```

Sprawdza stan każdej mikstury i aktualizuje jej UI.

6.26.4 Dokumentacja atrybutów składowych

6.26.4.1 audioSource

```
AudioSource PotionUIManager.audioSource [private]
```

Źródło dźwięku dla kliknięć w UI.

6.26.4.2 characterClass

```
CharacterClass PotionUIManager.characterClass [private]
```

Klasa postaci gracza (np. jej umiejętności, bonusy itp.).

6.26.4.3 idleManager

```
IdleManager PotionUIManager.idleManager [private]
```

[IdleManager](#) odpowiedzialny za logikę mikstur i fabryk.

6.26.4.4 numberOfPotions

```
int PotionUIManager.numberOfPotions = 6 [private]
```

Liczba mikstur dostępnych w grze.

6.26.4.5 potionUIs

```
List<PotionUI> PotionUIManager.potionUIs = new List<PotionUI>() [private]
```

Lista wszystkich elementów UI mikstur.

6.26.4.6 raptorCore

```
RaptorCore PotionUIManager.raptorCore [private]
```

Główny komponent gry, umożliwia dostęp do zasobów gracza.

6.26.4.7 scrollView

```
ScrollView PotionUIManager.scrollView [private]
```

ScrollView panelu z miksturami.

6.26.4.8 ui

```
VisualElement PotionUIManager.ui [private]
```

Root UI, do którego przypięte są wszystkie elementy.

Dokumentacja dla tej klasy została wygenerowana z pliku:

- [PotionUIManager.cs](#)

6.27 Dokumentacja klasy PrestigeSystem

Statyczny system prestiżu odpowiedzialny za obsługę resetu postępu i przyznawanie punktów prestiżu.

Statyczne metody publiczne

- static void [Init](#) ([RaptorCore](#) core)
Inicjalizuje system prestiżu i przypisuje referencję do [RaptorCore](#).
- static bool [CheckPrestige](#) (double val)
Sprawdza, czy gracz posiada wystarczającą ilość złota, aby wykonać prestiż.
- static void [ExecutePrestige](#) ()
Kalkuluje punkty prestiżu:
 - Przyznaje odpowiednie punkty prestiżu do kategorii *OneClickArmy*, *JackOfClicks*, *Automatron*
 - Zwiększa licznik prestiżu
 - Resetuje postać, zasoby, fabryki i GUI
 - Resetuje rdzeń gry
 - Aktualizuje UI strony prestiżu

Statyczne atrybuty prywatne

- static [RaptorCore _core](#)
- static bool [_initialized](#)

Flaga oznaczająca, czy system prestiżu został już zainicjalizowany.

6.27.1 Opis szczegółowy

Statyczny system prestiżu odpowiedzialny za obsługę resetu postępu i przyznawanie punktów prestiżu.

6.27.2 Dokumentacja funkcji składowych

6.27.2.1 CheckPrestige()

```
bool PrestigeSystem.CheckPrestige (  
    double val) [static]
```

Sprawdza, czy gracz posiada wystarczającą ilość złota, aby wykonać prestiż.

Parametry

<i>val</i>	Wymagana ilość złota do wykonania prestiżu.
------------	---

Zwraca

True, jeśli gracz posiada wystarczająco złota, false w przeciwnym wypadku.

6.27.2.2 ExecutePrestige()

```
void PrestigeSystem.ExecutePrestige () [static]
```

Kalkuluje punkty prestiżu:

- Przyznaje odpowiednie punkty prestiżu do kategorii OneClickArmy, JackOfClicks, Automatron
- Zwiększa licznik prestiżu
- Resetuje postać, zasoby, fabryki i GUI
- Resetuje rdzeń gry
- Aktualizuje UI strony prestiżu

.

6.27.2.3 `Init()`

```
void PrestigeSystem.Init (  
    RaptorCore core) [static]
```

Inicjalizuje system prestiżu i przypisuje referencję do [RaptorCore](#).

Parametry

<code>core</code>	Instancja RaptorCore zawierająca dane gracza.
-------------------	---

Wyjątki

<code>ArgumentNullException</code>	Rzucane, jeśli <code>core</code> jest null.
------------------------------------	---

6.27.3 Dokumentacja atrybutów składowych

6.27.3.1 `_core`

```
RaptorCore PrestigeSystem._core [static], [private]
```

Referencja do głównego rdzenia gry ([RaptorCore](#)), potrzebna do odczytu i modyfikacji złota, prestiżu i innych danych.

6.27.3.2 `_initialized`

```
bool PrestigeSystem._initialized [static], [private]
```

Flaga oznaczająca, czy system prestiżu został już zainicjalizowany.

Dokumentacja dla tej klasy została wygenerowana z pliku:

- [PrestigeSystem.cs](#)

6.28 Dokumentacja klasy `PrestigeUIManager`

Manager interfejsu użytkownika odpowiedzialny za system prestiżu. Obsługuje wyświetlanie statystyk prestiżu, pasek prestiżu oraz okno potwierdzenia resetu.

Metody publiczne

- [PrestigeUIManager](#) (VisualElement ui, AudioSource audio, [RaptorCore](#) raptorcore)
Konstruktor menedżera UI prestiżu.
- void [Initialize](#) ()
Inicjalizuje wszystkie referencje UI oraz podpiną zdarzenia przycisków.
- void [UpdateContentPage](#) ()
Aktualizuje zawartość zakładki prestiżu: liczniki prestiżu oraz punktów drzewek.
- void [EnablePrestige](#) ()
Aktywuje możliwość wywołania prestiżu przez kliknięcie paska doświadczenia. Zmienia również kolor paska na prestiżowy.
- void [DisablePrestige](#) ()
Dezaktywuje możliwość prestiżu i przywraca standardowy wygląd paska.
- void [Trigger](#) (ClickEvent evt)
Wyświetla okno potwierdzenia prestiżu po kliknięciu paska.
- void [Hide](#) ()
Ukrywa okno prestiżu bez wykonywania resetu.
- void [PrestigeTrigger](#) ()
Sprawdza warunki prestiżu i jeśli są spełnione, wykonuje metodę odpowiedzialną za rozdanie punktów z prestiżu oraz resetuje gre.

Atrybuty prywatne

- VisualElement [ui_ref](#)
Główna referencja do root VisualElement UI.
- VisualElement [PrestigeRoot](#)
Root okna potwierdzenia prestiżu (popup).
- VisualElement [ExpContainer](#)
Kontener paska doświadczenia, który po spełnieniu warunków staje się paskiem prestiżu.
- VisualElement [BarContainer](#)
Tło paska prestiżu.
- VisualElement [BarTexture](#)
Tekstura wypełnienia paska prestiżu.
- Button [PrestigeAccept](#)
Przycisk potwierdzający wykonanie prestiżu.
- Button [PrestigeCancel](#)
Przycisk anulujący okno prestiżu.
- AudioSource [audioSource](#)
Źródło dźwięku używane do kliknięć UI.
- [RaptorCore](#) rp
Referencja do głównego rdzenia gry ([RaptorCore](#)).
- VisualElement [contentPage](#)
Strona contentu odpowiadająca zakładce prestiżu (Content7).
- Button [PrestigeButtonMenu](#)
Przycisk w menu głównym otwierający zakładkę prestiżu (btn7).

6.28.1 Opis szczegółowy

Manager interfejsu użytkownika odpowiedzialny za system prestiżu. Obsługuje wyświetlanie statystyk prestiżu, pasek prestiżu oraz okno potwierdzenia resetu.

6.28.2 Dokumentacja konstruktora i destruktora

6.28.2.1 PrestigeUIManager()

```
PrestigeUIManager.PrestigeUIManager (  
    VisualElement ui,  
    AudioSource audio,  
    RaptorCore raptorcore)
```

Konstruktor managera UI prestiżu.

Parametry

<i>ui</i>	Root VisualElement UI.
<i>audio</i>	Źródło dźwięku do efektów UI.
<i>raptorcore</i>	Instancja RaptorCore .

6.28.3 Dokumentacja funkcji składowych

6.28.3.1 DisablePrestige()

```
void PrestigeUIManager.DisablePrestige ()
```

Dezaktywuje możliwość prestiżu i przywraca standardowy wygląd paska.

6.28.3.2 EnablePrestige()

```
void PrestigeUIManager.EnablePrestige ()
```

Aktywuje możliwość wywołania prestiżu przez kliknięcie paska doświadczenia. Zmienia również kolor paska na prestiżowy.

6.28.3.3 Hide()

```
void PrestigeUIManager.Hide ()
```

Ukrywa okno prestiżu bez wykonywania resetu.

6.28.3.4 Initialize()

```
void PrestigeUIManager.Initialize ()
```

Inicjalizuje wszystkie referencje UI oraz podpiną zdarzenia przycisków.

6.28.3.5 PrestigeTrigger()

```
void PrestigeUIManager.PrestigeTrigger ()
```

Sprawdza warunki prestiżu i jeśli są spełnione, wykonuje metodę odpowiedzialną za rozdanie punktów z prestiżu oraz resetuje gre.

6.28.3.6 Trigger()

```
void PrestigeUIManager.Trigger (  
    ClickEvent evt)
```

Wyświetla okno potwierdzenia prestiżu po kliknięciu paska.

Parametry

evt	Zdarzenie kliknięcia.
-----	-----------------------

6.28.3.7 UpdateContentPage()

```
void PrestigeUIManager.UpdateContentPage ()
```

Aktualizuje zawartość zakładki prestiżu: liczniki prestiżu oraz punktów drzewek.

6.28.4 Dokumentacja atrybutów składowych

6.28.4.1 audioSource

```
AudioSource PrestigeUIManager.audioSource [private]
```

Źródło dźwięku używane do kliknięć UI.

6.28.4.2 BarContainer

```
VisualElement PrestigeUIManager.BarContainer [private]
```

Tło paska prestiżu.

6.28.4.3 BarTexture

```
VisualElement PrestigeUIManager.BarTexture [private]
```

Tekstura wypełnienia paska prestiżu.

6.28.4.4 contentPage

`VisualElement PrestigeUIManager.contentPage [private]`

Strona contentu odpowiadająca zakładce prestiżu (Content7).

6.28.4.5 ExpContainer

`VisualElement PrestigeUIManager.ExpContainer [private]`

Kontener paska doświadczenia, który po spełnieniu warunków staje się paskiem prestiżu.

6.28.4.6 PrestigeAccept

`Button PrestigeUIManager.PrestigeAccept [private]`

Przycisk potwierdzający wykonanie prestiżu.

6.28.4.7 PrestigeButtonMenu

`Button PrestigeUIManager.PrestigeButtonMenu [private]`

Przycisk w menu głównym otwierający zakładkę prestiżu (btn7).

6.28.4.8 PrestigeCancel

`Button PrestigeUIManager.PrestigeCancel [private]`

Przycisk anulujący okno prestiżu.

6.28.4.9 PrestigeRoot

`VisualElement PrestigeUIManager.PrestigeRoot [private]`

Root okna potwierdzenia prestiżu (popup).

6.28.4.10 rp

`RaptorCore PrestigeUIManager.rp [private]`

Referencja do głównego rdzenia gry ([RaptorCore](#)).

6.28.4.11 ui_ref

VisualElement PrestigeUIManager.ui_ref [private]

Główna referencja do root VisualElement UI.

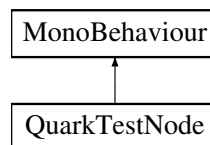
Dokumentacja dla tej klasy została wygenerowana z pliku:

- [PrestigeUIManager.cs](#)

6.29 Dokumentacja klasy QuarkTestNode

MonoBehaviour odpowiedzialny za testowanie struktury [QuarkType](#). Uruchamia zestaw testów jednostkowych sprawdzających poprawność:

Diagram dziedziczenia dla QuarkTestNode



Metody prywatne

- bool [IsApproximatelyEqual](#) ([QuarkType](#) a, [QuarkType](#) b, long mantissaTolerance=[MANTISSA_TOLERANCE](#))
Sprawdza przybliżoną równość dwóch wartości [QuarkType](#). Używane przy operacjach zmiennoprzecinkowych (np. dzielenie).
- void [Start](#) ()
Punkt startowy testów. Uruchamia wszystkie sekcje testowe i wypisuje podsumowanie.
- void [Assert](#) (bool condition, string message)
Własna implementacja asercji testowej. Zlicza testy oraz wypisuje wynik w konsoli Unity.
- void [TestNormalizationAndConstructors](#) ()
Testuje normalizację [QuarkType](#) oraz wszystkie konstruktory.
- void [TestComparisonOperators](#) ()
Testuje operatory porównań (>, <, ==, CompareTo).
- void [TestAddition](#) ()
Testuje dodawanie [QuarkType](#) + [QuarkType](#).
- void [TestSubtraction](#) ()
Testuje odejmowanie [QuarkType](#).
- void [TestMultiplication](#) ()
*Testuje mnożenie [QuarkType](#) * [QuarkType](#).*
- void [TestPrimitiveConversions](#) ()
Testuje konwersje z typów prymitywnych do [QuarkType](#).
- void [TestPrimitiveAddition](#) ()
Testuje dodawanie [QuarkType](#) z typami prymitywnymi.
- void [TestPrimitiveSubtraction](#) ()
Testuje odejmowanie [QuarkType](#) z typami prymitywnymi.
- void [TestPrimitiveMultiplication](#) ()

- Testuje mnożenie [QuarkType](#) z typami prymitywnymi.*
 - void [TestMultiplicationEXT](#) ()
Rozszerzone testy mnożenia (double, ułamki).
 - void [TestDivision](#) ()
Testuje operator dzielenia [QuarkType](#) / [QuarkType](#).
 - void [TestSignedAndRangeArithmetic](#) ()
Testuje znaki liczb oraz operacje na dużych zakresach.
 - void [TestPrecisionLossAndEdgeCases](#) ()
Testuje przypadki graniczne:
 - void [Update](#) ()
Pusta metoda Update – MonoBehaviour wymaga, ale nieużywana.

Atrybuty prywatne

- int [_testsRun](#) = 0
Licznik wszystkich wykonanych testów.
- int [_testsFailed](#) = 0
Licznik testów zakończonych niepowodzeniem.

Statyczne atrybuty prywatne

- const long [MANTISSA_TOLERANCE](#) = 1
Tolerancja porównania mantysy przy testach przybliżonych. Umożliwia różnicę ± 1 w najmniej znaczącej cyfrze ($1e-8$), co kompensuje błędy zaokrągleń.

6.29.1 Opis szczegółowy

MonoBehaviour odpowiedzialny za testowanie struktury [QuarkType](#). Uruchamia zestaw testów jednostkowych sprawdzających poprawność:

- normalizacji
- operatorów arytmetycznych
- porównań
- konwersji prymitywnych
- obsługi zakresów ekstremalnych i utraty precyzji

6.29.2 Dokumentacja funkcji składowych

6.29.2.1 Assert()

```
void QuarkTestNode.Assert (
    bool condition,
    string message) [private]
```

Własna implementacja asercji testowej. Zlicza testy oraz wypisuje wynik w konsoli Unity.

Parametry

<i>condition</i>	Warunek testu.
<i>message</i>	Opis testu.

6.29.2.2 IsApproximatelyEqual()

```
bool QuarkTestNode.IsApproximatelyEqual (
    QuarkType a,
    QuarkType b,
    long mantissaTolerance = MANTISSA_TOLERANCE) [private]
```

Sprawdza przybliżoną równość dwóch wartości [QuarkType](#). Używane przy operacjach zmiennoprzecinkowych (np. dzielenie).

Parametry

<i>a</i>	Pierwsza wartość.
<i>b</i>	Druga wartość.
<i>mantissaTolerance</i>	Dopuszczalna różnica mantysy.

Zwraca

True jeśli wartości są wystarczająco bliskie.

6.29.2.3 Start()

```
void QuarkTestNode.Start () [private]
```

Punkt startowy testów. Uruchamia wszystkie sekcje testowe i wypisuje podsumowanie.

6.29.2.4 TestAddition()

```
void QuarkTestNode.TestAddition () [private]
```

Testuje dodawanie [QuarkType](#) + [QuarkType](#).

6.29.2.5 TestComparisonOperators()

```
void QuarkTestNode.TestComparisonOperators () [private]
```

Testuje operatory porównań (>, <, ==, CompareTo).

6.29.2.6 TestDivision()

```
void QuarkTestNode.TestDivision () [private]
```

Testuje operator dzielenia [QuarkType](#) / [QuarkType](#).

6.29.2.7 TestMultiplication()

```
void QuarkTestNode.TestMultiplication () [private]
```

Testuje mnożenie [QuarkType](#) * [QuarkType](#).

6.29.2.8 TestMultiplicationEXT()

```
void QuarkTestNode.TestMultiplicationEXT () [private]
```

Rozszerzone testy mnożenia (double, ułamki).

6.29.2.9 TestNormalizationAndConstructors()

```
void QuarkTestNode.TestNormalizationAndConstructors () [private]
```

Testuje normalizację [QuarkType](#) oraz wszystkie konstruktory.

6.29.2.10 TestPrecisionLossAndEdgeCases()

```
void QuarkTestNode.TestPrecisionLossAndEdgeCases () [private]
```

Testuje przypadki graniczne:

- overflow wykładnika
- underflow do zera
- utratę precyzji

6.29.2.11 TestPrimitiveAddition()

```
void QuarkTestNode.TestPrimitiveAddition () [private]
```

Testuje dodawanie [QuarkType](#) z typami prymitywnymi.

6.29.2.12 TestPrimitiveConversions()

```
void QuarkTestNode.TestPrimitiveConversions () [private]
```

Testuje konwersje z typów prymitywnych do [QuarkType](#).

6.29.2.13 TestPrimitiveMultiplication()

```
void QuarkTestNode.TestPrimitiveMultiplication () [private]
```

Testuje mnożenie [QuarkType](#) z typami prymitywnymi.

6.29.2.14 TestPrimitiveSubtraction()

```
void QuarkTestNode.TestPrimitiveSubtraction () [private]
```

Testuje odejmowanie [QuarkType](#) z typami prymitywnymi.

6.29.2.15 TestSignedAndRangeArithmetic()

```
void QuarkTestNode.TestSignedAndRangeArithmetic () [private]
```

Testuje znaki liczb oraz operacje na dużych zakresach.

6.29.2.16 TestSubtraction()

```
void QuarkTestNode.TestSubtraction () [private]
```

Testuje odejmowanie [QuarkType](#).

6.29.2.17 Update()

```
void QuarkTestNode.Update () [private]
```

Pusta metoda Update – MonoBehaviour wymaga, ale nieużywana.

6.29.3 Dokumentacja atrybutów składowych

6.29.3.1 __testsFailed

```
int QuarkTestNode._testsFailed = 0 [private]
```

Licznik testów zakończonych niepowodzeniem.

6.29.3.2 `__testsRun`

```
int QuarkTestNode._testsRun = 0 [private]
```

Licznik wszystkich wykonanych testów.

6.29.3.3 `MANTISSA_TOLERANCE`

```
const long QuarkTestNode.MANTISSA_TOLERANCE = 1 [static], [private]
```

Tolerancja porównania mantysy przy testach przybliżonych. Umożliwia różnicę ± 1 w najmniej znaczącej cyfrze ($1e-8$), co kompensuje błędy zaokrągleń.

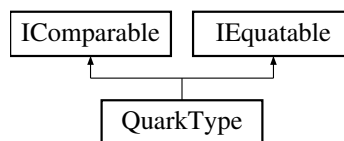
Dokumentacja dla tej klasy została wygenerowana z pliku:

- [QuarkTestNode.cs](#)

6.30 Dokumentacja klasy QuarkType

[QuarkType](#) to wysokozakresowy typ liczbowy oparty o:

Diagram dziedziczenia dla QuarkType



Metody publiczne

- [QuarkType](#) ()
Konstruktor domyślny – zero.
- [QuarkType](#) (long mantissa, long exponent)
Konstruktor bazowy. Przypisuje mantysę i wykładnik, a następnie normalizuje.
- [QuarkType](#) (double doubleValue)
Konstruktor z double. Konwertuje double do postaci (mantissa, exponent).
- [QuarkType](#) (long longValue)
Konstruktor z long.
- [QuarkType](#) (ulong ulongValue)
Konstruktor z ulong.
- void [Normalize](#) ()
Normalizuje mantysę do zakresu [1e8, 1e9) odpowiednio korygując wykładnik.
- override bool [Equals](#) (object obj)
Porównanie obiektowe.
- bool [Equals](#) ([QuarkType](#) other)
Porównanie wartości [QuarkType](#).
- override int [GetHashCode](#) ()
HashCode zgodny z Equals.

- int `CompareTo` (`QuarkType` other)
Porównanie do sortowania i operatorów < >.
- override string `ToString` ()
Zwraca tekstową reprezentację liczby `QuarkType`, używając skrótów (k, M, B, T, itd.) lub notacji wykładniczej dla bardzo dużych wykładników.
- void `SerializeToStream` (BinaryWriter writer)
Serializuje `QuarkType` do strumienia binarnego.
- void `DeserializeFromStream` (BinaryReader reader)
Deserializuje `QuarkType` ze strumienia binarnego.
- `QuarkType` `Ceil` ()
Zaokrągla wartość `QuarkType` w górę do najbliższej liczby całkowitej.
- `QuarkType` `Pow` (double power)
Podnosi `QuarkType` do potęgi zmiennoprzecinkowej.
- `QuarkType` `Floor` ()
Zaokrągla wartość `QuarkType` w dół do najbliższej liczby całkowitej.

Statyczne metody publiczne

- static implicit operator double (`QuarkType` q)
Niejawna konwersja `QuarkType` → double.
- static implicit operator `QuarkType` (int value)
Implicit konwersja z int.
- static implicit operator `QuarkType` (long value)
Implicit konwersja z long.
- static implicit operator `QuarkType` (uint value)
Implicit konwersja z uint.
- static implicit operator `QuarkType` (ulong value)
Implicit konwersja z ulong.
- static implicit operator `QuarkType` (float value)
Implicit konwersja z float.
- static bool operator== (`QuarkType` left, `QuarkType` right)
Operator równości.
- static bool operator!= (`QuarkType` left, `QuarkType` right)
Operator nierówności.
- static `QuarkType` operator- (`QuarkType` a)
Operator negacji.
- static `QuarkType` operator+ (`QuarkType` a, `QuarkType` b)
Dodawanie dwóch `QuarkType`.
- static `QuarkType` operator- (`QuarkType` a, `QuarkType` b)
Odejmowanie dwóch `QuarkType`.
- static `QuarkType` operator* (`QuarkType` a, `QuarkType` b)
Mnożenie dwóch `QuarkType`.
- static `QuarkType` operator/ (`QuarkType` a, `QuarkType` b)
Dzielenie dwóch `QuarkType`.
- static `QuarkType` operator+ (`QuarkType` a, double b)
Dodawanie `QuarkType` i wartości typu double. Double jest konwertowany do `QuarkType` przed operacją.
- static `QuarkType` operator+ (double a, `QuarkType` b)
Dodawanie wartości typu double i `QuarkType`.
- static `QuarkType` operator- (`QuarkType` a, double b)
Odejmowanie wartości double od `QuarkType`.

- static [QuarkType operator-](#) (double a, [QuarkType](#) b)
Odejmowanie [QuarkType](#) od wartości double.
- static [QuarkType operator*](#) ([QuarkType](#) a, double b)
Mnożenie [QuarkType](#) przez wartość double.
- static [QuarkType operator*](#) (double a, [QuarkType](#) b)
Mnożenie wartości double przez [QuarkType](#).
- static [QuarkType operator/](#) ([QuarkType](#) a, double b)
Dzielenie [QuarkType](#) przez wartość double.
- static [QuarkType operator/](#) (double a, [QuarkType](#) b)
Dzielenie wartości double przez [QuarkType](#).
- static [QuarkType operator++](#) ([QuarkType](#) a)
Operator inkrementacji. Zwiększa wartość [QuarkType](#) o 1.

Atrybuty publiczne

- long [Mantissa](#)
*Mantysa liczby (znak + 9 cyfr znaczących). Przechowuje wartość * 1e8.*
- long [Exponent](#)
Wykładnik potęgi dziesięciu.

Właściwości

- bool [IsNegative](#) [get]
Zwraca true jeśli liczba jest ujemna.
- static [QuarkType Zero](#) [get]
Stała zero.
- static [QuarkType One](#) [get]
Stała jeden.
- static [QuarkType NegativeOne](#) [get]
Stała minus jeden.

Metody prywatne

- int [CompareMagnitudeTo](#) ([QuarkType](#) other)
Porównuje wartości bez uwzględniania znaku.

Statyczne atrybuty prywatne

- const int [MantissaLength](#) = 9
Docelowa liczba cyfr mantysy.
- const long [NormalizeDivisor](#) = 100000000
Stała normalizacji (1e8).
- const long [ExponentRangeForPrecision](#) = [MantissaLength](#) + 1
Maksymalna różnica wykładników, przy której dodawanie zachowuje sensowną precyzję.

6.30.1 Opis szczegółowy

`QuarkType` to wysokozakresowy typ liczbowy oparty o:

- mantysę typu `long`
- wykładnik potęgi dziesięciu

Reprezentacja: $\text{value} = (\text{Mantissa} / 1e8) * 10^{\text{Exponent}}$

6.30.2 Dokumentacja konstruktora i destruktora

6.30.2.1 `QuarkType()` [1/5]

```
QuarkType.QuarkType ()
```

Konstruktor domyślny – zero.

6.30.2.2 `QuarkType()` [2/5]

```
QuarkType.QuarkType (  
    long mantissa,  
    long exponent)
```

Konstruktor bazowy. Przypisuje mantysę i wykładnik, a następnie normalizuje.

6.30.2.3 `QuarkType()` [3/5]

```
QuarkType.QuarkType (  
    double doubleValue)
```

Konstruktor z `double`. Konwertuje `double` do postaci (`mantissa`, `exponent`).

6.30.2.4 `QuarkType()` [4/5]

```
QuarkType.QuarkType (  
    long longValue)
```

Konstruktor z `long`.

6.30.2.5 `QuarkType()` [5/5]

```
QuarkType.QuarkType (  
    ulong ulongValue)
```

Konstruktor z `ulong`.

6.30.3 Dokumentacja funkcji składowych

6.30.3.1 Ceil()

`QuarkType` `QuarkType.Ceil ()`

Zaokrągla wartość `QuarkType` w górę do najbliższej liczby całkowitej.

6.30.3.2 CompareMagnitudeTo()

```
int QuarkType.CompareMagnitudeTo (  
    QuarkType other) [private]
```

Porównuje wartości bez uwzględniania znaku.

6.30.3.3 CompareTo()

```
int QuarkType.CompareTo (  
    QuarkType other)
```

Porównanie do sortowania i operatorów `<` `>`.

6.30.3.4 DeserializeFromStream()

```
void QuarkType.DeserializeFromStream (  
    BinaryReader reader)
```

Deserializuje `QuarkType` ze strumienia binarnego.

6.30.3.5 Equals() [1/2]

```
override bool QuarkType.Equals (  
    object obj)
```

Porównanie obiektowe.

6.30.3.6 Equals() [2/2]

```
bool QuarkType.Equals (  
    QuarkType other)
```

Porównanie wartości `QuarkType`.

6.30.3.7 Floor()

`QuarkType` `QuarkType.Floor ()`

Zaokrągla wartość `QuarkType` w dół do najbliższej liczby całkowitej.

6.30.3.8 GetHashCode()

```
override int QuarkType.GetHashCode ()
```

HashCode zgodny z Equals.

6.30.3.9 Normalize()

```
void QuarkType.Normalize ()
```

Normalizuje mantysę do zakresu [1e8, 1e9) odpowiednio korygując wykładnik.

6.30.3.10 operator double()

```
implicit QuarkType.operator double (  
    QuarkType q) [static]
```

Niejawna konwersja `QuarkType` → `double`.

6.30.3.11 operator QuarkType() [1/5]

```
implicit QuarkType.operator QuarkType (  
    float value) [static]
```

Implicit konwersja z `float`.

6.30.3.12 operator QuarkType() [2/5]

```
implicit QuarkType.operator QuarkType (  
    int value) [static]
```

Implicit konwersja z `int`.

6.30.3.13 operator QuarkType() [3/5]

```
implicit QuarkType.operator QuarkType (  
    long value) [static]
```

Implicit konwersja z `long`.

6.30.3.14 operator QuarkType() [4/5]

```
implicit QuarkType.operator QuarkType (  
    uint value) [static]
```

Implicit konwersja z `uint`.

6.30.3.15 operator QuarkType() [5/5]

```
implicit QuarkType.operator QuarkType (
    ulong value) [static]
```

Implicit konwersja z ulong.

6.30.3.16 operator"!=()

```
bool QuarkType.operator!= (
    QuarkType left,
    QuarkType right) [static]
```

Operator nierówności.

6.30.3.17 operator*() [1/3]

```
QuarkType QuarkType.operator* (
    double a,
    QuarkType b) [static]
```

Mnożenie wartości double przez QuarkType.

6.30.3.18 operator*() [2/3]

```
QuarkType QuarkType.operator* (
    QuarkType a,
    double b) [static]
```

Mnożenie QuarkType przez wartość double.

6.30.3.19 operator*() [3/3]

```
QuarkType QuarkType.operator* (
    QuarkType a,
    QuarkType b) [static]
```

Mnożenie dwóch QuarkType.

6.30.3.20 operator+() [1/3]

```
QuarkType QuarkType.operator+ (
    double a,
    QuarkType b) [static]
```

Dodawanie wartości typu double i QuarkType.

6.30.3.21 operator+() [2/3]

```
QuarkType QuarkType.operator+ (  
    QuarkType a,  
    double b) [static]
```

Dodawanie [QuarkType](#) i wartości typu double. Double jest konwertowany do [QuarkType](#) przed operacją.

6.30.3.22 operator+() [3/3]

```
QuarkType QuarkType.operator+ (  
    QuarkType a,  
    QuarkType b) [static]
```

Dodawanie dwóch [QuarkType](#).

6.30.3.23 operator++()

```
QuarkType QuarkType.operator++ (  
    QuarkType a) [static]
```

Operator inkrementacji. Zwiększa wartość [QuarkType](#) o 1.

6.30.3.24 operator-() [1/4]

```
QuarkType QuarkType.operator- (  
    double a,  
    QuarkType b) [static]
```

Odejmowanie [QuarkType](#) od wartości double.

6.30.3.25 operator-() [2/4]

```
QuarkType QuarkType.operator- (  
    QuarkType a) [static]
```

Operator negacji.

6.30.3.26 operator-() [3/4]

```
QuarkType QuarkType.operator- (  
    QuarkType a,  
    double b) [static]
```

Odejmowanie wartości double od [QuarkType](#).

6.30.3.27 operator-() [4/4]

```
QuarkType QuarkType.operator- (
    QuarkType a,
    QuarkType b) [static]
```

Odejmowanie dwóch [QuarkType](#).

6.30.3.28 operator/() [1/3]

```
QuarkType QuarkType.operator/ (
    double a,
    QuarkType b) [static]
```

Dzielenie wartości double przez [QuarkType](#).

6.30.3.29 operator/() [2/3]

```
QuarkType QuarkType.operator/ (
    QuarkType a,
    double b) [static]
```

Dzielenie [QuarkType](#) przez wartość double.

6.30.3.30 operator/() [3/3]

```
QuarkType QuarkType.operator/ (
    QuarkType a,
    QuarkType b) [static]
```

Dzielenie dwóch [QuarkType](#).

6.30.3.31 operator==(())

```
bool QuarkType.operator==(
    QuarkType left,
    QuarkType right) [static]
```

Operator równości.

6.30.3.32 Pow()

```
QuarkType QuarkType.Pow (
    double power)
```

Podnosi [QuarkType](#) do potęgi zmiennoprzecinkowej.

6.30.3.33 SerializeToStream()

```
void QuarkType.SerializeToStream (  
    BinaryWriter writer)
```

Serializuje [QuarkType](#) do strumienia binarnego.

6.30.3.34 ToString()

```
override string QuarkType.ToString ()
```

Zwraca tekstową reprezentację liczby [QuarkType](#), używając skrótów (k, M, B, T, itd.) lub notacji wykładniczej dla bardzo dużych wykładników.

6.30.4 Dokumentacja atrybutów składowych

6.30.4.1 Exponent

```
long QuarkType.Exponent
```

Wykładnik potęgi dziesięciu.

6.30.4.2 ExponentRangeForPrecision

```
const long QuarkType.ExponentRangeForPrecision = MantissaLength + 1 [static], [private]
```

Maksymalna różnica wykładników, przy której dodawanie zachowuje sensowną precyzję.

6.30.4.3 Mantissa

```
long QuarkType.Mantissa
```

Mantysa liczby (znak + 9 cyfr znaczących). Przechowuje wartość * 1e8.

6.30.4.4 MantissaLength

```
const int QuarkType.MantissaLength = 9 [static], [private]
```

Docelowa liczba cyfr mantysy.

6.30.4.5 NormalizeDivisor

```
const long QuarkType.NormalizeDivisor = 100000000 [static], [private]
```

Stała normalizacji (1e8).

6.30.5 Dokumentacja właściwości

6.30.5.1 IsNegative

```
bool QuarkType.IsNegative [get]
```

Zwraca true jeśli liczba jest ujemna.

6.30.5.2 NegativeOne

```
QuarkType QuarkType.NegativeOne [static], [get]
```

Stała minus jeden.

6.30.5.3 One

```
QuarkType QuarkType.One [static], [get]
```

Stała jeden.

6.30.5.4 Zero

```
QuarkType QuarkType.Zero [static], [get]
```

Stała zero.

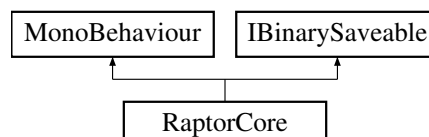
Dokumentacja dla tej klasy została wygenerowana z pliku:

- [QuarkType.cs](#)

6.31 Dokumentacja klasy RaptorCore

Główny rdzeń gry. Odpowiada za: walutę, kliknięcia, ticki idle, prestiż, serializację stanu gry.

Diagram dziedziczenia dla RaptorCore



Metody publiczne

- void [AddCurrency](#) ([QuarkType](#) currency)
Dodaje walutę do aktualnego zasobu.
- void [SubCurrency](#) ([QuarkType](#) currency)
*Odejmuje walutę z uwzględnieniem perka *alwaysWinner*.*
- void [SerializeToStream](#) ([BinaryWriter](#) writer)
Serializuje stan rdzenia gry do strumienia binarnego.
- void [DeserializeFromStream](#) ([BinaryReader](#) reader)
Odczytuje stan rdzenia gry ze strumienia binarnego.

Atrybuty publiczne

- [CharacterClass](#) [characterClass](#)
Klasa postaci – statystyki, talenty, modyfikatory.
- [IdleManager](#) [idleManager](#)
Manager trybu idle i fabryk.
- [QuarkType](#) [SkillMultiplier](#) = 0
Mnożnik pochodzący ze skilli.
- long [AutoPrestige](#) = 0
Punkty prestiżu drzewka Automatron.
- long [JackPrestige](#) = 0
Punkty prestiżu drzewka Jack of all Clicks.
- long [OneClickPrestige](#) = 0
Punkty prestiżu drzewka One Click Army.
- long [PrestigeCount](#) = 0
- double [Gold](#) = 0
Zdobyte złoto.
- [QuarkType](#) [potionClickBonus1](#) = 0
Bonus kliknięcia z mikstury typu 1.
- [QuarkType](#) [potionClickBonus2](#) = 0
Bonus kliknięcia z mikstury typu 2.
- int [clickingDebuff](#) = 1
Debuff kliknięć przy multitaskingu.

Funkcje pakietu

- void [ResetCore](#) ()
Resetuje stan rdzenia gry po wykonaniu prestiżu.

Właściwości

- int [SaveKey](#) [get]
Klucz zapisu używany w systemie zapisu binarnego.
- [ResourceManager](#) [ResourceManager](#) [get, private set]
Manager zasobów i fabryk.
- [SellingManager](#) [SellManager](#) [get, private set]
Manager sprzedaży zasobów.
- [SkillManager](#) [SkillManager](#) [get, private set]
Manager umiejętności i skill-checków.
- [QuarkType](#) [Currency](#) [get, set]
Aktualna waluta gracza dla aktywnego zasobu.

Typy prywatne

- enum `ClickSource` { `None` , `Mouse` , `Space` , `Enter` }
Źródło kliknięcia (mysz / klawiatura).

Metody prywatne

- void `Awake` ()
Inicjalizacja komponentów i managerów przed `Start()`.
- void `Start` ()
Inicjalizacja systemów gry oraz wczytanie zapisu.
- void `OnMouseDown` ()
Obsługa kliknięcia myszą na obiekcie.
- void `click` ()
Główna logika pojedynczego kliknięcia (normalnego lub krytycznego).
- void `FixedUpdate` ()
Tick logiki idle, autosave oraz pasywnych efektów.
- void `Update` ()
Obsługa klawiatury (spacja / enter).
- void `PerformClick` ()
Wykonuje kliknięcie po sprawdzeniu dostępności fabryki i stanu skill-checka.

Atrybuty prywatne

- Animator `anim`
Animator obsługujący animacje klikania.
- GameObject `skillCheck`
Obiekt UI używany do skill-checków.
- GameObject `CritGenerator`
Generator wizualizacji trafień krytycznych.
- `CritVisualGenerator` `critGen`
Komponent generujący wizualny efekt krytyka.
- AudioSource `audioSource`
Źródło dźwięku zwykłego kliknięcia.
- AudioSource `critSource`
Źródło dźwięku trafienia krytycznego.
- `QuarkType` `CBasevalue` = 1
Bazowa wartość kliknięcia.
- `QuarkType` `CMultiplier` = 1
Mnożnik kliknięcia.
- `QuarkType` `GBasevalue` = 0
Bazowa wartość produkcji idle.
- `QuarkType` `GMultiplier` = 1
Mnożnik produkcji idle.
- int `tickCount` = 0
Licznik ticków `FixedUpdate`.
- `ClickSource` `activeClickSource` = `ClickSource.None`
Aktualne źródło kliknięcia.
- float `sourceBlockEndTime` = 0f
Czas zakończenia blokady źródła kliknięcia.
- float `clickCooldown` = 0.5f
Cooldown pomiędzy kliknięciami.

6.31.1 Opis szczegółowy

Główny rdzeń gry. Odpowiada za: walutę, kliknięcia, ticki idle, prestiż, serializację stanu gry.

6.31.2 Dokumentacja składowych wyliczanych

6.31.2.1 ClickSource

```
enum RaptorCore.ClickSource [private]
```

Źródło kliknięcia (mysz / klawiatura).

Wartości wyliczeń

None	
Mouse	
Space	
Enter	

6.31.3 Dokumentacja funkcji składowych

6.31.3.1 AddCurrency()

```
void RaptorCore.AddCurrency (  
    QuarkType currency)
```

Dodaje walutę do aktualnego zasobu.

6.31.3.2 Awake()

```
void RaptorCore.Awake () [private]
```

Inicjalizacja komponentów i managerów przed [Start\(\)](#).

6.31.3.3 click()

```
void RaptorCore.click () [private]
```

Główna logika pojedynczego kliknięcia (normalnego lub krytycznego).

6.31.3.4 DeserializeFromStream()

```
void RaptorCore.DeserializeFromStream (  
    BinaryReader reader)
```

Odczytuje stan rdzenia gry ze strumienia binarnego.

Implementuje [IBinarySaveable](#).

6.31.3.5 FixedUpdate()

```
void RaptorCore.FixedUpdate () [private]
```

Tick logiki idle, autosave oraz pasywnych efektów.

6.31.3.6 OnMouseDown()

```
void RaptorCore.OnMouseDown () [private]
```

Obsługa kliknięcia myszą na obiekcie.

6.31.3.7 PerformClick()

```
void RaptorCore.PerformClick () [private]
```

Wykonuje kliknięcie po sprawdzeniu dostępności fabryki i stanu skill-checka.

6.31.3.8 ResetCore()

```
void RaptorCore.ResetCore () [package]
```

Resetuje stan rdzenia gry po wykonaniu prestiżu.

6.31.3.9 SerializeToStream()

```
void RaptorCore.SerializeToStream (  
    BinaryWriter writer)
```

Serializuje stan rdzenia gry do strumienia binarnego.

Implementuje [IBinarySaveable](#).

6.31.3.10 Start()

```
void RaptorCore.Start () [private]
```

Inicjalizacja systemów gry oraz wczytanie zapisu.

6.31.3.11 SubCurrency()

```
void RaptorCore.SubCurrency (  
    QuarkType currency)
```

Odejmuje walutę z uwzględnieniem perka alwaysWinner.

6.31.3.12 Update()

```
void RaptorCore.Update () [private]
```

Obsługa klawiatury (spacja / enter).

6.31.4 Dokumentacja atrybutów składowych

6.31.4.1 activeClickSource

```
ClickSource RaptorCore.activeClickSource = ClickSource.None [private]
```

Aktualne źródło kliknięcia.

6.31.4.2 anim

```
Animator RaptorCore.anim [private]
```

Animator obsługujący animacje klikania.

6.31.4.3 audioSource

```
AudioSource RaptorCore.audioSource [private]
```

Źródło dźwięku zwykłego kliknięcia.

6.31.4.4 AutoPrestige

```
long RaptorCore.AutoPrestige = 0
```

Punkty prestiżu drzewka Automatron.

6.31.4.5 CBasevalue

```
QuarkType RaptorCore.CBasevalue = 1 [private]
```

Bazowa wartość kliknięcia.

6.31.4.6 characterClass

```
CharacterClass RaptorCore.characterClass
```

Klasa postaci – statystyki, talenty, modyfikatory.

6.31.4.7 clickCooldown

```
float RaptorCore.clickCooldown = 0.5f [private]
```

Cooldown pomiędzy kliknięciami.

6.31.4.8 clickingDebuff

```
int RaptorCore.clickingDebuff = 1
```

Debuff kliknięć przy multitaskingu.

6.31.4.9 CMultiplier

```
QuarkType RaptorCore.CMultiplier = 1 [private]
```

Mnożnik kliknięcia.

6.31.4.10 critGen

```
CritVisualGenerator RaptorCore.critGen [private]
```

Komponent generujący wizualny efekt krytyka.

6.31.4.11 CritGenerator

```
GameObject RaptorCore.CritGenerator [private]
```

Generator wizualizacji trafień krytycznych.

6.31.4.12 critSource

```
AudioSource RaptorCore.critSource [private]
```

Źródło dźwięku trafienia krytycznego.

6.31.4.13 GBasevalue

```
QuarkType RaptorCore.GBasevalue = 0 [private]
```

Bazowa wartość produkcji idle.

6.31.4.14 GMultiplier

```
QuarkType RaptorCore.GMultiplier = 1 [private]
```

Mnożnik produkcji idle.

6.31.4.15 Gold

```
double RaptorCore.Gold = 0
```

Zdobyte złoto.

6.31.4.16 idleManager

```
IdleManager RaptorCore.idleManager
```

Manager trybu idle i fabryk.

6.31.4.17 JackPrestige

```
long RaptorCore.JackPrestige = 0
```

Punkty prestiżu drzewka Jack of all Clicks.

6.31.4.18 OneClickPrestige

```
long RaptorCore.OneClickPrestige = 0
```

Punkty prestiżu drzewka One Click Army.

6.31.4.19 potionClickBonus1

```
QuarkType RaptorCore.potionClickBonus1 = 0
```

Bonus kliknięcia z mikstury typu 1.

6.31.4.20 potionClickBonus2

```
QuarkType RaptorCore.potionClickBonus2 = 0
```

Bonus kliknięcia z mikstury typu 2.

6.31.4.21 PrestigeCount

```
long RaptorCore.PrestigeCount = 0
```

Liczba wykonanych prestiży.

6.31.4.22 skillCheck

```
GameObject RaptorCore.skillCheck [private]
```

Obiekt UI używany do skill-checków.

6.31.4.23 SkillMultiplier

`QuarkType` RaptorCore.SkillMultiplier = 0

Mnożnik pochodzący ze skilli.

6.31.4.24 sourceBlockEndTime

`float` RaptorCore.sourceBlockEndTime = 0f [private]

Czas zakończenia blokady źródła kliknięcia.

6.31.4.25 tickCount

`int` RaptorCore.tickCount = 0 [private]

Licznik ticków FixedUpdate.

6.31.5 Dokumentacja właściwości

6.31.5.1 Currency

`QuarkType` RaptorCore.Currency [get], [set]

Aktualna waluta gracza dla aktywnego zasobu.

6.31.5.2 ResourceManager

`ResourceManager` RaptorCore.ResourceManager [get], [private set]

Manager zasobów i fabryk.

6.31.5.3 SaveKey

`int` RaptorCore.SaveKey [get]

Klucz zapisu używany w systemie zapisu binarnego.

Implementuje [IBinarySaveable](#).

6.31.5.4 SellManager

`SellingManager` RaptorCore.SellManager [get], [private set]

Manager sprzedaży zasobów.

6.31.5.5 SkillManager

`SkillManager` `RaptorCore.SkillManager` [get], [private set]

Manager umiejętności i skill-checków.

Dokumentacja dla tej klasy została wygenerowana z pliku:

- [RaptorCore.cs](#)

6.32 Dokumentacja klasy Resource

Reprezentuje pojedynczy zasób w grze. Zawiera aktualną wartość, limit oraz modyfikatory limitu.

Metody publiczne

- [Resource](#) ()
Tworzy nowy zasób o podanej nazwie i nazwie wyświetlanej. Inicjalizuje wartości domyślne limitów.
- [Resource](#) (string _name, string _displayName)
Tworzy nowy zasób o podanej nazwie i nazwie wyświetlanej. Inicjalizuje wartości domyślne limitów.
- void [SerializeToStream](#) (BinaryWriter writer)
Zapisuje dane zasobu do strumienia binarnego. Używane przez system zapisu gry.
- void [DeserializeFromStream](#) (BinaryReader reader)
Odczytuje dane zasobu ze strumienia binarnego. Przywraca pełny stan zasobu.

Atrybuty publiczne

- string [name](#)
Nazwa zasobu używana w kodzie (unikalny identyfikator).
- string [displayName](#)
Nazwa wyświetlana w interfejsie użytkownika.
- [QuarkType](#) [BaseLimit](#)
Bazowy limit zasobu bez żadnych modyfikatorów.
- [QuarkType](#) [LvlBoost](#)
Mnożnik limitu wynikający z poziomu, fabryk lub progresji.
- [QuarkType](#) [SkillBoost](#)
Dodatkowy bonus do limitu wynikający z umiejętności lub perków.

Właściwości

- [QuarkType](#) [value](#) [get, set]
Aktualna ilość zasobu. Wartość jest automatycznie:
- [QuarkType](#) [Limit](#) [get]
Maksymalna możliwa ilość zasobu. Obliczana dynamicznie na podstawie:

Atrybuty prywatne

- [QuarkType](#) [_value](#)
Wewnętrzne pole przechowujące aktualną wartość zasobu.

6.32.1 Opis szczegółowy

Reprezentuje pojedynczy zasób w grze. Zawiera aktualną wartość, limit oraz modyfikatory limitu.

6.32.2 Dokumentacja konstruktora i destruktora

6.32.2.1 Resource() [1/2]

```
Resource.Resource ()
```

6.32.2.2 Resource() [2/2]

```
Resource.Resource (  
    string _name,  
    string _displayName)
```

Tworzy nowy zasób o podanej nazwie i nazwie wyświetlanej. Inicjalizuje wartości domyślne limitów.

Parametry

<code>_name</code>	Unikalna nazwa techniczna zasobu
<code>_displayName</code>	Nazwa wyświetlana w UI

6.32.3 Dokumentacja funkcji składowych

6.32.3.1 DeserializeFromStream()

```
void Resource.DeserializeFromStream (  
    BinaryReader reader)
```

Odczytuje dane zasobu ze strumienia binarnego. Przywraca pełny stan zasobu.

Parametry

<code>reader</code>	Strumień odczytu binarnego
---------------------	----------------------------

6.32.3.2 SerializeToStream()

```
void Resource.SerializeToStream (  
    BinaryWriter writer)
```

Zapisuje dane zasobu do strumienia binarnego. Używane przez system zapisu gry.

Parametry

<i>writer</i>	Strumień zapisu binarnego
---------------	---------------------------

6.32.4 Dokumentacja atrybutów składowych

6.32.4.1 _value

```
QuarkType Resource._value [private]
```

Wewnętrzne pole przechowujące aktualną wartość zasobu.

6.32.4.2 BaseLimit

```
QuarkType Resource.BaseLimit
```

Bazowy limit zasobu bez żadnych modyfikatorów.

6.32.4.3 displayName

```
string Resource.displayName
```

Nazwa wyświetlana w interfejsie użytkownika.

6.32.4.4 LvlBoost

```
QuarkType Resource.LvlBoost
```

Mnożnik limitu wynikający z poziomu, fabryk lub progresji.

6.32.4.5 name

```
string Resource.name
```

Nazwa zasobu używana w kodzie (unikalny identyfikator).

6.32.4.6 SkillBoost

```
QuarkType Resource.SkillBoost
```

Dodatkowy bonus do limitu wynikający z umiejętności lub perków.

6.32.5 Dokumentacja właściwości

6.32.5.1 Limit

`QuarkType Resource.Limit [get]`

Maksymalna możliwa ilość zasobu. Obliczana dynamicznie na podstawie:

- bazowego limitu
- bonusu z poziomu
- bonusu ze skilli

6.32.5.2 value

`QuarkType Resource.value [get], [set]`

Aktualna ilość zasobu. Wartość jest automatycznie:

- obcinana do zera
- ograniczana przez [Limit](#)

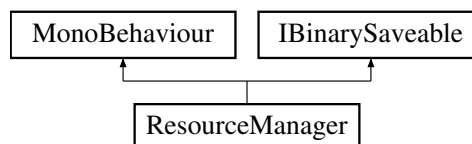
Dokumentacja dla tej klasy została wygenerowana z pliku:

- [Resource.cs](#)

6.33 Dokumentacja klasy ResourceManager

Zarządza wszystkimi zasobami w grze. Odpowiada za ich wartości, limity, zapis/odczyt, aktualizację UI oraz integrację z fabrykami.

Diagram dziedziczenia dla ResourceManager



Metody publiczne

- void [Initialize](#) ([IdleManager](#) _idleManager)
Inicjalizuje manager zasobów.
- void [LoadAllResources](#) ()
Ładuje wszystkie zasoby z zapisanych wartości.
- void [RegisterResource](#) ([Resource](#) resource)
Rejestruje nowy zasób w managerze.
- void [AddResource](#) (string resource, [QuarkType](#) amount)
Dodaje ilość do danego zasobu.
- [QuarkType](#) [GetResourceValue](#) (string resource)
Zwraca aktualną wartość zasobu.
- [QuarkType](#) [GetResourceValueDirect](#) (string resourceName)
Zwraca wartość zasobu bezpośrednio z pól zapisowych.
- bool [HasResource](#) (string resource, [QuarkType](#) amount)
Sprawdza czy gracz posiada wystarczającą ilość zasobu.
- bool [RemoveResource](#) (string resource, [QuarkType](#) amount)
Usuwa ilość zasobu jeśli jest dostępna.
- void [SetCurrentResource](#) (string resource)
Ustawia aktualnie wybrany zasób.
- void [SetResourceValueDirect](#) (string resourceName, [QuarkType](#) value)
Ustawia wartość zasobu bezpośrednio (debug / save).
- void [SetLvlBoostResource](#) (string resource, [QuarkType](#) limitvalue)
Mnoży boost poziomu dla zasobu.
- [QuarkType](#) [GetLimitResource](#) (string resource)
Zwraca aktualny limit zasobu.
- [QuarkType](#) [GetBaseLimitResource](#) (string resource)
Zwraca bazowy limit zasobu.
- void [IncrementSkillBoostResourceAll](#) (bool mode)
Dodaje lub usuwa boost skilla dla wszystkich zasobów.
- void [MullLimitResourceAll](#) ([QuarkType](#) MullValue)
Mnoży limit wszystkich zasobów.
- void [IncrementSkillBoostResource](#) (string resource, [QuarkType](#) AddValue, bool mode)
Dodaje lub usuwa boost skilla dla pojedynczego zasobu.
- void [SaveResourceValues](#) ()
Zapisuje wszystkie wartości zasobów do pól.
- bool [CanClickCurrentResource](#) ()
Sprawdza czy można kliknąć aktualny zasób.
- [Factory](#) [GetCurrentFactory](#) ()
Zwraca fabrykę przypisaną do aktualnego zasobu.
- void [SerializeToStream](#) ([BinaryWriter](#) writer)
Serializuje stan [ResourceManagera](#).
- void [DeserializeFromStream](#) ([BinaryReader](#) reader)
Deserializuje stan [ResourceManagera](#).

Atrybuty publiczne

- `QuarkType resource1Value` = 0
Aktualna wartość Resource1.
- `QuarkType resource2Value` = 0
Aktualna wartość Resource2.
- `QuarkType resource3Value` = 0
Aktualna wartość Resource3.
- `QuarkType resource4Value` = 0
Aktualna wartość Resource4.
- `QuarkType resource5Value` = 0
Aktualna wartość Resource5.
- `QuarkType resource6Value` = 0
Aktualna wartość Resource6.
- `QuarkType resource1Limit` = 1000
Podstawowy limit Resource1.
- `QuarkType resource2Limit` = 2500
Podstawowy limit Resource2.
- `QuarkType resource3Limit` = 5000
Podstawowy limit Resource3.
- `QuarkType resource4Limit` = 10000
Podstawowy limit Resource4.
- `QuarkType resource5Limit` = 25000
Podstawowy limit Resource5.
- `QuarkType resource6Limit` = 50000
Podstawowy limit Resource6.
- `string currentResource` = "Resource1"
Aktualnie wybrany zasób w UI.

Funkcje pakietu

- `void ResetResources ()`
Resetuje wszystkie zasoby do wartości początkowych (prestige).

Właściwości

- `int SaveKey` [get]
Klucz zapisu używany przez system zapisu binarnego.

Metody prywatne

- `void LoadResource (string name)`
Ładuje pojedynczy zasób na podstawie pól (refleksja).
- `void SaveResource (string name)`
Zapisuje wartości pojedynczego zasobu do odpowiednich pól ResourceManager.

Atrybuty prywatne

- [IdleManager](#) `idleManager`
Referencja do systemu idle (fabryk).
- `Dictionary< string, Resource > resources = new Dictionary<string, Resource>()`
Słownik wszystkich zarejestrowanych zasobów według nazwy.

6.33.1 Opis szczegółowy

Zarządza wszystkimi zasobami w grze. Odpowiada za ich wartości, limity, zapis/odczyt, aktualizację UI oraz integrację z fabrykami.

6.33.2 Dokumentacja funkcji składowych

6.33.2.1 AddResource()

```
void ResourceManager.AddResource (  
    string resource,  
    QuarkType amount)
```

Dodaje ilość do danego zasobu.

Parametry

<i>resource</i>	Nazwa zasobu
<i>amount</i>	Ilość do dodania

6.33.2.2 CanClickCurrentResource()

```
bool ResourceManager.CanClickCurrentResource ()
```

Sprawdza czy można kliknąć aktualny zasób.

6.33.2.3 DeserializeFromStream()

```
void ResourceManager.DeserializeFromStream (  
    BinaryReader reader)
```

Deserializuje stan ResourceManager.

Implementuje [IBinarySaveable](#).

6.33.2.4 GetBaseLimitResource()

```
QuarkType ResourceManager.GetBaseLimitResource (  
    string resource)
```

Zwraca bazowy limit zasobu.

6.33.2.5 GetCurrentFactory()

```
Factory ResourceManager.GetCurrentFactory ()
```

Zwraca fabrykę przypisaną do aktualnego zasobu.

6.33.2.6 GetLimitResource()

```
QuarkType ResourceManager.GetLimitResource (  
    string resource)
```

Zwraca aktualny limit zasobu.

6.33.2.7 GetResourceValue()

```
QuarkType ResourceManager.GetResourceValue (  
    string resource)
```

Zwraca aktualną wartość zasobu.

6.33.2.8 GetResourceValueDirect()

```
QuarkType ResourceManager.GetResourceValueDirect (  
    string resourceName)
```

Zwraca wartość zasobu bezpośrednio z pól zapisowych.

6.33.2.9 HasResource()

```
bool ResourceManager.HasResource (  
    string resource,  
    QuarkType amount)
```

Sprawdza czy gracz posiada wystarczającą ilość zasobu.

6.33.2.10 IncrementSkillBoostResource()

```
void ResourceManager.IncrementSkillBoostResource (  
    string resource,  
    QuarkType AddValue,  
    bool mode)
```

Dodaje lub usuwa boost skilla dla pojedynczego zasobu.

6.33.2.11 IncrementSkillBoostResourceAll()

```
void ResourceManager.IncrementSkillBoostResourceAll (  
    bool mode)
```

Dodaje lub usuwa boost skilla dla wszystkich zasobów.

6.33.2.12 Initialize()

```
void ResourceManager.Initialize (  
    IdleManager _idleManager)
```

Inicjalizuje manager zasobów.

Parametry

<code>_idleManager</code>	Referencja do IdleManagera
---------------------------	----------------------------

6.33.2.13 LoadAllResources()

```
void ResourceManager.LoadAllResources ()
```

Ładuje wszystkie zasoby z zapisanych wartości.

6.33.2.14 LoadResource()

```
void ResourceManager.LoadResource (  
    string name) [private]
```

Ładuje pojedynczy zasób na podstawie pól (refleksja).

Parametry

<code>name</code>	Nazwa zasobu
-------------------	--------------

6.33.2.15 MullLimitResourceAll()

```
void ResourceManager.MullLimitResourceAll (  
    QuarkType MullValue)
```

Mnoży limit wszystkich zasobów.

6.33.2.16 RegisterResource()

```
void ResourceManager.RegisterResource (  
    Resource resource)
```

Rejestruje nowy zasób w managerze.

Parametry

<code>resource</code>	Zasób do dodania
-----------------------	------------------

6.33.2.17 RemoveResource()

```
bool ResourceManager.RemoveResource (
    string resource,
    QuarkType amount)
```

Usuwa ilość zasobu jeśli jest dostępna.

6.33.2.18 ResetResources()

```
void ResourceManager.ResetResources () [package]
```

Resetuje wszystkie zasoby do wartości początkowych (prestige).

6.33.2.19 SaveResource()

```
void ResourceManager.SaveResource (
    string name) [private]
```

Zapisuje wartości pojedynczego zasobu do odpowiednich pól ResourceManager.

Parametry

<i>name</i>	Nazwa zasobu, np. "Resource1", "Resource2", ...
-------------	---

Funkcja używa refleksji, aby znaleźć pola odpowiadające zasobowi w klasie [ResourceManager](#) i zaktualizować je wartościami z obiektu [Resource](#) w słowniku 'resources'.
Jeśli zasób nie istnieje w słowniku, funkcja nic nie robi.

6.33.2.20 SaveResourceValues()

```
void ResourceManager.SaveResourceValues ()
```

Zapisuje wszystkie wartości zasobów do pól.

6.33.2.21 SerializeToStream()

```
void ResourceManager.SerializeToStream (
    BinaryWriter writer)
```

Serializuje stan ResourceManager.

Implementuje [IBinarySaveable](#).

6.33.2.22 SetCurrentResource()

```
void ResourceManager.SetCurrentResource (
    string resource)
```

Ustawia aktualnie wybrany zasób.

6.33.2.23 SetLvlBoostResource()

```
void ResourceManager.SetLvlBoostResource (
    string resource,
    QuarkType limitvalue)
```

Mnoży boost poziomu dla zasobu.

6.33.2.24 SetResourceValueDirect()

```
void ResourceManager.SetResourceValueDirect (
    string resourceName,
    QuarkType value)
```

Ustawia wartość zasobu bezpośrednio (debug / save).

6.33.3 Dokumentacja atrybutów składowych

6.33.3.1 currentResource

```
string ResourceManager.currentResource = "Resource1"
```

Aktualnie wybrany zasób w UI.

6.33.3.2 idleManager

```
IdleManager ResourceManager.idleManager [private]
```

Referencja do systemu idle (fabryk).

6.33.3.3 resource1Limit

```
QuarkType ResourceManager.resource1Limit = 1000
```

Podstawowy limit Resource1.

6.33.3.4 resource1Value

```
QuarkType ResourceManager.resource1Value = 0
```

Aktualna wartość Resource1.

6.33.3.5 resource2Limit

```
QuarkType ResourceManager.resource2Limit = 2500
```

Podstawowy limit Resource2.

6.33.3.6 resource2Value

`QuarkType ResourceManager.resource2Value = 0`

Aktualna wartość Resource2.

6.33.3.7 resource3Limit

`QuarkType ResourceManager.resource3Limit = 5000`

Podstawowy limit Resource3.

6.33.3.8 resource3Value

`QuarkType ResourceManager.resource3Value = 0`

Aktualna wartość Resource3.

6.33.3.9 resource4Limit

`QuarkType ResourceManager.resource4Limit = 10000`

Podstawowy limit Resource4.

6.33.3.10 resource4Value

`QuarkType ResourceManager.resource4Value = 0`

Aktualna wartość Resource4.

6.33.3.11 resource5Limit

`QuarkType ResourceManager.resource5Limit = 25000`

Podstawowy limit Resource5.

6.33.3.12 resource5Value

`QuarkType ResourceManager.resource5Value = 0`

Aktualna wartość Resource5.

6.33.3.13 resource6Limit

`QuarkType ResourceManager.resource6Limit = 50000`

Podstawowy limit Resource6.

6.33.3.14 resource6Value

`QuarkType` ResourceManager.resource6Value = 0

Aktualna wartość Resource6.

6.33.3.15 resources

`Dictionary<string, Resource>` ResourceManager.resources = new `Dictionary<string, Resource>()` [private]

Słownik wszystkich zarejestrowanych zasobów według nazwy.

6.33.4 Dokumentacja właściwości

6.33.4.1 SaveKey

`int` ResourceManager.SaveKey [get]

Klucz zapisu używany przez system zapisu binarnego.

Implementuje `IBinarySaveable`.

Dokumentacja dla tej klasy została wygenerowana z pliku:

- [ResourceManager.cs](#)

6.34 Dokumentacja klasy ResourceShopManager

Manager UI dla sklepu zasobów. Odpowiada za aktualizację wyświetlanych wartości zasobów, obsługę przycisków sprzedaży i suwaka procentowego sprzedaży.

Metody publiczne

- `ResourceShopManager` (`VisualElement` root, `RaptorCore` core, `CharacterClass` charClass, `AudioSource` audio)
Konstruktor `ResourceShopManager`.
- `void Initialize ()`
Inicjalizuje wszystkie elementy UI i przypisuje zdarzenia do przycisków.
- `void Update ()`
Aktualizuje UI (tekst slidera z procentową ilością sprzedaży).
- `void SetCurrencyText (List< QuarkType > list)`
Ustawia wyświetlanie ilości poszczególnych zasobów w UI sklepu.
- `void SetGoldText (string value)`
Ustawia wyświetlanie ilości złota gracza w UI sklepu.

Metody prywatne

- void [ClickSell1](#) ()

Wywoływana po kliknięciu przycisku sprzedaży pierwszego zasobu. Oblicza doświadczenie zdobyte przez postać i odtwarza dźwięk.

Atrybuty prywatne

- VisualElement [ui](#)
Root element UI.
- [RaptorCore](#) [raptorCore](#)
Referencja do rdzenia gry ([RaptorCore](#)).
- [CharacterClass](#) [characterClass](#)
Referencja do klasy postaci, do przyznawania EXP za sprzedaż.
- AudioSource [audioSource](#)
Źródło dźwięku odtwarzane przy sprzedaży zasobów.
- Label [res1curr](#)
Label do wyświetlania ilości pierwszego zasobu.
- Label [res2curr](#)
Label do wyświetlania ilości drugiego zasobu.
- Label [res3curr](#)
Label do wyświetlania ilości trzeciego zasobu.
- Label [res4curr](#)
Label do wyświetlania ilości czwartego zasobu.
- Label [res5curr](#)
Label do wyświetlania ilości piątego zasobu.
- Label [res6curr](#)
Label do wyświetlania ilości szóstego zasobu.
- Label [goldLabel](#)
Label wyświetlający ilość złota gracza.
- SliderInt [currencySlider](#)
Slider do wyboru procentowej ilości zasobu do sprzedaży.
- Label [sliderText](#)
Label wyświetlający aktualną wartość slidera w procentach.
- Button [sell1](#)
Przycisk do sprzedaży pierwszego zasobu.

6.34.1 Opis szczegółowy

Manager UI dla sklepu zasobów. Odpowiada za aktualizację wyświetlanych wartości zasobów, obsługę przycisków sprzedaży i suwaka procentowego sprzedaży.

6.34.2 Dokumentacja konstruktora i destruktora

6.34.2.1 ResourceShopManager()

```
ResourceShopManager.ResourceShopManager (  
    VisualElement root,  
    RaptorCore core,  
    CharacterClass charClass,  
    AudioSource audio)
```

Konstruktor [ResourceShopManager](#).

Parametry

<i>root</i>	Root element UI sklepu.
<i>core</i>	Referencja do RaptorCore .
<i>charClass</i>	Referencja do klasy postaci.
<i>audio</i>	Źródło dźwięku odtwarzane przy sprzedaży.

6.34.3 Dokumentacja funkcji składowych

6.34.3.1 ClickSell1()

```
void ResourceShopManager.ClickSell1 () [private]
```

Wywoływana po kliknięciu przycisku sprzedaży pierwszego zasobu. Oblicza doświadczenie zdobyte przez postać i odtwarza dźwięk.

6.34.3.2 Initialize()

```
void ResourceShopManager.Initialize ()
```

Inicjalizuje wszystkie elementy UI i przypisuje zdarzenia do przycisków.

6.34.3.3 SetCurrencyText()

```
void ResourceShopManager.SetCurrencyText (  
    List< QuarkType > list)
```

Ustawia wyświetlanie ilości poszczególnych zasobów w UI sklepu.

Parametry

<i>list</i>	Lista wartości zasobów w kolejności od Resource1 do Resource6.
-------------	--

6.34.3.4 SetGoldText()

```
void ResourceShopManager.SetGoldText (  
    string value)
```

Ustawia wyświetlanie ilości złota gracza w UI sklepu.

Parametry

<i>value</i>	Wartość złota w formie string.
--------------	--------------------------------

6.34.3.5 Update()

```
void ResourceShopManager.Update ()
```

Aktualizuje UI (tekst slidera z procentową ilością sprzedaży).

6.34.4 Dokumentacja atrybutów składowych

6.34.4.1 audioSource

```
AudioSource ResourceShopManager.audioSource [private]
```

Źródło dźwięku odtwarzane przy sprzedaży zasobów.

6.34.4.2 characterClass

```
CharacterClass ResourceShopManager.characterClass [private]
```

Referencja do klasy postaci, do przyznawania EXP za sprzedaż.

6.34.4.3 currencySlider

```
SliderInt ResourceShopManager.currencySlider [private]
```

Slider do wyboru procentowej ilości zasobu do sprzedaży.

6.34.4.4 goldLabel

```
Label ResourceShopManager.goldLabel [private]
```

Label wyświetlający ilość złota gracza.

6.34.4.5 raptorCore

```
RaptorCore ResourceShopManager.raptorCore [private]
```

Referencja do rdzenia gry ([RaptorCore](#)).

6.34.4.6 res1curr

Label ResourceShopManager.res1curr [private]

Label do wyświetlania ilości pierwszego zasobu.

6.34.4.7 res2curr

Label ResourceShopManager.res2curr [private]

Label do wyświetlania ilości drugiego zasobu.

6.34.4.8 res3curr

Label ResourceShopManager.res3curr [private]

Label do wyświetlania ilości trzeciego zasobu.

6.34.4.9 res4curr

Label ResourceShopManager.res4curr [private]

Label do wyświetlania ilości czwartego zasobu.

6.34.4.10 res5curr

Label ResourceShopManager.res5curr [private]

Label do wyświetlania ilości piątego zasobu.

6.34.4.11 res6curr

Label ResourceShopManager.res6curr [private]

Label do wyświetlania ilości szóstego zasobu.

6.34.4.12 sell1

Button ResourceShopManager.sell1 [private]

Przycisk do sprzedaży pierwszego zasobu.

6.34.4.13 sliderText

Label ResourceShopManager.sliderText [private]

Label wyświetlający aktualną wartość slidera w procentach.

6.34.4.14 ui

VisualElement ResourceShopManager.ui [private]

Root element UI.

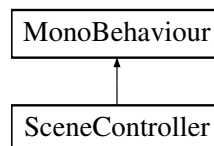
Dokumentacja dla tej klasy została wygenerowana z pliku:

- [ResourceShopManager.cs](#)

6.35 Dokumentacja klasy SceneController

Kontroler scen w grze. Zarządza przejściami między scenami z użyciem efektu fade in/out.

Diagram dziedziczenia dla SceneController



Metody publiczne

- void [LoadScene](#) (string name)
Ładuje nową scenę z efektem fade-out, a następnie fade-in.

Metody prywatne

- void [Awake](#) ()
Metoda wywoływana przy inicjalizacji obiektu. Pobiera komponent [SceneFade](#) z dzieci obiektu.
- IEnumerator [Start](#) ()
Start coroutine uruchamiana przy starcie sceny. Odpowiada za efekt fade-in przy wejściu do sceny.
- IEnumerator [LoadSceneCoroutine](#) (string name)
Coroutine ładująca scenę z efektem fade-out i fade-in.

Atrybuty prywatne

- float [fadeDuration](#)
Czas trwania efektu zanikania (fade) w sekundach.
- [SceneFade sceneFade](#)
Komponent obsługujący animacje zanikania sceny.

6.35.1 Opis szczegółowy

Kontroler scen w grze. Zarządza przejściami między scenami z użyciem efektu fade in/out.

6.35.2 Dokumentacja funkcji składowych

6.35.2.1 Awake()

```
void SceneController.Awake () [private]
```

Metoda wywoływana przy inicjalizacji obiektu. Pobiera komponent [SceneFade](#) z dzieci obiektu.

6.35.2.2 LoadScene()

```
void SceneController.LoadScene (  
    string name)
```

Ładuje nową scenę z efektem fade-out, a następnie fade-in.

Parametry

<i>name</i>	Nazwa sceny do załadowania.
-------------	-----------------------------

6.35.2.3 LoadSceneCoroutine()

```
IEnumerator SceneController.LoadSceneCoroutine (  
    string name) [private]
```

Coroutine ładująca scenę z efektem fade-out i fade-in.

Parametry

<i>name</i>	Nazwa sceny do załadowania.
-------------	-----------------------------

6.35.2.4 Start()

```
IEnumerator SceneController.Start () [private]
```

Start coroutine uruchamiana przy starcie sceny. Odpowiada za efekt fade-in przy wejściu do sceny.

6.35.3 Dokumentacja atrybutów składowych

6.35.3.1 fadeDuration

```
float SceneController.fadeDuration [private]
```

Czas trwania efektu zanikania (fade) w sekundach.

6.35.3.2 sceneFade

`SceneFade` `SceneController.sceneFade` [private]

Komponent obsługujący animacje zanikania sceny.

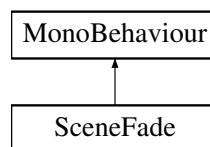
Dokumentacja dla tej klasy została wygenerowana z pliku:

- [SceneController.cs](#)

6.36 Dokumentacja klasy SceneFade

Klasa obsługująca efekt zanikania i pojawiania obrazu w scenie. Może być używana do przejść między scenami lub efektów fade-in/fade-out.

Diagram dziedziczenia dla SceneFade



Metody publiczne

- IEnumerator [FadeInCoroutine](#) (float duration)
Rozpoczyna efekt zanikania (fade-in) obrazu w określonym czasie. Po zakończeniu nakładka jest wyłączana.
- IEnumerator [FadeOutCoroutine](#) (float duration)
Rozpoczyna efekt pojawiania się (fade-out) obrazu w określonym czasie. Nakładka jest włączana przed rozpoczęciem efektu.

Metody prywatne

- void [Awake](#) ()
Metoda wywoływana przy inicjalizacji. Pobiera komponenty Image i RectTransform.
- void [LateUpdate](#) ()
Metoda wywoływana po wszystkich aktualizacjach. Dopasowuje skalę i rozmiar nakładki do tła.
- IEnumerator [FadeCoroutine](#) (Color startColor, Color targetColor, float duration)
Wspólna metoda do interpolacji koloru nakładki w czasie.

Atrybuty prywatne

- Image [scenelImage](#)
Komponent Image używany do wyświetlania nakładki fade.
- RectTransform [rectTransform](#)
RectTransform obrazu nakładki, do skalowania względem tła.
- RectTransform [backgroundTransform](#)
RectTransform tła, względem którego dopasowywana jest nakładka.

6.36.1 Opis szczegółowy

Klasa obsługująca efekt zanikania i pojawiania obrazu w scenie. Może być używana do przejść między scenami lub efektów fade-in/fade-out.

6.36.2 Dokumentacja funkcji składowych

6.36.2.1 Awake()

```
void SceneFade.Awake () [private]
```

Metoda wywoływana przy inicjalizacji. Pobiera komponenty Image i RectTransform.

6.36.2.2 FadeCoroutine()

```
IEnumerator SceneFade.FadeCoroutine (
    Color startColor,
    Color targetColor,
    float duration) [private]
```

Wspólna metoda do interpolacji koloru nakładki w czasie.

Parametry

<i>startColor</i>	Kolor początkowy.
<i>targetColor</i>	Kolor docelowy.
<i>duration</i>	Czas trwania efektu w sekundach.

Zwraca

Kolekcja IEnumerator do uruchomienia jako coroutine.

6.36.2.3 FadeInCoroutine()

```
IEnumerator SceneFade.FadeInCoroutine (
    float duration)
```

Rozpoczyna efekt zanikania (fade-in) obrazu w określonym czasie. Po zakończeniu nakładka jest wyłączana.

Parametry

<i>duration</i>	Czas trwania efektu w sekundach.
-----------------	----------------------------------

Zwraca

Kolekcja IEnumerator do uruchomienia jako coroutine.

6.36.2.4 FadeOutCoroutine()

```
IEnumerator SceneFade.FadeOutCoroutine (  
    float duration)
```

Rozpoczyna efekt pojawiania się (fade-out) obrazu w określonym czasie. Nakładka jest włączana przed rozpoczęciem efektu.

Parametry

<i>duration</i>	Czas trwania efektu w sekundach.
-----------------	----------------------------------

Zwraca

Kolekcja IEnumerator do uruchomienia jako coroutine.

6.36.2.5 LateUpdate()

```
void SceneFade.LateUpdate () [private]
```

Metoda wywoływana po wszystkich aktualizacjach. Dopasowuje skalę i rozmiar nakładki do tła.

6.36.3 Dokumentacja atrybutów składowych

6.36.3.1 backgroundTransform

```
RectTransform SceneFade.backgroundTransform [private]
```

RectTransform tła, względem którego dopasowywana jest nakładka.

6.36.3.2 rectTransform

```
RectTransform SceneFade.rectTransform [private]
```

RectTransform obrazu nakładki, do skalowania względem tła.

6.36.3.3 sceneImage

```
Image SceneFade.sceneImage [private]
```

Komponent Image używany do wyświetlania nakładki fade.

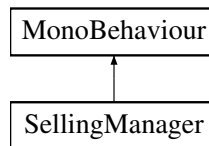
Dokumentacja dla tej klasy została wygenerowana z pliku:

- [SceneFade.cs](#)

6.37 Dokumentacja klasy SellingManager

Klasa odpowiedzialna za sprzedaż zasobów przez gracza i przeliczanie zysków w złocie. Uwzględnia bonusy z mikstur, prestiż i klasy postaci.

Diagram dziedziczenia dla SellingManager



Metody publiczne

- void `Initialize` (`RaptorCore` raptorCore, `ResourceManager` resManager, `IdleManager` idle, `CharacterClass` character)
Inicjalizuje manager sprzedaży, przypisując referencje do rdzenia gry, managera zasobów, idle managera i klasy postaci.
- double `SellMaterials` (float sellValue)
Sprzedaje aktualnie wybrany zasób w ilości określonej przez slider/parametr procentowy.
- double `SellResource` (string resource, float sellvalue, double pricePerUnit=1.0)
Sprzedaje konkretny zasób w podanej ilości procentowej i przelicza złoto, uwzględniając bonusy.

Atrybuty publiczne

- `QuarkType` potionSellBonus1 = 1
Bonus do sprzedaży zasobów przy aktywnej pierwszej miksturze sprzedażowej.
- `QuarkType` potionSellBonus2 = 1
Bonus do sprzedaży zasobów przy aktywnej drugiej miksturze sprzedażowej.

Metody prywatne

- void `Update` ()
Metoda wywoływana co klatkę. Jeżeli aktywny jest tryb "Michael Scott", sprzedaje automatycznie zasoby w pełnej ilości i przyznaje doświadczenie graczowi.

Atrybuty prywatne

- `RaptorCore` raptorCore
Referencja do głównego rdzenia gry, `RaptorCore`.
- `ResourceManager` resourceManager
Manager zasobów, służący do odczytu i modyfikacji ilości zasobów.
- `IdleManager` idleManager
Manager fabryk i mikstur (idle manager).
- `CharacterClass` characterClass
Obiekt klasy postaci gracza.

6.37.1 Opis szczegółowy

Klasa odpowiedzialna za sprzedaż zasobów przez gracza i przeliczanie zysków w złocie. Uwzględnia bonusy z mikstur, prestiż i klasy postaci.

6.37.2 Dokumentacja funkcji składowych

6.37.2.1 Initialize()

```
void SellingManager.Initialize (  
    RaptorCore raptorCore,  
    ResourceManager resManager,  
    IdleManager idle,  
    CharacterClass character)
```

Inicjalizuje manager sprzedaży, przypisując referencje do rdzenia gry, managera zasobów, idle managera i klasy postaci.

Parametry

<i>raptorCore</i>	Główny rdzeń gry.
<i>resManager</i>	Manager zasobów.
<i>idle</i>	Idle manager z fabrykami i miksturami.
<i>character</i>	Klasa postaci gracza.

6.37.2.2 SellMaterials()

```
double SellingManager.SellMaterials (  
    float sellValue)
```

Sprzedaje aktualnie wybrany zasób w ilości określonej przez slider/parametr procentowy.

Parametry

<i>sellValue</i>	Procent zasobu do sprzedania (0-100).
------------------	---------------------------------------

Zwraca

Wartość złota uzyskana ze sprzedaży.

6.37.2.3 SellResource()

```
double SellingManager.SellResource (
    string resource,
    float sellvalue,
    double pricePerUnit = 1::0)
```

Sprzedaje konkretny zasób w podanej ilości procentowej i przelicza złoto, uwzględniając bonusy.

Parametry

<i>resource</i>	Nazwa zasobu do sprzedaży.
<i>sellvalue</i>	Procent ilości zasobu do sprzedaży (0-100).
<i>pricePerUnit</i>	Cena jednostkowa zasobu (domyślnie 1.0).

Zwraca

Wartość złota uzyskana ze sprzedaży.

6.37.2.4 Update()

```
void SellingManager.Update () [private]
```

Metoda wywoływana co klatkę. Jeżeli aktywny jest tryb "Michael Scott", sprzedaje automatycznie zasoby w pełnej ilości i przyznaje doświadczenie graczowi.

6.37.3 Dokumentacja atrybutów składowych

6.37.3.1 characterClass

```
CharacterClass SellingManager.characterClass [private]
```

Obiekt klasy postaci gracza.

6.37.3.2 idleManager

```
IdleManager SellingManager.idleManager [private]
```

Manager fabryk i mikstur (idle manager).

6.37.3.3 potionSellBonus1

```
QuarkType SellingManager.potionSellBonus1 = 1
```

Bonus do sprzedaży zasobów przy aktywnej pierwszej miksturze sprzedażowej.

6.37.3.4 potionSellBonus2

`QuarkType` SellingManager.potionSellBonus2 = 1

Bonus do sprzedaży zasobów przy aktywnej drugiej miksturze sprzedażowej.

6.37.3.5 raptorCore

`RaptorCore` SellingManager.raptorCore [private]

Referencja do głównego rdzenia gry, `RaptorCore`.

6.37.3.6 resourceManager

`ResourceManager` SellingManager.resourceManager [private]

Manager zasobów, służący do odczytu i modyfikacji ilości zasobów.

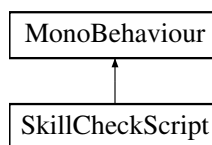
Dokumentacja dla tej klasy została wygenerowana z pliku:

- [SellManager.cs](#)

6.38 Dokumentacja klasy SkillCheckScript

Skrypt obsługujący mini-grę Skill Check w systemie klików. Gracz musi kliknąć odpowiednim przyciskiem myszy na nadlatujące klocki, aby zdobywać bonusy i utrzymywać combo.

Diagram dziedziczenia dla SkillCheckScript



Metody publiczne

- void `StartSkillCheck` ()
Rozpoczyna mini-grę Skill Check.

Metody prywatne

- void [Start](#) ()
Inicjalizacja komponentów.
- void [Update](#) ()
Główna pętla sprawdzająca kliknięcia gracza i kolizje klocków.
- void [CheckMouse0](#) ()
Sprawdza kliknięcie lewym przyciskiem myszy.
- void [CheckMouse1](#) ()
Sprawdza kliknięcie prawym przyciskiem myszy.
- void [ResetSkillCheck](#) (int value)
Resetuje mini-grę Skill Check. Przy poprawnym wyniku przyznaje bonusy i combo, w przeciwnym przypadku resetuje combo.
- void [ClearBricks](#) ()
Usuwa wszystkie klocki z mini-gry.
- IEnumerator [SpawnBricks](#) (int count, float [waitTime](#))
Coroutine odpowiadająca za spawnowanie klocków w określonym czasie.

Atrybuty prywatne

- GameObject [brickPrefab](#)
Prefab klocka, który będzie spawnowany w mini-grze.
- SpriteRenderer [flag](#)
Flaga pokazująca status mini-gry i jej kolory.
- [RaptorCore](#) [raptorCore](#)
Główny core gry, odpowiedzialny za zarządzanie walutą, umiejętnościami i bonusami.
- [CharacterClass](#) [characterClass](#)
Klasa postaci gracza, używana do sprawdzania i przyznawania bonusów oraz umiejętności.
- Queue< GameObject > [bricks](#) = new Queue<GameObject>()
Kolejka klocków aktualnie obecnych w mini-grze.
- int [kombo](#) = 0
Aktualne combo gracza.
- int [Bonus](#) = 5
Ilość bonusu przy poprawnym skill checku.
- float [waitTime](#) = 0.9f
Czas oczekiwania między spawnami kolejnych klocków.
- int [brickCount](#) = 10
Liczba klocków spawnowanych w mini-grze.
- GameObject [Layout](#)
Layout gry, potrzebny do obsługi wizualizacji combo.
- [ComboVisual](#) [combovisual](#)
Komponent odpowiedzialny za wizualizację combo.

6.38.1 Opis szczegółowy

Skrypt obsługujący mini-grę Skill Check w systemie klików. Gracz musi kliknąć odpowiednim przyciskiem myszy na nadlatujące klocki, aby zdobywać bonusy i utrzymywać combo.

6.38.2 Dokumentacja funkcji składowych

6.38.2.1 CheckMouse0()

```
void SkillCheckScript.CheckMouse0 () [private]
```

Sprawdza kliknięcie lewym przyciskiem myszy.

6.38.2.2 CheckMouse1()

```
void SkillCheckScript.CheckMouse1 () [private]
```

Sprawdza kliknięcie prawym przyciskiem myszy.

6.38.2.3 ClearBricks()

```
void SkillCheckScript.ClearBricks () [private]
```

Usuwa wszystkie klocki z mini-gry.

6.38.2.4 ResetSkillCheck()

```
void SkillCheckScript.ResetSkillCheck (  
    int value) [private]
```

Resetuje mini-grę Skill Check. Przy poprawnym wyniku przyznaje bonusy i combo, w przeciwnym przypadku resetuje combo.

Parametry

<i>value</i>	Wartość bonusu (dodatnia lub ujemna).
--------------	---------------------------------------

6.38.2.5 SpawnBricks()

```
IEnumerator SkillCheckScript.SpawnBricks (  
    int count,  
    float waitTime) [private]
```

Coroutine odpowiadająca za spawnowanie klocków w określonym czasie.

Parametry

<i>count</i>	Liczba klocków do spawnowania.
<i>waitTime</i>	Czas oczekiwania między kolejnymi klockami.

Zwraca

IEnumerator coroutine.

6.38.2.6 Start()

```
void SkillCheckScript.Start () [private]
```

Inicjalizacja komponentów.

6.38.2.7 StartSkillCheck()

```
void SkillCheckScript.StartSkillCheck ()
```

Rozpoczyna mini-grę Skill Check.

6.38.2.8 Update()

```
void SkillCheckScript.Update () [private]
```

Główna pętla sprawdzająca kliknięcia gracza i kolizje klocków.

6.38.3 Dokumentacja atrybutów składowych

6.38.3.1 Bonus

```
int SkillCheckScript.Bonus = 5 [private]
```

Ilość bonusu przy poprawnym skill checku.

6.38.3.2 brickCount

```
int SkillCheckScript.brickCount = 10 [private]
```

Liczba klocków spawnowanych w mini-grze.

6.38.3.3 brickPrefab

```
GameObject SkillCheckScript.brickPrefab [private]
```

Prefab klocka, który będzie spawnowany w mini-grze.

6.38.3.4 bricks

```
Queue<GameObject> SkillCheckScript.bricks = new Queue<GameObject>() [private]
```

Kolejka klocków aktualnie obecnych w mini-grze.

6.38.3.5 characterClass

`CharacterClass SkillCheckScript.characterClass [private]`

Klasa postaci gracza, używana do sprawdzania i przyznawania bonusów oraz umiejętności.

6.38.3.6 combovisual

`ComboVisual SkillCheckScript.combovisual [private]`

Komponent odpowiedzialny za wizualizację combo.

6.38.3.7 flag

`SpriteRenderer SkillCheckScript.flag [private]`

Flaga pokazująca status mini-gry i jej kolory.

6.38.3.8 kombo

`int SkillCheckScript.kombo = 0 [private]`

Aktualne combo gracza.

6.38.3.9 Layout

`GameObject SkillCheckScript.Layout [private]`

Layout gry, potrzebny do obsługi wizualizacji combo.

6.38.3.10 raptorCore

`RaptorCore SkillCheckScript.raptorCore [private]`

Główny core gry, odpowiedzialny za zarządzanie walutą, umiejętnościami i bonusami.

6.38.3.11 waitTime

`float SkillCheckScript.waitTime = 0.9f [private]`

Czas oczekiwania między spawnami kolejnych klocków.

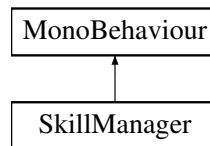
Dokumentacja dla tej klasy została wygenerowana z pliku:

- [SkillCheckScript.cs](#)

6.39 Dokumentacja klasy SkillManager

Manager umiejętności postaci gracza. Odpowiada za skill check, aktywne idle, efekty mikstur i tymczasowe buffy.

Diagram dziedziczenia dla SkillManager



Metody publiczne

- void **Initialize** ([RaptorCore](#) __raptorCore, [CharacterClass](#) charClass, [IdleManager](#) idle, [GameObject](#) skillCheckObj)
Inicjalizacja managera z głównymi komponentami gry.
- void **CheckAndTriggerSkillCheck** ()
Sprawdza, czy skill check może zostać aktywowany, i wywołuje go jeśli tak.
- void **TriggerSkillCheck** ()
Próbuje wywołać skill check z losową szansą i dodatkowymi boostami.
- void **UpdateSkillCheckCooldown** ()
Aktualizuje cooldown dla skill checka w każdej klatce.
- void **CheckReactionTest** (int tickCount)
Sprawdza i wywołuje skill check w trybie Reaction Test, w zależności od tickCount.
- IEnumerator **EnableActiveIdle** ()
Aktywuje tymczasowy buff dla fabryk powiązanych z aktualnym resource.
- double **GetActiveIdleValue** ()
Zwraca wartość produkcji powiązanej z aktywnym resource, przy użyciu active idle.
- void **EnableChickenDinner** (bool effect)
Aktywuje efekt "Chicken Dinner" w zależności od ustawień postaci.
- IEnumerator **ChickenDinnerEffect** (bool effect)
Coroutine odpowiadająca za dodanie i zdjęcie buffu Chicken Dinner.
- bool **IsSkillCheckActive** ()
Sprawdza, czy skill check jest aktualnie aktywny.
- void **RandomPotionEffect** ()
Losowo aktywuje efekt mikstury w grze, jeśli szansa na to pozwala.

Właściwości

- bool **SkillCheckCooldown** [get]
Właściwość informująca, czy skill check jest na cooldownzie.

Atrybuty prywatne

- [RaptorCore](#) `raptorCore`
- [CharacterClass](#) `characterClass`
Klasa postaci gracza, używana do sprawdzania umiejętności, buffów i efektów specjalnych.
- [IdleManager](#) `idleManager`
Manager fabryk, mikstur i produkcji idle.
- `GameObject` [skillCheck](#)
Obiekt mini-gry Skill Check.
- `bool` [skillCheckCooldown](#) = false
Flaga informująca, czy skill check jest aktualnie na cooldownzie.
- `int` [skillCheckCooldownCount](#) = 0
Licznik używany do odmierzania cooldownu skill checka.

6.39.1 Opis szczegółowy

Manager umiejętności postaci gracza. Odpowiada za skill check, aktywne idle, efekty mikstur i tymczasowe buffy.

6.39.2 Dokumentacja funkcji składowych

6.39.2.1 CheckAndTriggerSkillCheck()

```
void SkillManager.CheckAndTriggerSkillCheck ()
```

Sprawdza, czy skill check może zostać aktywowany, i wywołuje go jeśli tak.

6.39.2.2 CheckReactionTest()

```
void SkillManager.CheckReactionTest (  
    int tickCount)
```

Sprawdza i wywołuje skill check w trybie Reaction Test, w zależności od tickCount.

Parametry

<i>tickCount</i>	Licznik ticków gry.
------------------	---------------------

6.39.2.3 ChickenDinnerEffect()

```
IEnumerator SkillManager.ChickenDinnerEffect (
    bool effect)
```

Coroutine odpowiadająca za dodanie i zdjęcie buffu Chicken Dinner.

Parametry

<i>effect</i>	Czy efekt ma być aktywny.
---------------	---------------------------

6.39.2.4 EnableActiveIdle()

```
IEnumerator SkillManager.EnableActiveIdle ()
```

Aktywuje tymczasowy buff dla fabryk powiązanych z aktualnym resource.

6.39.2.5 EnableChickenDinner()

```
void SkillManager.EnableChickenDinner (
    bool effect)
```

Aktywuje efekt "Chicken Dinner" w zależności od ustawień postaci.

6.39.2.6 GetActiveIdleValue()

```
double SkillManager.GetActiveIdleValue ()
```

Zwraca wartość produkcji powiązanej z aktywnym resource, przy użyciu active idle.

6.39.2.7 Initialize()

```
void SkillManager.Initialize (
    RaptorCore _raptorCore,
    CharacterClass charClass,
    IdleManager idle,
    GameObject skillCheckObj)
```

Inicjalizacja menedżera z głównymi komponentami gry.

6.39.2.8 IsSkillCheckActive()

```
bool SkillManager.IsSkillCheckActive ()
```

Sprawdza, czy skill check jest aktualnie aktywny.

Zwraca

True jeśli skill check jest aktywny, w przeciwnym razie false.

6.39.2.9 RandomPotionEffect()

```
void SkillManager.RandomPotionEffect ()
```

Losowo aktywuje efekt mikstury w grze, jeśli szansa na to pozwala.

6.39.2.10 TriggerSkillCheck()

```
void SkillManager.TriggerSkillCheck ()
```

Próbuje wywołać skill check z losową szansą i dodatkowymi boostami.

6.39.2.11 UpdateSkillCheckCooldown()

```
void SkillManager.UpdateSkillCheckCooldown ()
```

Aktualizuje cooldown dla skill checka w każdej klatce.

6.39.3 Dokumentacja atrybutów składowych

6.39.3.1 characterClass

```
CharacterClass SkillManager.characterClass [private]
```

Klasa postaci gracza, używana do sprawdzania umiejętności, buffów i efektów specjalnych.

6.39.3.2 idleManager

```
IdleManager SkillManager.idleManager [private]
```

Manager fabryk, mikstur i produkcji idle.

6.39.3.3 raptorCore

```
RaptorCore SkillManager.raptorCore [private]
```

Główny core gry, odpowiedzialny za walutę, skill multiplier i inne zasoby.

6.39.3.4 skillCheck

```
GameObject SkillManager.skillCheck [private]
```

Obiekt mini-gry Skill Check.

6.39.3.5 skillCheckCooldown

```
bool SkillManager.skillCheckCooldown = false [private]
```

Flaga informująca, czy skill check jest aktualnie na cooldownzie.

6.39.3.6 skillCheckCooldownCount

```
int SkillManager.skillCheckCooldownCount = 0 [private]
```

Licznik używany do odmierzenia cooldownu skill checka.

6.39.4 Dokumentacja właściwości

6.39.4.1 SkillCheckCooldown

```
bool SkillManager.SkillCheckCooldown [get]
```

Właściwość informująca, czy skill check jest na cooldownzie.

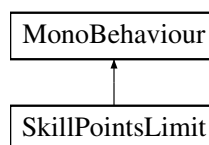
Dokumentacja dla tej klasy została wygenerowana z pliku:

- [SkillManager.cs](#)

6.40 Dokumentacja klasy SkillPointsLimit

Manager przycisku Skill Points. Odpowiada za wyświetlanie liczby punktów umiejętności, zmianę koloru przycisku w zależności od dostępności oraz zwiększanie i zmniejszanie punktów postaci.

Diagram dziedziczenia dla SkillPointsLimit



Metody publiczne

- void [UpdateButton](#) ()
Aktualizuje wyświetlanie przycisku Skill Points w zależności od liczby punktów postaci. Zmienia kolor przycisku w zależności od dostępności punktów.
- void [DecSkillPoints](#) ()
Zmniejsza liczbę punktów postaci o 1 i aktualizuje przycisk.
- void [IncSkillPoints](#) ()
Zwiększa liczbę punktów postaci o 1 i aktualizuje przycisk.

Atrybuty publiczne

- VisualElement [ui](#)
Root UI dokumentu.

Metody prywatne

- void [Awake](#) ()
Inicjalizacja komponentów i ustawienie kolorów przycisku.

Atrybuty prywatne

- VisualElement [critContainer](#)
- Button [Skillbtn](#)
Przycisk UI reprezentujący Skill Points.
- [CharacterClass](#) [characterClass](#)
Referencja do klasy postaci gracza.
- Color [availableColor](#)
Kolor przycisku, gdy punkty są dostępne.
- Color [unavailableColor](#)
Kolor przycisku, gdy punkty są niedostępne.

6.40.1 Opis szczegółowy

Manager przycisku Skill Points. Odpowiada za wyświetlanie liczby punktów umiejętności, zmianę koloru przycisku w zależności od dostępności oraz zwiększanie i zmniejszanie punktów postaci.

6.40.2 Dokumentacja funkcji składowych

6.40.2.1 Awake()

```
void SkillPointsLimit.Awake () [private]
```

Inicjalizacja komponentów i ustawienie kolorów przycisku.

6.40.2.2 DecSkillPoints()

```
void SkillPointsLimit.DecSkillPoints ()
```

Zmniejsza liczbę punktów postaci o 1 i aktualizuje przycisk.

6.40.2.3 IncSkillPoints()

```
void SkillPointsLimit.IncSkillPoints ()
```

Zwiększa liczbę punktów postaci o 1 i aktualizuje przycisk.

6.40.2.4 UpdateButton()

```
void SkillPointsLimit.UpdateButton ()
```

Aktualizuje wyświetlanie przycisku Skill Points w zależności od liczby punktów postaci. Zmienia kolor przycisku w zależności od dostępności punktów.

6.40.3 Dokumentacja atrybutów składowych

6.40.3.1 availableColor

```
Color SkillPointsLimit.availableColor [private]
```

Kolor przycisku, gdy punkty są dostępne.

6.40.3.2 characterClass

```
CharacterClass SkillPointsLimit.characterClass [private]
```

Referencja do klasy postaci gracza.

6.40.3.3 critContainer

```
VisualElement SkillPointsLimit.critContainer [private]
```

Kontener wizualny dla efektów krytycznych (CRIT), jeśli potrzebny do interfejsu.

6.40.3.4 Skillbtn

```
Button SkillPointsLimit.Skillbtn [private]
```

Przycisk UI reprezentujący Skill Points.

6.40.3.5 ui

```
VisualElement SkillPointsLimit.ui
```

Root UI dokumentu.

6.40.3.6 unavailableColor

```
Color SkillPointsLimit.unavailableColor [private]
```

Kolor przycisku, gdy punkty są niedostępne.

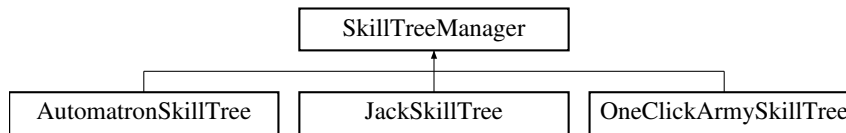
Dokumentacja dla tej klasy została wygenerowana z pliku:

- [SkillPointsLimit.cs](#)

6.41 Dokumentacja klasy SkillTreeManager

Bazowa klasa dla drzewek umiejętności. Zarządza przyciskami, zależnościami, liniami wizualnymi między umiejętnościami i zapis/odczyt stanu nauczonych umiejętności.

Diagram dziedziczenia dla SkillTreeManager



Metody publiczne

- **SkillTreeManager** (VisualElement root, [CharacterClass](#) charClass, Dictionary< string, List< string > > treeDependencies, string containerName, HashSet< string > learnedSet, [SkillPointsLimit](#) skillPointsLimit)
Konstruktor bazowy [SkillTreeManager](#).
- void **ClearTree** ()
Czyści nauczzone umiejętności.
- void **InitializeSkills** (string[] skillIds)
Tworzy przyciski dla wszystkich umiejętności w drzewku.
- void **RegisterTooltip** (string skillId, string title, string description, string lore)
Rejestruje tooltip dla przycisku umiejętności.
- void **DrawLine** (string fromId, string toId, Color color)
Rysuje pojedynczą linię między dwoma przyciskami.
- void **DrawAllLines** ()
Rysuje wszystkie linie zależności w drzewku.
- void **UpdateAll** ()
Aktualizuje wszystkie przyciski wizualnie.
- void **SerializeToStream** (BinaryWriter writer)
Zapisuje stan nauczonych umiejętności do strumienia.
- void **DeserializeFromStream** (BinaryReader reader)
Wczytuje stan nauczonych umiejętności ze strumienia.

Metody chronione

- void **OnSkillClicked** (string id)
Wywoływane po kliknięciu w umiejętność
- bool **CanUnlock** (string id)
Sprawdza, czy umiejętność może zostać nauczona.
- void **UpdateVisual** (string id)
Aktualizuje wizualną reprezentację przycisku umiejętności.
- virtual void **OnSkillUnlearned** (string id)
Wywoływane przy "odnauczaniu" umiejętności (klik prawym przyciskiem).

Atrybuty chronione

- VisualElement `ui`
Root UI dokumentu.
- CharacterClass `characterClass`
Klasa postaci, której drzewko dotyczy.
- Dictionary< string, Button > `buttons` = new()
Mapa przycisków umiejętności według ID.
- HashSet< string > `learned` = new()
Zbiór nauczonych umiejętności.
- Dictionary< string, List< string > > `dependencies` = new()
Zależności umiejętności.
- VisualElement `skillTreeContainer`
Kontener UI dla całego drzewka.
- VisualElement `lineLayer`
Warstwa linii rysowanych między umiejętnościami.
- Color `lockedColor` = new Color(0.3f, 0.3f, 0.3f)
Kolor przycisku zablokowanego.
- Color `availableColor` = new Color(0.8f, 0.6f, 0.3f)
Kolor przycisku dostępnego do nauki.
- Color `learnedColor` = new Color(0.2f, 0.8f, 0.2f)
Kolor przycisku nauczonego.
- SkillPointsLimit `skillPointsLimit`
Referencja do managera punktów umiejętności.

Metody prywatne

- void `InitializeLineLayer` (VisualElement layer)
Inicjalizacja warstwy linii między przyciskami.
- void `DrawLineBetweenButtons` (Button from, Button to, Color color, VisualElement targetLayer)
Rysuje linię łączącą dwa przyciski w drzewku umiejętności.

6.41.1 Opis szczegółowy

Bazowa klasa dla drzewek umiejętności. Zarządza przyciskami, zależnościami, liniami wizualnymi między umiejętnościami i zapis/odczyt stanu nauczonych umiejętności.

6.41.2 Dokumentacja konstruktora i destruktor

6.41.2.1 SkillTreeManager()

```
SkillTreeManager.SkillTreeManager (
    VisualElement root,
    CharacterClass charClass,
    Dictionary< string, List< string > > treeDependencies,
    string containerName,
    HashSet< string > learnedSet,
    SkillPointsLimit skillPointsLimit)
```

Konstruktor bazowy [SkillTreeManager](#).

Parametry

<i>root</i>	Root UI
<i>charClass</i>	Klasa postaci
<i>treeDependencies</i>	Zależności w drzewku
<i>containerName</i>	Nazwa kontenera UI dla drzewa
<i>learnedSet</i>	Zbiór już nauczonych umiejętności
<i>skillPointsLimit</i>	Manager punktów umiejętności

6.41.3 Dokumentacja funkcji składowych

6.41.3.1 CanUnlock()

```
bool SkillTreeManager.CanUnlock (  
    string id) [abstract], [protected]
```

Sprawdza, czy umiejętność może zostać nauczona.

Parametry

<i>id</i>	ID umiejętności
-----------	-----------------

6.41.3.2 ClearTree()

```
void SkillTreeManager.ClearTree ()
```

Czyści nauczzone umiejętności.

6.41.3.3 DeserializeFromStream()

```
void SkillTreeManager.DeserializeFromStream (  
    BinaryReader reader)
```

Wczytuje stan nauczonych umiejętności ze strumienia.

6.41.3.4 DrawAllLines()

```
void SkillTreeManager.DrawAllLines ()
```

Rysuje wszystkie linie zależności w drzewku.

6.41.3.5 DrawLine()

```
void SkillTreeManager.DrawLine (
    string fromId,
    string toId,
    Color color)
```

Rysuje pojedynczą linię między dwoma przyciskami.

6.41.3.6 DrawLineBetweenButtons()

```
void SkillTreeManager.DrawLineBetweenButtons (
    Button from,
    Button to,
    Color color,
    VisualElement targetLayer) [private]
```

Rysuje linię łączącą dwa przyciski w drzewku umiejętności.

Parametry

<i>from</i>	Przycisk źródłowy, od którego wychodzi linia
<i>to</i>	Przycisk docelowy, do którego linia prowadzi
<i>color</i>	Kolor linii
<i>targetLayer</i>	Warstwa UI, na której linia ma być narysowana

6.41.3.7 InitializeLineLayer()

```
void SkillTreeManager.InitializeLineLayer (
    VisualElement layer) [private]
```

Inicjalizacja warstwy linii między przyciskami.

6.41.3.8 InitializeSkills()

```
void SkillTreeManager.InitializeSkills (
    string[] skillIds)
```

Tworzy przyciski dla wszystkich umiejętności w drzewku.

Parametry

<i>skillIds</i>	Lista ID umiejętności
-----------------	-----------------------

6.41.3.9 OnSkillClicked()

```
void SkillTreeManager.OnSkillClicked (
    string id) [abstract], [protected]
```

Wywoływane po kliknięciu w umiejętność

Parametry

<i>id</i>	ID umiejętności
-----------	-----------------

6.41.3.10 OnSkillUnlearned()

```
virtual void SkillTreeManager.OnSkillUnlearned (
    string id) [protected], [virtual]
```

Wywoływane przy "odnauczaniu" umiejętności (klik prawym przyciskiem).

Parametry

<i>id</i>	ID umiejętności
-----------	-----------------

6.41.3.11 RegisterTooltip()

```
void SkillTreeManager.RegisterTooltip (
    string skillId,
    string title,
    string description,
    string lore)
```

Rejestruje tooltip dla przycisku umiejętności.

Parametry

<i>skillId</i>	ID umiejętności
<i>title</i>	Tytuł tooltipa
<i>description</i>	Opis umiejętności
<i>lore</i>	Lore / historia

6.41.3.12 SerializeToStream()

```
void SkillTreeManager.SerializeToStream (
    BinaryWriter writer)
```

Zapisuje stan nauczonych umiejętności do strumienia.

6.41.3.13 UpdateAll()

```
void SkillTreeManager.UpdateAll ()
```

Aktualizuje wszystkie przyciski wizualnie.

6.41.3.14 UpdateVisual()

```
void SkillTreeManager.UpdateVisual (  
    string id) [abstract], [protected]
```

Aktualizuje wizualną reprezentację przycisku umiejętności.

Parametry

<i>id</i>	ID umiejętności
-----------	-----------------

6.41.4 Dokumentacja atrybutów składowych

6.41.4.1 availableColor

```
Color SkillTreeManager.availableColor = new Color(0.8f, 0.6f, 0.3f) [protected]
```

Kolor przycisku dostępnego do nauki.

6.41.4.2 buttons

```
Dictionary<string, Button> SkillTreeManager.buttons = new() [protected]
```

Mapa przycisków umiejętności według ID.

6.41.4.3 characterClass

```
CharacterClass SkillTreeManager.characterClass [protected]
```

Klasa postaci, której drzewko dotyczy.

6.41.4.4 dependencies

```
Dictionary<string, List<string> > SkillTreeManager.dependencies = new() [protected]
```

Zależności umiejętności.

6.41.4.5 learned

```
HashSet<string> SkillTreeManager.learned = new() [protected]
```

Zbiór nauczonych umiejętności.

6.41.4.6 learnedColor

`Color SkillTreeManager.learnedColor = new Color(0.2f, 0.8f, 0.2f) [protected]`

Kolor przycisku nauczonego.

6.41.4.7 lineLayer

`VisualElement SkillTreeManager.lineLayer [protected]`

Warstwa linii rysowanych między umiejętnościami.

6.41.4.8 lockedColor

`Color SkillTreeManager.lockedColor = new Color(0.3f, 0.3f, 0.3f) [protected]`

Kolor przycisku zablokowanego.

6.41.4.9 skillPointsLimit

`SkillPointsLimit SkillTreeManager.skillPointsLimit [protected]`

Referencja do menedżera punktów umiejętności.

6.41.4.10 skillTreeContainer

`VisualElement SkillTreeManager.skillTreeContainer [protected]`

Kontener UI dla całego drzewka.

6.41.4.11 ui

`VisualElement SkillTreeManager.ui [protected]`

Root UI dokumentu.

Dokumentacja dla tej klasy została wygenerowana z pliku:

- [SkillTreesUI.cs](#)

6.42 Dokumentacja klasy SkillTreesUIManager

Menedżer wszystkich trzech drzewek umiejętności w UI. Odpowiada za inicjalizację, aktualizację, serializację i rysowanie linii między przyciskami.

Metody publiczne

- **SkillTreesUIManager** (VisualElement **ui**, CharacterClass **characterClass**, HashSet< string > **learnedOneClick**, HashSet< string > **learnedJack**, HashSet< string > **learnedAutomatron**, SkillPointsLimit **skillPointsLimit**)
Konstruktor menedżera drzewek umiejętności.
- void **ClearTrees** ()
Czyści wszystkie drzewka umiejętności (resetuje odblokowane umiejętności).
- void **InitializeAllTrees** ()
Inicjalizuje wszystkie drzewka umiejętności i rejestruje tooltipy oraz linie.
- void **Update** ()
Aktualizuje wizualnie wszystkie drzewka umiejętności.
- void **SerializeToStream** (BinaryWriter **writer**)
Serializacja wszystkich drzewek do strumienia binarnego.
- void **DeserializeFromStream** (BinaryReader **reader**)
Deserializacja wszystkich drzewek z binarnego strumienia.

Atrybuty publiczne

- VisualTreeAsset **tooltipAsset**
Asset UI dla tooltipów umiejętności (VisualTreeAsset), używany do wyświetlania informacji o skillach.

Metody prywatne

- void **InitializeOneClickArmyTree** ()
Inicjalizacja drzewka One Click Army: ustawienie przycisków, tooltipów i połączeń linii.
- void **InitializeJackTree** ()
Inicjalizacja drzewka Jack of All Clicks.
- void **InitializeAutomatronTree** ()
Inicjalizacja drzewka Automatron.

Atrybuty prywatne

- VisualElement **ui**
Root VisualElement UI, w którym znajdują się przyciski wszystkich drzewek umiejętności.
- CharacterClass **characterClass**
Obiekt klasy postaci gracza. Przechowuje informacje o punktach umiejętności, aktywnych skryptach itd.
- OneClickArmySkillTree **oneClickArmyTree**
Instancja drzewka One Click Army. Zarządza przyciskami, tooltipami i liniami dla tej klasy.
- JackSkillTree **jackTree**
Instancja drzewka Jack of All Clicks. Zarządza przyciskami, tooltipami i liniami dla tej klasy.
- AutomatronSkillTree **automatronTree**
Instancja drzewka Automatron. Zarządza przyciskami, tooltipami i liniami dla tej klasy.
- HashSet< string > **learnedOneClick**
Zestaw odblokowanych umiejętności drzewka One Click Army.
- HashSet< string > **learnedJack**
Zestaw odblokowanych umiejętności drzewka Jack of All Clicks.
- HashSet< string > **learnedAutomatron**
Zestaw odblokowanych umiejętności drzewka Automatron.
- SkillPointsLimit **skillPointsLimit**
Obiekt odpowiedzialny za limity punktów umiejętności i ich wizualizację w UI.

6.42.1 Opis szczegółowy

Menedżer wszystkich trzech drzewek umiejętności w UI. Odpowiada za inicjalizację, aktualizację, serializację i rysowanie linii między przyciskami.

6.42.2 Dokumentacja konstruktora i destruktora

6.42.2.1 SkillTreesUIManager()

```
SkillTreesUIManager.SkillTreesUIManager (  
    VisualElement ui,  
    CharacterClass characterClass,  
    HashSet< string > learnedOneClick,  
    HashSet< string > learnedJack,  
    HashSet< string > learnedAutomatron,  
    SkillPointsLimit skillPointsLimit)
```

Konstruktor menedżera drzewek umiejętności.

Parametry

<i>ui</i>	Root VisualElement UI, w którym są przyciski drzewek
<i>characterClass</i>	Obiekt reprezentujący klasę gracza
<i>learnedOneClick</i>	Zestaw odblokowanych umiejętności One Click Army
<i>learnedJack</i>	Zestaw odblokowanych umiejętności Jack of All Clicks
<i>learnedAutomatron</i>	Zestaw odblokowanych umiejętności Automatron
<i>skillPointsLimit</i>	Obiekt zarządzający punktami umiejętności

6.42.3 Dokumentacja funkcji składowych

6.42.3.1 ClearTrees()

```
void SkillTreesUIManager.ClearTrees ()
```

Czyści wszystkie drzewka umiejętności (resetuje odblokowane umiejętności).

6.42.3.2 DeserializeFromStream()

```
void SkillTreesUIManager.DeserializeFromStream (  
    BinaryReader reader)
```

Deserializacja wszystkich drzewek z binarnego strumienia.

Parametry

<i>reader</i>	BinaryReader do odczytu
---------------	-------------------------

6.42.3.3 InitializeAllTrees()

```
void SkillTreesUIManager.InitializeAllTrees ()
```

Inicjalizuje wszystkie drzewka umiejętności i rejestruje tooltipy oraz linie.

6.42.3.4 InitializeAutomatronTree()

```
void SkillTreesUIManager.InitializeAutomatronTree () [private]
```

Inicjalizacja drzewka Automatron.

6.42.3.5 InitializeJackTree()

```
void SkillTreesUIManager.InitializeJackTree () [private]
```

Inicjalizacja drzewka Jack of All Clicks.

6.42.3.6 InitializeOneClickArmyTree()

```
void SkillTreesUIManager.InitializeOneClickArmyTree () [private]
```

Inicjalizacja drzewka One Click Army: ustawienie przycisków, tooltipów i połączeń linii.

6.42.3.7 SerializeToStream()

```
void SkillTreesUIManager.SerializeToStream (  
    BinaryWriter writer)
```

Serializacja wszystkich drzewek do strumienia binarnego.

Parametry

<i>writer</i>	BinaryWriter do zapisu
---------------	------------------------

6.42.3.8 Update()

```
void SkillTreesUIManager.Update ()
```

Aktualizuje wizualnie wszystkie drzewka umiejętności.

6.42.4 Dokumentacja atrybutów składowych

6.42.4.1 automatronTree

```
AutomatronSkillTree SkillTreesUIManager.automatronTree [private]
```

Instancja drzewka Automatron. Zarządza przyciskami, tooltipami i liniami dla tej klasy.

6.42.4.2 characterClass

`CharacterClass SkillTreesUIManager.characterClass [private]`

Obiekt klasy postaci gracza. Przechowuje informacje o punktach umiejętności, aktywnych skryptach itd.

6.42.4.3 jackTree

`JackSkillTree SkillTreesUIManager.jackTree [private]`

Instancja drzewka Jack of All Clicks. Zarządza przyciskami, tooltipami i liniami dla tej klasy.

6.42.4.4 learnedAutomatron

`HashSet<string> SkillTreesUIManager.learnedAutomatron [private]`

Zestaw odblokowanych umiejętności drzewka Automatron.

6.42.4.5 learnedJack

`HashSet<string> SkillTreesUIManager.learnedJack [private]`

Zestaw odblokowanych umiejętności drzewka Jack of All Clicks.

6.42.4.6 learnedOneClick

`HashSet<string> SkillTreesUIManager.learnedOneClick [private]`

Zestaw odblokowanych umiejętności drzewka One Click Army.

6.42.4.7 oneClickArmyTree

`OneClickArmySkillTree SkillTreesUIManager.oneClickArmyTree [private]`

Instancja drzewka One Click Army. Zarządza przyciskami, tooltipami i liniami dla tej klasy.

6.42.4.8 skillPointsLimit

`SkillPointsLimit SkillTreesUIManager.skillPointsLimit [private]`

Obiekt odpowiedzialny za limity punktów umiejętności i ich wizualizację w UI.

6.42.4.9 tooltipAsset

`VisualTreeAsset SkillTreesUIManager.tooltipAsset`

Asset UI dla tooltipów umiejętności (VisualTreeAsset), używany do wyświetlania informacji o skillach.

6.42.4.10 ui

VisualElement SkillTreesUIManager.ui [private]

Root VisualElement UI, w którym znajdują się przyciski wszystkich drzewek umiejętności.

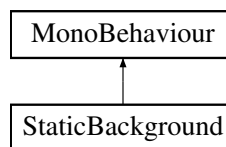
Dokumentacja dla tej klasy została wygenerowana z pliku:

- [SkillTreesUIManager.cs](#)

6.43 Dokumentacja klasy StaticBackground

Klasa odpowiedzialna za dynamiczną zmianę tła w UI w zależności od aktualnie wybranej fabryki. Sprawdza co klatkę, która fabryka jest w fokusie, i aktualizuje RawImage tła odpowiednią teksturą.

Diagram dziedziczenia dla StaticBackground



Metody prywatne

- void [Start](#) ()
Inicjalizuje komponenty RawImage i RectTransform oraz ustawia początkową skalę i tło.
- void [Update](#) ()
Sprawdza w każdej klatce, czy aktualna fabryka się zmieniła. Jeśli tak, aktualizuje tło gry.
- void [UpdateBackground](#) ()
Ustawia odpowiednią teksturę tła w zależności od aktualnej fabryki.

Atrybuty prywatne

- Texture2D[] [backgrounds](#)
Tablica tekstur tła, każda odpowiada innej fabryce.
- [RaptorCore](#) [raptorCore](#)
Referencja do rdzenia gry, który przechowuje informacje o aktualnie aktywnej fabryce.
- string [currentFactory](#)
Nazwa aktualnie wyświetlanej fabryki, używana do porównania i zmiany tła.
- RawImage [rawImage](#)
Komponent RawImage, który wyświetla teksturę tła w UI.
- RectTransform [rectTransform](#)
RectTransform obiektu, wykorzystywany do ustawienia skali i pozycji tła.

6.43.1 Opis szczegółowy

Klasa odpowiedzialna za dynamiczną zmianę tła w UI w zależności od aktualnie wybranej fabryki. Sprawdza co klatkę, która fabryka jest w fokusie, i aktualizuje RawImage tła odpowiednią teksturą.

6.43.2 Dokumentacja funkcji składowych

6.43.2.1 Start()

```
void StaticBackground.Start () [private]
```

Inicjalizuje komponenty RawImage i RectTransform oraz ustawia początkową skalę i tło.

6.43.2.2 Update()

```
void StaticBackground.Update () [private]
```

Sprawdza w każdej klatce, czy aktualna fabryka się zmieniła. Jeśli tak, aktualizuje tło gry.

6.43.2.3 UpdateBackground()

```
void StaticBackground.UpdateBackground () [private]
```

Ustawia odpowiednią teksturę tła w zależności od aktualnej fabryki.

6.43.3 Dokumentacja atrybutów składowych

6.43.3.1 backgrounds

```
Texture2D [] StaticBackground.backgrounds [private]
```

Tablica tekstur tła, każda odpowiada innej fabryce.

6.43.3.2 currentFactory

```
string StaticBackground.currentFactory [private]
```

Nazwa aktualnie wyświetlanej fabryki, używana do porównania i zmiany tła.

6.43.3.3 raptorCore

```
RaptorCore StaticBackground.raptorCore [private]
```

Referencja do rdzenia gry, który przechowuje informacje o aktualnie aktywnej fabryce.

6.43.3.4 rawImage

```
RawImage StaticBackground.rawImage [private]
```

Komponent RawImage, który wyświetla teksturę tła w UI.

6.43.3.5 rectTransform

`RectTransform StaticBackground.rectTransform` [private]

RectTransform obiektu, wykorzystywany do ustawienia skali i pozycji tła.

Dokumentacja dla tej klasy została wygenerowana z pliku:

- [StaticBackground.cs](#)

6.44 Dokumentacja klasy Tooltip

Klasa statyczna odpowiedzialna za wyświetlanie tooltipów w UI. Umożliwia inicjalizację tooltipu, jego pokazanie, ukrycie, przesuwanie oraz rejestrowanie elementów, które będą wywoływać tooltip po najechaniu myszką.

Statyczne metody publiczne

- static void [Init](#) (VisualElement uiRoot, VisualTreeAsset tooltipAsset)
Inicjalizuje tooltip, dodaje go do drzewa UI i ustawia początkowe właściwości.
- static void [SizeRefresh](#) (GeometryChangedEvent evt)
Aktualizuje wymiary tooltipu po zmianie rozmiaru UI.
- static void [Show](#) (string title, string content, Vector2 position, string lore="Lorem ipsum dolor sit amet consectetur adipiscing elit. Consectetur adipiscing elit quisque faucibus ex sapien vitae. Ex sapien vitae pellentesque sem placerat in id. Placerat in id cursus mi pretium tellus duis. Pretium tellus duis convallis tempus leo eu aenean.")
Wyświetla tooltip w podanej pozycji z określonym tytułem, treścią i lore.
- static void [Move](#) (Vector2 position)
Przesuwa tooltip do podanej pozycji kursora. [Tooltip](#) jest przesuwany tak, aby nie wychodził poza obszar UI.
- static void [Hide](#) ()
Ukrywa tooltip, zmienia jego wymiary na 0 i ustawia niewidoczność.
- static void [Register](#) (VisualElement element, string Title, string [Content](#), string lore)
Rejestruje element UI, aby wyświetlał tooltip przy najechaniu myszką.

Statyczne atrybuty prywatne

- static TemplateContainer [root](#)
Główny kontener instancji tooltipu.
- static VisualElement [Frame](#)
Ramka wizualna tooltipu, w której wyświetlane są wszystkie elementy.
- static Label [Title](#)
Label wyświetlający tytuł tooltipu.
- static Label [Content](#)
Label wyświetlający główną treść tooltipu.
- static Label [Lore](#)
Label wyświetlający lore lub dodatkowy opis tooltipu.

6.44.1 Opis szczegółowy

Klasa statyczna odpowiedzialna za wyświetlanie tooltipów w UI. Umożliwia inicjalizację tooltipu, jego pokazanie, ukrycie, przesuwanie oraz rejestrowanie elementów, które będą wywoływać tooltip po najechaniu myszką.

6.44.2 Dokumentacja funkcji składowych

6.44.2.1 Hide()

```
void Tooltip.Hide () [static]
```

Ukrywa tooltip, zmienia jego wymiary na 0 i ustawia niewidoczność.

6.44.2.2 Init()

```
void Tooltip.Init (  
    VisualElement uiRoot,  
    VisualTreeAsset tooltipAsset) [static]
```

Inicjalizuje tooltip, dodaje go do drzewa UI i ustawia początkowe właściwości.

Parametry

<i>uiRoot</i>	Root UI, do którego zostanie dodany tooltip.
<i>tooltipAsset</i>	VisualTreeAsset z layoutem tooltipu.

6.44.2.3 Move()

```
void Tooltip.Move (  
    Vector2 position) [static]
```

Przesuwa tooltip do podanej pozycji kursora. [Tooltip](#) jest przesuwany tak, aby nie wychodził poza obszar UI.

Parametry

<i>position</i>	Pozycja kursora myszki w UI.
-----------------	------------------------------

6.44.2.4 Register()

```
void Tooltip.Register (
    VisualElement element,
    string Title,
    string Content,
    string lore) [static]
```

Rejestruje element UI, aby wyświetlał tooltip przy najechaniu myszką.

Parametry

<i>element</i>	Element UI, który ma wywoływać tooltip.
<i>Title</i>	Tytuł tooltipu.
<i>Content</i>	Główna treść tooltipu.
<i>lore</i>	Dodatkowy opis lub lore tooltipu.

6.44.2.5 Show()

```
void Tooltip.Show (
    string title,
    string content,
    Vector2 position,
    string lore = "Lorem ipsum dolor sit amet consectetur adipiscing elit. Consectetur adipiscing elit quisque fau
[static]
```

Wyświetla tooltip w podanej pozycji z określonym tytułem, treścią i lore.

Parametry

<i>title</i>	Tytuł tooltipu.
<i>content</i>	Główna treść tooltipu.
<i>position</i>	Pozycja kursora myszki w UI.
<i>lore</i>	Dodatkowy opis lub lore tooltipu. Domyślny lorem ipsum.

6.44.2.6 SizeRefresh()

```
void Tooltip.SizeRefresh (
    GeometryChangedEvent evt) [static]
```

Aktualizuje wymiary tooltipu po zmianie rozmiaru UI.

Parametry

<i>evt</i>	Event zawierający nowe wymiary obszaru.
------------	---

6.44.3 Dokumentacja atrybutów składowych

6.44.3.1 Content

Label Tooltip.Content [static], [private]

Label wyświetlający główną treść tooltipu.

6.44.3.2 Frame

VisualElement Tooltip.Frame [static], [private]

Ramka wizualna tooltipu, w której wyświetlane są wszystkie elementy.

6.44.3.3 Lore

Label Tooltip.Lore [static], [private]

Label wyświetlający lore lub dodatkowy opis tooltipu.

6.44.3.4 root

TemplateContainer Tooltip.root [static], [private]

Główny kontener instancji tooltipu.

6.44.3.5 Tittle

Label Tooltip.Tittle [static], [private]

Label wyświetlający tytuł tooltipu.

Dokumentacja dla tej klasy została wygenerowana z pliku:

- [ToolTipStatic.cs](#)

Rozdział 7

Dokumentacja plików

7.1 Dokumentacja pliku ActivePotionsUI.cs

Komponenty

- class [ActivePotionsUI](#)

Zarządza wyświetlaniem aktywnych mikstur (Potions) w UI gry. Pobiera dane z [IdleManager](#) i dynamicznie aktualizuje panel mikstur.

7.2 Dokumentacja pliku BrickMovementScript.cs

Komponenty

- class [BrickMovementScript](#)

Odpowiada za ruch boczka oraz wykrywanie kolizji z granicami. Skrypt losuje kierunek ruchu boczka podczas aktywacji "SkillChecka", zmienia jego wygląd oraz aktualizuje status overlapowania w zależności od kolizji z obszarami granicznymi.

7.3 Dokumentacja pliku BuffManager.cs

Komponenty

- class [BuffManager](#)

Zarządza buffami wyświetlanymi w interfejsie użytkownika.

7.4 Dokumentacja pliku BuffVisual.cs

Komponenty

- class [BuffVisual](#)

Reprezentuje pojedynczy buff wyświetlany w interfejsie użytkownika.

7.5 Dokumentacja pliku CharacterClass.cs

Komponenty

- class [CharacterClass](#)
Główna klasa reprezentująca postać gracza w systemie RPG.

7.6 Dokumentacja pliku ComboVisual.cs

Komponenty

- class [ComboVisual](#)
Odpowiada za wizualne wyświetlanie aktualnego mnożnika combo.

7.7 Dokumentacja pliku ContentPageManager.cs

Komponenty

- class [ContentPageManager](#)
Zarządza przełączaniem stron zawartości w interfejsie użytkownika.

7.8 Dokumentacja pliku CritVisualGenerator.cs

Komponenty

- class [CritVisualGenerator](#)
Generator wizualizacji efektu krytycznego trafienia.

7.9 Dokumentacja pliku DynamicBackground.cs

Komponenty

- class [DynamicBackground](#)
Dynamicznie zmienia tło UI z efektami fade i opcjonalnym zoomem.

7.10 Dokumentacja pliku Factory.cs

Komponenty

- class [Factory](#)
Reprezentuje fabrykę produkującą określony zasób w grze.

7.11 Dokumentacja pliku FactoryUIManager.cs

Komponenty

- class [FactoryUIManager](#)
Zarządza interfejsem użytkownika fabryk w grze.
- class [FactoryUIManager.FactoryUI](#)
Przechowuje referencje do elementów UI jednej fabryki.
- class [FactoryUIManager.FactoryUpgradeLevels](#)
Przechowuje poziomy ulepszeń fabryki.

7.12 Dokumentacja pliku IdleManager.cs

Komponenty

- class [IdleManager](#)
Klasa odpowiedzialna za zarządzanie fabrykami, zasobami i miksturami w grze. Obsługuje ticki produkcji, kupno i sprzedaż fabryk oraz aktywację efektów mikstur. Implementuje [IBinarySaveable](#) do zapisu i wczytywania stanu gry.

7.13 Dokumentacja pliku LayoutController.cs

Komponenty

- class [LayoutController](#)
Kontroler głównego layoutu gry. Zarządza wszystkimi menedżerami UI, zasobami i tooltipami.

7.14 Dokumentacja pliku LevelUIManager.cs

Komponenty

- class [LevelUIManager](#)
Menedżer UI poziomu postaci. Aktualizuje pasek doświadczenia i wyświetla aktualny poziom postaci.

7.15 Dokumentacja pliku MainMenuController.cs

Komponenty

- class [MainMenuController](#)
Kontroler głównego menu gry, obsługuje przyciski Start, Ustawienia, Wyjście oraz slidery i toggle dla ustawień audio i ekranu.

7.16 Dokumentacja pliku OptionsManager.cs

Komponenty

- class [OptionsManager](#)

Zarządza menu opcji gry, w tym ustawieniami audio, wyświetlania i zapisu.

7.17 Dokumentacja pliku Potion.cs

Komponenty

- class [Potion](#)

Reprezentuje miksturę/potion w grze, którą można kupić i aktywować. Mikstury mają koszty w określonych zasobach, czas trwania oraz przypisany efekt.

7.18 Dokumentacja pliku PotionUIManager.cs

Komponenty

- class [PotionUIManager](#)

Zarządza wyświetlaniem panelu mikstur w UI, aktualizuje ceny, efekty i przyciski zakupu.

- class [PotionUIManager.PotionUI](#)

Reprezentuje pojedynczy element UI dla mikstury.

7.19 Dokumentacja pliku PrestigeSystem.cs

Komponenty

- class [PrestigeSystem](#)

Statyczny system prestiżu odpowiedzialny za obsługę resetu postępu i przyznawanie punktów prestiżu.

7.20 Dokumentacja pliku PrestigeUIManager.cs

Komponenty

- class [PrestigeUIManager](#)

Manager interfejsu użytkownika odpowiedzialny za system prestiżu. Obsługuje wyświetlanie statystyk prestiżu, pasek prestiżu oraz okno potwierdzenia resetu.

7.21 Dokumentacja pliku QuarkTestNode.cs

Komponenty

- class [QuarkTestNode](#)

MonoBehaviour odpowiedzialny za testowanie struktury [QuarkType](#). Uruchamia zestaw testów jednostkowych sprawdzających poprawność:

7.22 Dokumentacja pliku QuarkType.cs

Komponenty

- class [QuarkType](#)
QuarkType to wysokozakresowy typ liczbowy oparty o:

7.23 Dokumentacja pliku RaptorCore.cs

Komponenty

- class [RaptorCore](#)
Główny rdzeń gry. Odpowiada za: walutę, kliknięcia, ticki idle, prestiż, serializację stanu gry.

7.24 Dokumentacja pliku Resource.cs

Komponenty

- class [Resource](#)
Reprezentuje pojedynczy zasób w grze. Zawiera aktualną wartość, limit oraz modyfikatory limitu.

7.25 Dokumentacja pliku ResourceManager.cs

Komponenty

- class [ResourceManager](#)
Zarządza wszystkimi zasobami w grze. Odpowiada za ich wartości, limity, zapis/odczyt, aktualizację UI oraz integrację z fabrykami.

7.26 Dokumentacja pliku ResourceShopManager.cs

Komponenty

- class [ResourceShopManager](#)
Manager UI dla sklepu zasobów. Odpowiada za aktualizację wyświetlanych wartości zasobów, obsługę przycisków sprzedaży i suwaka procentowego sprzedaży.

7.27 Dokumentacja pliku SaveManager.cs

Komponenty

- interface [IBinarySaveable](#)
Interfejs dla wszystkich klas, które można zapisać/odczytać w formacie binarnym.
- class [AutoSaveSystem](#)
Statyczny system autozapisu gry. Umożliwia zapis i odczyt wszystkich obiektów implementujących [IBinarySaveable](#), obsługę autopoziomu zapisu i domyślnej ścieżki do pliku.

7.28 Dokumentacja pliku SceneController.cs

Komponenty

- class [SceneController](#)

Kontroler scen w grze. Zarządza przejściami między scenami z użyciem efektu fade in/out.

7.29 Dokumentacja pliku SceneFade.cs

Komponenty

- class [SceneFade](#)

Klasa obsługująca efekt zanikania i pojawiania obrazu w scenie. Może być używana do przejść między scenami lub efektów fade-in/fade-out.

7.30 Dokumentacja pliku SellManager.cs

Komponenty

- class [SellingManager](#)

Klasa odpowiedzialna za sprzedaż zasobów przez gracza i przeliczanie zysków w złocie. Uwzględnia bonusy z mikstur, prestiż i klasy postaci.

7.31 Dokumentacja pliku SkillCheckScript.cs

Komponenty

- class [SkillCheckScript](#)

Skrypt obsługujący mini-grę Skill Check w systemie klików. Gracz musi kliknąć odpowiednim przyciskiem myszy na nadlatujące klocki, aby zdobywać bonusy i utrzymywać combo.

7.32 Dokumentacja pliku SkillManager.cs

Komponenty

- class [SkillManager](#)

Manager umiejętności postaci gracza. Odpowiada za skill check, aktywne idle, efekty mikstur i tymczasowe buffy.

7.33 Dokumentacja pliku SkillPointsLimit.cs

Komponenty

- class [SkillPointsLimit](#)

Manager przycisku Skill Points. Odpowiada za wyświetlanie liczby punktów umiejętności, zmianę koloru przycisku w zależności od dostępności oraz zwiększanie i zmniejszanie punktów postaci.

7.34 Dokumentacja pliku SkillTreesUI.cs

Komponenty

- class [SkillTreeManager](#)
Bazowa klasa dla drzewek umiejętności. Zarządza przyciskami, zależnościami, liniami wizualnymi między umiejętnościami i zapis/odczyt stanu nauczonych umiejętności.
- class [OneClickArmySkillTree](#)
Drzewko umiejętności typu "One Click Army". Zarządza odblokowywaniem i wizualizacją poszczególnych umiejętności dla tej gałęzi.
- class [JackSkillTree](#)
Drzewko umiejętności typu "Jack of All Clicks". Zarządza odblokowywaniem i wizualizacją poszczególnych umiejętności dla tej gałęzi.
- class [AutomatronSkillTree](#)
Drzewko umiejętności typu "Automatron". Zarządza odblokowywaniem i wizualizacją poszczególnych umiejętności dla tej gałęzi.

7.35 Dokumentacja pliku SkillTreesUIManager.cs

Komponenty

- class [SkillTreesUIManager](#)
Menedżer wszystkich trzech drzewek umiejętności w UI. Odpowiada za inicjalizację, aktualizację, serializację i rysowanie linii między przyciskami.

7.36 Dokumentacja pliku StaticBackground.cs

Komponenty

- class [StaticBackground](#)
Klasa odpowiedzialna za dynamiczną zmianę tła w UI w zależności od aktualnie wybranej fabryki. Sprawdza co klatkę, która fabryka jest w fokusie, i aktualizuje RawImage tła odpowiednią teksturą.

7.37 Dokumentacja pliku ToolTipStatic.cs

Komponenty

- class [Tooltip](#)
Klasa statyczna odpowiedzialna za wyświetlanie tooltipów w UI. Umożliwia inicjalizację tooltipu, jego pokazanie, ukrycie, przesuwanie oraz rejestrowanie elementów, które będą wywoływać tooltip po najechaniu myszką.

Skorowidz

- [_AutoSave](#)
 - [AutoSaveSystem, 24](#)
 - [_core](#)
 - [PrestigeSystem, 137](#)
 - [_initialized](#)
 - [PrestigeSystem, 137](#)
 - [_testsFailed](#)
 - [QuarkTestNode, 146](#)
 - [_testsRun](#)
 - [QuarkTestNode, 146](#)
 - [_value](#)
 - [Resource, 168](#)
- [activeClickSource](#)
 - [RaptorCore, 162](#)
- [ActiveIdle](#)
 - [CharacterClass, 36](#)
- [activeIdle](#)
 - [CharacterClass, 48](#)
- [activePotionDisplays](#)
 - [ActivePotionsUI, 16](#)
- [activePotionsPanel](#)
 - [ActivePotionsUI, 16](#)
- [ActivePotionsUI, 13](#)
 - [activePotionDisplays, 16](#)
 - [activePotionsPanel, 16](#)
 - [cincoPotionSprite, 16](#)
 - [CreatePotionDisplay, 14](#)
 - [cuatroPotionSprite, 16](#)
 - [dosPotionSprite, 16](#)
 - [GetPotionColor, 14](#)
 - [GetPotionSprite, 15](#)
 - [idleManager, 17](#)
 - [RebuildPanel, 15](#)
 - [RefreshPanel, 15](#)
 - [seisPotionSprite, 17](#)
 - [Start, 15](#)
 - [tresPotionSprite, 17](#)
 - [uiDocument, 17](#)
 - [unoPotionSprite, 17](#)
 - [Update, 15](#)
 - [UpdateActivePotionsDisplay, 16](#)
 - [UpdatePotionTimers, 16](#)
- [ActivePotionsUI.cs, 221](#)
- [AddCurrency](#)
 - [RaptorCore, 160](#)
- [Addict](#)
 - [CharacterClass, 36](#)
- [AddResource](#)
 - [ResourceManager, 172](#)
- [AlanWake](#)
 - [CharacterClass, 37](#)
- [AlwaysWinner](#)
 - [CharacterClass, 37](#)
- [alwaysWinner](#)
 - [CharacterClass, 48](#)
- [anim](#)
 - [RaptorCore, 162](#)
- [ApplyBuff](#)
 - [BuffManager, 28](#)
- [ApplySkillChanges](#)
 - [CharacterClass, 37](#)
- [Assert](#)
 - [QuarkTestNode, 144](#)
- [audioSource](#)
 - [ContentPageManager, 58](#)
 - [FactoryUIManager, 82](#)
 - [LayoutController, 105](#)
 - [OptionsManager, 125](#)
 - [PotionUIManager, 134](#)
 - [PrestigeUIManager, 140](#)
 - [RaptorCore, 162](#)
 - [ResourceShopManager, 181](#)
- [AutomatronSkillTree, 17](#)
 - [AutomatronSkillTree, 19](#)
 - [CanUnlock, 20](#)
 - [GetDependencies, 20](#)
 - [OnSkillClicked, 20](#)
 - [UpdateVisual, 20](#)
- [automatronTree](#)
 - [SkillTreesUIManager, 212](#)
- [AutoPrestige](#)
 - [RaptorCore, 162](#)
- [AutoSave](#)
 - [AutoSaveSystem, 24](#)
- [AutoSaveSystem, 21](#)
 - [_AutoSave, 24](#)
 - [AutoSave, 24](#)
 - [DeleteSave, 22](#)
 - [GetAutoSave, 22](#)
 - [LazyGetAutoSave, 22](#)
 - [LoadAll, 22](#)
 - [loaded, 24](#)
 - [LoadGame, 23](#)
 - [path, 24](#)
 - [SaveAll, 23](#)
 - [SaveFileName, 24](#)
 - [SaveGame, 23](#)
 - [SavePath, 24](#)

- SavePathS, [23](#)
- SetAutoSave, [23](#)
- AutoSavetoggle
 - MainMenuController, [115](#)
- autoSaveToggle
 - OptionsManager, [125](#)
- availableColor
 - SkillPointsLimit, [202](#)
 - SkillTreeManager, [208](#)
- Awake
 - LayoutController, [103](#)
 - MainMenuController, [113](#)
 - RaptorCore, [160](#)
 - SceneController, [184](#)
 - SceneFade, [186](#)
 - SkillPointsLimit, [201](#)
- backButton
 - MainMenuController, [115](#)
- backgrounds
 - DynamicBackground, [64](#)
 - StaticBackground, [215](#)
- backgroundTransform
 - SceneFade, [187](#)
- backSettings
 - OptionsManager, [125](#)
- BarContainer
 - PrestigeUIManager, [140](#)
- barMask
 - LevelUIManager, [110](#)
- BarTexture
 - PrestigeUIManager, [140](#)
- barTexture
 - LevelUIManager, [110](#)
- baseCost
 - Factory, [68](#)
- BaseLimit
 - Resource, [168](#)
- baseMultiplierCost
 - FactoryUIManager.FactoryUpgradeLevels, [86](#)
- baseProduction
 - Factory, [68](#)
- baseProductionCost
 - FactoryUIManager.FactoryUpgradeLevels, [86](#)
- Bonus
 - SkillCheckScript, [194](#)
- boostedChance
 - CharacterClass, [48](#)
- boughtCount
 - Potion, [129](#)
- brickCount
 - SkillCheckScript, [194](#)
- BrickMovementScript, [25](#)
 - OnTriggerEnter2D, [26](#)
 - OnTriggerExit2D, [26](#)
 - overlap, [27](#)
 - speed, [27](#)
 - sr, [27](#)
 - Start, [26](#)
 - Update, [26](#)
- BrickMovementScript.cs, [221](#)
- brickPrefab
 - SkillCheckScript, [194](#)
- bricks
 - SkillCheckScript, [194](#)
- btn1
 - ContentPageManager, [58](#)
- btn2
 - ContentPageManager, [58](#)
- btn3
 - ContentPageManager, [59](#)
- btn4
 - ContentPageManager, [59](#)
- btn5
 - ContentPageManager, [59](#)
- btn6
 - ContentPageManager, [59](#)
- btn7
 - ContentPageManager, [59](#)
- buffBar
 - BuffManager, [29](#)
- bufficons
 - LayoutController, [105](#)
- BuffManager, [27](#)
 - ApplyBuff, [28](#)
 - buffBar, [29](#)
 - BuffManager, [28](#)
 - buffs, [29](#)
 - RegisterBuff, [28](#)
 - RemoveBuff, [29](#)
- buffManager
 - CharacterClass, [48](#)
 - LayoutController, [105](#)
- BuffManager.cs, [221](#)
- buffs
 - BuffManager, [29](#)
- BuffVisual, [29](#)
 - BuffVisual, [30](#)
 - Hide, [30](#)
 - root, [31](#)
 - Show, [30](#)
 - skillId, [31](#)
 - valueLabel, [31](#)
- BuffVisual.cs, [221](#)
- buttons
 - SkillTreeManager, [208](#)
- buyButton
 - PotionUIManager.PotionUI, [131](#)
- buyButtons
 - FactoryUIManager.FactoryUI, [71](#)
- BuyFactory
 - FactoryUIManager, [76](#)
 - IdleManager, [91](#)
- BuyFactoryMax
 - FactoryUIManager, [76](#)
- BuyPotion
 - IdleManager, [91](#)

- PotionUIManager, [133](#)
- CalculateUpgradeCost
 - FactoryUIManager, [77](#)
- CanClickCurrentResource
 - ResourceManager, [172](#)
- CanUnlock
 - AutomatronSkillTree, [20](#)
 - JackSkillTree, [100](#)
 - OneClickArmySkillTree, [120](#)
 - SkillTreeManager, [205](#)
- CBasevalue
 - RaptorCore, [162](#)
- Ceil
 - QuarkType, [151](#)
- ChampionOfClicks
 - CharacterClass, [37](#)
- championOfClicks
 - CharacterClass, [48](#)
- ChangeBackground
 - DynamicBackground, [63](#)
- ChangeBackgroundRoutine
 - DynamicBackground, [63](#)
- changeInterval
 - DynamicBackground, [64](#)
- CharacterClass, [31](#)
 - Activeldle, [36](#)
 - activeldle, [48](#)
 - Addict, [36](#)
 - AlanWake, [37](#)
 - AlwaysWinner, [37](#)
 - alwaysWinner, [48](#)
 - ApplySkillChanges, [37](#)
 - boostedChance, [48](#)
 - buffManager, [48](#)
 - ChampionOfClicks, [37](#)
 - championOfClicks, [48](#)
 - ChickenDinner, [38](#)
 - chickenDinner, [49](#)
 - ChristmasBonus, [38](#)
 - christmasBonus, [49](#)
 - criticalChance, [49](#)
 - CriticalMass, [38](#)
 - currentExp, [49](#)
 - DeathDose, [38](#)
 - deathDose, [49](#)
 - DeserializeFromStream, [39](#)
 - EnableAlanWake, [39](#)
 - EnableExpBoost, [39](#)
 - EnableGenius, [39](#)
 - Entrepreneur, [39](#)
 - expBoost, [49](#)
 - FailToWin, [40](#)
 - failToWin, [49](#)
 - Failure, [40](#)
 - FailureGrind, [40](#)
 - failureGrind, [49](#)
 - GainExp, [40](#)
 - Genius, [41](#)
 - GetCriticalChance, [41](#)
 - GetCurrentExp, [41](#)
 - GetLevel, [41](#)
 - GetMaxExpCap, [41](#)
 - GetSkillCheckChance, [41](#)
 - HardWorker, [41](#)
 - hardWorker, [50](#)
 - HungryForMore, [42](#)
 - HungryWolf, [42](#)
 - hungryWolf, [50](#)
 - JustBastard, [42](#)
 - komboBoost, [50](#)
 - level, [50](#)
 - LuckyBastard, [42](#)
 - MarketplaceGenius, [43](#)
 - marketplaceGenius, [50](#)
 - maxExpCap, [50](#)
 - maxLvlCap, [50](#)
 - MichealScott, [43](#)
 - michealScott, [50](#)
 - MortalClicker, [43](#)
 - mortalClicker, [51](#)
 - Multitasking, [43](#)
 - multitasking, [51](#)
 - NewLevelCap, [44](#)
 - NoMatterWhat, [44](#)
 - noMatterWhat, [51](#)
 - OneForEveryone, [44](#)
 - oneForEveryone, [51](#)
 - PassiveAgressive, [44](#)
 - passiveAgressive, [51](#)
 - Pedator, [45](#)
 - potionBoost, [51](#)
 - potionRandomChance, [51](#)
 - productionIdleAgressiveBonus, [51](#)
 - productionIdleBonus, [52](#)
 - productionIdlePedatorBonus, [52](#)
 - PushToTheLimit, [45](#)
 - Raptorcore, [52](#)
 - ReactionTest, [45](#)
 - reactionTest, [52](#)
 - ResetCharacter, [45](#)
 - SaveKey, [54](#)
 - sellingBonus, [52](#)
 - sellingHardBonus, [52](#)
 - SerializeToStream, [46](#)
 - SetLevel, [46](#)
 - Shark, [46](#)
 - SkillBasedClicking, [46](#)
 - skillCheckChance, [52](#)
 - skillCheckReduce, [52](#)
 - skillpoints, [53](#)
 - skillPointsLimit, [53](#)
 - Start, [47](#)
 - Symbiosis, [47](#)
 - symbiosis, [53](#)
 - T1, [53](#)
 - T2, [53](#)

- T3, [53](#)
- UnskilledPredator, [47](#)
- unskilledPredator, [53](#)
- Update, [47](#)
- UpdateTreeCounter, [47](#)
- WhatEyesDontSee, [48](#)
- whatEyesDontSee, [53](#)
- characterClass
 - IdleManager, [96](#)
 - LayoutController, [106](#)
 - LevelUIManager, [110](#)
 - PotionUIManager, [134](#)
 - RaptorCore, [162](#)
 - ResourceShopManager, [181](#)
 - SellingManager, [190](#)
 - SkillCheckScript, [194](#)
 - SkillManager, [199](#)
 - SkillPointsLimit, [202](#)
 - SkillTreeManager, [208](#)
 - SkillTreesUIManager, [212](#)
- CharacterClass.cs, [222](#)
- CheckAndTriggerSkillCheck
 - SkillManager, [197](#)
- CheckMouse0
 - SkillCheckScript, [193](#)
- CheckMouse1
 - SkillCheckScript, [193](#)
- CheckPrestige
 - PrestigeSystem, [136](#)
- CheckReactionTest
 - SkillManager, [197](#)
- ChickenDinner
 - CharacterClass, [38](#)
- chickenDinner
 - CharacterClass, [49](#)
- ChickenDinnerEffect
 - SkillManager, [197](#)
- ChristmasBonus
 - CharacterClass, [38](#)
- christmasBonus
 - CharacterClass, [49](#)
- cincoPotionSprite
 - ActivePotionsUI, [16](#)
- ClearBricks
 - SkillCheckScript, [193](#)
- ClearTree
 - SkillTreeManager, [205](#)
- ClearTrees
 - SkillTreesUIManager, [211](#)
- click
 - RaptorCore, [160](#)
- clickCooldown
 - RaptorCore, [162](#)
- clickerObject
 - FactoryUIManager, [82](#)
 - LayoutController, [106](#)
- clickingDebuff
 - RaptorCore, [163](#)
- ClickSell1
 - ResourceShopManager, [180](#)
- ClickSource
 - RaptorCore, [160](#)
- CMultiplier
 - RaptorCore, [163](#)
- comboContainer
 - ComboVisual, [55](#)
- comboLabel
 - ComboVisual, [55](#)
- ComboVisual, [54](#)
 - comboContainer, [55](#)
 - comboLabel, [55](#)
 - isVisible, [56](#)
 - Start, [55](#)
 - ui, [56](#)
 - UpdateCombo, [55](#)
- combovisual
 - SkillCheckScript, [195](#)
- ComboVisual.cs, [222](#)
- CompareMagnitudeTo
 - QuarkType, [151](#)
- CompareTo
 - QuarkType, [151](#)
- Content
 - Tooltip, [219](#)
- contentPage
 - PrestigeUIManager, [140](#)
- contentPage1
 - ContentPageManager, [59](#)
- contentPage2
 - ContentPageManager, [59](#)
- contentPage3
 - ContentPageManager, [59](#)
- contentPage4
 - ContentPageManager, [60](#)
- contentPage5
 - ContentPageManager, [60](#)
- contentPage6
 - ContentPageManager, [60](#)
- contentPage7
 - ContentPageManager, [60](#)
- ContentPageManager, [56](#)
 - audioSource, [58](#)
 - btn1, [58](#)
 - btn2, [58](#)
 - btn3, [59](#)
 - btn4, [59](#)
 - btn5, [59](#)
 - btn6, [59](#)
 - btn7, [59](#)
 - contentPage1, [59](#)
 - contentPage2, [59](#)
 - contentPage3, [59](#)
 - contentPage4, [60](#)
 - contentPage5, [60](#)
 - contentPage6, [60](#)
 - contentPage7, [60](#)

- ContentPageManager, 58
 - Initialize, 58
 - ShowPage, 58
 - ui, 60
- contentPageManager
 - LayoutController, 106
- ContentPageManager.cs, 222
- cost1
 - Potion, 129
- cost2
 - Potion, 129
- costLabel
 - FactoryUIManager.FactoryUI, 71
 - PotionUIManager.PotionUI, 131
- costMultiplier
 - Factory, 69
- costOverEstimate
 - Factory, 67
- count
 - Factory, 69
- countLabel
 - FactoryUIManager.FactoryUI, 71
- CreatePotionDisplay
 - ActivePotionsUI, 14
- critContainer
 - CritVisualGenerator, 62
 - SkillPointsLimit, 202
- critGen
 - RaptorCore, 163
- CritGenerator
 - RaptorCore, 163
- criticalChance
 - CharacterClass, 49
- CriticalMass
 - CharacterClass, 38
- critSource
 - LayoutController, 106
 - OptionsManager, 125
 - RaptorCore, 163
- CritVisualGenerator, 60
 - critContainer, 62
 - SpawnCrit, 61
 - Start, 61
 - ui, 62
- CritVisualGenerator.cs, 222
- cuatroPotionSprite
 - ActivePotionsUI, 16
- Currency
 - RaptorCore, 165
- currencyIcon
 - FactoryUIManager, 82
- currencySlider
 - ResourceShopManager, 181
- currentCost
 - Factory, 70
- currentExp
 - CharacterClass, 49
- currentFactory
 - StaticBackground, 215
- currentFactoryIndex
 - FactoryUIManager, 82
- currentIndex
 - DynamicBackground, 64
- currentResource
 - ResourceManager, 176
- CycleToNextFactory
 - FactoryUIManager, 77
- CycleToPreviousFactory
 - FactoryUIManager, 77
- DeathDose
 - CharacterClass, 38
- deathDose
 - CharacterClass, 49
- DecSkillPoints
 - SkillPointsLimit, 201
- DeleteSave
 - AutoSaveSystem, 22
- DeleteSaveButton
 - MainMenuController, 115
- deleteSaveButton
 - OptionsManager, 125
- dependencies
 - SkillTreeManager, 208
- DeserializeFromStream
 - CharacterClass, 39
 - Factory, 67
 - FactoryUIManager, 77
 - FactoryUIManager.FactoryUpgradeLevels, 86
 - IBinarySaveable, 88
 - IdleManager, 91
 - LayoutController, 103
 - QuarkType, 151
 - RaptorCore, 160
 - Resource, 167
 - ResourceManager, 172
 - SkillTreeManager, 205
 - SkillTreesUIManager, 211
- detailsPanel
 - FactoryUIManager.FactoryUI, 71
- DisablePotionEffect
 - IdleManager, 91
- DisablePrestige
 - PrestigeUIManager, 139
- displayName
 - Resource, 168
- Dokumentacja katalogu Assets, 11
- Dokumentacja katalogu Scripts, 11
- dosPotionSprite
 - ActivePotionsUI, 16
- DrawAllLines
 - SkillTreeManager, 205
- DrawLine
 - SkillTreeManager, 205
- DrawLineBetweenButtons
 - SkillTreeManager, 206
- duration

- Potion, [129](#)
- DynamicBackground, [62](#)
 - backgrounds, [64](#)
 - ChangeBackground, [63](#)
 - ChangeBackgroundRoutine, [63](#)
 - changeInterval, [64](#)
 - currentIndex, [64](#)
 - enableZoom, [64](#)
 - fadeDuration, [64](#)
 - maxZoomValue, [64](#)
 - rawImage, [65](#)
 - rectTransform, [65](#)
 - Start, [63](#)
 - ZoomEffect, [64](#)
 - zoomValue, [65](#)
- DynamicBackground.cs, [222](#)
- effect
 - Potion, [129](#)
- effectDescription
 - Potion, [129](#)
- effectLabel
 - PotionUIManager.PotionUI, [131](#)
- EnableActiveIdle
 - SkillManager, [198](#)
- EnableAlanWake
 - CharacterClass, [39](#)
- EnableChickenDinner
 - SkillManager, [198](#)
- EnableExpBoost
 - CharacterClass, [39](#)
- EnableGenius
 - CharacterClass, [39](#)
- EnableOptions
 - OptionsManager, [123](#)
- EnablePotionEffect
 - IdleManager, [92](#)
- EnablePrestige
 - PrestigeUIManager, [139](#)
- enableZoom
 - DynamicBackground, [64](#)
- Enter
 - RaptorCore, [160](#)
- Entrepreneur
 - CharacterClass, [39](#)
- Equals
 - QuarkType, [151](#)
- ExecutePrestige
 - PrestigeSystem, [136](#)
- exit
 - OptionsManager, [126](#)
- ExitGame
 - OptionsManager, [123](#)
- expBarContainer
 - LevelUIManager, [111](#)
- expBoost
 - CharacterClass, [49](#)
- ExpContainer
 - PrestigeUIManager, [141](#)
- Exponent
 - QuarkType, [156](#)
- ExponentRangeForPrecision
 - QuarkType, [156](#)
- factories
 - IdleManager, [96](#)
- Factory, [65](#)
 - baseCost, [68](#)
 - baseProduction, [68](#)
 - costMultiplier, [69](#)
 - costOverEstimate, [67](#)
 - count, [69](#)
 - currentCost, [70](#)
 - DeserializeFromStream, [67](#)
 - Factory, [66](#)
 - GetProduction, [68](#)
 - GetSellValue, [68](#)
 - isUnlocked, [69](#)
 - name, [69](#)
 - productionMultiplier, [69](#)
 - resource, [69](#)
 - SerializeToStream, [68](#)
 - unlockCost, [69](#)
- Factory.cs, [222](#)
- factoryLabel
 - FactoryUIManager, [83](#)
- FactoryUIManager, [73](#)
 - audioSource, [82](#)
 - BuyFactory, [76](#)
 - BuyFactoryMax, [76](#)
 - CalculateUpgradeCost, [77](#)
 - clickerObject, [82](#)
 - currencyIcon, [82](#)
 - currentFactoryIndex, [82](#)
 - CycleToNextFactory, [77](#)
 - CycleToPreviousFactory, [77](#)
 - DeserializeFromStream, [77](#)
 - factoryLabel, [83](#)
 - FactoryUIManager, [76](#)
 - factoryUIs, [83](#)
 - factoryUpgradeLevels, [83](#)
 - GetFactory, [77](#)
 - idleManager, [83](#)
 - images, [83](#)
 - Initialize, [78](#)
 - InitializeFactories, [78](#)
 - InitializeUnlockButtons, [78](#)
 - IsFactoryUnlocked, [78](#)
 - nextFactoryBtn, [83](#)
 - numberOfFactories, [83](#)
 - prevFactoryBtn, [83](#)
 - raptorCore, [84](#)
 - ResetFactoryGUI, [78](#)
 - scrollView, [84](#)
 - SellFactory, [79](#)
 - SellFactoryMax, [79](#)
 - SerializeToStream, [79](#)
 - shopIcon, [84](#)

- SwitchToFactory, 79
- ToggleFactoryDetails, 80
- ui, 84
- UnlockFactory, 80
- unlockFactory2Btn, 84
- unlockFactory3Btn, 84
- unlockFactory4Btn, 84
- unlockFactory5Btn, 84
- unlockFactory6Btn, 85
- UnlockFactoryFromButton, 80
- Update, 80
- UpdateButton, 81
- UpdateFactoryUI, 81
- UpdateNavigationButtons, 81
- UpdateUnlockButton, 81
- UpdateUnlockButtons, 81
- UpgradeFactoryBaseProduction, 82
- UpgradeFactoryMultiplier, 82
- factoryUIManager
 - LayoutController, 106
- FactoryUIManager.cs, 223
- FactoryUIManager.FactoryUI, 70
 - buyButtons, 71
 - costLabel, 71
 - countLabel, 71
 - detailsPanel, 71
 - isVisible, 71
 - multiplierLevelLabel, 71
 - nameLabel, 71
 - productionLevelLabel, 71
 - sellButtons, 72
 - toggleButton, 72
 - unlockButton, 72
 - upgradeMainBtn, 72
 - upgradeMultiplierBtn, 72
 - upgradePanel, 72
 - upgradeProductionBtn, 72
 - upgradesVisible, 72
- FactoryUIManager.FactoryUpgradeLevels, 85
 - baseMultiplierCost, 86
 - baseProductionCost, 86
 - DeserializeFromStream, 86
 - multiplierLevel, 86
 - productionLevel, 86
 - SerializeToStream, 86
- factoryUIs
 - FactoryUIManager, 83
- factoryUpgradeLevels
 - FactoryUIManager, 83
- FadeCoroutine
 - SceneFade, 186
- fadeDuration
 - DynamicBackground, 64
 - SceneController, 184
- FadeInCoroutine
 - SceneFade, 186
- FadeOutCoroutine
 - SceneFade, 186
- FailToWin
 - CharacterClass, 40
- failToWin
 - CharacterClass, 49
- Failure
 - CharacterClass, 40
- FailureGrind
 - CharacterClass, 40
- failureGrind
 - CharacterClass, 49
- FixedUpdate
 - RaptorCore, 160
- flag
 - SkillCheckScript, 195
- Floor
 - QuarkType, 151
- Frame
 - Tooltip, 219
- fullScreen
 - MainMenuController, 115
 - OptionsManager, 126
- GainExp
 - CharacterClass, 40
- GBasevalue
 - RaptorCore, 163
- Genius
 - CharacterClass, 41
- GetActiveldleValue
 - SkillManager, 198
- GetAutoSave
 - AutoSaveSystem, 22
- GetBaseLimitResource
 - ResourceManager, 172
- GetCriticalChance
 - CharacterClass, 41
- GetCurrentExp
 - CharacterClass, 41
- GetCurrentFactory
 - ResourceManager, 172
- GetDependencies
 - AutomatronSkillTree, 20
 - JackSkillTree, 100
 - OneClickArmySkillTree, 120
- GetFactory
 - FactoryUIManager, 77
 - IdleManager, 92
- GetFactoryCount
 - IdleManager, 92
- GetFocusedFactory
 - IdleManager, 92
- GetHashCode
 - QuarkType, 151
- GetLevel
 - CharacterClass, 41
- GetLimitResource
 - ResourceManager, 173
- GetMaxExpCap
 - CharacterClass, 41

- GetPotion
 - IdleManager, [93](#)
- GetPotionColor
 - ActivePotionsUI, [14](#)
- GetPotionSprite
 - ActivePotionsUI, [15](#)
- GetProduction
 - Factory, [68](#)
- GetResourceValue
 - ResourceManager, [173](#)
- GetResourceValueDirect
 - ResourceManager, [173](#)
- GetSellValue
 - Factory, [68](#)
- GetSkillCheckChance
 - CharacterClass, [41](#)
- GetUnlockedFactoryCount
 - IdleManager, [93](#)
- GMultiplier
 - RaptorCore, [163](#)
- Gold
 - RaptorCore, [163](#)
- goldLabel
 - ResourceShopManager, [181](#)
- HardWorker
 - CharacterClass, [41](#)
- hardWorker
 - CharacterClass, [50](#)
- HasResource
 - ResourceManager, [173](#)
- Hide
 - BuffVisual, [30](#)
 - PrestigeUIManager, [139](#)
 - Tooltip, [217](#)
- HungryForMore
 - CharacterClass, [42](#)
- HungryWolf
 - CharacterClass, [42](#)
- hungryWolf
 - CharacterClass, [50](#)
- IBinarySaveable, [87](#)
 - DeserializeFromStream, [88](#)
 - SaveKey, [88](#)
 - SerializeToStream, [88](#)
- IdleManager, [88](#)
 - BuyFactory, [91](#)
 - BuyPotion, [91](#)
 - characterClass, [96](#)
 - DeserializeFromStream, [91](#)
 - DisablePotionEffect, [91](#)
 - EnablePotionEffect, [92](#)
 - factories, [96](#)
 - GetFactory, [92](#)
 - GetFactoryCount, [92](#)
 - GetFocusedFactory, [92](#)
 - GetPotion, [93](#)
 - GetUnlockedFactoryCount, [93](#)
 - InitFactories, [93](#)
 - InitPotions, [94](#)
 - InitResources, [94](#)
 - OnDisable, [94](#)
 - OnEnable, [94](#)
 - potionFactoryBonus1, [96](#)
 - potionFactoryBonus2, [96](#)
 - potions, [96](#)
 - ProcessTick, [94](#)
 - raptorCore, [96](#)
 - ResetFactory, [94](#)
 - resources, [97](#)
 - SaveKey, [97](#)
 - SellFactory, [94](#)
 - SerializeToStream, [95](#)
 - Start, [95](#)
 - tickCoroutine, [97](#)
 - tickInterval, [97](#)
 - TickLoop, [95](#)
 - UnlockFactory, [95](#)
- idleManager
 - ActivePotionsUI, [17](#)
 - FactoryUIManager, [83](#)
 - LayoutController, [106](#)
 - PotionUIManager, [134](#)
 - RaptorCore, [164](#)
 - ResourceManager, [176](#)
 - SellingManager, [190](#)
 - SkillManager, [199](#)
- IdleManager.cs, [223](#)
- images
 - FactoryUIManager, [83](#)
 - LayoutController, [106](#)
- IncrementSkillBoostResource
 - ResourceManager, [173](#)
- IncrementSkillBoostResourceAll
 - ResourceManager, [173](#)
- IncSkillPoints
 - SkillPointsLimit, [201](#)
- Init
 - PrestigeSystem, [136](#)
 - Tooltip, [217](#)
- InitFactories
 - IdleManager, [93](#)
- Initialize
 - ContentPageManager, [58](#)
 - FactoryUIManager, [78](#)
 - LevelUIManager, [110](#)
 - OptionsManager, [123](#)
 - PotionUIManager, [133](#)
 - PrestigeUIManager, [139](#)
 - ResourceManager, [173](#)
 - ResourceShopManager, [180](#)
 - SellingManager, [189](#)
 - SkillManager, [198](#)
- InitializeAllTrees
 - SkillTreesUIManager, [211](#)
- InitializeAudioSettings

- OptionsManager, [123](#)
- InitializeAutomatronTree
 - SkillTreesUIManager, [212](#)
- InitializeDisplaySettings
 - OptionsManager, [123](#)
- InitializeFactories
 - FactoryUIManager, [78](#)
- InitializeJackTree
 - SkillTreesUIManager, [212](#)
- InitializeLineLayer
 - SkillTreeManager, [206](#)
- InitializeManagers
 - LayoutController, [103](#)
- InitializeOneClickArmyTree
 - SkillTreesUIManager, [212](#)
- InitializePotions
 - PotionUIManager, [133](#)
- InitializeSaveSettings
 - OptionsManager, [123](#)
- InitializeSkills
 - SkillTreeManager, [206](#)
- InitializeTooltips
 - LayoutController, [104](#)
- InitializeUnlockButtons
 - FactoryUIManager, [78](#)
- InitPotions
 - IdleManager, [94](#)
- InitResources
 - IdleManager, [94](#)
- Instance
 - LayoutController, [108](#)
- isActive
 - Potion, [130](#)
- IsApproximatelyEqual
 - QuarkTestNode, [144](#)
- IsFactoryUnlocked
 - FactoryUIManager, [78](#)
- IsNegative
 - QuarkType, [157](#)
- IsSkillCheckActive
 - SkillManager, [198](#)
- isUnlocked
 - Factory, [69](#)
- isVisible
 - ComboVisual, [56](#)
 - FactoryUIManager.FactoryUI, [71](#)
- JackPrestige
 - RaptorCore, [164](#)
- JackSkillTree
 - CanUnlock, [100](#)
 - GetDependencies, [100](#)
 - JackSkillTree, [99](#)
 - OnSkillClicked, [100](#)
 - UpdateVisual, [100](#)
- jackTree
 - SkillTreesUIManager, [213](#)
- JustBastard
 - CharacterClass, [42](#)
- kombo
 - SkillCheckScript, [195](#)
- komboBoost
 - CharacterClass, [50](#)
- LateUpdate
 - SceneFade, [187](#)
- Layout
 - SkillCheckScript, [195](#)
- LayoutController, [101](#)
 - audioSource, [105](#)
 - Awake, [103](#)
 - bufficons, [105](#)
 - buffManager, [105](#)
 - characterClass, [106](#)
 - clickerObject, [106](#)
 - contentPageManager, [106](#)
 - critSource, [106](#)
 - DeserializeFromStream, [103](#)
 - factoryUIManager, [106](#)
 - idleManager, [106](#)
 - images, [106](#)
 - InitializeManagers, [103](#)
 - InitializeTooltips, [104](#)
 - Instance, [108](#)
 - learnedAutomatron, [106](#)
 - learnedJackOfAllClicks, [107](#)
 - learnedOneClickArmy, [107](#)
 - levelUIManager, [107](#)
 - OnEnable, [104](#)
 - optionsManager, [107](#)
 - potionUIManager, [107](#)
 - prestigeUIManager, [107](#)
 - raptorCore, [107](#)
 - ResetGui, [104](#)
 - resourceShopManager, [107](#)
 - SaveKey, [108](#)
 - SerializeToStream, [104](#)
 - SetCurrencyText, [104](#)
 - SetGoldText, [105](#)
 - skillPointsLimit, [108](#)
 - skillTreesUIManager, [108](#)
 - tooltipAsset, [108](#)
 - ui, [108](#)
 - Update, [105](#)
 - UpdateUI, [105](#)
- LayoutController.cs, [223](#)
- LazyGetAutoSave
 - AutoSaveSystem, [22](#)
- learned
 - SkillTreeManager, [208](#)
- learnedAutomatron
 - LayoutController, [106](#)
 - SkillTreesUIManager, [213](#)
- learnedColor
 - SkillTreeManager, [208](#)
- learnedJack
 - SkillTreesUIManager, [213](#)
- learnedJackOfAllClicks

- LayoutController, [107](#)
- learnedOneClick
 - SkillTreesUIManager, [213](#)
- learnedOneClickArmy
 - LayoutController, [107](#)
- level
 - CharacterClass, [50](#)
- LevelUIManager, [109](#)
 - barMask, [110](#)
 - barTexture, [110](#)
 - characterClass, [110](#)
 - expBarContainer, [111](#)
 - Initialize, [110](#)
 - LevelUIManager, [110](#)
 - lvlNumber, [111](#)
 - ui, [111](#)
 - Update, [110](#)
 - UpdateLvlBar, [110](#)
- levelUIManager
 - LayoutController, [107](#)
- LevelUIManager.cs, [223](#)
- Limit
 - Resource, [169](#)
- lineLayer
 - SkillTreeManager, [209](#)
- linkedFactory
 - Potion, [130](#)
- LoadAll
 - AutoSaveSystem, [22](#)
- LoadAllResources
 - ResourceManager, [174](#)
- loaded
 - AutoSaveSystem, [24](#)
- LoadGame
 - AutoSaveSystem, [23](#)
- LoadResource
 - ResourceManager, [174](#)
- LoadScene
 - SceneController, [184](#)
- LoadSceneCoroutine
 - SceneController, [184](#)
- LoadSettings
 - OptionsManager, [124](#)
- lockedColor
 - SkillTreeManager, [209](#)
- Lore
 - Tooltip, [219](#)
- LuckyBastard
 - CharacterClass, [42](#)
- LvlBoost
 - Resource, [168](#)
- lvlNumber
 - LevelUIManager, [111](#)
- mainContainer
 - MainMenuController, [115](#)
- mainMenu
 - MainMenuController, [115](#)
 - OptionsManager, [126](#)
- MainMenuController, [111](#)
 - AutoSavetoggle, [115](#)
 - Awake, [113](#)
 - backButton, [115](#)
 - DeleteSaveButton, [115](#)
 - fullScreen, [115](#)
 - mainContainer, [115](#)
 - mainMenu, [115](#)
 - musicSlider, [115](#)
 - musicSource, [116](#)
 - OnBackButtonClicked, [113](#)
 - OnClick, [113](#)
 - OnEnable, [113](#)
 - OnExitButtonClicked, [113](#)
 - OnMusicVolumeChanged, [113](#)
 - OnSettingsButtonClicked, [114](#)
 - OnSfxVolumeChanged, [114](#)
 - OnStartButtonClicked, [114](#)
 - quitButton, [116](#)
 - resolutionList, [116](#)
 - resolutions, [116](#)
 - sceneController, [116](#)
 - SetFullScreen, [114](#)
 - SetResolution, [114](#)
 - settingsButton, [116](#)
 - settingsContainer, [116](#)
 - sfxSlider, [116](#)
 - sfxSource, [117](#)
 - startButton, [117](#)
- MainMenuController.cs, [223](#)
- Mantissa
 - QuarkType, [156](#)
- MANTISSA_TOLERANCE
 - QuarkTestNode, [147](#)
- MantissaLength
 - QuarkType, [156](#)
- MarketplaceGenius
 - CharacterClass, [43](#)
- marketplaceGenius
 - CharacterClass, [50](#)
- maxExpCap
 - CharacterClass, [50](#)
- maxLvlCap
 - CharacterClass, [50](#)
- maxZoomValue
 - DynamicBackground, [64](#)
- menu
 - OptionsManager, [126](#)
- MichealScott
 - CharacterClass, [43](#)
- michealScott
 - CharacterClass, [50](#)
- MortalClicker
 - CharacterClass, [43](#)
- mortalClicker
 - CharacterClass, [51](#)
- Mouse
 - RaptorCore, [160](#)

- Move
 - Tooltip, 217
- MullLimitResourceAll
 - ResourceManager, 174
- multiplierLevel
 - FactoryUIManager.FactoryUpgradeLevels, 86
- multiplierLevelLabel
 - FactoryUIManager.FactoryUI, 71
- Multitasking
 - CharacterClass, 43
- multitasking
 - CharacterClass, 51
- musicSlider
 - MainMenuController, 115
 - OptionsManager, 126
- musicSource
 - MainMenuController, 116
- name
 - Factory, 69
 - Potion, 130
 - Resource, 168
- nameLabel
 - FactoryUIManager.FactoryUI, 71
 - PotionUIManager.PotionUI, 131
- NegativeOne
 - QuarkType, 157
- NewLevelCap
 - CharacterClass, 44
- nextFactoryBtn
 - FactoryUIManager, 83
- NoMatterWhat
 - CharacterClass, 44
- noMatterWhat
 - CharacterClass, 51
- None
 - RaptorCore, 160
- Normalize
 - QuarkType, 152
- NormalizeDivisor
 - QuarkType, 156
- numberOfFactories
 - FactoryUIManager, 83
- numberOfPotions
 - PotionUIManager, 134
- OnBackButtonClicked
 - MainMenuController, 113
- OnClick
 - MainMenuController, 113
- OnDisable
 - IdleManager, 94
- One
 - QuarkType, 157
- OneClickArmySkillTree, 117
 - CanUnlock, 120
 - GetDependencies, 120
 - OneClickArmySkillTree, 119
 - OnSkillClicked, 120
 - UpdateVisual, 120
- oneClickArmyTree
 - SkillTreesUIManager, 213
- OneClickPrestige
 - RaptorCore, 164
- OneForEveryone
 - CharacterClass, 44
- oneForEveryone
 - CharacterClass, 51
- OnEnable
 - IdleManager, 94
 - LayoutController, 104
 - MainMenuController, 113
- OnExitButtonClicked
 - MainMenuController, 113
- OnMouseDown
 - RaptorCore, 161
- OnMusicVolumeChanged
 - MainMenuController, 113
 - OptionsManager, 124
- OnSettingsButtonClicked
 - MainMenuController, 114
- OnSfxVolumeChanged
 - MainMenuController, 114
 - OptionsManager, 124
- OnSkillClicked
 - AutomatronSkillTree, 20
 - JackSkillTree, 100
 - OneClickArmySkillTree, 120
 - SkillTreeManager, 206
- OnSkillUnlearned
 - SkillTreeManager, 207
- OnStartButtonClicked
 - MainMenuController, 114
- OnTriggerEnter2D
 - BrickMovementScript, 26
- OnTriggerExit2D
 - BrickMovementScript, 26
- operator double
 - QuarkType, 152
- operator QuarkType
 - QuarkType, 152
- operator!=
 - QuarkType, 153
- operator+
 - QuarkType, 153, 154
- operator++
 - QuarkType, 154
- operator-
 - QuarkType, 154
- operator/
 - QuarkType, 155
- operator==
 - QuarkType, 155
- operator*
 - QuarkType, 153
- optionBtn
 - OptionsManager, 126

- options
 - OptionsManager, 126
- OptionsManager, 121
 - audioSource, 125
 - autoSaveToggle, 125
 - backSettings, 125
 - critSource, 125
 - deleteSaveButton, 125
 - EnableOptions, 123
 - exit, 126
 - ExitGame, 123
 - fullScreen, 126
 - Initialize, 123
 - InitializeAudioSettings, 123
 - InitializeDisplaySettings, 123
 - InitializeSaveSettings, 123
 - LoadSettings, 124
 - mainMenu, 126
 - menu, 126
 - musicSlider, 126
 - OnMusicVolumeChanged, 124
 - OnSfxVolumeChanged, 124
 - optionBtn, 126
 - options, 126
 - OptionsManager, 123
 - resolutionList, 126
 - resolutions, 127
 - resume, 127
 - ResumeGame, 124
 - ReturnSettings, 124
 - SetFullScreen, 124
 - SetResolution, 125
 - settings, 127
 - sfxSlider, 127
 - ui, 127
- optionsManager
 - LayoutController, 107
- OptionsManager.cs, 224
- overlap
 - BrickMovementScript, 27
- PassiveAgressive
 - CharacterClass, 44
- passiveAgressive
 - CharacterClass, 51
- path
 - AutoSaveSystem, 24
- Pedator
 - CharacterClass, 45
- PerformClick
 - RaptorCore, 161
- Potion, 127
 - boughtCount, 129
 - cost1, 129
 - cost2, 129
 - duration, 129
 - effect, 129
 - effectDescription, 129
 - isActive, 130
 - linkedFactory, 130
 - name, 130
 - Potion, 128
 - resourceType1, 130
 - resourceType2, 130
 - timeRemaining, 130
- Potion.cs, 224
- potionBoost
 - CharacterClass, 51
- potionClickBonus1
 - RaptorCore, 164
- potionClickBonus2
 - RaptorCore, 164
- potionFactoryBonus1
 - IdleManager, 96
- potionFactoryBonus2
 - IdleManager, 96
- potionPanel
 - PotionUIManager.PotionUI, 131
- potionRandomChance
 - CharacterClass, 51
- potions
 - IdleManager, 96
- potionSellBonus1
 - SellingManager, 190
- potionSellBonus2
 - SellingManager, 190
- PotionUIManager, 132
 - audioSource, 134
 - BuyPotion, 133
 - characterClass, 134
 - idleManager, 134
 - Initialize, 133
 - InitializePotions, 133
 - numberOfPotions, 134
 - PotionUIManager, 133
 - potionUIs, 134
 - raptorCore, 135
 - scrollView, 135
 - ui, 135
 - Update, 134
 - UpdatePotionUI, 134
- potionUIManager
 - LayoutController, 107
- PotionUIManager.cs, 224
- PotionUIManager.PotionUI, 131
 - buyButton, 131
 - costLabel, 131
 - effectLabel, 131
 - nameLabel, 131
 - potionPanel, 131
- potionUIs
 - PotionUIManager, 134
- Pow
 - QuarkType, 155
- PrestigeAccept
 - PrestigeUIManager, 141
- PrestigeButtonMenu

- PrestigeUIManager, 141
- PrestigeCancel
 - PrestigeUIManager, 141
- PrestigeCount
 - RaptorCore, 164
- PrestigeRoot
 - PrestigeUIManager, 141
- PrestigeSystem, 135
 - _core, 137
 - _initialized, 137
 - CheckPrestige, 136
 - ExecutePrestige, 136
 - Init, 136
- PrestigeSystem.cs, 224
- PrestigeTrigger
 - PrestigeUIManager, 139
- PrestigeUIManager, 137
 - audioSource, 140
 - BarContainer, 140
 - BarTexture, 140
 - contentPage, 140
 - DisablePrestige, 139
 - EnablePrestige, 139
 - ExpContainer, 141
 - Hide, 139
 - Initialize, 139
 - PrestigeAccept, 141
 - PrestigeButtonMenu, 141
 - PrestigeCancel, 141
 - PrestigeRoot, 141
 - PrestigeTrigger, 139
 - PrestigeUIManager, 139
 - rp, 141
 - Trigger, 140
 - ui_ref, 141
 - UpdateContentPage, 140
- prestigeUIManager
 - LayoutController, 107
- PrestigeUIManager.cs, 224
- prevFactoryBtn
 - FactoryUIManager, 83
- ProcessTick
 - IdleManager, 94
- productionIdleAgressiveBonus
 - CharacterClass, 51
- productionIdleBonus
 - CharacterClass, 52
- productionIdlePedatorBonus
 - CharacterClass, 52
- productionLevel
 - FactoryUIManager.FactoryUpgradeLevels, 86
- productionLevelLabel
 - FactoryUIManager.FactoryUI, 71
- productionMultiplier
 - Factory, 69
- PushToTheLimit
 - CharacterClass, 45
- QuarkTestNode, 142
 - _testsFailed, 146
 - _testsRun, 146
 - Assert, 144
 - IsApproximatelyEqual, 144
 - MANTISSA_TOLERANCE, 147
 - Start, 144
 - TestAddition, 144
 - TestComparisonOperators, 144
 - TestDivision, 145
 - TestMultiplication, 145
 - TestMultiplicationEXT, 145
 - TestNormalizationAndConstructors, 145
 - TestPrecisionLossAndEdgeCases, 145
 - TestPrimitiveAddition, 145
 - TestPrimitiveConversions, 145
 - TestPrimitiveMultiplication, 146
 - TestPrimitiveSubtraction, 146
 - TestSignedAndRangeArithmetic, 146
 - TestSubtraction, 146
 - Update, 146
- QuarkTestNode.cs, 224
- QuarkType, 147
 - Ceil, 151
 - CompareMagnitudeTo, 151
 - CompareTo, 151
 - DeserializeFromStream, 151
 - Equals, 151
 - Exponent, 156
 - ExponentRangeForPrecision, 156
 - Floor, 151
 - GetHashCode, 151
 - IsNegative, 157
 - Mantissa, 156
 - MantissaLength, 156
 - NegativeOne, 157
 - Normalize, 152
 - NormalizeDivisor, 156
 - One, 157
 - operator double, 152
 - operator QuarkType, 152
 - operator!=, 153
 - operator+, 153, 154
 - operator++, 154
 - operator-, 154
 - operator/, 155
 - operator==, 155
 - operator*, 153
 - Pow, 155
 - QuarkType, 150
 - SerializeToStream, 155
 - ToString, 156
 - Zero, 157
- QuarkType.cs, 225
- quitButton
 - MainMenuController, 116
- RandomPotionEffect
 - SkillManager, 198
- RaptorCore, 157

- activeClickSource, [162](#)
- AddCurrency, [160](#)
- anim, [162](#)
- audioSource, [162](#)
- AutoPrestige, [162](#)
- Awake, [160](#)
- CBasevalue, [162](#)
- characterClass, [162](#)
- click, [160](#)
- clickCooldown, [162](#)
- clickingDebuff, [163](#)
- ClickSource, [160](#)
- CMultiplier, [163](#)
- critGen, [163](#)
- CritGenerator, [163](#)
- critSource, [163](#)
- Currency, [165](#)
- DeserializeFromStream, [160](#)
- Enter, [160](#)
- FixedUpdate, [160](#)
- GBasevalue, [163](#)
- GMultiplier, [163](#)
- Gold, [163](#)
- idleManager, [164](#)
- JackPrestige, [164](#)
- Mouse, [160](#)
- None, [160](#)
- OneClickPrestige, [164](#)
- OnMouseDown, [161](#)
- PerformClick, [161](#)
- potionClickBonus1, [164](#)
- potionClickBonus2, [164](#)
- PrestigeCount, [164](#)
- ResetCore, [161](#)
- ResourceManager, [165](#)
- SaveKey, [165](#)
- SellManager, [165](#)
- SerializeToStream, [161](#)
- skillCheck, [164](#)
- SkillManager, [165](#)
- SkillMultiplier, [164](#)
- sourceBlockEndTime, [165](#)
- Space, [160](#)
- Start, [161](#)
- SubCurrency, [161](#)
- tickCount, [165](#)
- Update, [161](#)
- Raptorcore
 - CharacterClass, [52](#)
- raptorCore
 - FactoryUIManager, [84](#)
 - IdleManager, [96](#)
 - LayoutController, [107](#)
 - PotionUIManager, [135](#)
 - ResourceShopManager, [181](#)
 - SellingManager, [191](#)
 - SkillCheckScript, [195](#)
 - SkillManager, [199](#)
 - StaticBackground, [215](#)
- RaptorCore.cs, [225](#)
- rawImage
 - DynamicBackground, [65](#)
 - StaticBackground, [215](#)
- ReactionTest
 - CharacterClass, [45](#)
- reactionTest
 - CharacterClass, [52](#)
- RebuildPanel
 - ActivePotionsUI, [15](#)
- rectTransform
 - DynamicBackground, [65](#)
 - SceneFade, [187](#)
 - StaticBackground, [215](#)
- RefreshPanel
 - ActivePotionsUI, [15](#)
- Register
 - Tooltip, [217](#)
- RegisterBuff
 - BuffManager, [28](#)
- RegisterResource
 - ResourceManager, [174](#)
- RegisterTooltip
 - SkillTreeManager, [207](#)
- RemoveBuff
 - BuffManager, [29](#)
- RemoveResource
 - ResourceManager, [174](#)
- res1curr
 - ResourceShopManager, [181](#)
- res2curr
 - ResourceShopManager, [182](#)
- res3curr
 - ResourceShopManager, [182](#)
- res4curr
 - ResourceShopManager, [182](#)
- res5curr
 - ResourceShopManager, [182](#)
- res6curr
 - ResourceShopManager, [182](#)
- ResetCharacter
 - CharacterClass, [45](#)
- ResetCore
 - RaptorCore, [161](#)
- ResetFactory
 - IdleManager, [94](#)
- ResetFactoryGUI
 - FactoryUIManager, [78](#)
- ResetGui
 - LayoutController, [104](#)
- ResetResources
 - ResourceManager, [175](#)
- ResetSkillCheck
 - SkillCheckScript, [193](#)
- resolutionList
 - MainMenuController, [116](#)
 - OptionsManager, [126](#)

- resolutions
 - MainMenuController, [116](#)
 - OptionsManager, [127](#)
- Resource, [166](#)
 - _value, [168](#)
 - BaseLimit, [168](#)
 - DeserializeFromStream, [167](#)
 - displayName, [168](#)
 - Limit, [169](#)
 - LvlBoost, [168](#)
 - name, [168](#)
 - Resource, [167](#)
 - SerializeToStream, [167](#)
 - SkillBoost, [168](#)
 - value, [169](#)
- resource
 - Factory, [69](#)
- Resource.cs, [225](#)
- resource1Limit
 - ResourceManager, [176](#)
- resource1Value
 - ResourceManager, [176](#)
- resource2Limit
 - ResourceManager, [176](#)
- resource2Value
 - ResourceManager, [176](#)
- resource3Limit
 - ResourceManager, [177](#)
- resource3Value
 - ResourceManager, [177](#)
- resource4Limit
 - ResourceManager, [177](#)
- resource4Value
 - ResourceManager, [177](#)
- resource5Limit
 - ResourceManager, [177](#)
- resource5Value
 - ResourceManager, [177](#)
- resource6Limit
 - ResourceManager, [177](#)
- resource6Value
 - ResourceManager, [177](#)
- ResourceManager, [169](#)
 - AddResource, [172](#)
 - CanClickCurrentResource, [172](#)
 - currentResource, [176](#)
 - DeserializeFromStream, [172](#)
 - GetBaseLimitResource, [172](#)
 - GetCurrentFactory, [172](#)
 - GetLimitResource, [173](#)
 - GetResourceValue, [173](#)
 - GetResourceValueDirect, [173](#)
 - HasResource, [173](#)
 - idleManager, [176](#)
 - IncrementSkillBoostResource, [173](#)
 - IncrementSkillBoostResourceAll, [173](#)
 - Initialize, [173](#)
 - LoadAllResources, [174](#)
 - LoadResource, [174](#)
 - MullLimitResourceAll, [174](#)
 - RaptorCore, [165](#)
 - RegisterResource, [174](#)
 - RemoveResource, [174](#)
 - ResetResources, [175](#)
 - resource1Limit, [176](#)
 - resource1Value, [176](#)
 - resource2Limit, [176](#)
 - resource2Value, [176](#)
 - resource3Limit, [177](#)
 - resource3Value, [177](#)
 - resource4Limit, [177](#)
 - resource4Value, [177](#)
 - resource5Limit, [177](#)
 - resource5Value, [177](#)
 - resource6Limit, [177](#)
 - resource6Value, [177](#)
 - resources, [178](#)
 - SaveKey, [178](#)
 - SaveResource, [175](#)
 - SaveResourceValues, [175](#)
 - SerializeToStream, [175](#)
 - SetCurrentResource, [175](#)
 - SetLvlBoostResource, [175](#)
 - SetResourceValueDirect, [176](#)
- resourceManager
 - SellingManager, [191](#)
- ResourceManager.cs, [225](#)
- resources
 - IdleManager, [97](#)
 - ResourceManager, [178](#)
- ResourceShopManager, [178](#)
 - audioSource, [181](#)
 - characterClass, [181](#)
 - ClickSell1, [180](#)
 - currencySlider, [181](#)
 - goldLabel, [181](#)
 - Initialize, [180](#)
 - raptorCore, [181](#)
 - res1curr, [181](#)
 - res2curr, [182](#)
 - res3curr, [182](#)
 - res4curr, [182](#)
 - res5curr, [182](#)
 - res6curr, [182](#)
 - ResourceShopManager, [180](#)
 - sell1, [182](#)
 - SetCurrencyText, [180](#)
 - SetGoldText, [180](#)
 - sliderText, [182](#)
 - ui, [182](#)
 - Update, [181](#)
- resourceShopManager
 - LayoutController, [107](#)
- ResourceShopManager.cs, [225](#)
- resourceType1
 - Potion, [130](#)

- resourceType2
 - Potion, [130](#)
- resume
 - OptionsManager, [127](#)
- ResumeGame
 - OptionsManager, [124](#)
- ReturnSettings
 - OptionsManager, [124](#)
- root
 - BuffVisual, [31](#)
 - Tooltip, [219](#)
- rp
 - PrestigeUIManager, [141](#)
- SaveAll
 - AutoSaveSystem, [23](#)
- SaveFileName
 - AutoSaveSystem, [24](#)
- SaveGame
 - AutoSaveSystem, [23](#)
- SaveKey
 - CharacterClass, [54](#)
 - IBinarySaveable, [88](#)
 - IdleManager, [97](#)
 - LayoutController, [108](#)
 - RaptorCore, [165](#)
 - ResourceManager, [178](#)
- SaveManager.cs, [225](#)
- SavePath
 - AutoSaveSystem, [24](#)
- SavePathS
 - AutoSaveSystem, [23](#)
- SaveResource
 - ResourceManager, [175](#)
- SaveResourceValues
 - ResourceManager, [175](#)
- SceneController, [183](#)
 - Awake, [184](#)
 - fadeDuration, [184](#)
 - LoadScene, [184](#)
 - LoadSceneCoroutine, [184](#)
 - sceneFade, [184](#)
 - Start, [184](#)
- sceneController
 - MainMenuController, [116](#)
- SceneController.cs, [226](#)
- SceneFade, [185](#)
 - Awake, [186](#)
 - backgroundTransform, [187](#)
 - FadeCoroutine, [186](#)
 - FadeInCoroutine, [186](#)
 - FadeOutCoroutine, [186](#)
 - LateUpdate, [187](#)
 - rectTransform, [187](#)
 - sceneImage, [187](#)
- sceneFade
 - SceneController, [184](#)
- SceneFade.cs, [226](#)
- sceneImage
 - SceneFade, [187](#)
- scrollView
 - FactoryUIManager, [84](#)
 - PotionUIManager, [135](#)
- seisPotionSprite
 - ActivePotionsUI, [17](#)
- sell1
 - ResourceShopManager, [182](#)
- sellButtons
 - FactoryUIManager.FactoryUI, [72](#)
- SellFactory
 - FactoryUIManager, [79](#)
 - IdleManager, [94](#)
- SellFactoryMax
 - FactoryUIManager, [79](#)
- sellingBonus
 - CharacterClass, [52](#)
- sellingHardBonus
 - CharacterClass, [52](#)
- SellingManager, [188](#)
 - characterClass, [190](#)
 - idleManager, [190](#)
 - Initialize, [189](#)
 - potionSellBonus1, [190](#)
 - potionSellBonus2, [190](#)
 - raptorCore, [191](#)
 - resourceManager, [191](#)
 - SellMaterials, [189](#)
 - SellResource, [189](#)
 - Update, [190](#)
- SellManager
 - RaptorCore, [165](#)
- SellManager.cs, [226](#)
- SellMaterials
 - SellingManager, [189](#)
- SellResource
 - SellingManager, [189](#)
- SerializeToStream
 - CharacterClass, [46](#)
 - Factory, [68](#)
 - FactoryUIManager, [79](#)
 - FactoryUIManager.FactoryUpgradeLevels, [86](#)
 - IBinarySaveable, [88](#)
 - IdleManager, [95](#)
 - LayoutController, [104](#)
 - QuarkType, [155](#)
 - RaptorCore, [161](#)
 - Resource, [167](#)
 - ResourceManager, [175](#)
 - SkillTreeManager, [207](#)
 - SkillTreesUIManager, [212](#)
- SetAutoSave
 - AutoSaveSystem, [23](#)
- SetCurrencyText
 - LayoutController, [104](#)
 - ResourceShopManager, [180](#)
- SetCurrentResource
 - ResourceManager, [175](#)

- SetFullScreen
 - MainMenuController, [114](#)
 - OptionsManager, [124](#)
- SetGoldText
 - LayoutController, [105](#)
 - ResourceShopManager, [180](#)
- SetLevel
 - CharacterClass, [46](#)
- SetLvlBoostResource
 - ResourceManager, [175](#)
- SetResolution
 - MainMenuController, [114](#)
 - OptionsManager, [125](#)
- SetResourceValueDirect
 - ResourceManager, [176](#)
- settings
 - OptionsManager, [127](#)
- settingsButton
 - MainMenuController, [116](#)
- settingsContainer
 - MainMenuController, [116](#)
- sfxSlider
 - MainMenuController, [116](#)
 - OptionsManager, [127](#)
- sfxSource
 - MainMenuController, [117](#)
- Shark
 - CharacterClass, [46](#)
- shopIcon
 - FactoryUIManager, [84](#)
- Show
 - BuffVisual, [30](#)
 - Tooltip, [218](#)
- ShowPage
 - ContentPageManager, [58](#)
- SizeRefresh
 - Tooltip, [218](#)
- SkillBasedClicking
 - CharacterClass, [46](#)
- SkillBoost
 - Resource, [168](#)
- Skillbtn
 - SkillPointsLimit, [202](#)
- skillCheck
 - RaptorCore, [164](#)
 - SkillManager, [199](#)
- skillCheckChance
 - CharacterClass, [52](#)
- SkillCheckCooldown
 - SkillManager, [200](#)
- skillCheckCooldown
 - SkillManager, [199](#)
- skillCheckCooldownCount
 - SkillManager, [200](#)
- skillCheckReduce
 - CharacterClass, [52](#)
- SkillCheckScript, [191](#)
 - Bonus, [194](#)
 - brickCount, [194](#)
 - brickPrefab, [194](#)
 - bricks, [194](#)
 - characterClass, [194](#)
 - CheckMouse0, [193](#)
 - CheckMouse1, [193](#)
 - ClearBricks, [193](#)
 - combovisual, [195](#)
 - flag, [195](#)
 - kombo, [195](#)
 - Layout, [195](#)
 - raptorCore, [195](#)
 - ResetSkillCheck, [193](#)
 - SpawnBricks, [193](#)
 - Start, [193](#)
 - StartSkillCheck, [194](#)
 - Update, [194](#)
 - waitTime, [195](#)
- SkillCheckScript.cs, [226](#)
- skillId
 - BuffVisual, [31](#)
- SkillManager, [196](#)
 - characterClass, [199](#)
 - CheckAndTriggerSkillCheck, [197](#)
 - CheckReactionTest, [197](#)
 - ChickenDinnerEffect, [197](#)
 - EnableActiveldle, [198](#)
 - EnableChickenDinner, [198](#)
 - GetActiveldleValue, [198](#)
 - idleManager, [199](#)
 - Initialize, [198](#)
 - IsSkillCheckActive, [198](#)
 - RandomPotionEffect, [198](#)
 - RaptorCore, [165](#)
 - raptorCore, [199](#)
 - skillCheck, [199](#)
 - SkillCheckCooldown, [200](#)
 - skillCheckCooldown, [199](#)
 - skillCheckCooldownCount, [200](#)
 - TriggerSkillCheck, [199](#)
 - UpdateSkillCheckCooldown, [199](#)
- SkillManager.cs, [226](#)
- SkillMultiplier
 - RaptorCore, [164](#)
- skillpoints
 - CharacterClass, [53](#)
- SkillPointsLimit, [200](#)
 - availableColor, [202](#)
 - Awake, [201](#)
 - characterClass, [202](#)
 - critContainer, [202](#)
 - DecSkillPoints, [201](#)
 - IncSkillPoints, [201](#)
 - Skillbtn, [202](#)
 - ui, [202](#)
 - unavailableColor, [202](#)
 - UpdateButton, [201](#)
- skillPointsLimit

- CharacterClass, [53](#)
- LayoutController, [108](#)
- SkillTreeManager, [209](#)
- SkillTreesUIManager, [213](#)
- SkillPointsLimit.cs, [226](#)
- skillTreeContainer
 - SkillTreeManager, [209](#)
- SkillTreeManager, [203](#)
 - availableColor, [208](#)
 - buttons, [208](#)
 - CanUnlock, [205](#)
 - characterClass, [208](#)
 - ClearTree, [205](#)
 - dependencies, [208](#)
 - DeserializeFromStream, [205](#)
 - DrawAllLines, [205](#)
 - DrawLine, [205](#)
 - DrawLineBetweenButtons, [206](#)
 - InitializeLineLayer, [206](#)
 - InitializeSkills, [206](#)
 - learned, [208](#)
 - learnedColor, [208](#)
 - lineLayer, [209](#)
 - lockedColor, [209](#)
 - OnSkillClicked, [206](#)
 - OnSkillUnlearned, [207](#)
 - RegisterTooltip, [207](#)
 - SerializeToStream, [207](#)
 - skillPointsLimit, [209](#)
 - skillTreeContainer, [209](#)
 - SkillTreeManager, [204](#)
 - ui, [209](#)
 - UpdateAll, [207](#)
 - UpdateVisual, [208](#)
- SkillTreesUI.cs, [227](#)
- SkillTreesUIManager, [209](#)
 - automatronTree, [212](#)
 - characterClass, [212](#)
 - ClearTrees, [211](#)
 - DeserializeFromStream, [211](#)
 - InitializeAllTrees, [211](#)
 - InitializeAutomatronTree, [212](#)
 - InitializeJackTree, [212](#)
 - InitializeOneClickArmyTree, [212](#)
 - jackTree, [213](#)
 - learnedAutomatron, [213](#)
 - learnedJack, [213](#)
 - learnedOneClick, [213](#)
 - oneClickArmyTree, [213](#)
 - SerializeToStream, [212](#)
 - skillPointsLimit, [213](#)
 - SkillTreesUIManager, [211](#)
 - tooltipAsset, [213](#)
 - ui, [213](#)
 - Update, [212](#)
- skillTreesUIManager
 - LayoutController, [108](#)
- SkillTreesUIManager.cs, [227](#)
- sliderText
 - ResourceShopManager, [182](#)
- sourceBlockEndTime
 - RaptorCore, [165](#)
- Space
 - RaptorCore, [160](#)
- SpawnBricks
 - SkillCheckScript, [193](#)
- SpawnCrit
 - CritVisualGenerator, [61](#)
- speed
 - BrickMovementScript, [27](#)
- sr
 - BrickMovementScript, [27](#)
- Start
 - ActivePotionsUI, [15](#)
 - BrickMovementScript, [26](#)
 - CharacterClass, [47](#)
 - ComboVisual, [55](#)
 - CritVisualGenerator, [61](#)
 - DynamicBackground, [63](#)
 - IdleManager, [95](#)
 - QuarkTestNode, [144](#)
 - RaptorCore, [161](#)
 - SceneController, [184](#)
 - SkillCheckScript, [193](#)
 - StaticBackground, [215](#)
- startButton
 - MainMenuController, [117](#)
- StartSkillCheck
 - SkillCheckScript, [194](#)
- StaticBackground, [214](#)
 - backgrounds, [215](#)
 - currentFactory, [215](#)
 - raptorCore, [215](#)
 - rawImage, [215](#)
 - rectTransform, [215](#)
 - Start, [215](#)
 - Update, [215](#)
 - UpdateBackground, [215](#)
- StaticBackground.cs, [227](#)
- SubCurrency
 - RaptorCore, [161](#)
- SwitchToFactory
 - FactoryUIManager, [79](#)
- Symbiosis
 - CharacterClass, [47](#)
- symbiosis
 - CharacterClass, [53](#)
- T1
 - CharacterClass, [53](#)
- T2
 - CharacterClass, [53](#)
- T3
 - CharacterClass, [53](#)
- TestAddition
 - QuarkTestNode, [144](#)
- TestComparisonOperators

- QuarkTestNode, [144](#)
- TestDivision
 - QuarkTestNode, [145](#)
- TestMultiplication
 - QuarkTestNode, [145](#)
- TestMultiplicationEXT
 - QuarkTestNode, [145](#)
- TestNormalizationAndConstructors
 - QuarkTestNode, [145](#)
- TestPrecisionLossAndEdgeCases
 - QuarkTestNode, [145](#)
- TestPrimitiveAddition
 - QuarkTestNode, [145](#)
- TestPrimitiveConversions
 - QuarkTestNode, [145](#)
- TestPrimitiveMultiplication
 - QuarkTestNode, [146](#)
- TestPrimitiveSubtraction
 - QuarkTestNode, [146](#)
- TestSignedAndRangeArithmetic
 - QuarkTestNode, [146](#)
- TestSubtraction
 - QuarkTestNode, [146](#)
- tickCoroutine
 - IdleManager, [97](#)
- tickCount
 - RaptorCore, [165](#)
- tickInterval
 - IdleManager, [97](#)
- TickLoop
 - IdleManager, [95](#)
- timeRemaining
 - Potion, [130](#)
- Tittle
 - Tooltip, [219](#)
- toggleButton
 - FactoryUIManager.FactoryUI, [72](#)
- ToggleFactoryDetails
 - FactoryUIManager, [80](#)
- ToolTipStatic.cs, [227](#)
- Tooltip, [216](#)
 - Content, [219](#)
 - Frame, [219](#)
 - Hide, [217](#)
 - Init, [217](#)
 - Lore, [219](#)
 - Move, [217](#)
 - Register, [217](#)
 - root, [219](#)
 - Show, [218](#)
 - SizeRefresh, [218](#)
 - Tittle, [219](#)
- tooltipAsset
 - LayoutController, [108](#)
 - SkillTreesUIManager, [213](#)
- ToString
 - QuarkType, [156](#)
- tresPotionSprite
 - ActivePotionsUI, [17](#)
- Trigger
 - PrestigeUIManager, [140](#)
- TriggerSkillCheck
 - SkillManager, [199](#)
- ui
 - ComboVisual, [56](#)
 - ContentPageManager, [60](#)
 - CritVisualGenerator, [62](#)
 - FactoryUIManager, [84](#)
 - LayoutController, [108](#)
 - LevelUIManager, [111](#)
 - OptionsManager, [127](#)
 - PotionUIManager, [135](#)
 - ResourceShopManager, [182](#)
 - SkillPointsLimit, [202](#)
 - SkillTreeManager, [209](#)
 - SkillTreesUIManager, [213](#)
- ui_ref
 - PrestigeUIManager, [141](#)
- uiDocument
 - ActivePotionsUI, [17](#)
- unavailableColor
 - SkillPointsLimit, [202](#)
- unlockButton
 - FactoryUIManager.FactoryUI, [72](#)
- unlockCost
 - Factory, [69](#)
- UnlockFactory
 - FactoryUIManager, [80](#)
 - IdleManager, [95](#)
- unlockFactory2Btn
 - FactoryUIManager, [84](#)
- unlockFactory3Btn
 - FactoryUIManager, [84](#)
- unlockFactory4Btn
 - FactoryUIManager, [84](#)
- unlockFactory5Btn
 - FactoryUIManager, [84](#)
- unlockFactory6Btn
 - FactoryUIManager, [85](#)
- UnlockFactoryFromButton
 - FactoryUIManager, [80](#)
- unoPotionSprite
 - ActivePotionsUI, [17](#)
- UnskilledPredator
 - CharacterClass, [47](#)
- unskilledPredator
 - CharacterClass, [53](#)
- Update
 - ActivePotionsUI, [15](#)
 - BrickMovementScript, [26](#)
 - CharacterClass, [47](#)
 - FactoryUIManager, [80](#)
 - LayoutController, [105](#)
 - LevelUIManager, [110](#)
 - PotionUIManager, [134](#)
 - QuarkTestNode, [146](#)

- RaptorCore, 161
- ResourceShopManager, 181
- SellingManager, 190
- SkillCheckScript, 194
- SkillTreesUIManager, 212
- StaticBackground, 215
- UpdateActivePotionsDisplay
 - ActivePotionsUI, 16
- UpdateAll
 - SkillTreeManager, 207
- UpdateBackground
 - StaticBackground, 215
- UpdateButton
 - FactoryUIManager, 81
 - SkillPointsLimit, 201
- UpdateCombo
 - ComboVisual, 55
- UpdateContentPage
 - PrestigeUIManager, 140
- UpdateFactoryUI
 - FactoryUIManager, 81
- UpdateLvlBar
 - LevelUIManager, 110
- UpdateNavigationButtons
 - FactoryUIManager, 81
- UpdatePotionTimers
 - ActivePotionsUI, 16
- UpdatePotionUI
 - PotionUIManager, 134
- UpdateSkillCheckCooldown
 - SkillManager, 199
- UpdateTreeCounter
 - CharacterClass, 47
- UpdateUI
 - LayoutController, 105
- UpdateUnlockButton
 - FactoryUIManager, 81
- UpdateUnlockButtons
 - FactoryUIManager, 81
- UpdateVisual
 - AutomatronSkillTree, 20
 - JackSkillTree, 100
 - OneClickArmySkillTree, 120
 - SkillTreeManager, 208
- UpgradeFactoryBaseProduction
 - FactoryUIManager, 82
- UpgradeFactoryMultiplier
 - FactoryUIManager, 82
- upgradeMainBtn
 - FactoryUIManager.FactoryUI, 72
- upgradeMultiplierBtn
 - FactoryUIManager.FactoryUI, 72
- upgradePanel
 - FactoryUIManager.FactoryUI, 72
- upgradeProductionBtn
 - FactoryUIManager.FactoryUI, 72
- upgradesVisible
 - FactoryUIManager.FactoryUI, 72
- value
 - Resource, 169
- valueLabel
 - BuffVisual, 31
- waitTime
 - SkillCheckScript, 195
- WhatEyesDontSee
 - CharacterClass, 48
- whatEyesDontSee
 - CharacterClass, 53
- Zero
 - QuarkType, 157
- ZoomEffect
 - DynamicBackground, 64
- zoomValue
 - DynamicBackground, 65